

国外计算机科学经典教材

Scientific Computing with MATLAB

MATLAB 科学计算

(德) Alfio Quarteroni
Fausto Saleri
李敏波

著
译



清华大学出版社

Scientific Computing with MATLAB

本书介绍了如何利用计算机解决若干类数学问题的数值计算方法。书中主要阐述了如何计算连续函数的零点和积分，如何求解线性系统，如何利用多项式处理函数逼近，以及如何构造微分方程的精确近似解。为了让所述内容更加生动和具体，书中始终都结合 MATLAB 编程环境来进行阐述。书中对所介绍的全部算法都作了程序演示，以便读者可以对这些算法的理论性能，如稳定性、准确性和复杂性作出实时的定量评估。对于在练习和例题中出现的一些问题，本书也给出了解决办法，这些问题大多来自于具体的实际应用。对于本书未涉及到的相关问题，在每章的结束都给出了相关参考文献，以便读者进行更深入的理解和学习。

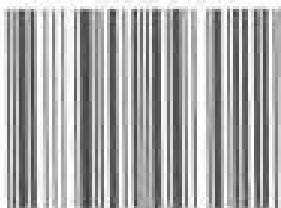
本书主要内容

- 非线性方程
- 函数和数据的逼近
- 数值微分与数值积分
- 线性系统
- 特征值和特征向量
- 常微分方程
- 边值问题数值方法

本书读者对象

本书适合作为高等院校计算机科学与工程专业的教材。

ISBN 7-302-09302-4



9 787302 093022 >

定价：26.00 元

国外计算机科学经典教材

MATLAB 科学计算

(德) Alfio Quarteroni 著
Fausto Saleri
李敏波 译

清华大学出版社

北 京

Alfio Quarteroni, Fausto Saleri

Scientific Computing with MATLAB

EISBN: 3-540-44363-0

Copyright© 2002 by Springer Press Ltd.

Authorized translation from the English language edition published by Springer Press Ltd

All rights reserved. For Sale in the People's Republic of China only.

Chinese simplified language edition published by Tsinghua University Press.

本书中文简体字版由 Springer 出版公司授权清华大学出版社出版。未经出版者书面许可,不得以任何方式复制或抄袭本书内容。

北京市版权局著作权合同登记号 图字: 01-2004-1049

版权所有,翻印必究。举报电话: 010-62782989 13901104297 13801310933

本书封面贴有清华大学出版社防伪标签,无标签者不得销售。

本书防伪标签采用清华大学核研院专有核径迹膜防伪技术,用户可通过在图案表面涂抹清水,图案消失,水干后图案复现;或将表面膜揭下,放在白纸上用彩笔涂抹,图案在白纸上再现的方法识别真伪。

图书在版编目(CIP)数据

MATLAB 科学计算/(德)夸特罗尼,(德)色拉瑞著;李敏波译.-北京:清华大学出版社,2005.1

书名原文:Scientific Computing with MATLAB

(国外计算机科学经典教材)

ISBN 7-302-09302-4

I. M… II. ①夸… ②色… ③李… III. 计算机辅助计算—软件包, MATLAB—教材 IV. TP391.75

中国版本图书馆 CIP 数据核字(2004)第 089269 号

出 版 者:清华大学出版社 地 址:北京清华大学学研大厦

<http://www.tup.com.cn> 邮 编:100084

社 总 机:010-62770175 客 户 服 务:010-62776969

组稿编辑:曹 康

文稿编辑:侯 斌

封面设计:康 博

版式设计:康 博

印 刷 者:北京市通州大中印刷厂

装 订 者:三河市化甲屯小学装订二厂

发 行 者:新华书店总店北京发行所

开 本:185×260 印张:14.75 字数:306 千字

版 次:2005 年 1 月第 1 版 2005 年 1 月第 1 次印刷

书 号:ISBN 7-302-09302-4/TP·6528

印 数:1~4000

定 价:26.00 元

本书如存在文字不清、漏印以及缺页、倒页、脱页等印装质量问题,请与清华大学出版社出版部联系调换。联系电话:(010)62770175-3103 或(010)62795704

出版说明

近年来,我国高等学校的计算机学科教育进行了较大的改革,急需一批门类齐全、具有国际水平的计算机经典教材,以适应当前的教学需要。引进国外经典教材,可以了解并吸收国际先进的教学思想和教学方法,使我国的计算机学科教育能够与国际接轨,从而培育更多具有国际水准的计算机专业人才,增强我国信息产业的核心竞争力。Pearson、Thomson、McGraw-Hill、Springer、John Wiley 等出版集团都是全球最有影响的图书出版机构,它们在高等教育领域也都有着不凡的表现,为全世界的高等学校计算机教学提供了大量的优秀教材。为了满足我国高等学校计算机学科的教学需要,我社计划从这些知名的国外出版集团引进计算机学科经典教材。

为了保证引进版教材的质量,我们在全国范围内组织并成立了“清华大学计算机外版教材编审委员会”(以下简称“编委会”),旨在对引进教材进行审定、对教材翻译质量进行评审。“编委会”成员皆为全国各类重点院校教学与科研第一线的知名教授,其中许多教授为各校相关院、系的院长或系主任。“编委会”一致认为,引进版教材要能够满足国内各高校计算机教学与国际接轨的需要,要有特色风格,有创新性、先进性、示范性和一定的前瞻性,是真正的经典教材。为了保证外版教材的翻译质量,我们聘请了高校计算机相关专业教学与科研第一线的教师及相关领域的专家担纲译者,其中许多译者为海外留学回国人员。为了尽可能地保留与发扬教材原著的精华,在经过翻译和编辑加工之后,由“编委会”成员对文稿进行审定,以最大程度地弥补和修正在前面一系列加工过程中对教材造成的误差和瑕疵。

由于时间紧迫和能力所限,本套外版教材在出版过程中还可能存在一些不足和遗憾,欢迎广大师生批评指正。同时,也欢迎读者朋友积极向我们推荐各类优秀的国外计算机教材,共同为我国高等学校的计算机教育事业贡献力量。

清华大学出版社

国外计算机科学经典教材

编审委员会

主任委员:

孙家广 清华大学教授

副主任委员:

周立柱 清华大学教授

委员（按姓氏笔画排序）:

王成山	天津大学教授
王 珊	中国人民大学教授
冯少荣	厦门大学教授
冯全源	西南交通大学教授
刘乐善	华中科技大学教授
刘腾红	中南财经政法大学教授
占根林	南京师范大学教授
孙吉贵	吉林大学教授
阮秋琦	北京交通大学教授
何 晨	上海交通大学教授
吴百锋	复旦大学教授
李 彤	云南大学教授
沈钧毅	西安交通大学教授
邵志清	华东理工大学教授
陈 纯	浙江大学教授
陈 钟	北京大学教授
陈道蓄	南京大学教授
周伯生	北京航空航天大学教授
孟祥旭	山东大学教授
姚淑珍	北京航空航天大学教授
徐佩霞	中国科学技术大学教授
徐晓飞	哈尔滨工业大学教授
秦小麟	南京航空航天大学教授
钱培德	苏州大学教授
曹元大	北京理工大学教授
龚声蓉	苏州大学教授
谢希仁	中国人民解放军理工大学教授

前 言

本书主要介绍科学计算方法，列举了若干类数学问题的数值解法，这些问题利用解析方法一般难以求解。书中主要介绍了如何计算连续函数的零点和积分，如何求解线性系统，如何利用多项式法处理函数逼近，以及如何构造微分方程精确的近似解问题。

为此，第1章中主要介绍了计算机在对实数、复数、向量和矩阵进行存储和运算时所遵循的运行规则。

为了使所叙述的内容更加生动和具体，书中自始至终都结合 MATLAB 编程环境来进行阐述。读者会逐渐地熟悉 MATLAB¹主要的函数、语句和结构。本书对每章中提到的算法都将给出具体程序实现，以便读者对这些算法的理论性能，如稳定性、准确性和复杂性作出实时的定量评估。书中对于在习题和例题中提出的若干问题也给出了解决办法，这些问题多来自于某些特定的实际应用。

在每章的最后，都有一节专门指出未提及的相关内容，同时给出相关参考文献，以便读者进行更深入的理解和学习。

书中会经常提到参考书 [QSS00]，它对本书所提到的问题进行了更深入的讲解，并对有关结论进行了理论证明。而关于 MATLAB 更为详尽的介绍，我们向读者推荐 [HH00]。本书中出现的所有程序均可从 mox.polimi.it/Springer 站点下载。

本书对读者并无特别要求，读者只需具备微积分的基础知识即可阅读本书。

尽管如此，在第1章中还是对微积分和几何学中的基本概念进行了回顾，这些概念在全书中都有广泛的应用。

最后，作者非常感谢海德堡 Springer-Verlag 的 Thanh-Ha Le Thi，Springer-Italia 的 Francesca Bonadei 和 Marina Forlizzi 在整个项目中给予的友好合作。同时感谢 Cardiff 大学的 Eastham 教授对全书手稿所做的文字编辑工作，他提出的许多宝贵意见，对全书的多处内容进行了改进。

Alfio Quarteroni, Fausto Saleri

2003 年 5 月于米兰和洛桑

¹ MATLAB 是 The MathWorks 公司的商标，公司地址：24 Prime Park Way, Natick MA 01760，电话：001+508-647-7000，传真：001+508-647-7001。

目 录

第 1 章	绪论	1
1.1	实数	1
1.1.1	实数的表示	1
1.1.2	浮点数的运算	3
1.2	复数	5
1.3	矩阵	7
1.4	实函数	12
1.4.1	零点	13
1.4.2	多项式	14
1.4.3	积分和微分	16
1.5	误差	18
	代价	21
1.6	MATLAB 简介	23
1.6.1	MATLAB 语句	24
1.6.2	MATLAB 编程	25
1.7	补充说明	29
1.8	习题	29
第 2 章	非线性方程	31
2.1	二分法	32
2.2	Newton 法	35
2.3	固定点迭代	39
2.4	补充说明	43
2.5	习题	45
第 3 章	函数和数据的逼近	48
3.1	插值	50
3.1.1	Lagrangian 多项式插值	51
3.1.2	Chebyshev 插值	55
3.1.3	三角插值和 FFT	56
3.2	分段线性插值	61

3.3 样条函数逼近	62
3.4 最小平方法	64
3.5 补充说明	67
3.6 习题	68
第 4 章 数值微分与数值积分	70
4.1 函数导数的逼近	71
4.2 数值积分	73
4.2.1 中点公式	73
4.2.2 梯形公式	76
4.2.3 Simpson 公式	79
4.3 Simpson 自适应算法	81
4.4 补充说明	84
4.5 习题	85
第 5 章 线性系统	88
5.1 LU 因式分解法	90
5.2 主元素技术	97
5.3 LU 因式分解的精确度	99
5.4 三对角系统的解法	102
5.5 迭代方法	104
5.6 迭代法的终止条件	109
5.7 Richardson 方法	111
5.8 补充说明	114
5.9 习题	115
第 6 章 特征值和特征向量	118
6.1 幂法	121
6.2 幂法的变形	124
6.3 计算移位量的方法	126
6.4 计算全部特征值的方法	128
6.5 补充说明	129
6.6 习题	129
第 7 章 常微分方程	132
7.1 柯西问题	133
7.2 欧拉方法	134
7.3 Crank-Nicolson 方法	139

7.4	零稳定性	141
7.5	无边界区间上的稳定性	142
7.6	高阶方法	151
7.7	预测纠正法	152
7.8	微分方程系统	154
7.9	补充说明	158
7.10	习题	159
第 8 章	边值问题数值方法	162
8.1	边值问题逼近	165
8.1.1	有限差分逼近	165
8.1.2	有限元法逼近	167
8.2	二维有限差分	170
8.3	补充说明	178
8.4	习题	178
第 9 章	习题解答	180
9.1	第 1 章	180
9.2	第 2 章	182
9.3	第 3 章	187
9.4	第 4 章	191
9.5	第 5 章	195
9.6	第 6 章	201
9.7	第 7 章	204
9.8	第 8 章	212
参考书目		217

第 1 章 绪 论

本书将系统地应用基本的数学概念，这些概念读者应该已经知道，但可能不会立即回想起它们。

因此本章将主要帮助读者回顾这些概念，并介绍属于数值分析领域的新概念。同时将使用 MATLAB(MATrix LABoratory)工具来帮助阐述这些概念的含义和作用，MATLAB 提供了用于科学计算的可编程、可视化的集成环境。1.6 节对 MATLAB 进行简要的介绍，这些介绍足以使读者读懂书中的后续内容，但我们还是向有兴趣的读者推荐手册 [HH00]，该手册对 MATLAB 语言进行了完整的描述和说明。

本章将扼要地介绍微积分学、线性代数学和几何学中的基本概念，不过将采用一种更有利于把它们应用到科学计算之中的方式来介绍它们。

1.1 实数

许多人都知道实数集 $\mathbf{R}$ ，但也许很少人能知道计算机处理实数的方式。一方面，计算机的资源是有限的，因此它只能表示出实数集 $\mathbf{R}$ 的有限维的子集 F 。子集 F 中的数被称为浮点数。另一方面，正如将要在 1.1.2 小节中所讲的，用于描述 F 的性质与用于描述实数集 $\mathbf{R}$ 的性质是不同的。原因是任意实数 x 原则上都被机器截取了一定位数，这就产生了一个新的数值(称为浮点数)，记为 $f(x)$ ，而这个浮点数并不一定要与原来的数 x 保持完全一致。

1.1.1 实数的表示

为了更好地理解集合 $\mathbf{R}$ 和 F 之间的区别，我们通过几个 MATLAB 实例来说明计算机(例如一台 PC 机)处理实数的方式。至于是采用 MATLAB 语言，还是采用其他的语言，完全根据使用的方便程度来决定。实际上计算结果主要取决于计算机的工作方式，而很少取决于所采用的编程语言。考虑有理数 $x=1/7$ ，它表示成小数为 $0.142\ 857$ 。由于它的小数位数是无限的，因此它的十进制表示也是无限的。为了得到它的计算机表示，在提示符后输入分数 $1/7$ ，得到：

```
>>1/7
ans=
    0.1429
```

得到的结果是一个小数点后只有四位的数，而且它的最后一位与初始数字的对应位不同。如果把输入换成 $1/3$ ，我们将会得到 0.3333，此时第四位小数仍然与初始数值的对应位保持一致。这是由于实数在计算机内是四舍五入的。这意味着：首先，只能返回一个固定位数的十进制数；其次，对于最后一位数字来说，如果与其相邻的下一位数字大于或等于 5，那么最后一位数字便加 1。

需要注意的一点是，只使用一个 4 位十进制数去表示一个实数是存在问题的。实际上，数字在内部是用 16 位十进制数来表示的，我们所作看到的结果也只是 MATLAB 多种输出格式中的一种。同样的一个数值，根据其所做的特定格式的说明的不同，便可以得到它的不同表示方式。例如，对于数 $1/7$ 来说，有下列一些可能的输出格式：

format long	输出	0.14285714285714
format short e	输出	1.4286e-01
format long e	输出	1.428571428571428e-01
format short g	输出	0.14286
format long g	输出	0.142857142857143

可以看出一些表示方式比其他几种方式更加接近计算机的内部表示。实际上，计算机通常采用下列方式来存储一个实数：

$$x = (-1)^s \cdot (0.a_1a_2\cdots a_t) \cdot \beta^e = (-1)^s \cdot m \cdot \beta^{e-t}, \quad a_1 \neq 0 \quad (1.1)$$

其中 s 只能取 0 或 1， β (大于或等于 2 的整数) 是计算机采用的底数，整数 m 称为尾数，尾数的长度 t 是所存储数据 a_i 的最大位数，范围介于 0 和 $\beta - 1$ 之间，整数 e 称为指数。长型 e 数据采用的表示方法就非常类似于这种形式，其中 e 代表指数，表示指数大小的数位前面加有正负运算符号，写于字符 e 的右侧。式(1.1)中所给数值的小数点位置是不固定的，称之为浮点数。数位 $a_1a_2\cdots a_p$ (其中 $p \leq t$) 称为数 x 的有效数位。

$a_1 \neq 0$ 这一条件保证了同一个数值不会出现多种表示形式。例如，如果没有这一限制条件，数 $1/10$ 既可以表示为(以 10 为底) 0.1×10^0 ，也可以表示成 0.01×10^1 等其他形式。

因此，集合 F 可以由底数 β ，有效位数 t 的大小和指数 e 的变化范围 (L, U) (其中 $L < 0, U > 0$) 完全描述确定。此时集合可表示为 $F(\beta, t, L, U)$ 。例如，在 MATLAB 中集合 F 可以表示为 $F(2, 53, -1021, 1024)$ (实际上，以 2 为底的含有 53 个有效数位的数值与 MATLAB 用长型所显示出来的以 10 为底的含有 15 个有效数位的数值是一致的)。

当用集合 F 中的数 $f(x)$ 来代替一个不等于 0 的实数 x 时，不可避免地会造成舍入误差。由于满足下式：

$$\frac{|x - fl(x)|}{|x|} \leq \frac{1}{2} \varepsilon_M \quad (1.2)$$

所以舍入误差一般都比较小, 公式中 $\varepsilon_M = \beta^{1-t}$ 代表距 1 最近的浮点数与 1 之间的差值。注意 ε_M 是 β 和 t 的函数。在 MATLAB 中, 可以通过 eps 命令来得到 ε_M , 结果为 $\varepsilon_M = 2^{-52} \approx 2.22 \times 10^{-16}$ 。需要指出, 在式(1.2)中描述的是 x 的相对误差, 它比绝对误差 $|x - fl(x)|$ 有着更实际的意义。实际上, 后者未考虑 x 量值本身的作用, 而前者考虑了这个问题。

0 不属于集合 F , 因为如果 0 属于集合 F , 则在式(1.1)中 $a_1 = 0$, 此时必须对它进行单独处理。此外, 由于 L 和 U 都是有限数, 所以那些绝对值极大或极小的数值都不能表示出来。如果精确地描述, F 集中最小的和最大的正实数分别表示为:

$$x_{\min} = \beta^{L-1}, \quad x_{\max} = \beta^U (1 - \beta^{-t})$$

在 MATLAB 中, 可以利用 realmin 和 realmax 函数来分别得到它们, 结果如下:

$$\begin{aligned} x_{\min} &= 2.225\,073\,858\,507\,201 \times 10^{-308} \\ x_{\max} &= 1.797\,693\,134\,862\,315\,8 \times 10^{308} \end{aligned}$$

遇到比 $x_{\min}$ 还小的正数时会输出下溢信息, 同时把这个数按 0 处理或采用特定方式处理(可参照 [QSS00] 第二章)。对于那些比 $x_{\max}$ 还大的数, 会给出上溢信息, 并把它存储在变量 Inf 中(计算机用 Inf 变量来表示无穷大 $+\infty$)。集合 F 中的元素在 $x_{\min}$ 处更为密集, 在接近 $x_{\max}$ 处密集度减小。实际上, 集合 F 中与 $x_{\max}$ 距离最近的数(在 $x_{\max}$ 的左侧)和与 $x_{\min}$ 最近的数(在其右侧)分别是:

$$\begin{aligned} x_{\min}^- &= 1.797\,693\,134\,862\,315\,7 \times 10^{308} \\ x_{\min}^+ &= 2.225\,073\,858\,507\,202 \times 10^{-308} \end{aligned}$$

可见 $x_{\min}^+ - x_{\min} \approx 10^{-323}$, 而 $x_{\max} - x_{\max}^- \approx 10^{292}$ (!)。但从式(1.2)可以看出这两种情况下的相对距离都是很小的。

1.1.2 浮点数的运算

由于 F 集合只是集合 $\mathbf{R}$ 的一个特定的子集, 所以在集合 $\mathbf{R}$ 的运算中适用的方法并不完全适用于浮点数的代数运算。具体来讲, 交换律对于加法运算 ($fl(x+y) = fl(y+x)$) 与乘法运算 ($fl(xy) = fl(yx)$) 仍然适用, 但是其他规则如结合律和分配律已不再适用了。而且 0 也不再是惟一的。实际上, 当把变量 a 赋值为 1 并执行下面

的表达式时:

```
>>a=1; b=1; while a+b~=a; b=b/2; end
```

可见, 循环每执行一次, 只要此时 a 与 b 相加之和不等 a , 变量 b 便会被除以 2。如果按通常的方式对一个实数进行这种操作, 这个程序将永远不会停止, 而在本例中, 程序在运行一定次数以后便会结束, 并返回 b 的值: $1.1102e-16 = \varepsilon_M/2$ 。因此我们至少可以看到: 虽然 b 此时并不等于 0, 但却有 $a+b=a$ 成立。这种情况之所以能够发生, 原因就在于集合 F 是由相互独立的数值所构成的; 当把两个数 a 和 b 相加时, 其中 $b < a$, 并且 b 小于 ε_M 时, 总会得到 $a+b$ 等于 a 的结果。

当上溢或下溢的情况发生时, 结合律便不再适用了。设 $a=1.0e+308$, $b=1.1e+308$, $c=-1.001e+308$, 用下面两种方式分别来进行加法运算, 可以得到:

$$a+(b+c)=1.0990e+308, \quad (a+b)+c=\text{Inf}$$

这个例子清楚地说明了当两个符号相反、绝对值相近的数值进行加法运算时会出现什么情况。此处得到的结果是错误的, 这种情况被称为有效数位的丢失或消去。例如在计算表达式 $((1+x)-1)/x$ (很明显, 对于任意 $x \neq 0$, 结果应为 1) 的结果时:

```
>>x=1.e-15; ((1+x)-1)/x  
ans=  
1.1102
```

可见结果很不准确, 相对误差竟大于 11%!

再看另一个数值相消的例子, 计算下列函数:

$$f(x)=x^7-7x^6+21x^5-35x^4+35x^3-21x^2+7x-1 \quad (1.3)$$

在横轴上区间 $[1-2 \times 10^{-8}, 1+2 \times 10^{-8}]$ 内均匀分布的 401 个点上, 得到的是如图 1.1 所示的杂乱曲线 (实际上该运算就是 $(x-1)^7$, 它是连续的, 在 $x=1$ 这样小的邻域内等于零函数)。在 1.4 节, 会给出描绘曲线所用的命令。

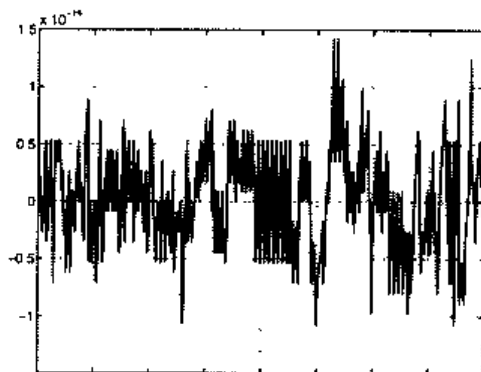


图 1.1 由消去误差导致的函数(1.3)的波动曲线

最后需要注意的一个比较有趣的问题是：在集合 F 中并没有包括那些含义不明确的表达形式，如 $0/0$ 或 ∞/∞ ，遇到这类表达形式，将会给出提示信息 NaN(not a number, 在 MATLAB 中用 NaN 来表示)，对此声明通常的数值计算规则对它们并不适用。

注 1.1 尽管舍入误差通常都很小，但如果它们在长型和复数型的运算规则中不断累加时，也可能导致灾难性的后果。有两个真实的事例足以说明问题，一个是发生于 1996 年 6 月 4 日的阿里娅娜火箭的爆炸事故，这次事故就是由于控制台上计算机的溢出导致的；另一个是 1991 年海湾战争期间，曾有一枚美国的爱国者导弹落入了美军军营，这也是由于计算导弹飞行轨道时产生的舍入误差引起的。

另一个例子倒没有多少灾难性(但同样会引起不少麻烦)，它就是下面的序列：

$$z_2 = 2, \quad z_{n+1} = 2^{n-1/2} \sqrt{1 - \sqrt{1 - 4^{1-n} z_n^2}}, \quad n = 2, 3, \dots \quad (1.4)$$

当 n 趋于无穷时，序列将收敛于 π 。当利用 MATLAB 来计算 z_n 时， π 和 z_n 之间的相对误差在前 16 次迭代运算中是逐渐减少的，但当迭代运算继续时，由于舍入误差的影响，相对误差会逐渐增加(见图 1.2)。

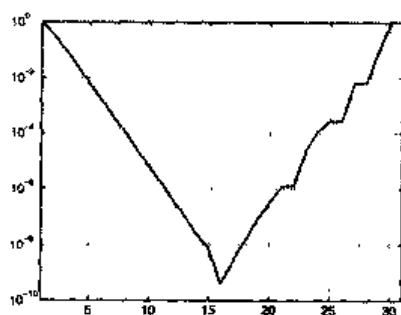


图 1.2 相对误差 $|\pi - z_n|/\pi$ 与 n 的对数曲线

参见习题 1.1~1.2。

1.2 复数

复数集用 $\mathbf{C}$ 来表示，复数的形式为 $z = x + iy$ ，其中 $i = \sqrt{-1}$ 是虚数单位(此时有 $i^2 = -1$)，而 $x = \text{Re}(z)$ 和 $y = \text{Im}(z)$ 分别是 z 的实部和虚部。复数在计算机上通常采用实数对的形式来表示。

在 MATLAB 中，变量 i 和 j 如果没有经过重新定义，则它们代表虚数单位。要引入一个实部为 x ，虚部为 y 的复数，只需写成 $x + i*y$ ；也可以采用另一种方式，使用命令 `complex(x, y)` 来实现。再来看一下复数 z 的三角表示法(或称极坐标表示法)：

$$z = \rho e^{i\theta} = \rho(\cos\theta + i\sin\theta), \quad (1.5)$$

其中 $\rho = \sqrt{x^2 + y^2}$ 是复数的绝对值(可通过 `abs(z)` 命令来得到), θ 为相角, 它是在复平面 (x, y) 上向量 z 与 x 轴之间的夹角。 θ 可以通过键入 `angle(z)` 命令来得到。于是式 (1.5) 可以表示为:

$$\text{abs}(z) * (\cos(\text{angle}(z)) + i * \sin(\text{angle}(z)))$$

通过 `compass(z)` 命令可以得到一个或多个复数的极坐标形式, 其中 z 既可以是单一的复数, 也可以是一个复向量, 该向量的元素为复数。例如键入下列语句:

```
>>z=3+i*3;compass(z);
```

可以得到如图 1.3 所示的图表。

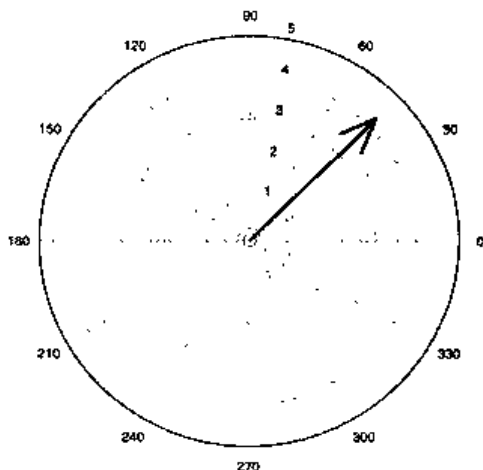


图 1.3 MATLAB 的 compass 命令的输出

对于任意给定的复数 z , 可以利用 `real(z)` 命令来求得实部, 利用 `imag(z)` 命令求得虚部, 并可以利用 `conj(z)` 函数来方便地得到 z 的复共轭 $\bar{z} = x - iy$ 。

在 MATLAB 进行所有的运算之前都暗含着一个假设: 所有操作数和结果一律按复数对待, 因此有时会有一些令人意外的现象出现。例如, 如果利用 MATLAB 命令 $(-5)^{(1/3)}$ 来求 -5 的立方根, 此时得到的结果并不是 $-1.7099\cdots$, 而是复数 $0.8500 + 1.4809i$ (设符号 $^$ 代表幂指数运算)。实际上, 所有以 $\rho e^{i(\theta + 2k\pi)}$ (其中 k 为整数) 这一形式表示的数与 z 是难以区分的。计算 $\sqrt[3]{z}$ 可以得到 $\sqrt[3]{\rho} e^{i(\theta/3 + 2k\pi/3)}$, 也就是下列三种不同形式:

$$z_1 = \sqrt[3]{\rho} e^{i\theta/3}, \quad z_2 = \sqrt[3]{\rho} e^{i(\theta/3 + 2\pi/3)}, \quad z_3 = \sqrt[3]{\rho} e^{i(\theta/3 + 4\pi/3)}$$

ATLAB 将从其中选择一个作为输出, 选择的方法是在复平面上从实轴开始逆

时针旋转扫描, 最先遇到的那个值便作为结果输出。由于 $z=-5$ 的极坐标表示形式是 $\rho e^{i\theta}$, 其中 $\rho=5$, $\theta=-\pi$, 则三个根分别为(图 1.4 表示出了三个根在 Gauss 平面上的分布):

$$z_1 = \sqrt[3]{5} (\cos(-\pi/3) + i \sin(-\pi/3)) \simeq 0.8550 - 1.4809i$$

$$z_2 = \sqrt[3]{5} (\cos(\pi/3) + i \sin(\pi/3)) \simeq 0.8550 + 1.4809i$$

$$z_3 = \sqrt[3]{5} (\cos(-\pi) + i \sin(-\pi)) \simeq -1.71$$

可见 MATLAB 选择了第二个根作为输出(参见图 1.4)。

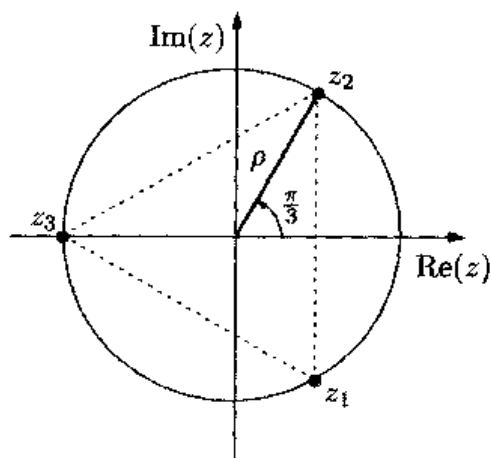


图 1.4 实数-5 的立方根在复平面上的表示

最后, 由式(1.5)可得 Euler 公式:

$$\cos(\theta) = \frac{1}{2}(e^{i\theta} + e^{-i\theta}), \quad \sin(\theta) = \frac{1}{2i}(e^{i\theta} - e^{-i\theta}) \quad (1.6)$$

1.3 矩阵

设 n 和 m 是正整数, 一个 m 行 n 列的矩阵由 $m \times n$ 个元素 a_{ij} 组成, 其中 $i=1, \dots, m$, $j=1, \dots, n$, 用下面的阵列来表示:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \quad (1.7)$$

矩阵 A 可简写为 $A=(a_{ij})$ 。若 A 中元素为实数, 可记为 $A \in \mathbf{R}^{m \times n}$, 如果为复数, 可记为 $A \in \mathbf{C}^{m \times n}$ 。

n 维方阵即是 $m=n$ 时的矩阵。从矩阵中单独提出一列元素，则称之为列向量，而从矩阵中单独提出一行元素称为行向量。

为了在 MATLAB 中引入矩阵，需要逐行地写入所有元素，给出表示矩阵的符号，同时行与行之间要互相间隔开。例如键入命令

```
>>A=[1 2 3;4 5 6]
```

得到

```
A=
     1     2     3
     4     5     6
```

它是一个 2×3 的矩阵，矩阵的元素就是输入的数值。零矩阵 0 即是矩阵中所有元素 $a_{ij}=0$ (其中 $i=1, \dots, m, j=1, \dots, n$) 的矩阵；一个 $m \times n$ 阶的矩阵可以用 MATLAB 命令 `zeros(m, n)` 得到。键入命令 `eye(m, n)` 可构造一个对角阵，这个对角阵主对角线的元素都为 1，其余元素都为 0。

一个 $m \times n$ 矩阵 A 的主对角线是由元素 $a_{ii}(i=1, \dots, \min(m, n))$ 所构成的对角线。特别地，执行命令 `eye(n)` (即 `eye(n, n)` 的简化形式) 可以得到一个主对角线均为 1 的 n 维方阵，称为单位阵，记为 I 。

定义下列运算：

1. 如果 $A=(a_{ij})$, $B=(b_{ij})$ 是 $m \times n$ 矩阵， A 与 B 加法运算定义为： $A+B=(a_{ij}+b_{ij})$ ；
2. 矩阵 A 与一个实数或复数 λ 之间的乘法运算定义为 $\lambda A=(\lambda a_{ij})$ ；
3. 两个矩阵进行乘法运算，要求这两个矩阵有相邻的公共维，即如果矩阵 A 是 $m \times p$ 的，矩阵 B 是 $p \times n$ 的， p 为正整数，则可以进行乘法运算。相乘后的矩阵 $C=AB$ 是一个 $m \times n$ 的矩阵，其元素

$$c_{ij} = \sum_{k=1}^p a_{ik} b_{kj}, \quad \text{其中 } i=1, \dots, m, \quad j=1, \dots, n$$

下面给出一个矩阵加法和乘法运算的例子：

```
>>A=[1 2 3;4 5 6]; B=[7 8 9;10 11 12]; C=[13 14;15 16;17 18];
>>A+B
ans=
     8     10     12
    14     16     18
>>A*C
ans=
    94    100
   229    224
```

需要注意，如果对两个维数不匹配的矩阵进行操作，MATLAB 将给出诊断信息。

例如:

```
>>A+C
???Error using ==>+
Matrix dimensions must agree.
>> A*B
???Error using ==>*
Inner matrix dimensions must agree.
```

注 1.2 对于任意给定的矩阵 A , MATLAB 的 `dim=size(A)` 命令会返回一个二元变量 `dim=[m, n]`, 它给出的是矩阵 A 的行数和列数。更进一步, `whos` 命令可以列出当前工作会话中所有参与运算的变量类型。例如, 如果已经按前述方法定义了矩阵 A 、 B 和 C , 则执行 `whos` 命令会得到如下信息:

```
Name      Size      Bytes Class
A          2x3       48   double array
B          2x3       48   double array
C          2x3       48   double array
Grand total is 18 elements using 144 bytes
```

如果 A 是一个 n 阶方阵, 则它的逆矩阵(假设逆矩阵存在)也是一个 n 阶方阵, 记为 A^{-1} , 并满足关系式 $AA^{-1}=A^{-1}A=I$ 。可以通过 `inv(A)` 函数求得 A^{-1} 。对于矩阵 A , 当且仅当它的行列式的值, 即 `det(A)` 函数返回的值不为 0 时, 逆矩阵存在。而行列式不为 0 则要求矩阵 A 的列向量之间是线性无关的(见 1.3.1 小节)。一个矩阵 A 的行列式可用递归表达式(Laplace 公式)定义如下:

$$\det(A) = \begin{cases} a_{11} & \text{如果 } n=1 \\ \sum_{j=1}^n \Delta_{ij} a_{ij}, & n>1, \quad \forall i=1, \dots, n \end{cases} \quad (1.8)$$

其中 $\Delta_{ij} = (-1)^{i+j} \det(A_{ij})$, A_{ij} 是矩阵 A 中去掉第 i 行第 j 列的所有元素之后, 由余下的元素所构成的矩阵。特别地,

在 $A \in \mathbf{R}^{1 \times 1}$ 时, 有 $\det(A) = a_{11}$

在 $A \in \mathbf{R}^{2 \times 2}$ 时, 有 $\det(A) = a_{11} a_{22} - a_{12} a_{21}$

在 $A \in \mathbf{R}^{3 \times 3}$ 时, 有 $\det(A) = a_{11} a_{22} a_{33} + a_{31} a_{12} a_{23} + a_{21} a_{13} a_{32} - a_{11} a_{23} a_{32} - a_{21} a_{12} a_{33} - a_{31} a_{13} a_{22}$

矩阵乘法运算有下列性质: 如果 $A=BC$, 则 $\det(A)=\det(B) \cdot \det(C)$ 。

对于一个 2×2 矩阵, 计算它的逆矩阵和行列式的值可以采用下面的指令:

```
>>A=[1 2; 3 4];
>>inv(A)
ans=
    -2.0000    1.0000
```

```

1.5000    -0.5000
>>det(A)
ans=
    -2

```

如果对于一个奇异矩阵进行求逆操作, MATLAB 将会返回诊断信息, 同时给出一个所有元素均为 Inf 的矩阵, 正如在下面的例子所看到的:

```

>>A=[1 2; 0 0]
>>inv(A)
Warning: Matrix is singular to working precision.
Ans=
    Inf    Inf
    Inf    Inf

```

对于特殊形式的方阵, 计算其逆矩阵和行列式非常简单。特别当 A 是一个对角阵时, 即矩阵 A 除主对角线元素以外其余元素均为 0, 则行列式的值为 $\det(A) = a_{11}a_{22}\dots a_{nn}$ 。特别地, 当且仅当 A 的对角线上的所有元素 $a_{kk} \neq 0$ 时, A 为非奇异矩阵。此时 A 的逆矩阵仍是对角阵, 其所有元素是矩阵 A 的对应元素的倒数。

设 v 是 n 维向量, MATLAB 的 `diag(v)` 函数会生成一个对角阵, 对角阵的元素与向量 v 的对应元素相同。更一般地, `diag(v, m)` 函数会生成一个 $n+\text{abs}(m)$ 维的方阵, 该方阵的上方第 m 个对角线(即由元素 $i, i+m$ 构成的对角线)的元素与向量 v 相同, 而其元素均为零。需要注意的是该函数只有在 m 为负时才有效, 这时实际起作用的是处于下方对角线上的元素。

例如, 取 $v=[1\ 2\ 3]$, 则

```

>>A=diag(v, -1)
A=
    0    0    0    0
    1    0    0    0
    0    2    0    0
    0    0    3    0

```

特殊矩阵还有上三角阵和下三角阵。如果一个 n 维方阵 A 的主对角线以上的(以下的)所有元素均为 0, 则称方阵 A 为下三角阵(上三角阵)。计算上三角阵和下三角阵的行列式比较简单, 只须把主对角线元素逐个相乘即可。

通过 `tril(A)` 和 `triu(A)` 函数, 可以从一个 n 维方阵中提取出它的下三角阵和上三角阵。更一般地, 函数 `tril(A, m)` 和 `triu(A, m)`, 其中 m 取值范围从 $-n$ 到 n , 还允许从矩阵 A 中分别提取出第 m 条对角线及其以上(下)的元素。

例如设矩阵 $A=[3\ 1\ 2; -1\ 3\ 4; -2\ -1\ 3]$, 利用函数 $L1=\text{tril}(A)$ 可以得到:

```

L1=
    3    0    0

```

$$\begin{bmatrix} -1 & 3 & 0 \\ -2 & -1 & 3 \end{bmatrix}$$

而利用函数 $L2=\text{tril}(A, -1)$ 可以得到:

$$L2 = \begin{bmatrix} 0 & 0 & 0 \\ -1 & 0 & 0 \\ -2 & -1 & 0 \end{bmatrix}$$

最后回顾一下转置矩阵的概念, 矩阵 $A \in \mathbf{R}^{m \times n}$ 的转置矩阵 $A^T \in \mathbf{R}^{n \times m}$ 就是矩阵 A 的行列互换之后得到的矩阵。当 $A=A^T$ 时, 称矩阵 A 为对称阵。在 MATLAB 中, 矩阵 A 的转置矩阵采用符号 A' 来表示。

1.3.1 向量

本书采用黑体符号来表示向量, 符号 $\mathbf{v}$ 表示一个列向量, 向量的第 i 个元素为 v_i , 当所有的元素都是实数时, 可以记作 $\mathbf{v} \in \mathbf{R}^n$ 。

在 MATLAB 中认为向量是一类特殊的矩阵。引入一个列向量需要在方括号内键入每个元素的值, 每个元素之间采用分号隔开; 对于行向量, 同样要输入每个元素的值, 但此时每个元素之间采用空格或逗号作为间隔。例如, 输入 $\mathbf{v}=[1; 2; 3]$ 和 $\mathbf{w}=[1 \ 2 \ 3]$ 便分别定义了列向量 $\mathbf{v}$ 和行向量 $\mathbf{w}$, 它们都是一维向量。 $\text{zeros}(n, 1)$ 函数(或 $\text{zeros}(1, n)$ 函数)可以生成一个所有元素均为 0 的 n 维列向量(行向量), 记为 $\mathbf{0}$ 。类似地, $\text{ones}(n, 1)$ 函数可以生成一个所有元素都为 1 的 n 维列向量, 记为 $\mathbf{1}$ 。而函数 $\mathbf{v}=[]$ 则初始化了一个空向量。

对于一个向量组 $\{\mathbf{y}_1, \dots, \mathbf{y}_m\}$, 如果方程 $\alpha_1 \mathbf{y}_1 + \dots + \alpha_m \mathbf{y}_m = \mathbf{0}$ 只有在 $\alpha_1, \dots, \alpha_m$ 全等于零时才成立, 则称这个向量组是线性无关的。在 n 维空间 $\mathbf{R}^n$ (或 $\mathbf{C}^n$) 中的一个线性无关的向量组 $\beta = \{\mathbf{y}_1, \dots, \mathbf{y}_n\}$ 就是空间 $\mathbf{R}^n$ (或 $\mathbf{C}^n$) 的一个基, 也就是说, n 维空间 $\mathbf{R}^n$ 中的任意一个向量 $\mathbf{w}$ 都可以表示成下列形式:

$$\mathbf{w} = \sum_{k=1}^n w_k \mathbf{y}_k$$

并且此时系数矩阵 $\{w_k\}$ 是唯一的, 称之为向量 $\mathbf{w}$ 在基 β 下的坐标。例如, $\mathbf{R}^n$ 空间的规范基是一组向量 $\{\mathbf{e}_1, \dots, \mathbf{e}_n\}$, 其中向量 $\mathbf{e}_i$ 的第 i 个元素为 1, 而 $\mathbf{e}_i$ 的其余元素为 0, 规范基虽然不是 $\mathbf{R}^n$ 空间的唯一的基, 但它是经常被采用的一种形式。

n 维空间 $\mathbf{R}^n$ 中的向量 $\mathbf{v}, \mathbf{w}$ 的点积定义为:

$$(\mathbf{v}, \mathbf{w}) = \mathbf{w}^T \mathbf{v} = \sum_{k=1}^n v_k w_k$$

$\{v_k\}$ 和 $\{w_k\}$ 分别是向量 $\mathbf{v}$ 和 $\mathbf{w}$ 的坐标。相应的 MATLAB 命令是 $\mathbf{w}' * \mathbf{v}$, 这里符号 $'$ 表示向量的转置。向量 $\mathbf{v}$ 的长度(或称为模)定义为:

$$\|v\| = \sqrt{(v, v)} = \sqrt{\sum_{k=1}^n v_k^2}$$

模的计算可以用 `norm(v)` 函数来实现。

MATLAB 命令 `x.*y` 或 `x.^2` 表明这些运算是在元素与元素之间进行的。例如，定义向量：

```
>>v=[1;2;3]; w=[4;5;6];
```

指令

```
>>w'*v
```

```
ans=
```

```
32
```

可以得到点积，而

```
>>w.*v
```

```
ans=
```

```
4 10 18
```

则返回一个向量，向量的第 i 个元素就是 x 与 y 的第 i 个元素相乘之后的值 $x_i y_i$ 。

最后回顾一下特征值的概念，对于一个数 λ (实数或复数) 和矩阵 $A \in \mathbf{R}^{n \times n}$ ，如果存在一个向量 $v \in \mathbf{R}^n$ ，且 $v \neq 0$ 满足等式 $Av = \lambda v$ ，则称 λ 为矩阵 A 的特征值，向量 v 称为关于 λ 的特征向量。

一般来说，特征值的计算是比较困难的。而对于对角阵或者三角阵来说则比较容易，它们的特征值就是主对角线上所有元素的乘积。

参见习题 1.3~1.5。

1.4 实函数

本节讨论一下实函数。对于一个定义在区间 (a, b) 上的函数 f 来说，一般会对它进行的运算和操作主要有：求零点、求积分，求导以及分析函数的性能。

`fplot(fun, lims)` 函数可以在区间 $(\text{lims}(1), \text{lims}(2))$ 上绘出 `fun` 函数的图形(它是以字符串的形式存储的)。例如，要在区间 $(-5, 5)$ 上绘出 $f(x)=1/(1+x^2)$ 的图形，可以输入：

```
>>fun='1/(1+x^2)';lims=[-5, 5]; fplot(fun, lims);
```

或者直接输入：

```
>>fplot('1/(1+x^2)', [-5 5]);
```

MATLAB 在横轴上对函数进行非均匀采样, 并以 0.2% 的误差再现出实际的函数波形。要提高绘图的精确程度可以使用函数:

```
>> fplot(fun, lims, tol, n, 'LineStyle', P1, P2, ...)
```

其中 `tol` 代表期望的误差限度, 参数 $n(\geq 1)$ 表示绘图点的数目不能少于 $n+1$ 个。`'linespec'` 声明绘图所用线条的形式(例如 `'--'` 代表虚线, `'.'` 代表点线), 而参数 $p_1, p_2, \dots$ 可以直接传递给函数 `fun`。如果 `tol`, 或 `'Linespec'` 都采用默认值, 则传递的是一个空矩阵(`[]`)。

在初始化了 x 的值以后, 输入 `y=eval(fun)` 可以计算出表达式 `fun` 在 x 点处的值, 结果存入变量 y 中。注意 x 和 y 都可能是一个向量。该指令的约束条件是 `fun` 函数必须以 x 作为自变量。当 `fun` 函数的自变量的名称不同时(常发生于变量是由内函数所定义的情况), 此时要用 `feval` 函数来代替 `eval` 函数(见注 1.4)。

最后指出, 如果需要在曲线后面加上如图 1.1 那样的背景坐标, 只须在 `fplot` 命令后面加上 `grid on` 语句即可。

注 1.3 通常情况下, 允许一个函数有可变个输入变量会非常有用。使用 `varargin` 变量可以实现这一点, 它是一个可以包含任意个变量的单元阵列。变量 `varargin` 必须被声明为最后一个输入变量, 并且在它之前的所有变量也将存储在这个以 `varargin` 命名的阵列中。例如, 下列程序(见 1.6.2 节)

```
function L=mytril(varargin)
L=tril(varargin{:});
```

`varargin` 变量中包括了所有的输入变量。`mytril` 函数是利用以逗号相间隔的 `varargin` 阵列来给 `tril` 函数传递参数的。指令

```
L=mytril([1 2 3; 4 5 6; 7 8 9], -2)
```

```
L=
0   0   0
0   0   0
7   0   0
```

得到的结果是一个单元阵列, 它包含的值是 `[1 2 3; 3 4 5; 6 7 8]` 和 `-2`。

1.4.1 零点

如果一个实函数 f 满足 $f(\alpha)=0$, 则称 α 为函数 f 的零点。如果 $f'(\alpha) \neq 0$, 则该零点称为一阶零点, 否则称为多重零点。

从函数的图形上可以大致推断出零点的位置(当然有一定的误差)。对于一个给定的函数来说, 要计算出它的所有零点并不是非常容易的。 n 阶实系数多项式的表达式为:

$$p_n(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n = \sum_{k=0}^n a_kx^k, \quad a_k \in \mathbf{R}, \quad a_n \neq 0$$

当 $n=1$ 时可求得惟一的零点是 $\alpha = -a_0/a_1$ (此时 p_1 代表的是一条直线); 当 $n=2$ 时有两个零点, 分别为 α_+ 和 α_- (此时 p_2 代表的是一条抛物线)。

$$\alpha_{\pm} = \frac{-a_1 \pm \sqrt{a_1^2 - 4a_0a_2}}{2a_2}$$

当 $n \geq 5$ 时, 对任意的一个多项式 p_n 求解零点时, 便很难再找到确切的解析式了。

还有, 函数零点的个数在求解之前一般是未知的。但对多项式情况例外, 多项式的零点个数就等于多项式的阶数。而且如果 $\alpha = x + iy$ 是一个多项式的零点, 那么它的复共轭 $\bar{\alpha} = x - iy$ 也是多项式的一个零点。

在 MATLAB 中利用 `fzero(fun, x0)` 函数可以计算出指定函数 `fun` 在给定的点 x_0 附近的零点, 这时得到的是一个近似的结果。 x_0 为向量时, 用于指定间隔区间, x_0 为标量时, 则作为搜索起点。还可以使用 `fzero(fun, [x0, x1])` 函数让系统在以 x_0, x_1 为端点的区间内进行搜索, 此时要求函数 f 在点 x_0 和 x_1 处不同号。

例如, 函数 $f(x) = x^2 - 1 + e^x$, 从图像上可以看出函数在 $(-1, 1)$ 区间上有两个零点。可以通过下列指令来计算它们:

```
>> fun = 'x^2 - 1 + exp(x)';
>> fzero(fun, 1)
Zero found in the interval: [-0.28, 1.9051].
ans =
    6.0953e-18
>> fzero(fun, -1)
Zero found in the interval: [-1.2263, -0.68].
ans =
   -0.7146
```

第 2 章将会给出求解任意函数的零点可以采用的几种方法。

1.4.2 多项式

多项式是一类特殊的函数, MATLAB 有一个专用的工具箱 `polyfun` 来处理它们。函数 `polyval` 用于计算一个多项式在一点或几点的值, 它的输入参数是向量 p 和向量 x , p 中的元素是多项式按降阶排列时各项的系数, 从 a_n 至 a_0 , 而 x 中的元素是对多项式进行求值运算的横坐标点。运算结果可以通过下列指令保存到向量 y 中:


```
>>y=polyval(p, x)
```

例如, 对 $p(x)=x^7+3x^2-1$, 在点 $x_k=-1+k*0.25$ ($k=0, \dots, 8$)处求值, 可以采用下列指令:

```
>>p=[1 0 0 0 3 0 -1]; x=[-1;0.25:1];
>>y=polyval(p, x)
y=
Columns 1 through 7
    1.0000    0.5540   -0.2578   -0.8126   -1.0000   -0.8124   -0.2422
Columns 8 through 9
    0.8210    3.0000
```

当然, 也可以选用 `fplot` 函数来求解。但此时需要输入多项式完整的解析表达式, 而不再只是简单的输入系数了。

再来回顾一下, 如果 α 满足 $p(\alpha)=0$, 则称 α 为 p 的零点, 或称为代数方程 $p(x)=0$ 的根。函数 `roots` 可以近似计算出一个多项式的零点, 它只需输入向量 p 即可。

例如, 计算 $p(x)=x^3-6x^2+11x-6$ 的零点, 可以输入:

```
>>p=[1 -6 11 -6]; format long;
>> roots(p)
ans=
    3.000000000000000
    2.000000000000000
    1.000000000000000
```

遗憾的是, 结果并不总是准确的。例如, 对于多项式 $p(x)=(x-1)^7$, 它的零点是 $\alpha=1$, 而程序却会输出很令人意外的结果:

```
>>p=[1 -7 21 -35 35 -21 7 -1];
>> roots(p)
ans=
    1.0088
    1.0055+0.0069i
    1.0055-0.0069i
    0.9980+0.0085i
    0.9921-0.0085i
    0.9921+0.0038i
    0.9921-0.0038i
```

这是因为多项式的系数 p 的符号是交替变换的, 产生了消去误差从而导致了结果错误。

下面介绍函数 $p=\text{conv}(p1, p2)$, 其中向量 $p1, p2$ 中包含的是两个多项式的系数, 而函数返回的就是这两个多项式进行乘法运算以后得到的新多项式的系数。类似地, 函数 $[q, r]=\text{deconv}(p1, p2)$ 返回的是 $p1$ 除以 $p2$ 以后所得多项式的各项系数, 即 $p1=\text{conv}(p2, q)+r$, 也就是说, q 和 r 分别是经过除法运算之后得到的商式和余式。

下面给出一个例子，考虑多项式 $p_1(x)=x^4-1$ 和 $p_2(x)=x^3-1$ 之间的乘法和除法运算：

```
>>p1=[1 0 0 0 -1];
>>p2=[1 0 0 -1];
>>p=conv(p1, p2)
p=
1 0 0 -1 -1 0 0 1
>>[q, r]=deconv(p1, p2)
q=
1 0
r=
0 0 0 1 -1
```

可以看出多项式 $p(x)=p_1(x)p_2(x)=x^7-x^4-x^3+1$ ， $q(x)=x$ ，并且 $r(x)=x-1$ ，所以有 $p_1(x)=q(x)p_2(x)+r(x)$ 。

最后介绍 **polyint(p)** 函数和 **polyder(p)** 函数，其中向量 p 的元素同样代表多项式的系数，这两个函数分别返回多项式经过积分运算以后的系数(在 $x=0$ 时为 0)和多项式经过求导运算以后的系数。

如果向量 x 代表由横轴的截点所构成的向量，向量 p (各个分量为 p_i) 表示多项式 P (形式分别为 P_i) 的各项系数，则可以把上面介绍的函数总结在表 1.1 中：

表 1.1 函数和输出

函 数	输 出
$y=\text{polyval}(p, x)$	$P(x)$ 的值
$z=\text{roots}(p)$	P 的根，使得 $P(z)=0$
$p=\text{conv}(p_1, p_2)$	多项式 P_1P_2 的系数
$[q, r]=\text{deconv}(p_1, p_2)$	q 为 Q 的系数， r 为 R 的系数 使得 $P_1=Q P_2+R$
$y=\text{polyder}(p)$	$P'(x)$ 的系数
$y=\text{polyint}(p)$	$\int P(x)dx$ 的系数

多项式 P 在第 $n+1$ 个节点也有值存在时，**polyfit** 函数可求出 $n+1$ 阶拟合多项式的系数(见 3.1.1 节)。

1.4.3 积分和微分

本书经常会引用下面两个结论：

1. 积分基本定理：在区间 $[a, b)$ 上，如果 f 是连续函数，那么函数 $F(x) = \int_a^x f(t)dt$ 是可微函数，称为 f 的原函数，它满足下列条件：

$$\forall x \in [a, b], F'(x) = f(x)$$

2. 积分第一中值定理: 如果 f 是定义在区间 $[a, b]$ 上的连续函数, 对于 $x_1, x_2 \in [a, b]$, 则 $\exists \xi \in (x_1, x_2)$ 使得下式成立:

$$f(\xi) = \frac{1}{x_2 - x_1} \int_{x_1}^{x_2} f(t) dt$$

有时尽管知道积分是存在的, 也可能难以计算出来。比如如果不知道怎样有效地计算对数的值, 即使已经知道了 $\ln|x|$ 是 $1/x$ 的积分也是用处不大的。在第 4 章会介绍几种在指定了误差限的情况下计算任意连续函数积分的方法, 而无需知道原函数的有关知识。

再来回顾一下微分的概念, 对于定义在区间 $[a, b]$ 上的函数 f , 如果在任意一点 $\bar{x} \in (a, b)$ 下列极限的值都存在并有限:

$$f'(\bar{x}) = \lim_{h \rightarrow 0} \frac{1}{h} (f(\bar{x} + h) - f(\bar{x})) \quad (1.9)$$

则称原函数 f 是可导的。

如果 $\bar{x} = a$, 则定义中要求 h 为正, 如果 $\bar{x} = b$, 则 h 为负。只要导数存在, 它的值就是函数的曲线在点 $\bar{x}$ 处的斜率。如果一个连续函数在区间 $[a, b]$ 的任意一点都可导, 则称函数属于空间 $C^1([a, b])$ 。更一般地, 如果一个函数 p 阶可导 (p 为正整数), 则称这个函数属于 $C^p([a, b])$ 。特别地, $C^1([a, b])$ 代表的是由 $[a, b]$ 上的连续函数所构成的空间。

还有一个经常会用到的结论是中值定理: 对于 $f \in C^1([a, b])$ 存在 $\xi \in (a, b)$, 使得下式

$$f'(\xi) = (f(b) - f(a)) / (b - a)$$

成立。

最后回顾一下 Taylor 级数的知识, 对于一个连续函数, 如果在 x_0 的邻域内存在 $n+1$ 阶导数, 则函数可以在点 x_0 处展开成 Taylor 级数:

$$\begin{aligned} T_n(x) &= f(x_0) + (x - x_0)f'(x_0) + \cdots + \frac{1}{n!}(x - x_0)^n f^{(n)}(x_0) \\ &= \sum_{k=0}^n \frac{(x - x_0)^k}{k!} f^{(k)}(x_0) \end{aligned}$$

MATLAB 工具箱 Symbolic 提供的 diff、int 和 taylor 函数, 可分别用于计算一

个给定函数的导数、不定积分和相关的 Taylor 多项式。特别地，我们要对其进行运算操作的函数，系统已经定义了字符串 f 来表示它，函数 $\text{diff}(f, n)$ 将给出 f 的 n 阶导数的值，函数 $\text{int}(f)$ 将给出不定积分的值，函数 $\text{taylor}(f, x, n+1)$ 将给出在 $x_0=0$ 的邻域内的 n 阶 Taylor 多项式。变量 x 必须用 `syms x` 语句声明为一个符号。这样便不再考虑它本身的值，而直接对它进行代数运算。

如果要对函数 $f(x)=(x^2+2x+2)/(x^2-1)$ 进行上述操作，可以输入下列语句：

```
>>f='(x^2+2*x+2)/(x^2-1)';
>>syms x
>>diff(f)
(2*x+2)/(x^2-1)-2*(x^2+2*x+2)/(x^2-1)^2*x
>>int(f)
x+5/2*log(x-1)-1/2*log(1+x)
>>taylor(f, x, 6)
-2-2*x-3*x^2-2*x^3-3*x^4-2*x^5
```

函数 `funtool` 允许对任意一个函数采用简化的符号运算，如图 1.5 所示。



图 1.5 funtool 函数的图形界面

参见习题 1.6~1.7。

1.5 误差

俗话说“人非圣贤，孰能无过”，在数值计算中误差也一样是难以避免的。正如前面章节中提到的最简单的一个示例一样，只要用计算机来表示一个实数，便会引入误差。所以这里的问题不是试图去消除误差，而是要把误差控制在一定的范围内。

一般来讲，在对一个问题进行近似和求解过程中产生的误差可以分为几个等级(如图 1.6 所示)。

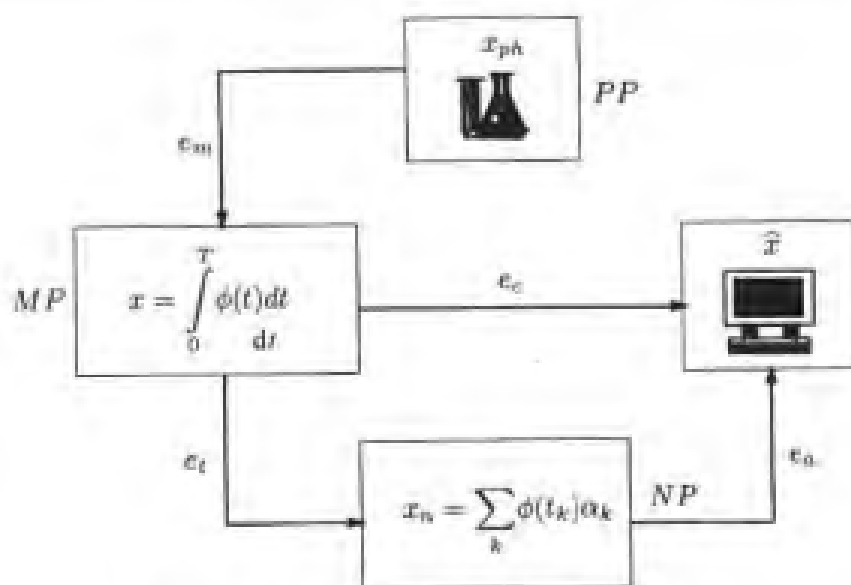


图 1.6 计算机处理过程中所产生误差的分类

处在最高一级上的是误差 e_m ，它是在把一个实际的物理模型(pp 代表物理问题， x_{ph} 代表这个物理问题的结果)抽象成一个数学模型(MP ，其解为 x)的时候产生的。这类误差会限制数学模型在某些实际情况中的应用，而且它也不在科学计算所能控制的范围之内。

数学模型(不论是以图 1.6 所示的积分的形式、还是以代数方程或微分方程的形式，或者是以线性或非线性系统的形式来表述的)一般都很难直接求得显式解，而计算机是采用数值计算的方法来求解的，这就势必会引入误差，至少会引起舍入误差的产生和积累，这里把产生的所有误差称为 e_a 。

另一方面，计算机在处理任何一个含有无穷级数的算式时，只能通过近似的方法来解决，这就进一步就引入了误差。比如在对级数求和的时候，只能选择一个合适的截取点，采取有限项来近似求解。

下面有必要对数值问题(NP)进行一下介绍，数值问题的解 x_n 与 x 的真值之间的差值 e_t 称为截断误差。这类误差只有在有限维的数学模型中(如在求解一个线性系统时)才不会产生。误差 e_a 和 e_t 共同组成了计算误差 e_c ，其大小正是我们所关心的。

绝对计算误差是 x 与 $\hat{x}$ 之间的差值， x 是数学模型的解的真值， $\hat{x}$ 是经过数值计算之后得到的解，

$$e_c^{abs} = |x - \hat{x}|$$

而相对计算误差是(如果 $x \neq 0$):

$$e_c^{rel} = |x - \hat{x}| / |x|$$

其中 $|\cdot|$ 代表模值, 或度量大小的其他量值, 它与 x 的均值有关。

数值运算过程通常是数学模型的一个近似, 它把数学模型作为一个带有离散化参数的函数来看待, 离散化程度用 h 来衡量, 设 h 为正。当 h 趋于 0 时, 如果数值处理过程能够返回数学模型的真值解, 则称数值计算是收敛的。而且, 如果(绝对或相对)误差可以表示成 h 的一个函数。如

$$e_c = Ch^p$$

其中 C 与 h 无关, p 为正数, 此时称该方法 p 阶收敛。

例 1.1 采用式(1.9)所示的增量比来估计函数 f 在点 $\bar{x}$ 处的导数值。显然, 如果 f 在 $\bar{x}$ 点可微, 则用增量比来替代 f' 时所产生的误差在 $h \rightarrow 0$ 时也趋于 0。但是, 正如将在 4.1 节所叙述的, 只有在 $f \in C^2$ 处于点 $\bar{x}$ 邻域内时, 误差才可以看作是 Ch 。

在研究数值计算的收敛性时, 经常是按照对数尺度来给出误差曲线的。把误差作为 h 的一个函数, 坐标系内纵轴是 $\log(e_c)$ 而横轴代表 $\log(h)$ 。这种表示方法的优点是很直观的, 如果 $e_c = Ch^p$, 则有 $\log e_c = \log C + p \log h$, 可见取对数以后, 指数运算转化成了线性运算。所以如果要对这两种方法作一个比较, 那就是如果曲线在对数坐标系内斜率越大, 则它在普通坐标系内阶数也越高。在 MATLAB 中, 按对数尺度来绘出图形很容易实现: 只需键入 $\log \log(x, y)$, 向量 x 、 y 分别包含了在横纵坐标上要显示出的数据。

例如, 图 1.7 给出了两种不同方法下的误差状态的两条直线。实线表示一阶的近似, 而虚线则表示二阶时的情况。

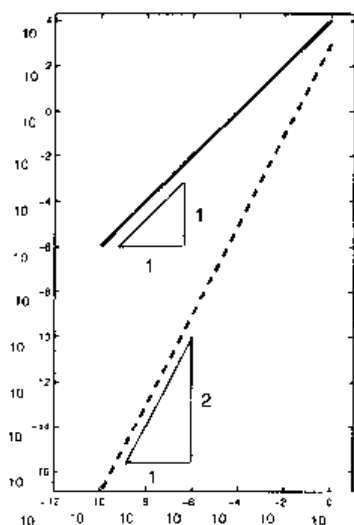


图 1.7 以对数尺度来表示的图形

还有另一种绘图的方法, 在给定了离散化参数 h_i 值时, 只要已知了这时的相对误差 e_i 就可以确定应该采用多大的阶数, $i=1, \dots, N$: 这里假设 e_i 等于 Ch_i^p , 而 C

与 i 无关。可通过下式得到 p 值:

$$p_i = \log(e_i / e_{i-1}) / \log(h_i / h_{i-1}) \quad i = 2, \dots, N \quad (1.11)$$

实际上, 由于许多未知的因素, 误差的大小是不容易确定的。因此有必要介绍一下那些可以用来估计出误差大小的可计算的量, 称为误差估计。在 2.2, 2.3 和 4.3 节中, 会给出几个相关的示例。

代价

一般来说, 计算机是采用一定的算法来进行求解的, 这种算法是利用有限的语句表示出针对有限序列所进行的精确直接的运算操作。人们感兴趣的也是那些仅包含有限次运算步骤的方法。

一种算法的计算代价就是: 执行这个运算操作所需要进行的浮点运算(简称 ops)的次数。计算机的速度通常是用计算机在 1 秒内能够执行的浮点运算的次数来衡量的(flops)。这里经常会用到下面一些缩写的短语, Mega-flops= 10^6 flops, Giga-flops= 10^9 flops, Tera-flops= 10^{12} flops。现在速度最快的计算机可以达到 40Tera-flops。MATLAB 的早期版本可以通过每秒执行的指令次数来统计出每秒执行的浮点运算的次数。在 MATLAB 6 以后的版本便去掉了这一功能。然而本书中有时仍会用到 flops 函数, 但这仅是针对 5.3 版本而言的。

一般来讲, 对于一种给定算法, 精确地求出它所需要进行的运算次数并没有多大必要。然而, 确定维数的大小则非常有用, 维数的大小是参数 d 的函数, 而参数 d 与问题的维数大小有关。因此说如果一种算法的运算次数与 d 无关, 则称它有连续复杂度, 即 $O(1)$ 运算; 如果它需要 $O(d)$ 次运算, 则称它有线性复杂度; 或者更一般地, 如果它需要 $O(d^m)$ 次运算, m 为正整数, 则称它有多项式复杂度。其他算法可能有指数复杂度($O(c^d)$ 次运算), 甚至有阶乘复杂度($O(d!)$ 次运算)。其中符号 $O(d^m)$ 的含义是: 在 d 较大时, 运算次数可以看作是按 d^m 来递增的。

例 1.2 (矩阵和向量相乘) 设 A 是一个 n 阶方阵, 并设 $v \in \mathbb{R}^n$: 则乘积 Av 的第 j 个元素为:

$$a_{j1}v_1 + a_{j2}v_2 + \cdots + a_{jn}v_n$$

它需要进行 n 次乘法运算和 $n-1$ 次加法运算, 完成全部元素的运算需要 $n(2n-1)$ 次浮点运算。所以, 这个算法需要的运算次数是 $O(n^2)$ ops, 可见它关于参数 n 有平方级复杂度。采用这一算法进行两个 n 维矩阵的相乘运算时运算量为 $O(n^3)$ 。然而 Strassen 指出存在一种算法, 它最多只需要 $O(n^{\log_2 7})$ 次运算, Winograd 和 Coppersmith 也指出存在另一种算法, 它最多需要的运算次数是 $O(n^{2.376})$ ops。

例 1.3 (计算矩阵的行列式的值)如前所述,可以采用递归公式(1.8)来计算 n 阶方阵的行列式的值。这一算法的复杂度为 n 的阶乘,它只能适用于矩阵阶数比较小的情况。例如,如果 $n=24$,一台运算次数可以达到 1Peta-flop(即每秒达到 10^{15} ops)的计算机需要 20 年才能完成这个计算,所以需要找出更有效率的算法。后来人们找到了一种通过矩阵之间相乘来计算出行列式值的算法,并且它的复杂度为 $O(n^{\log_2 7})$ ops,正如前面提到的 Strassen 算法(参见 [BB96])。

运算次数并不是衡量算法性能的惟一参数。还有一个有关的因素是计算机访问内存时需要占用的时间(它依赖于算法所采用的编码方式)。衡量算法的另一个指标是 CPU 时间(CPU 即中央处理单元),可以用 MATLAB 的 `cputime` 函数来得到这一参数的值。输入和输出阶段之间占用的整个时间可以利用 `etime` 函数来获得。

例 1.4 为了计算出矩阵和向量相乘时所需要的时间,可以输入下列程序:

```
>>A=rand(n, n);v=rand(n); T=[]; sizeA=[]; count=1;
>>for k=1:step:n
    AA=A(1:k, 1:k);vv=v(1:k)';
    t=cputime; b=AA*vv; tt=cputime-t;
    T=[T, tt];sizeA=[sizeA, k];count=count+1;
end
```

其中出现在 for 循环中的指令 `a:step:b` 可以生成所有表达形式为 $a+step*k$ 的数值,这里正整数 k 的范围是从下限 0 开始的,上限是在满足表达式 $a+step*kmax$ 的值不大于 b 时 k 所能取到的最大值 $kmax$ (如 $a=1, b=2000, step=50$)。函数 `rand(n, m)` 定义了一个 $n \times m$ 的随机矩阵。最后,向量 T 的元素就是 CPU 计算一次单个矩阵与向量之间的乘积所占用的时间,而函数 `cputime` 返回的是从 MATLAB 程序开始运行时 CPU 所使用的时间,以秒为单位。因此,执行完某个程序所用的时间是 CPU 实际占用的时间与执行这个程序之前预先计算出的时间的差,预先计算的时间存储在变量 t 中。得到图 1.8 所用的函数是 `plot(sizeA, T, 'o')`,图中显示 CPU 所用的时间是按矩阵阶数 n 的平方来递增的。

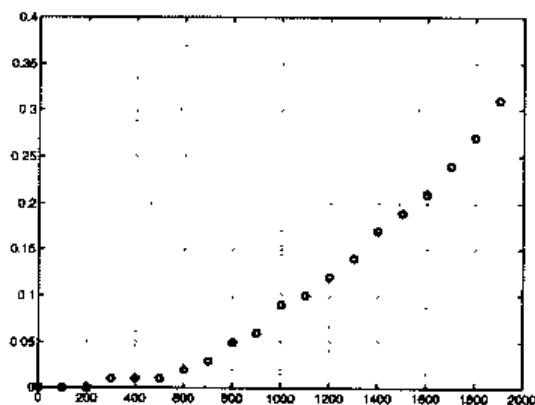


图 1.8 矩阵与向量之间的乘积运算: CPU 使用的时间(以秒为单位)与矩阵维数 n 之间的关系(计算机主频为 433Mhz)

1.6 MATLAB 简介

MATLAB 是用于科学计算和可视化显示集成环境。它是用 C 语言编写，由 Mathwork 公司发布的(www.mathworks.com)。

主程序一般放在 MATLAB 根目录下的子目录 bin 中。在安装以后，工作区内会出现提示符>>，对 MATLAB 的操作可以通过在提示符后输入指令来实现。例如，在个人电脑上运行 MATLAB 之后会显示：

```
<MATLAB>
Copyright 1984-2001 The MathWorks, Inc.
Version 6.1.0.450 Release 12.1
May 18 2001
```

To get started, select "MATLAB Help" from the Help menu.

>>

在按回车键后，系统会对提示符后面的所有字符进行解释¹。MATLAB 会精确地检查输入的字符是否与已经定义过的字符一致，或是与 MATLAB 中的程序或函数的名称一致。

如果在检查中发现有错误，MATLAB 会返回一个错误警告。如果没有发现错误，便会执行所有的指令，给出一个输出结果并显示出来。在任何情况下，只要系统执行完当前指令便会再次出现提示符，提示用户系统已准备好执行新指令了。如果想停止 MATLAB 程序的执行，可以输入 quit 指令(或 exit 指令)并按回车键。需要说明，在本书叙述中使用到的程序、函数和命令这三个短语的含义是等同的。当输入的表达式恰好与 MATLAB 所定义的基本结构一致时(如默认的一个数值或字符串)，它们会立即以默认的变量 ans 返回到输出中。下面给出一个例子：

```
>>'home'
ans=
home
```

这时如果再键入一个不同的字符串(或数值)，则变量 ans 将更新为这个新值。

如果在字符串后面加上分号，便不会再自动显示出中间结果。如输入 'home'；MATLAB 只会给出提示符(但已经把 'home' 值赋给了变量 ans)。

更一般地，运算符 "=" 允许把一个值(或一个字符串)赋给一个给定的变量。例如把字符串 'Welcome to NYC' 赋给变量 a 可以输入：

```
>>a='Welcome to NYC';
```

¹ 一个 MATLAB 程序并不是像其他语言那样必须要经过编译，如 Fortran 或 C 语言，但有时出于快速执行的需要，也会用 mcc 命令来调用 MATLAB 编译器。

正如上面所看到的,变量的类型无须预先声明, MATLAB 会自动地、动态地分配变量类型。比如在上面的例子中,如果继续输入 $a=5$, 变量 a 此时的值就是一个数值而不再是字符串了。当然,用法上带来的方便也需要付出一些代价。如果把一个名字为 `quit` 的变量赋值为 5, 则 MATLAB 中的 `quit` 函数的原有功能便被禁用了。所以应避免使用与 MATLAB 函数名称相同的变量名。但如果在变量名称前面使用 `clear` 命令(例如用于 `quit` 函数), 便可以把以前的赋值全都消掉, 恢复 `quit` 函数的初始含义。

在 `save` 命令后面加上 `fname`, 便可以把工作空间中的所有变量以二进制的格式保存到 MATLAB 文件 `fname.mat` 中。重新调出这些数据可以利用 `load fname.mat` 命令。在保存或调出时如果省略了文件名, 系统将使用默认的文件名 `matlab.mat`。要保存变量 $v_1, v_2, \dots, v_n$, 可以采用下列指令:

```
save fname v1v2...vn
```

`help` 命令可以列出所有函数和系统定义的变量, 还包括所谓的工具箱, 即一组用于专门用途的函数。其中也包含几个常用的基本函数, 如正弦(`sin(a)`)、余弦(`cos(a)`)、平方根(`sqrt(a)`)和指数(`exp(a)`)。

变量和命令的名称中不能出现一些特殊字符, 如算术运算符(+、-、*、/), 逻辑运算符与(&)、或(|)、非(~), 关系运算符大于(>)、大于等于($\geq$)、小于(<)、小于等于($\leq$)、等于(=)。最后, 变量名不能以数字、括号或标点符号开头。

1.6.1 MATLAB 语句

作为一种特殊的编程语言, MATLAB 语言允许用户自己编写新程序。尽管在使用本书中所介绍的程序时并不需要对程序本身了解更多, 但读者还是可以对这些程序做一些修改, 或是重新编写这些程序。

MATLAB 包含标准语句, 如条件语句和循环语句。

条件语句 `if-elseif-else` 一般采用下列形式:

```
if cond(1)
    statement(1)
elseif cond(2)
    statement(2)
.
.
.
else
    statement(n)
end
```

其中 `cond(1), cond(2), \dots` 代表 MATLAB 条件表达式, 它们返回的值只能是 0 或 1(假或真)。当某个条件表达式返回 1 时, 则与之相对应的语句便会执行。如果所有的条

件表达式返回的值都为 0，系统将执行 `statement(n)` 语句。这也意味着只要某个条件表达式 `cond(k)` 的返回值为 0，控制流程便会转到下一行。

例如，计算二次多项式 ax^2+bx+c 的根，可以使用下列程序(其中命令 `disp(·)` 的作用只是显示出写在括号内的字符)。

```
>> if a~=0
    sq=sqrt(b*b-4*a*c);
    x(1)=0.5*(-b+sq)/a;
    x(2)=0.5*(-b-sq)/a;
elseif b~=0
    x(1)=-c/b;
(1.12)
elseif c~=0
    disp('Impossible equation');
else
    disp('The given equation is an identity')
end
```

注意只有在输入了 `end` 之后，MATLAB 才会执行所有的语句。

MATLAB 允许使用两种循环，`for` 循环(类似于 Fortran 中的 `do` 循环或 C 语言中的 `for` 循环)和 `while` 循环。在使用 `for` 循环时，循环指针与给定的一个行向量的每个元素进行比较，只要向量中存在一个元素与之相等，就执行相应的指令。例如，计算 Fibonacci 序列 $\{f_i = f_{i-1} + f_{i-2}\}$ 的前六项时，设 $f_1 = 0$ ， $f_2 = 1$ ，可以使用下列语句：

```
>>f(1)=0;f(2)=1;
>>for i=[3 4 5 6]
    f(i)=f(i-1)+f(i-2);
end
```

分号可以用来分隔同一行中的不同 MATLAB 语句。也可以采用 `>>for i=3:6` 来代替上面的第二条指令。而对于 `while` 循环，只要给定的条件表达式 `cond` 的值为真，便会一直执行。如果上面的例子用 `while` 循环来实现，可以写成下列形式：

```
>>f(1)=0;f(2)=1;k=3;
>>while k<=6
    f(k)=f(k-1)+f(k-2);k=k+1;
end
```

另外还有一些不常用到的语句，如 `switch`，`case`，`otherwise`。感兴趣的读者可以通过 `help` 命令来了解它们的含义。

1.6.2 MATLAB 编程

下面简要介绍一下如何编写 MATLAB 程序。正如前面提到的那样，MATLAB 允许用户自己编写新程序。新程序必须存放在一个扩展名为 `.m` 的文件中，该文件称

为 M 文件。M 文件要存放在 MATLAB 的自动搜索路径中，自动搜索路径的列表可以由 `path` 命令得到(使用 `help` 命令可以查到添加自动搜索路径的方法)。MATLAB 的第一个搜索路径是当前的工作路径。

M 文件有两种形式：命令式(script)和函数式(function)，认识到它们之间的区别非常重要。命令式只是把执行的 MATLAB 指令编辑到一个 M 文件中，并可以随时调用。例如，一个指令序列(1.12)只需把它拷贝到 `equation.m` 中，便转变成了一个命令式(命名为 `equation`)。而在执行它时只须在提示符后面输入指令 `equation`。下面给出两个例子：

```
>>a=1; b=1; c=1;
>> equation
ans=
-0.5000+0.8660i -0.5000 - 0.8660i

>>a=0; b=1; c=1
>>equation
ans=
-1
```

比命令式更灵活的形式是函数。函数通常是以下列形式定义在 `m` 文件(通常命名为 `name.m`)中的：

```
function[out1, ..., outn]=name(in1, ..., inm)
```

其中 `out1, ..., outn` 是输出变量，`in1, ..., inm` 是输入变量。

下面这个名称为 `det23.m` 的文件，定义了一个名称为 `det23` 的新函数，功能是依据 1.3 节中给出的公式来计算 2 阶或 3 阶矩阵的行列式的值。

```
Function det=det23(A)
%DET23 computes the determinant of a square matrix
% of dimension 2 or 3
[n, m]=size(A);
if n==m
    if n==2
        det=A(1, 1)*A(2, 2)-A(2, 1)*A(1, 2);
    elseif n==3
        det=A(1, 1)*det23(A[2, 3], [2, 3])-A(1, 2)*det23(A([2, 3], [1, 3]))+...
            A(1, 3)*det23(A[2, 3], [1, 2]);
    else
        disp('Only 2x2 or 3x3 matrices');
    end
else
    disp('Only square matrices');
end
return
```

省略号...用在一行的末尾,说明该表达式没有写完,下一行为续行。而指令 $A([i, j], [k, l])$ 生成了一个 2×2 矩阵,它以原矩阵 A 的第 i, j 行与第 k, l 列交叉点处的值作为矩阵的元素。

在调用函数时, MATLAB 会创建一个本地工作空间。该函数中使用的变量不能用作全局变量,除非它们是作为输入变量使用的。而且该函数中用到的变量如果不是作为输出变量,则在执行以后这些变量将会被消除。符号 % 表示在它之后的语句为注释说明语句。

一般来说,程序是运行到最后一行才结束的,return 语句可用于提前退出程序(在满足特定的条件时)。例如,要近似求解黄金分割 $\alpha = 1.618\ 033\ 988\ 7\dots$,它是比例式 f_k / f_{k-1} 当 $k \rightarrow \infty$ 时的极限,迭代重复这个算式直到两次得到的结果之差小于 10^{-4} ,可以采用下列函数:

```
function [golden, k]=fibonacci
f(1)=0;f(2)=1;goldenold=0;kmax=100;tol=1.e-04;
for k=3:kmax
    f(k)=f(k-1)+f(k-2);
    golden=f(k)/f(k-1);
    if abs(golden-goldenold)<=tol
        return
    end
    goldenold=golden;
end
return
```

然后可以输入

```
>>[alpha, niter]=fibonacci
alpha=
    1.61805555555556
niter=
    14
```

在 14 次迭代运算之后,函数会返回一个近似值,它得到了 α 的前 5 位有效数字。

MATLAB 函数的输入输出参数的个数是可以改变的,我们可以对 Fibonacci 函数做如下修改:

```
function [golden, k]=fibonacci(tol, kmax)
if nargin == 0
    kmax= 100;tol=1.e-04;% default values
elseif nargin == 1
    kmax=100;% default value only for kmax
end
f(1)=0;f(2)=1;goldenold = 0;
for k=3:kmax
    f(k)=f(k-1)+f(k-2);
```

```

golden=f(k)/f(k-1);
if abs(golden-goldenold)<=tol
    return
end
goldenold=golden;
end
return

```

`nargin` 函数可以给出输入参数的个数。这个新的 Fibonacci 函数, 可以规定内部迭代运算允许进行的最大次数(`kmax`)和一个具体的误差限 `tol`。如果没有输入这个参数, 系统会返回错误信息(在本例中, `kmax=100`, `tol=1.e-0.4`)。下面再给出一个例子:

```

>>[alpha, niter]=fibonacci(1.e-6, 200)
alpha=
    1.61803381340013
niter=
    19

```

注意, 在规定了更为严格的误差限之后, 得到的新的 α 近似值的有效数字已升为 8 位。

`nargin` 函数可以求出一个给定函数的输入变量的个数。见下面的例子:

```

>> nargin('fibonacci')
ans=
    2

```

注 1.4 (内函数)`inline` 函数最简单的用法为 `g=inline(expr, arg1, arg2, ..., argn)`, 上式定义了一个以 `arg1, arg2, ..., argn` 作为自变量的函数 `g`, 其中字符串 `expr` 代表函数 `g` 的表达式。例如, `g=inline('sin(r)', 'r')` 表达的函数为 $g(r)=\sin(r)$ 。该命令的简化形式为 `g=inline(expr)`, 它假设 `expr` 是默认变量 `x` 的函数。并且对于一个经过声明之后的内函数, 可以利用 `feval` 函数在任意一组变量处对其进行求值运算。例如, 在点 $z=[0 \ 1]$ 处求 `g` 的值可以输入:

```

>> feval('g', z);

```

注意使用 `feval` 函数时, 自变量名称(此处为 `z`)并不一定要与当初 `inline` 命令定义时使用的那个名称(`r`)一致, 这一点不同于 `eval` 函数。

在简要介绍了 MATLAB 之后, 建议读者使用 `help` 命令来进一步了解 MATLAB 的相关知识, 并通过本书所介绍的一些程序来掌握各种算法的实现。

参见习题 1.8~1.11。

1.7 补充说明

[Üeb97] 和 [QSS00] 这两本专著系统地讨论了有关浮点数运算的问题, 可以作为相关问题的参考资料。

关于运算复杂度方面的问题可参考 [Pan92]。

有关 MATLAB 更为系统的介绍, 有兴趣的读者可参阅 MATLAB 手册 [HH00], 亦可参阅 [HLR01] 或 [EKH02] 这两本专著。

1.8 习题

习题 1.1 在集合 $F(2, 2, -2, 2)$ 中有多少个数? 该集合的 ε_M 值是多少?

习题 1.2 证明集合 $F(\beta, t, L, U)$ 中包含的元素个数为 $2(\beta-1)\beta^{t-1}(U-L+1)$ 。

习题 1.3 编写 MATLAB 程序, 生成一个上(下)三角阵, 维数为 10, 主对角线上元素为 2, 主对角线上(下)面的对角线元素为 -3。

习题 1.4 把习题 1.3 中的矩阵的第 3 行和第 7 行相交换, 编写出相应的 MATLAB 程序, 并编写出实现第 4 列和第 8 列交换的程序。

习题 1.5 验证在空间 $\mathbf{R}^4$ 中的下列向量是否线性相关。

$$v_1 = [0 \ 1 \ 0 \ 1], \quad v_2 = [1 \ 2 \ 3 \ 4], \quad v_3 = [1 \ 0 \ 1 \ 0], \quad v_4 = [0 \ 0 \ 1 \ 1]$$

习题 1.6 利用 MATLAB 的专用工具箱, 写出下列函数并计算它们的一、二阶导数和积分。

$$g(x) = \sqrt{x^2 + 1}, \quad f(x) = \sin(x^3) + \cosh(x)$$

习题 1.7 任给 n 维向量 v , 利用函数 $C = \text{poly}(v)$ 可以拟合出多项式 $p(x) = \sum_{k=0}^{n+1} c(k)x^{n+1-k}$ 的第 $n+1$ 个系数, 它等于 $\prod_{k=1}^n (x - v(k))$ 。如果采用精确的运算可得出 $v = \text{roots}(\text{poly}(c))$ 。但是由于舍入误差的影响, 实际计算得出的结果并不是这样的。利用函数 $\text{roots}(\text{poly}([1:n]))$ 来检验这一现象, n 的范围从 0~25。

习题 1.8 编写出计算下列序列的程序:

$$I_0 = \frac{1}{e}(e-1)$$

$$I_{n+1} = 1 - (n+1)I_n, \quad n = 0, 1, \dots$$

把数值计算的得到的结果与极限 $I_n \rightarrow 0$ (当 $n \rightarrow \infty$ 时) 相比较。

习题 1.9 分析迭代序列(1.4)用于 MATLAB 计算时的性能, 并解释产生错误的原因。

习题 1.10 考虑下面计算 π 值的算法。首先在 $[0, 1]$ 区间内产生 n 个随机数对 $\{(x_k, y_k)\}$, 统计出它们落在单位圆的第一个四分之一区域内的个数 m , 显然, π 值就等于序列 $\pi_n = 4m/n$ 的极限。编写出计算这个序列的 MATLAB 程序, 并观察在 n 值增大时误差的变化情况。

习题 1.11 编写 MATLAB 程序, 计算二项式系数 $\binom{n}{k} = n!/(k!(n-k)!)$, 其中 n 和 k 是自然数, 并满足 $k \leq n$ 。

第 2 章 非线性方程

计算一个函数 f 的零点(即求方程 $f(x)=0$ 的根)是在科学计算中经常遇到的问题。然而通常情况下,这类问题不可能通过有限次数值运算来完成。例如,如 1.4.1 小节所示,当 f 是一个阶数高于 4 的多项式时,其零点并不存在解析式形式的解。而当 f 不是多项式的情况,求解零点问题就更加复杂了。

因此我们采用迭代法来求解这类问题。这种方法从一个或几个初始值开始,建立了收敛序列 $x^{(k)}$,并且该序列能够收敛到所求函数 f 的零点 α 。

问题 2.1(投资基金) 每年年初,一个银行客户存入 v 欧元作为投资基金,在第 n 年的年末,取出 M 欧元的资本。要求计算这项投资平均每年的利息 I 。由下列 M 和 I 的关系式:

$$M = v \sum_{k=1}^n (1+I)^k = v \frac{1+I}{I} [(1+I)^n - 1]$$

可推出 I 是代数方程的根:

$$f(I) = 0 \quad \text{其中 } f(I) = M - v \frac{1+I}{I} [(1+I)^n - 1]$$

该问题的解答参见例 2.1。

问题 2.2(气体状态方程) 要确定在温度 T 和压力 p 下,气体所占的体积 V 。状态方程(即 p 、 V 、 T 之间相关联的方程)是:

$$\left[p + a(N/V)^2 \right] (V - Nb) = kNT \quad (2.1)$$

其中 a 和 b 是系数,其大小取决于气体特性, N 是容积 V 中包含的分子数, k 是玻尔兹曼常数。因此需要解一个非线性方程,该方程的根是 V (参见习题 2.2)。

问题 2.3(静力学) 考虑一个如图 2.1 所示的四个刚性链杆 a_i 组成的机械系统。对于任意允许的角度 β , 求出相应的杆 a_1 和 a_2 间角度 α 的值。由向量等式:

$$\mathbf{a}_1 - \mathbf{a}_2 - \mathbf{a}_3 - \mathbf{a}_4 = \mathbf{0}$$

并且杆 a_1 总是位于 x 轴上,可推出 β 和 α 具有下列关系:

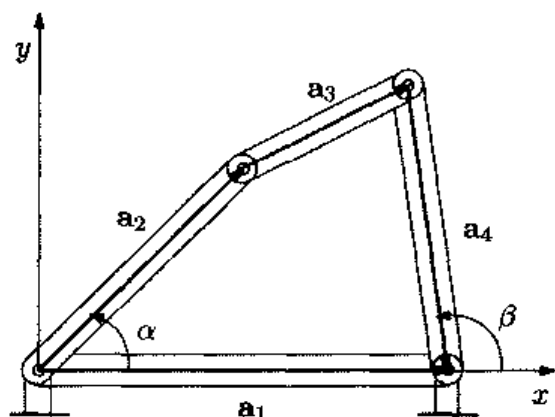


图 2.1 题 2.3 的四杆系统

$$\frac{a_1}{a_2} \cos(\beta) - \frac{a_1}{a_4} \cos(\alpha) - \cos(\beta - \alpha) = -\frac{a_1^2 + a_2^2 - a_3^2 + a_4^2}{2a_2a_4} \quad (2.2)$$

这里, a_i 是已知的第 i 个杆的长度。上式称为 Freudenstein 方程, 可以将其改写成如下形式: $f(\alpha) = 0$, 其中 $f(x) = (a_1/a_2)\cos(\beta) - (a_1/a_4)\cos(x) - \cos(\beta - x) + (a_1^2 + a_2^2 - a_3^2 + a_4^2)/(2a_2a_4)$ 。显式形式的解只能在 β 取特殊值时得到。需要指出的是, 并不是对所有的 β 值都存在解, 同时解可能不惟一。对于任意给定的介于 0 和 π 之间的 β , 需要利用数值方法来求解该方程(参见习题 2.9)。

2.1 二分法

设 f 是区间 $[a, b]$ 上的连续函数, 且满足 $f(a)f(b) < 0$ 。那么 f 必然在 (a, b) 上至少有一个零点。为简化起见, 设零点是惟一的, 记为 α 。

(对于多个零点的情形, 通过命令 `fplot`, 将区间划分成多个小区间使每个小区间内只包含其中的一个零点。)

二分法的思路就是平分给定的区间, 选择 f 有符号变化的子区间(这样子区间内会始终包含 α)。这个程序反复迭代, 直到最后选择的区间小到能够达到所期望的精度为止。由于当 k 趋向于无穷时, 子区间的长度趋向于零, 所以子区间 $I^{(k)}$ 的中点形成的序列 $\{x^{(k)}\}$ 必然趋向于 α 。二分法的迭代可参照图 2.2。

具体说来, 二分法首先需要设下列初始值:

$$a^{(0)} = a, \quad b^{(0)} = b, \quad I^{(0)} = (a^{(0)}, b^{(0)}), \quad x^{(0)} = (a^{(0)} + b^{(0)})/2$$

在第 $k \geq 1$ 步, 选择区间 $I^{(k-1)} = (a^{(k-1)}, b^{(k-1)})$ 的子区间 $I^{(k)} = (a^{(k)}, b^{(k)})$, 方法如下:

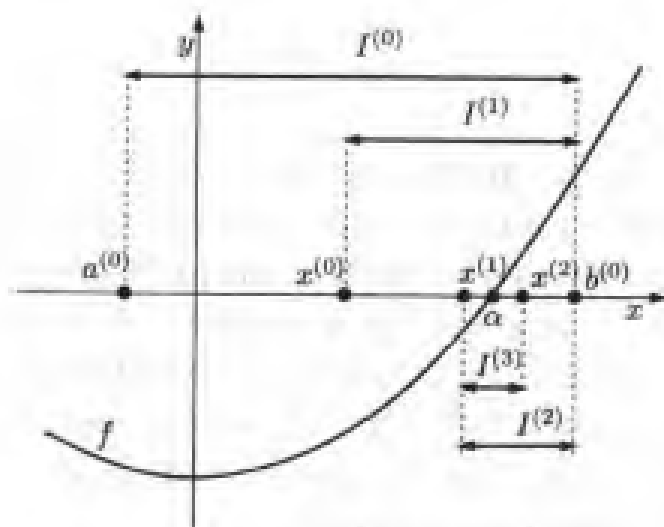


图 2.2 二分法的几次迭代

给定 $x^{(k+1)} = (a^{(k+1)} + b^{(k+1)})/2$, 如果 $f(x^{(k+1)}) = 0$, 则 $\alpha = x^{(k+1)}$, 该方法终止执行; 否则,

如果 $f(a^{(k+1)})f(x^{(k+1)}) < 0$ 令 $a^{(k+1)} = a^{(k+1)}$, $b^{(k+1)} = x^{(k+1)}$

如果 $f(x^{(k+1)})f(b^{(k+1)}) < 0$ 令 $a^{(k+1)} = x^{(k+1)}$, $b^{(k+1)} = b^{(k+1)}$

再定义 $x^{(k)} = (a^{(k)} + b^{(k)})/2$, 且 k 以步长 1 增长。

举例来说, 如图 2.2 所示的情形, 其中选取 $f(x) = x^2 - 1$, 通过选取 $a^{(0)} = -0.25$ 和 $b^{(0)} = 1.25$, 可得

$$\begin{aligned} I^{(0)} &= (-0.25, 1.25), & x^{(0)} &= 0.5 \\ I^{(1)} &= (0.5, 1.25), & x^{(1)} &= 0.875 \\ I^{(2)} &= (0.875, 1.25), & x^{(2)} &= 1.0625 \\ I^{(3)} &= (0.875, 1.0625), & x^{(3)} &= 0.96875 \end{aligned}$$

注意每一个子区间 $I^{(k)}$ 均包含零点 α 。而且, 由于每一步均将 $I^{(k)}$ 的长度 $|I^{(k)}| = b^{(k)} - a^{(k)}$ 二分, 所以序列 $\{x^{(k)}\}$ 必定收敛到 α 。由于 $|I^{(k)}| = (1/2)^k |I^{(0)}|$, 第 k 步的误差满足下式:

$$|e^{(k)}| = |x^{(k)} - \alpha| < \frac{1}{2} |I^{(k)}| = \left(\frac{1}{2}\right)^{k+1} (b - a)$$

对于给定的误差容限 ε , 为保证 $|e^{(k)}| < \varepsilon$ 时执行 $k_{\min}$ 次迭代就能达到要求, 则最小整数 $k_{\min}$ 应满足下列不等式:

$$k_{\min} > \log_2 \left(\frac{b-a}{\varepsilon} \right) - 1 \quad (2.3)$$

显然，上面的不等式为一般形式，而不是像以前那样只对特定选择的 f 成立。

程序 1 是二分法的一个实现：fun 是指定函数 f 的字符串(或内嵌函数)， a 和 b 是所要查找的区间的边界，tol 是误差容限 ε ，nmax 是指定的最大迭代数目。当 fun 是一个内嵌函数(或是一个定义于 m 文件中的函数)时，除了与独立函数相关的一个变量之外，它还可以接受定义函数 fun 所需要引入的其他辅助变量。

输出变量 zero 包含的是 α 的近似值，残差 res 是在 zero 和 niter 情况下 f 的值，其中 niter 是执行的总迭代的次数。命令 find($fx == 0$) 可以求出与零元素相对应的向量 fx 的指数，而命令 sign(fx) 可以得到 fx 的符号。

程序 1—bisection: 二分法

```
function [zero,res, niter]-bisection (fun,a,b,tol, nmax,varargin)
% BISECTION Find function zeros.
% ZERO=BISECTION(FUN,A,B,TOL,NMAX) tries to find a zero ZERO of the
% continuous function FUN in the interval [A,B] using the bisection
% method.
% FUN accepts real scalar input x and returns a real scalar value.
% If the search fails an error message is displayed. FUN can also
% be an inline object.
% ZERO=BISECTION(FUN,A,B,TOL,NMAX,P1,P2,...) passes parameters P1,
% P2,... to the function FUN(X,P1,P2,...).
% [ZERO,RES,NITER]=BISECTION(FUN,...) returns the value of the
% residual in ZERO and the iteration number at which ZERO was computed
x= [a, (a+b)*0.5, b];
fx=feval(fun,x,varargin{:});
if fx(1)*fx(3)>0
    error(' The sign of the function at the extrema of the interval must
be different');
elseif fx(1)==0
    zero = a; res = 0; niter= 0;
    return
elseif fx(3) == 0
    zero = b; res = 0; niter =0;
    return
end
niter= 0;
I= (b- a)*0.5;
while I >= tol & niter <= nmax
    niter= niter + 1;
    if sign(fx(1))*sign(fx(2)) < 0
        x(3)= x(2); x(2)= x(1)+(x(3)-x(1))*0.5;
        fx = feval(fun,x,varargin{:}); I = (x(3)-x(1))*0.5;
```

```

elseif sign(fx(2))*sign(fx(3)) < 0
    x(1)= x(2); x(2)= x(1)+-(x(3)-x(1))*0.5;
    fx= feval(fun,x,varargin{:}); I= (x(3)-x(1))*0.5;
else
    x(2) = x(find(fx==0)); I = 0;
end
end
end
if niter > nmax
    fprintf(['bisection stopped without converging to the desired
    tolerance',...
    'because the maximum number of iterations was reached\n']);
end
zero = x(2); x= x(2); res= reval(fun,x);

```

例 2.1 利用二分法求解问题 2.1, 设 v 等于 1000 欧元, 5 年后 M 等于 6000 欧元。则函数 f 的曲线可通过下列指令得到:

```

>> f = inline('M-v*(1+i).^((1+i).^5-1)./I','I','M','v');
>> fplot(f,[0.01,0.3],[[],[],[],6000,1000])

```

可见区间(0.01, 0.1)内 f 有惟一的零点, 近似等于 0.06。如果设 $\text{tol}=10^{-12}$, $a=0.01$ 和 $b=0.1$, 执行程序 1, 经过 36 次迭代运算后, 该法收敛到数值 0.061402, 这与式(2.3)中取 $k_{\min}=36$ 时的近似值符合得很好。因此可得出结论, 利息率 I 约等于 6.14%。

虽然简单, 但二分法并不能保证误差是单调递减的, 只能保证查找区间每经过一次迭代就变为原来的二分之一。因此, 如果把控制区间长度 $I^{(k)}$ 作为迭代运算终止的惟一标准, 就有可能得不到精确的 α 的逼近值。

实际上, 这种方法并没有正确地考虑 f 的实际特性。一个明显的例子是即使 f 是一个线性函数, 利用这种方法也不可能经过一次迭代运算就得到收敛值(除非零点 α 是初始查找区间的中点)。

参见习题 2.1~2.5。

2.2 Newton 法

计算函数 f 的零点的一个更为有效的方法是利用 f 的可微性。在这种情况下, 公式:

$$y(x) = f(x^{(k)}) + f'(x^{(k)})(x - x^{(k)})$$

给出了曲线 $(x, f(x))$ 在点 $x^{(k)}$ 处的切线方程。

如果假设 $x^{(k+1)}$ 满足 $y(x^{(k+1)})=0$, 可得

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}, \quad \text{设 } f'(x^{(k)}) \neq 0 \quad (2.4)$$

利用上式, 可以计算出序列 $x^{(k)}$ 的值, 该序列由初始推测值 $x^{(0)}$ 开始。该方法称为 Newton 法, 它通过在小区间内利用 f 的切线来替代 f , 从而分别计算出 f 的零点(参见图 2.3)。

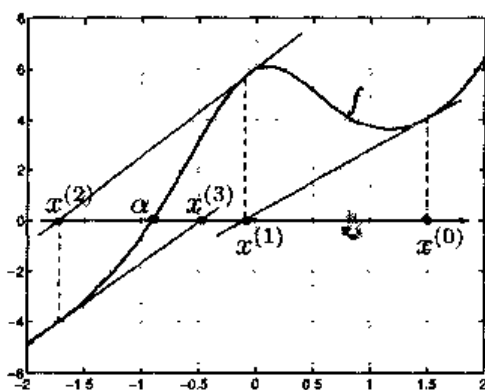


图 2.3 对于函数 $f(x) = x + e^x + 10/(1+x^2) - 5$, 在初始推测值 $x^{(0)}$ 的情况下利用 Newton 法进行的第一步迭代运算

实际上, 通过在特定点 $x^{(k)}$ 的邻域内将 f 展开成 Taylor 级数, 可以得到:

$$f(x^{(k+1)}) = f(x^{(k)}) + \delta^{(k)} f'(x^{(k)}) + O((\delta^{(k)})^2) \quad (2.5)$$

其中 $\delta^{(k)} = x^{(k+1)} - x^{(k)}$ 。设 $f(x^{(k+1)})$ 等于零, 且忽略余项 $O((\delta^{(k)})^2)$, 可得函数 $x^{(k+1)}$, 它是 $x^{(k)}$ 的函数, 如式(2.4)所示。由此(2.4)可认为是(2.5)的近似形式。

显然, 当 f 是线性时, (2.4)只需经过一步运算就可得到收敛值, 即 $f(x) = a_1 x + a_0$; 此时:

$$x^{(1)} = x^{(0)} - \frac{a_1 x^{(0)} + a_0}{a_1} = -\frac{a_0}{a_1}$$

例 2.2 用 Newton 法解题 2.1, 采用初始数据 $x^{(0)} = 0.3$ 。经过 6 次迭代运算后两次相邻迭代间的差值小于等于 10^{-12} 。

一般来讲, Newton 法并非对所有可能的 $x^{(0)}$ 收敛, 而只是对那些足够靠近 α 的 $x^{(0)}$ 的值才有效。初看起来, 这个条件似乎毫无意义; 但实际上, 为了计算出 α (未知的), 应该从一个足够靠近 α 的值开始!

在实际中, 选取的初始值 $x^{(0)}$ 可以通过二分法的几次迭代得到或者通过对 f 曲线的研究得到。如果正确选择 $x^{(0)}$ 且 α 是一个一阶零点(见 1.4.1 节), 那么 Newton

法是收敛的。更进一步, 在 f 是高至二阶的连续可微的特殊情况下, 有下列收敛结果(参见习题 2.8):

$$\lim_{k \rightarrow \infty} \frac{x^{(k+1)} - \alpha}{(x^{(k)} - \alpha)^2} = \frac{f''(\alpha)}{2f'(\alpha)} \quad (2.6)$$

因此, Newton 法一般是二次收敛的, 或者收敛阶数为 2, 因为对于足够大的 k 值, 第 $(k+1)$ 步的误差相当于第 k 步误差的平方乘以一个与 k 无关的常数。

在零点的阶数 m 大于 1 的情况下, Newton 法的收敛阶数降为 1(参见习题 2.15)。在这种情况下, 应如下改进原来的方法以使其恢复到 2 阶:

$$x^{(k+1)} = x^{(k)} - m \frac{f(x^{(k)})}{f'(x^{(k)})}, \quad \text{设 } f'(x^{(k)}) \neq 0 \quad (2.7)$$

显然, 改进 Newton 法需要预先知道 m 的知识。否则就需要采用一种自适应 Newton 法, 它仍是 2 阶的, 具体可参见 [QSS00], 第 6.6.2 节。

例 2.3 函数 $f(x) = (x-1)\log(x)$ 有惟一的 $m=2$ 重零点 $\alpha=1$ 。分别利用 Newton 法(2.4)和改进 Newton 法(2.7)进行计算。在图 2.4 中描述了利用两种方法所得的误差相对于迭代次数的变化。注意经典 Newton 法只能是线性收敛的。

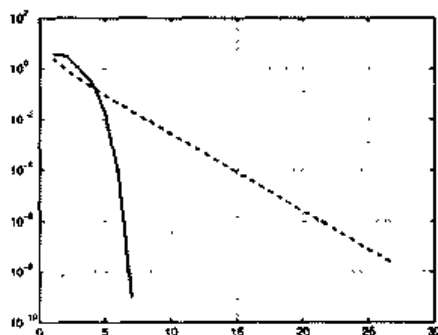


图 2.4 例 2.3 中函数的误差与迭代次数的关系。虚线对应于 Newton 法(2.4), 实线应于改进的 Newton 法(2.7)($m=2$)

理论上, 一个有收敛性的 Newton 法只有经过无限次迭代运算才可以返回零点 α 。但在实际中, 只需要 α 的近似值达到指定的误差容限 ε 要求即可。这样为使迭代终止所需的最小运算次数 $k_{\min}$ 应满足下列不等式:

$$|e^{(k_{\min})}| = |\alpha - x^{(k_{\min})}| < \varepsilon$$

然而, 由于误差是未知的, 所以需要用一个合适的误差估计来代替它, 这就要找到

一个能够很容易计算出来的量,以便通过它估计出真正的误差。在 2.3 节的结尾可看到一个适用于 Newton 法的误差估计,该估计可通过两次相邻的迭代之差得到。即只要在第 $k_{\min}$ 步满足下列条件,迭代就应该终止:

$$|x^{(k_{\min})} - x^{(k_{\min}-1)}| < \varepsilon \quad (2.8)$$

注 2.1(一般情况)迭代终止的标准(2.8)可能并不适用于除了 Newton 法以外的方法。还可以选择的是误差估计方法,改方法是基于第 k 步的残差,其定义为 $r^{(k)} = f(x^{(k)})$ (注意当 $x^{(k)}$ 是函数的零点时残差是零)。

当运算进行到第一个满足下列条件的 $k_{\min}$ 时,迭代终止:

$$|f(x^{(k_{\min})})| < \varepsilon$$

只有当 f 的变化形式在零点 α 的邻域 I_a 内接近线性时(参见图 2.5),利用残差作为误差估计才能满足条件。否则对于 $x \in I_a$, 如 $|f'(x)| \gg 1$, 则误差的估计会偏高,如果 $|f'(x)| \ll 1$, 则对误差的估计会偏低(参见习题 2.6)。

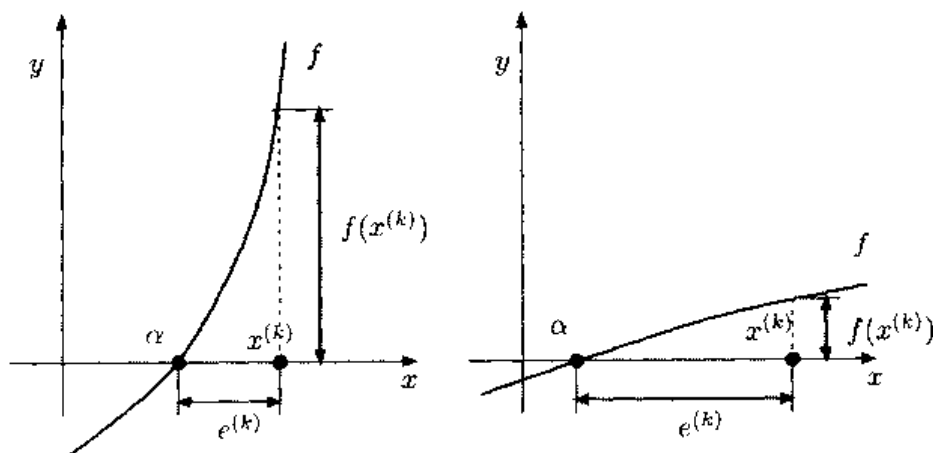


图 2.5 欠误差估计时残差的两种情况: $|f'(x)| \gg 1$ (左侧), $|f'(x)| \ll 1$ (右侧), x 在 a 的邻域内

程序 2 是 Newton 法(2.4)的一个实现。只要简单地用 f/m 来替代 f' , 就可得到 Newton 法的改进形式。输入的参数 f 和 df 是定义函数 f 及其一阶导数的字符, 而 x_0 是初始猜测值。当随后的相邻两步的迭代之差的绝对值小于给定的误差容限 tol , 或达到了最大的迭代数目 nmax 时, 该法的迭代将终止。

程序 2—netwon: Newton 法

```
function[zero,res,niter]=Newton(f,df,x0,tol,nmax,varargin)
%NEWTON Find function zeros.
```



```

%ZERO=NEWTON(FUN,DFUN,X0,TOL,NMAX) tries to find the zero ZERO of
%the continuous and differentiable function FUN nearest to X0 using the
%Newton method. FUN and its derivative DFUN accept real scalar input x
%and returns
%a real scalar value. If the search fails an error message is displayed.
%FUN and DFUN can also be inline objects.
%ZERO=NEWTON(FUN,DFUN,X0,TOL,NMAX,P1,P2,...) passes parameters
%P1,P2,... to functions: FUN(X,P1,P2,...) and DFUN(X,P1,P2,...).
%[ZERO,RES,NITER]= NEWTON(FUN,...) returns the value of the
%residual in ZERO and the iteration number at which ZERO was computed.
x = x0;
fx = feval(f,x,varargin{:});
dfx = feval(df,x,varargin{:});
niter = 0;
diff = tol+1;
while diff>= tol & niter <=nmax
    niter = niter + 1;
    diff=- fx/dfx;
    x = x + diff;
    diff = abs(diff);
    fx = feval(f,x,varargin{:});
    dfx = feval(df,x,varargin{:});
end
if niter > nmax
    fprintf(['newton stopped without converging to the desired
tolerance',...
'because the maximum number of iterations was reached\n']);
end
zero = x; res = fx;

```

小结

1. 计算一个函数零点所采用的方法通常是迭代类型的;
 2. 二分法通过在每次迭代时对区间的长度一分为二而生成一个区间序列的方法来计算一个函数 f 的零点。只要 f 在初始区间是连续的且在该区间的两个端点有相反的符号, 这种方法就是收敛的。
 3. Newton 法通过 f 及其导数的值来计算 f 的零点 α 。其收敛的必要条件是初始数据在 α 的合适(足够小)的邻域内。
 4. Newton 法只有当 α 是 f 的一阶零点时是二次收敛的, 否则是线性收敛。
- 参见习题 2.6~2.14。

2.3 固定点迭代

使用小型计算器可以验证, 对实数值 1, 重复利用 cosine 键, 可得出下列实数

序列:

$$x^{(1)} = \cos(1) = 0.54030230586814$$

$$x^{(2)} = \cos(x^{(1)}) = 0.85755321584639$$

$$x^{(10)} = \cos(x^{(9)}) = 0.74423735490056$$

$$x^{(20)} = \cos(x^{(19)}) = 0.7391843997149$$

其值应趋向于 $\alpha = 0.73908513\cdots$ 。由于可以构造出 $x(k+1) = \cos(x(k))$, 其中 $k=0, 1, \cdots (x(0)=1)$, 极限 α 满足方程 $\cos(\alpha) = \alpha$, 此时 α 称为余弦函数的固定点。那么为了计算出给定函数的零点, 又该如何利用迭代法呢, 在前一个例子中, α 不仅是一个余弦函数的固定点, 同时也是函数 $f(x) = x - \cos(x)$ 的零点, 因此前述方法也可以认为是一种计算 f 的零点的方法。另外, 并非每个函数都有固定点。例如, 如果利用指数函数来重复先前的例子, 仍设 $x(0)=1$, 则仅仅经过 4 步运算就会出现溢出 (参见图 2.6)。

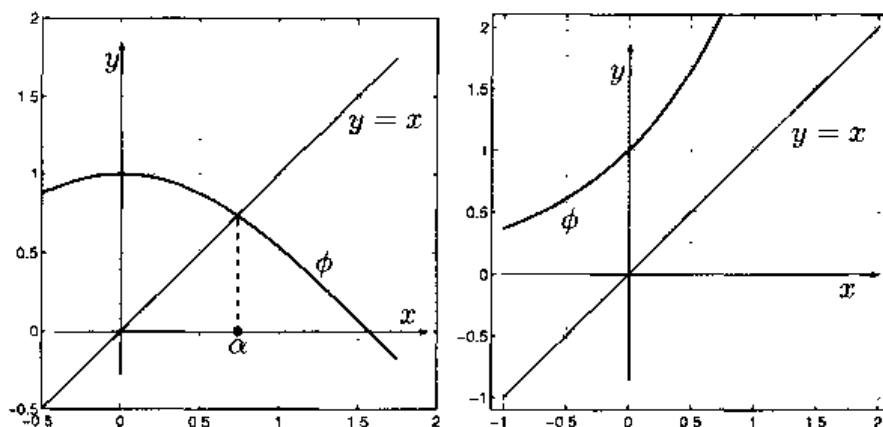


图 2.6 函数 $\phi(x) = \cos x$ 有且只有一个固定点(左图), 而函数 $\phi(x) = e^x$ 没有固定点(右图)

考虑下列问题, 可以使上述想法更加直观和清晰。给定函数 $\phi: [a, b] \rightarrow \mathbf{R}$, 寻找 $\alpha \in [a, b]$ 使得:

$$\alpha = \phi(\alpha)$$

如果满足上式的 α 存在, 就称为 ϕ 的一个固定点, 它可以通过下列算法计算:

$$x^{(k+1)} = \phi(x^{(k)}), \quad k \geq 0 \quad (2.9)$$

其中 $x(0)$ 是初始猜测值。该算法称为固定点迭代, ϕ 称为迭代函数。因此上例就是

用 $\phi(x) = \cos(x)$ 进行固定点迭代的一个实例。

式(2.9)的几何解释如图 2.7(左图)所示。可先推测 ϕ 是一个连续的函数且数列 $\{x^{(k)}\}$ 的极限存在, 那么这个极限就是 ϕ 的一个固定点。命题 2.1 和 2.2 中给出了这个结果更为精确的表述。

例 2.4 Newton 法(2.4)可以认为是固定点迭代的一种算法, 其迭代函数是:

$$\phi(x) = x - \frac{f(x)}{f'(x)} \quad (2.10)$$

从现在起, 这个函数用 ϕ_N 表示(这里 N 代表 Newton)。由于一般迭代项 $x^{(k+1)}$ 与 $x^{(k)}$ 及 $x^{(k-1)}$ 都有关, 因此这不是二分法的情形。

正如图 2.7(右图)中所示的那样, 固定点迭代并不收敛。实际上, 有下列结论成立。

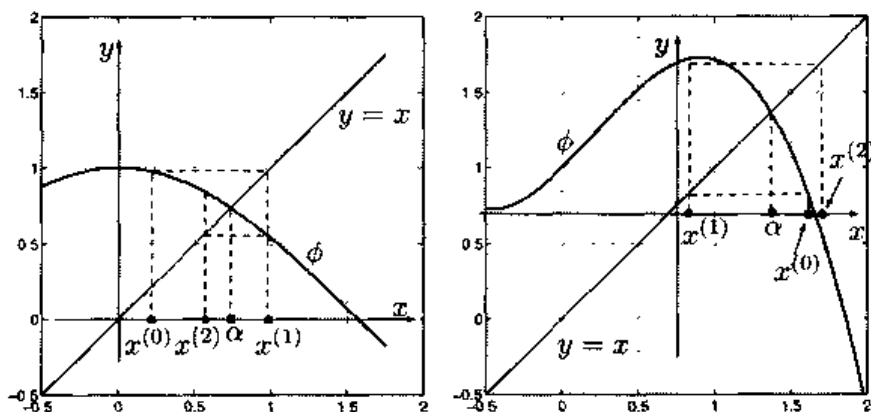


图 2.7 两个微分迭代函数的几个固定点迭代的示意图。左侧的迭代收敛到固定点 α , 而右侧的迭代产生一个发散的数列

命题 2.1 假设(2.9)中的迭代方程满足下列性质:

1. 对所有 $x \in [a, b]$, 有 $\phi(x) \in [a, b]$;
2. ϕ 在区间 $[a, b]$ 可微;
3. 对所有的 $x \in [a, b]$, $\exists K > 1$ 使得 $|\phi'(x)| \leq K$ 。

则无论区间 $[a, b]$ 上的初始数据 $x^{(0)}$ 如何选择, ϕ 有惟一的固定点 $\alpha \in [a, b]$, 且由式(2.9)定义的数列收敛到 α 。更进一步

$$\lim_{k \rightarrow \infty} \frac{x^{(k+1)} - \alpha}{x^{(k)} - \alpha} = \phi'(\alpha) \quad (2.11)$$

从式(2.11)可推断出固定点迭代至少是线性收敛的, 即对于足够大的 k , 在第 $k+1$ 步的误差相当于在第 k 步的误差乘以一个与 k 无关的常数 $\phi'(\alpha)$, 其绝对值严格小于 1。

例 2.5 函数 $\phi(x) = \cos(x)$ 满足命题 2.1 的所有假设。实际上, $|\phi'(\alpha)| = |\sin(\alpha)| \approx 0.67 < 1$, 这样由连续性可知存在一个 α 的邻域 I_α , 对于所有的 $x \in I_\alpha$, 均有 $|\phi'|$

$| \phi'(x) | < 1$ 成立。函数 $\phi(x) = x^2 - 1$, 虽然有两个固定点 $\alpha_{\pm} = (1 \pm \sqrt{5})/2$, 但由于 $| \phi'(\alpha_{\pm}) | = | 1 \pm \sqrt{5} | > 1$, 故不满足假设条件, 对应的固定点迭代不会收敛。

Newton 法并不是惟一的具有二次收敛特性的迭代算法。实际上, 有下列性质成立。

命题 2.2 设满足命题 2.1 的所有假定条件。另外假设 ϕ 是二次可微的且

$$\phi(\alpha) = 0, \quad \phi''(\alpha) \neq 0$$

那么固定点迭代式(2.9)是二阶收敛的, 并且

$$\lim_{k \rightarrow \infty} \frac{x^{(k+1)} - \alpha}{(x^{(k)} - \alpha)^2} = \frac{1}{2} \phi''(\alpha) \quad (2.12)$$

例 2.4 说明固定点迭代式(2.9)也可以用来计算函数 f 的零点。其实, 对于任意给定的 f , 由式(2.10)定义的函数 ϕ 并不是惟一存在的迭代函数。

例如, 对方程 $\log(x) = \gamma$ 的解, 设 $f(x) = \log(x) - \gamma$, 由式(2.10)可推出迭代方程:

$$\phi_N(x) = x(1 - \log(x) + \gamma)$$

另一种固定点迭代算法可通过在方程 $f(x) = 0$ 的两边各添加一个 x 来得到。相应的迭代方程是 $\phi_1(x) = x + \log(x) - \gamma$ 。选择迭代函数 $\phi_2(x) = x \log(x) / \gamma$ 可得到改进的方法。这些方法并非都是收敛的。例如, 当 $\gamma = -2$ 时, 对应于迭代函数 ϕ_N 和 ϕ_2 的方法都是收敛的, 而由于在固定点 α 的一个邻域内有 $| \phi_1'(x) | > 1$, 则使对应于迭代函数 ϕ_1 的方法不收敛。

2.3.1 如何终止固定点迭代

一般来说, 当相邻两次的迭代之差的绝对值小于给定的误差容限 ε 时, 固定点迭代将终止。

由于 $\alpha = \phi(\alpha)$ 且 $x^{(k+1)} = \phi(x^{(k)})$, 利用中值定理(见 1.4.3 节)可得:

$$\alpha - x^{(k+1)} = \phi(\alpha) - \phi(x^{(k)}) = \phi'(\xi^{(k)}) (\alpha - x^{(k)}), \quad \text{其中 } \xi^{(k)} \in I_{\alpha, x^{(k)}}$$

$I_{\alpha, x^{(k)}}$ 是以 α 和 $x^{(k)}$ 为端点的区间。利用恒等式

$$\alpha - x^{(k)} = (\alpha - x^{(k+1)}) + (x^{(k+1)} - x^{(k)})$$

可得

$$\alpha - x^{(k)} = \frac{1}{1 - \phi'(\xi^{(k)})} (x^{(k+1)} - x^{(k)}) \quad (2.13)$$

因此, 如果在 α 的邻域内有 $\phi'(x) \approx 0$, 相邻两步迭代之差会产生一个较为满意的误差估计, 这对于包括 Newton 法在内的二阶方法是成立的。但这个近似值在 ϕ' 接近于 1 时, 将不再满足要求。

例 2.6 对于 $m=11$ 和 $m=21$, 利用 Newton 法计算函数 $f(x)=(x-1)^{m-1}\log(x)$ 的零点 $\alpha=1$, m 为其重数。在这种情况下 Newton 法一阶收敛; 可以证明(参见习题 2.15) $\phi'_N=1-1/m$ 也是一种固定点迭代的算法, 其中 ϕ_N 是所用方法的迭代函数。随着 m 的增加, 由两个相邻的迭代之差得出的误差估计的精确度下降。这可由图 2.8 中的数值结果得到验证, 该图中, 对 $m=11$ 和 $m=21$ 分别比较了真实误差和估计误差之间的差异。随着 m 的增长, 这两个方程之间的差值也逐渐增大。

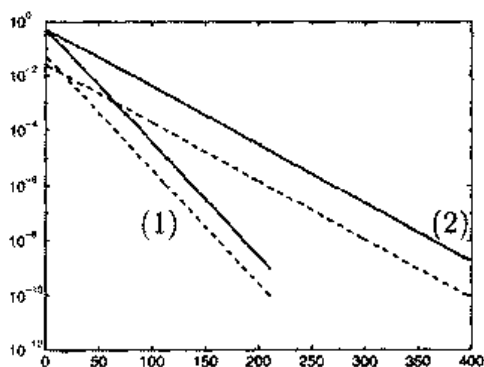


图 2.8 例 2.6 中, 误差的绝对值(实线)和两次相邻迭代之差的绝对值(虚线)分别对应于迭代次数的曲线图。其中曲线 1 取 $m=11$, 曲线 2 取 $m=21$

小结

1. 满足 $\phi(\alpha)=\alpha$ 的数字 α 叫作 ϕ 的固定点。对于它的计算可利用所谓的固定点迭代法: $x^{(k+1)} = \phi(x^{(k)})$;
 2. 固定点迭代在 ϕ 及其一阶导数符合假设条件的情况下收敛。一般地, 收敛是线性的, 但在 $\phi'(\alpha)=0$ 的特殊情况下, 固定点迭代二次收敛;
 3. 固定点迭代也可用来计算一个函数的零点。
- 参见习题 2.15~2.18。

2.4 补充说明

计算一个函数零点的最复杂的方法要结合不同的算法。特别指出的是,

MATLAB 函数 `fzero`(参见 1.4.1 节)采用了所谓的 Dekker-Brent 法(参见 [QSS00], 6.2.3 节)。在它的基本形式 `fzero(fun, x0)` 里, 要计算函数 `fun` 的零点, 其中 `fun` 既可以是代表 x 的函数的字符串, 也可以是内嵌函数的名字或 `m` 文件的名字。

例如, 也可通过 `fzero` 来解例 2.1 的问题, 利用初始值 $x_0=0.3$ (如 Newton 法所做的那样), 通过下列指令

```
>> f = inline('6000-1000*(1+1)/1*((1+1)^5-1)','1'); x0 = 0.3;
>> [alpha,res,flag,iter] = fzero(f,x0);
```

可得 $\alpha=0.06140241153653$, 残量 $\text{res}=9.0949\text{e}-13$, 在 $\text{iter}=29$ 迭代次数下。当 `flag` 是负的, 它表示 `fzero` 不能找到零点。Newton 法 6 次迭代就收敛到值 0.06140241153652 , 其残量等于 $2.3646\text{e}-11$ 。

本章中多次提到如何计算一个多项式的零点的问题。此问题的解决需要特别算法, 该方法基于 deflation 技术, 是一种能自动排除已经计算过的零点的程序。关于其他方法, 简要介绍一下 Sturm 法、Müller 法、Newton-Hörner 法(参见 [Atk89] 或 [QSS00]) 和 Bairstow 法([RR85])。另一种方法是将求解函数零点的问题转化为求解特殊矩阵(称为伴随矩阵)特征值的问题, 然后再利用适当的方法来计算。MATLAB 中的 `roots` 函数采用了这种方法, 这已在 1.4.2 节介绍过。

Newton 法和固定点迭代法经过扩展, 可以用来计算具有下列形式的非线性系统的根(参见 [QSS00], 第 7 章):

$$\begin{cases} f_1(x_1, x_2, \dots, x_n) = 0 \\ f_2(x_1, x_2, \dots, x_n) = 0 \\ \vdots \\ f_n(x_1, x_2, \dots, x_n) = 0 \end{cases}$$

其中 $f_1, \dots, f_n$ 是非线性函数。其他方法诸如 Broyden 和准 Newton 法, 可看成是 Newton 法的一般化(参见 [DS83])。

MATLAB 指令

```
zero = fsolve('fun', x0)
```

可以用来计算一个非线性系统的单极点, 该系统由用户函数 `fun` 定义, 以向量 $\mathbf{x}_0$ 作为初始猜测值。对于输入向量 $\mathbf{x}$ 的任意值, 函数 `fun` 返回了 n 个值 $f_1(\mathbf{x}), \dots, f_n(\mathbf{x})$ 。

例如, 考虑下列系统:

$$\begin{cases} x^2 + y^2 = 1 \\ \sin(\pi x/2) + y^3 = 0 \end{cases} \quad (2.14)$$

其解为(0.4761, -0.8794)和(-0.4761, 0.8794)。

相应的 MATLAB 用户函数, 即 systemnl, 其定义如下:

```
function fx = systemnl(x)
fx(1) = x(1)^2 + x(2)^2 - 1;
fx(2) = sin(pi*0.5*x(1)) + x(2)^3;
```

因此求解此题的 MATLAB 指令是:

```
>> x0 = [1 1]';
>> alpha = fsolve('systemnl', x0)
alpha =
    0.4761   -0.8794
```

利用此程序仅能找到两个根中的一个, 另一个可通过利用初始数据 $-x_0$ 计算得到。

2.5 习题

习题 2.1 给定函数 $f(x) = \cosh x + \cos x - \gamma$, 对于 $\gamma = 1, 2, 3$ 找出包含函数 f 零点的区间。然后通过二分法计算零点, 误差容限为 10^{-10} 。

习题 2.2 设气体为二氧化碳(CO_2), 式(2.1)中的系数 a 和 b 取下列值: $a = 0.401 \text{ Pa} \cdot \text{m}^6$, $b = 42.7 \times 10^{-6} \text{ m}^3$ (Pa 表示 Pascal)。用二分法计算在温度 $T = 300 \text{ K}$ 且压强为 $p = 3.5 \times 10^7 \text{ Pa}$ 的条件下, 1000 个 CO_2 分子所占的体积。误差容限为 10^{-12} (玻尔兹曼常数是 $k = 1.3806503 \cdot 10^{-23} \text{ Joule} \cdot \text{K}^{-1}$)。

习题 2.3 一个物体位于一个斜面上, 该斜面的坡度以恒定速度 ω 变化。 t 秒后, 物体的位置是

$$s(t, \omega) = \frac{g}{2\omega^2} [\sinh(\omega t) - \sin(\omega t)]$$

这里, $g = 9.8 \text{ m/s}^2$ 表示重力加速度。设物体已在 1 秒内移动了 1 米, 计算相应的 ω 的值, 误差容限为 10^{-5} 。

习题 2.4 证明不等式(2.3)。

习题 2.5 思考在程序 1 中为什么使用了指令 $x(2) = x(1) + (x(3) - x(1)) * 0.5$, 而

不是利用比较常用的 $x(2)=(x(1)+x(3))*0.5$ 来计算中点。

习题 2.6 利用 Newton 法解习题 2.1。并解释当 $\gamma=2$ 时该方法不精确的原因。

习题 2.7 利用 Newton 法计算一个正数 α 的平方根。进一步地, 可用相似的方法计算 α 的立方根。

习题 2.8 假设 Newton 法收敛, 当 α 是 $f(x)=0$ 的单根且 f 在 α 的邻域内二阶连续可微时, 证明式(2.6)成立。

习题 2.9 利用 Newton 法求解问题 2.3, 对于 $\beta \in [0, 2\pi/3]$, 误差容限是 10^{-5} 。设杆的长度为 $a_1=10$ cm, $a_2=13$ cm, $a_3=8$ cm, $a_4=10$ cm。由下列两个初始值 $x^{(0)}=-0.1$ 和 $x^{(0)}=2\pi/3$ 开始, 分别求出 β 值。

习题 2.10 注意函数 $f(x)=e^x-2x^2$ 有 3 个零点, $\alpha_1<0$, α_2 和 α_3 为正数。求出当 $x^{(0)}$ 为何值时, Newton 法收敛到 α_1 。

习题 2.11 利用 Newton 法计算 $f(x)=x^3-3x^22^{-x}+3x4^{-x}-8^{-x}$ 在 $[0,1]$ 内的零点, 并解释为什么收敛不是二次的。

习题 2.12 一颗炮弹以速度 v_0 和角度 α 在高度为 h 的炮管中发射, 当 α 满足 $\sin(\alpha)=\sqrt{2gh/v_0^2}$ 时达到其最大射程, 这里 $g=9.8\text{m/s}^2$ 是重力加速度。用 Newton 法计算 α , 设 $v_0=10\text{m/s}$, $h=1\text{m}$ 。

习题 2.13 用 Newton 法求解问题 2.1, 误差容限是 10^{-12} , 设 $M=6000$ 欧元, $v=1000$ 欧元且 $n=5$ 。初始推测值可通过在区间 $(0.01, 0.1)$ 上利用二分法经过 5 次迭代后得到。

习题 2.14 如图 2.9 所示的一个拐角走廊。能从一端到另一端在地面上滑动通过的杆的最大长度 L 由下列方程给出

$$L = l_2 / (\sin(\pi - \gamma - \alpha)) + l_1 / \sin(\alpha)$$

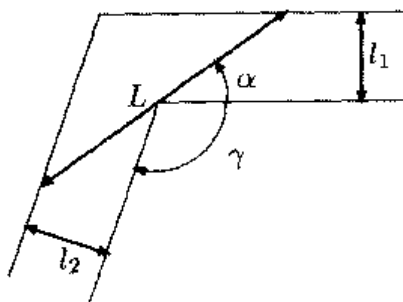


图 2.9 通道滑动杆的问题

这里 α 是非线性方程的解

$$l_2 \frac{\cos(\pi - \gamma - \alpha)}{\sin^2(\pi - \gamma - \alpha)} - l_1 \frac{\cos(\alpha)}{\sin^2(\alpha)} = 0 \quad (2.15)$$

用 Newton 法计算 α , 取 $l_2=10$, $l_1=8$, $\gamma=3\pi/5$ 。

习题 2.15 在考虑一个固定点迭代时, 设 ϕ_N 为 Newton 法的迭代函数。取 $\phi'_N(\alpha) = 1 - 1/m$, 这里 α 是 f 的一个 m 重零点。证明 Newton 法在 α 是 $f(x)=0$ 的单根时二次收敛, 否则是线性收敛的。

习题 2.16 从 $f(x)=x^3+4x^2-10$ 的曲线中推断出此函数有惟一实零点 α 。为计算 α 利用下列固定点迭代: 给定 $x^{(0)}$, 定义 $x^{(k+1)}$ 使得:

$$x^{(k+1)} = \frac{2(x^{(k)})^3 + 4(x^{(k)})^2 + 10}{3(x^{(k)})^2 + 8x^{(k)}}, \quad k \geq 0$$

并分析它对 α 的收敛性。

习题 2.17 分析固定点迭代法的收敛性

$$x^{(k+1)} = \frac{x^{(k)} [(x^{(k)})^2 + 3a]}{3(x^{(k)})^2 + a}, \quad k \geq 0$$

并计算正数 a 的平方根。

习题 2.18 利用基于残差(参见注 2.1)的终止标准来重复习题 2.11 所执行的计算, 并比较哪一个结果更为精确。

第 3 章 函数和数据的逼近

逼近一个函数 f 就是利用 n 个简单形式的替换函数 $\tilde{f}$ 来代替原来的函数 f 。这种方法在数值积分中经常使用, 正如将要在第 4 章中介绍的那样, 一般并不是直接来计算 $\int_a^b f(x)dx$, 而是通过计算 $\int_a^b \tilde{f}(x)dx$ 来代替, 其中 $\tilde{f}$ 是一个易于积分的函数(如多项式)。还有的情况是, 函数 f 可能只在一些孤立点处有值。在这种情况下, 我们的目标是构造一个连续函数 $\tilde{f}$, 此函数是能够代表有限数据集合的经验定律来表示出的。下面给出两个例子来介绍这种方法。

问题 3.1(气候学) 接近地面的大气温度取决于那里的碳酸浓度 K 。在表 3.1(摘自《哲学杂志》41 期, 237 页(1986))中, 描述了在 3 个不同的 K 值下, 地球不同纬度上的平均温度相对于实际的平均温度(以参考值 $K=1$ 作为标准)的变化情况。此时, 利用已知数据可以构造一个函数, 该函数能够提供在任意允许的纬度和其他 K 值下的平均温度的近似值。

问题 3.2(财政学) 图 3.1 描述了两年来在苏黎士股票交易市场中的一种股票的价格变化情况。该曲线是通过用直线连接每天休市时所报出的价格来得到的。这种简单的表示, 实际隐含了一个假设, 即价格在一天时间内是线性变化的(这种近似叫做复合线性插值)。我们想知道从这张图中能否预测出在最后一次报价后的一小段时间内的股票价格。在第 3.4 节可看到这种预测可利用一种称为最小平方数据逼近的特殊方法来得到(参见例题 3.8)。

表 3.1 对于不同纬度的不同的碳酸浓度值 K , 地球上平均每年的温度变化

纬度	$K=0.67$	$K=1.5$	$K=2.0$	$K=3.0$
65	-3.1	3.52	6.05	9.3
55	-3.22	3.62	6.02	9.3
45	-3.3	3.65	5.92	9.17
35	-3.32	3.52	5.7	8.82
25	-3.17	3.47	5.3	8.1
15	-3.07	3.25	5.02	7.52
5	-3.02	3.15	4.95	7.3
-5	-3.02	3.15	4.97	7.35
-15	-3.12	3.2	5.07	7.62
-25	-3.2	3.27	5.35	8.22

(续表)

纬度	$K=0.67$	$K=1.5$	$K=2.0$	$K=3.0$
-35	-3.35	3.52	5.62	8.8
-45	-3.37	3.7	5.95	9.25
-55	-3.25	3.7	6.1	9.5

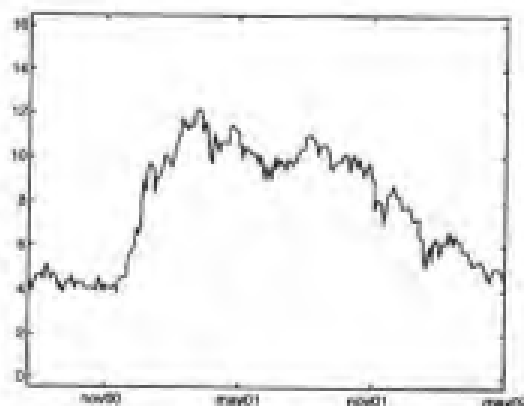
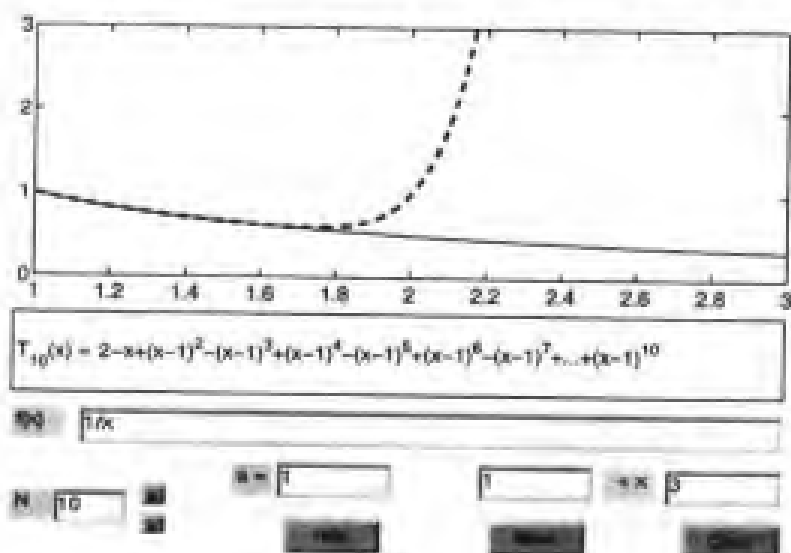


图 3.1 两年来某种股票的价格变化

众所周知,一个函数 f 可以在给定的区间内用 1.4.3 节介绍的 Taylor 多项式代替。这种算法所花费的代价也比较大,因为它需要已知在给定点 x_0 处关于 f 及其直到 n 阶(多项式的次数)的导数的知识。另一个更大的缺点是从点 x_0 展开的 Taylor 多项式在远离点 x_0 处也许不能精确地表示 f 。例如,图 3.2 比较了函数 $f(x)=1/x$ 和它在点

图 3.2 函数 $f(x)=1/x$ (实线)和它在点 $x_0=1$ 处的 10 次 Taylor 展开式(虚线)的比较。

这里也给出了 Taylor 展开式的解析形式

$x_0=1$ 处的 10 阶 Taylor 展开式的曲线。这幅图也显示出了 MATLAB 函数 `taylor` 的图形界面,该函数可以计算得到任意给定函数的任意阶数的 Taylor 展开式。函数

和 Taylor 展开式之间在 $x_0=1$ 的一个非常小的邻域内符合得非常好, 但当 $x-x_0$ 变大时, 二者之间的一致性就会变得很差。好在其他函数如指数函数并非如此, 指数函数只要阶数 n 足够大, 它在 $x_0=0$ 处的 Taylor 展开式对于所有的 $x \in \mathbf{R}$ 逼近精度都非常高。

在本章, 我们会介绍基于另一种途径的逼近方法。

3.1 插值

从问题 3.1 和 3.2 中可看出, 在一些应用中, 对于函数的了解可能只限于该函数在一些离散点的取值。此时面对的(一般)情况是: 已知 $n+1$ 个数对 $\{x_i, f(x_i)\}$, 其中 $i=0, \dots, n$, 并且点 x_i 互不相同, 称之为节点。

例如对于表 3.1 的情况, n 等于 12, 把第一列中给出的纬度值作为节点 x_i , 而 $f(x_i)$ 是在其余几列中相应的(温度)值。

此时自然会要求逼近函数 $\tilde{f}$ 满足下列关系:

$$\tilde{f}(x_i) = f(x_i), \quad i = 0, 1, \dots, n \quad (3.1)$$

这里 $\tilde{f}$ 称为 f 的插值, 方程(3.1)是插值条件。

下面给出几种比较重要的插值:

——多项式插值:

$$\tilde{f}(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

——三角插值:

$$\tilde{f}(x) = a_{-M}e^{-iMx} + \dots + a_0 + a_Me^{iMx}\sqrt{b^2 - 4ac}$$

其中 M 是整数, 如果 n 是偶数, 则 M 等于 $n/2$, 如果 n 是奇数, 则 M 等于 $(n-1)/2$, i 是虚数单位;

——有理数插值

$$\tilde{f}(x) = \frac{a_0 + a_1x + \dots + a_kx^k}{a_{k+1} + a_{k+2}x + \dots + a_{k+n-1}x^n}$$

为简单起见, 本书只讨论那些与 $n+1$ 个未知系数 α_i 是线性依赖关系的插值。多项式和三角插值都属于这一类, 而有理数插值不在其中。

3.1.1 Lagrangian 多项式插值

下面重点讨论多项式插值。有以下结论成立:

命题 3.1 对于任意的数对集合 $\{x_i, f(x_i)\}$, 其中 $i=0, \dots, n$, x_i 是不同的节点, 存在惟一的阶数小于或等于 n 的多项式, 记为 $\Pi_n f$, 称之为 $f(x_i)$ 在节点 x_i 的插值多项式, 使得下式成立:

$$\Pi_n f(x_i) = f(x_i), \quad i=0, \dots, n \quad (3.2)$$

当 $f(x_i)$ (其中 $i=0, \dots, n$) 的取值来自于连续函数 f 时, $\Pi_n f$ 称为 f 的插值多项式 (简称 f 的插值)。

下面使用反证法来验证惟一性, 假设存在两个 n 阶多项式, $\Pi_n f$ 和 $\Pi'_n f$, 均满足节点关系 (3.2)。它们的差 $\Pi_n f - \Pi'_n f$, 也将是一个 n 次多项式, 并且在 $n+1$ 个节点取值为零。由一个著名的代数定理可知, 这样的多项式只能取值为零, 所以 $\Pi'_n f$ 必定与 $\Pi_n f$ 一致。

为得到 $\Pi_n f$ 的表达式, 先从如下特殊情形开始, 此时 $f(x_i)$ 在所有 $i \neq k$ 以外的点处取值为零, 其中 k 是固定值, 并且 $f(x_k) = 1$ 。再设 $\varphi_k(x) = \Pi_n f(x)$, 则必有 (见图 3.3)

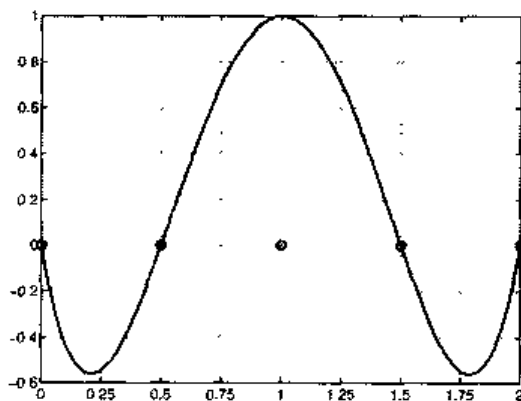


图 3.3 与 5 个平均分布的节点的集合相对应的多项式 $\varphi_2 \in P_4$

$$\varphi_k \in P_n, \quad \varphi_k(x_i) = \delta_{jk} = \begin{cases} 1 & j=k \\ 0 & j \neq k \end{cases}$$

(δ_{jk} 是 Kronecker 符号)。

函数 φ_k 可以写成下列形式

$$\varphi_k(x) = \prod_{\substack{j=0 \\ j \neq k}}^n \frac{x - x_j}{x_k - x_j}, \quad k = 0, \dots, n \quad (3.3)$$

现在讨论一般形式, 这里 $\{f(x_i), i=0, \dots, n\}$ 是一个任意值的集合。利用一个显然的迭加准则即可求得下列 $\Pi_n f$ 的表达式:

$$\Pi_n f(x) = \sum_{k=0}^n f(x_k) \varphi_k(x) \quad (3.4)$$

实际上, 这个展开式满足插值条件(3.2), 这是因为

$$\Pi_n f(x_i) = \sum_{k=0}^n f(x_k) \varphi_k(x_i) = \sum_{k=0}^n f(x_k) \delta_{ik} = f(x_i), \quad i = 0, \dots, n$$

由于这些函数的重要地位, 函数 φ_k 称为 Lagrange 特征多项式, 且式(3.4)是插值的 Lagrange 形式。

在 MATLAB 里, 可把 $n+1$ 个数对 $\{(x_i, f(x_i))\}$ 存储在向量 x 和 y 中, 然后使用指令 $c = \text{polyfit}(x, y, n)$ 给出插值多项式的系数。具体说来, $c(1)$ 将给出 x^n 的系数, $c(2)$ 将给出 x^{n-1} 的系数, $\dots c(n+1)$ 将给出 $\Pi_n f(0)$ 的值(有关此命令的更多用法可参见 3.4 节)。正如在第 1 章所见到的, 可用指令 $p = \text{polyval}(c, z)$ 来计算 $p(j)$ 的值, 该值通过在 $z(j)$, $j=1, \dots, m$ 的插值多项式得到, 后者是 m 个任意点的集合。

在可以得到函数 f 的显式形式的情况下, 可以利用指令 $y = \text{eval}(f)$ 来得到由 f 在一些特定节点处(应存于向量 x 中)的取值组成的向量 y 。

例 3.1 为得到问题 3.1 的数据相对于值 $K=0.67$ (表 3.1 的第一列)的插值多项式, 只选用纬度 65、35、5、-25、-55 的温度值, 可以利用下列 MATLAB 指令:

```
>>> x = [-55 -25 5 35 65]; y = [-3.25 -3.2 -3.02 -3.32 -3.1];
>> format short e; c = polyfit(x, y, 4)
c =
    8.2819e-08   -4.5267e-07   -3.4684e-04    3.7757e-04   -3.0132e+00
```

插值多项式的图形可以利用下列指令得到:

```
>> z = linspace(x(1), x(end), 100);
>> p = polyval(c, z); plot(z, p, x, y, 'o');
```

为了得到一个平滑的曲线, 我们求出了在区间 $[-55, 65]$ 内的平均分布的 101 个点的值(实际上, MATLAB 的图形总是由相邻两点间的分段线性插值构成)。注意指令 $x(\text{end})$ 直接给出了向量 x 的最后一个元素, 而无需事先给出向量长度。在图 3.4 中黑点对应于那些用来构造插值多项式的值, 而圆圈对应于那些没有使用的值。我们可

以定性地看到曲线与分布的数据间的一致性。

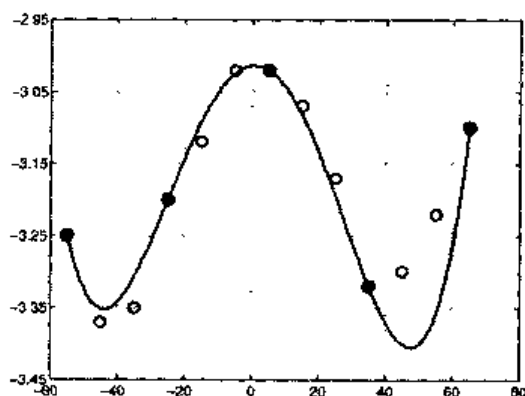


图 3.4 例 3.1 中介绍的 4 阶插值多项式

使用下列结果可以计算利用插值多项式 $\Pi_n f$ 来代替 f 时所产生的误差值:

命题 3.2 设 I 是一有界区间, 考虑 I 中的 $n+1$ 个不同的插值点 $\{x_i, i=0, \dots, n\}$, 设 f 在 I 中 $n+1$ 阶连续可微。那么 $\forall x \in I \exists \xi \in I$, 使得下式成立:

$$E_n f(x) = f(x) - \Pi_n f(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i) \quad (3.5)$$

显然, $E_n f(x_i) = 0, i=0, \dots, n$ 。

式(3.5)在节点为均匀分布的情况下可化为更简洁的表述形式, 即当 $x_i = x_{i-1} + h$ 时, 其中 $i=1, \dots, n, h>0$ 和 x_0 均为给定的数值。如习题 3.1 所述, $\forall x \in (x_0, x_n)$, 可以证明:

$$\left| \prod_{i=0}^n (x - x_i) \right| \leq n! \frac{h^{n+1}}{4} \quad (3.6)$$

因此,

$$\max_{x \in I} |E_n f(x)| \leq \frac{\max_{x \in I} |f^{(n+1)}(x)|}{4(n+1)!} h^{n+1} \quad (3.7)$$

尽管当 $n \rightarrow \infty$ 时, $h^{n+1}/[4(n+1)!]$ 趋向于零, 但是不能从式(3.7)中推断出此时误差也趋于 0。实际上, 正如例 3.2 所示的那样, 存在函数 f , 其极限是无穷大, 即

$$\lim_{n \rightarrow \infty} \max_{x \in I} |E_n f(x)| = \infty$$

这个令人意外的结果表明, 通过增加插值多项式的阶数 n , 并不一定能得到函数 f 的更精确的插值。例如, 如果使用了表 3.1 中的第一列的所有数据, 可得到如图 3.5 所示的插值多项式 $\Pi_{12}f$, 其在区间左侧的图形远不如图 3.4 中利用更少数目节点的图形令人满意。对于特殊类的函数可能会出现更差的结果, 正如将要在下面的例子中所描述的那样。

例 3.2 (Runge) 如果函数 $f(x)=1/(1+x^2)$ 在区间 $I=(-5, 5)$ 上利用均匀分布的节点进行插值, 当 $n \rightarrow \infty$ 时, 误差 $\max_{x \in I} |E_n f(x)|$ 趋向于无穷。这是由于当 $n \rightarrow \infty$ 时, $\max_{x \in I} |f^{(n+1)}(x)|$ 的数量级超出了 $h^{n+1}/[4(n+1)]$ 的无穷小阶。这个结论能够通过计算 f 及其高至 21 阶导的最大值来验证, 可以利用下列指令来实现:

```
>> syms x; n=20; f='1/(1+x^2)'; df=diff(f,1);
>> cdf=char(df);
>> for i=1:n+1, df=diff(df,1); cdfn=char(df);
    x=fzero(cdfn,0); M(i)=abs(eval(cdf)); cdf=cdfn;
end
```

函数 $f^{(n)}$ 的绝对值, $n=1, \dots, 21$, 存储在向量 M 中。注意函数 `char` 把符号表达式 `df` 转换成能通过函数 `fzero` 计算的字符串。特别地, 当 $n=3, 9, 15, 21$ 时, $f^{(n)}$ 的绝对值分别为:

```
>> M([3,9,15,21])=
ans =
    4.6686e+00    3.2426e+05    1.2160e+12    4.8421e+19
```

而相应的 $\prod_{i=0}^n (x-x_i)/(n+1)!$ 的最大值是

```
>> z=linspace(-5,5,10000);
>> for n=0:20; h=10/(n+1); x=[-5:h:5];
    c=poly(x); r(n+1)=max(polyval(c,z)); r(n+1)=r(n+1)/prod([1:n+2]); end
>> r([3,9,15,21])
ans=
    2.8935e+00    5.1813e-03    8.5854e-07    2.1461e-11
```

其中 $c=\text{poly}(x)$ 是一个向量, 它以多项式的系数作为元素, 而 x 是以该多项式的根作为元素的向量。由此可以得出 $\max_{x \in I} |E_n f(x)|$ 的取值分别为:

```
>> format short e;
    1.3509e+01    1.6801e+03    1.0442e+06    1.0399e+09
```

它们分别对应于 $n=3, 9, 15, 21$ 。

函数 f 的插值多项式曲线中会出现一些振荡, 尤其是在靠近区间端点附近(见图 3.5, 右侧)的曲线更是如此, 这表明函数并非是完全收敛的。这种现象叫做 Runge 现象。

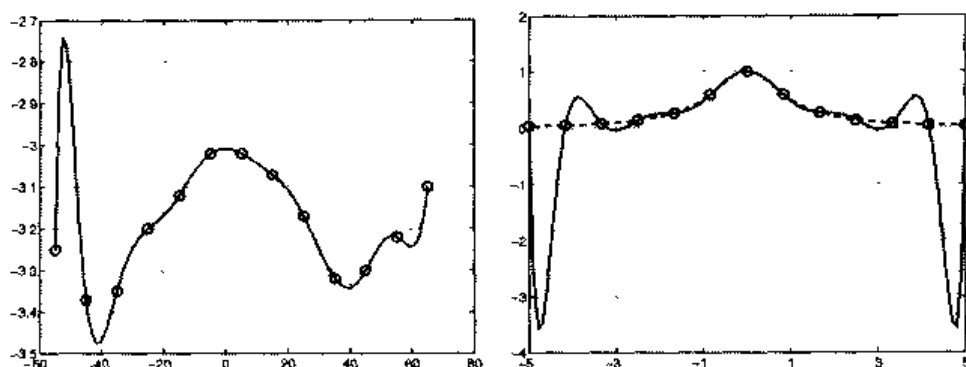


图 3.5 Runge 现象的两个例子: 左侧, 利用表 3.1 中 $K=6.7$ 的数据计算得到的 $\Pi_{12}f$; 右侧, 函数 $f(x)=1/(1+x^2)$ 的 13 个均匀分布的节点(虚线)计算得到的 $\Pi_{12}f$ (实线)

注 3.1 可以证明下列不等式成立:

$$\max_{x \in I} |f'(x) - (\Pi_n f)'(x)| \leq Ch^n \max_{x \in I} |f^{(n+1)}(x)|$$

其中 C 是与 h 无关的常数。因此如果用 $\Pi_n f$ 的一阶导来逼近 f 的一阶导, 则相对于 h 的收敛性便会减少一阶。在 MATLAB 中, $(\Pi_n f)'$ 可以用指令 $[d] = \text{polyder}(c)$ 来得到, 其中 c 是存储插值多项式系数的输入向量, 而 d 是存储它的一阶导的系数的输出向量(参见 1.4.2 节)。

参见习题 3.1~3.4。

3.1.2 Chebyshev 插值

利用合适的节点分布就可避免 Runge 现象。特别指出, 在任意区间 $[a, b]$ 内, 考虑所谓的 Chebyshev 节点(见图 3.6(a)):

$$x_i = \frac{a+b}{2} + \frac{b-a}{2} \hat{x}_i, \text{ 其中 } \hat{x}_i = -\cos(\pi i/n), \quad i=0, \dots, n$$

实际上, 对于特殊的节点分布能够证明, 如果 f 在区间 $[a, b]$ 是连续的可微的函数, 对于所有的 $x \in [a, b]$, 当 $n \rightarrow \infty$ 时, $\Pi_n f$ 收敛到 f 。

Chebyshev 节点, 即平均分布在半个单位圆上的节点, 它们在坐标轴上的坐标, 位于区间 $[a, b]$ 内并且在区间端点处较为密集(见图 3.6)。

另一种在区间 $[a, b]$ 上非一致分布的节点, 具有与 Chebyshev 节点相同的性质, 这些节点表示为:

$$\bar{x}_i = \frac{a+b}{2} - \frac{b-a}{2} \cos\left(\frac{2i+1}{n+1} \frac{\pi}{2}\right), \quad i=0, \dots, n$$

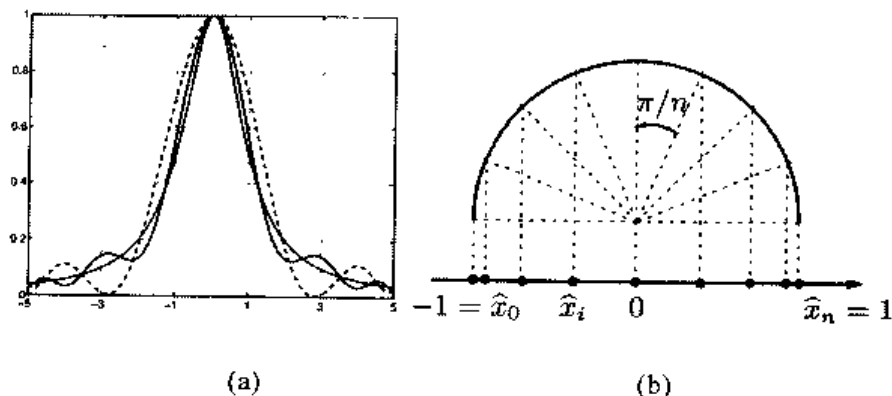


图 3.6 (a)图中比较了函数 $f(x)=1/(1+x^2)$ (细实线) 的 Chebyshev 8 次 (虚线) 和 12 次 (实线) 插值多项式的曲线。注意随着次数的增加, 曲线的伪振荡幅度减弱。
(b) 图中显示了在区间 $[-1, 1]$ 的 Chebyshev 节点的分布情况

例 3.3 重新考虑 Runge 例子中的函数 f , 计算其在 Chebyshev 节点的插值多项式。这可通过下列 MATLAB 指令得到:

```
>> xc=-cos(pi*[0:n]/n); x=(a+b)*0.5+(b-a)*xc*0.5;
```

其中 $n+1$ 是节点的数目, 而 a 和 b 是插值区间 (下面将选择 $a=-5$ 和 $b=5$) 的端点。然后可以通过下列指令来计算插值多项式:

```
>> f='1./(1+x.^2)'; y=eval(f); c=polyfit(x,y,n);
```

现在, 在区间 $[-5, 5]$ 内均匀分布的 1001 个点上, 计算 f 与其 Chebyshev 插值之间误差的绝对值, 并取其误差的最大值:

```
>> x=linspace(-5,5,1000); p=polyval(c,x); fx=eval(f); err=
max(abs(p-fx));
```

如表 3.2 所示, 随着 n 的增加, 最大误差是逐渐下降的

表 3.2 Runge 函数 $f(x)=1/(1+x^2)$ 的 Chebyshev 插值误差

N	5	10	20	40
E_n	0.6386	0.1322	0.0177	0.0003

3.1.3 三角插值和 FFT

为了逼近一个周期函数 $f: [0, 2\pi] \rightarrow R$, 即满足 $f(0)=f(2\pi)$, 我们利用一个三角多项式 $\tilde{f}$ 来实现, 其中 f 是通过在 $n+1$ 个节点 $x_j=2\pi j/(n+1)$, $j=0, \dots, n$, 上进行插值得到的, 即

$$\tilde{f}(x_j)=f(x_j), \quad j=0, \dots, n \quad (3.8)$$

三角插值 $\tilde{f}$ 是由正弦和余弦函数的线性组合得到的。

特别地, 如果 n 是偶数, $\tilde{f}$ 会有下列形式:

$$\tilde{f}(x) = \frac{a_0}{2} + \sum_{k=1}^M [a_k \cos(kx) + b_k \sin(kx)] \quad (3.9)$$

其中 $M=n/2$, 如果 n 为奇数:

$$\tilde{f}(x) = \frac{a_0}{2} + \sum_{k=1}^M [a_k \cos(kx) + b_k \sin(kx)] + a_{M+1} \cos((M+1)x) \quad (3.10)$$

其中 $M=(n-1)/2$ 。可将式(3.9)改写为下列形式:

$$\tilde{f}(x) = \sum_{k=-M}^M c_k e^{ikx} \quad (3.11)$$

i 是虚数单位。系数 c_k 与系数 a_k 和 b_k 的关系如下:

$$a_k = c_k + c_{-k}, \quad b_k = i(c_k - c_{-k}), \quad k=0, \dots, M \quad (3.12)$$

实际上, 由式(1.5)可得 $e^{ikx} = \cos(kx) + i\sin(kx)$ 且

$$\begin{aligned} \sum_{k=-M}^M c_k e^{ikx} &= \sum_{k=-M}^M c_k (\cos(kx) + i\sin(kx)) \\ &= \sum_{k=1}^M (c_k (\cos(kx) + i\sin(kx)) + c_{-k} (\cos(kx) - i\sin(kx))) + c_0 \end{aligned}$$

因此再由式(3.12)可以推导出式(3.9)。

类似地, 当 n 是奇数时, 式(3.10)变为:

$$\tilde{f}(x) = \sum_{k=-\{M+1\}}^{M+1} c_k e^{ikx} \quad (3.13)$$

其中当 $k=0, \dots, M$ 时系数 c_k 与前面相同, 而 $c_{M+1} = c_{-(M+1)} = a_{M+1}/2$ 。两种情况下, 均有:

$$\tilde{f}(x) = \sum_{k=-(M+\mu)}^{M+\mu} c_k e^{ikx} \quad (3.14)$$

n 为奇数时, $\mu=0$; 当 n 为偶数时, $\mu=1$ 。

由于形式上与傅立叶级数相似, 所以 $\tilde{f}$ 称为(实)离散傅里叶变换(DFT)。在节点 $x_j = jh$ 上加上插值条件, 其中 $h = 2\pi/(n+1)$, 可以得到:

$$\sum_{k=-(M+\mu)}^{M+\mu} c_k e^{ikjh} = f(x_j), \quad j=0, \dots, n \quad (3.15)$$

为计算系数 $\{c_k\}$, 用 $e^{-imx_j} = e^{-imjh}$ 乘以方程(3.15), 这里 m 是 0 和 n 之间的整数, 并且对 j 求和:

$$\sum_{j=0}^n \sum_{k=-(M+\mu)}^{M+\mu} c_k e^{ikjh} e^{-imjh} = \sum_{j=0}^n f(x_j) e^{-imjh} \quad (3.16)$$

现在利用下列恒等式:

$$\sum_{j=0}^n e^{ijh(k-m)} = (n+1) \delta_{km}$$

其中 δ_{km} 是 Kronecker 符号。这个恒等式在 $k=m$ 时显然成立。当 $k \neq m$ 时, 有

$$\sum_{j=0}^n e^{ijh(k-m)} = \frac{1 - \left(e^{i(k-m)h}\right)^{n+1}}{1 - e^{i(k-m)h}}$$

等式右侧的分子是零, 这是因为:

$$1 - e^{i(k-m)h(n+1)} = 1 - e^{i(k-m)2\pi} = 1 - \cos((k-m)2\pi) - i \sin((k-m)2\pi)$$

因此, 由(3.16)式可以得到 $\tilde{f}$ 系数的显式表达式:

$$c_m = \frac{1}{n+1} \sum_{j=0}^n f(x_j) e^{-imjh}, \quad m=-(M+\mu), \dots, M+\mu$$

所有系数 c_m 的计算, 可以利用快速傅立叶变换(FFT)来进行 $n \log_2 n$ 阶计算来完成, MATLAB 中的 `fft`(见例 3.4)函数可以实现上述计算。利用反变换可以得到相似的结论, 此时系数 $\{c_k\}$ 经过反变换可以得到 $\{f(x_j)\}$ 的值。MATLAB 程序 `ifft` 可以实现快速傅立叶反变换。

例 3.4 考虑函数 $f(x) = x(x-2\pi)e^{-x}$, 其中 $x \in [0, 2\pi]$ 。利用 MATLAB 程序 `fft`

时, 首先要求出 f 在节点 $x_j = j\pi/5$, $j=0, \dots, 9$ 处的值。然后利用下列指令(注意此处 $*$ 表示向量中元素间的乘积):

```
>> x=pi/5*[0:9]; y=x.*(x-2*pi).*exp(-x); Y=fft(y);
```

可以得到:

```
>>Y =
Columns 1 through 3
-6.5203e+00    -4.6728e-01+4.2001e+00i    1.2681e+00+1.6211e+00i
Columns 4 through 6
    1.0985e+00+6.0080e-01i    0.2585e-01+2.1398e-01i    8.7010e-01-1.3887e-16i
Columns 7 through 6
    9.2585e-01-2.1398e-01i    1.0985e+00-6.0080e-01i    1.2681e+00-1.6211e+00i
Column 10
-4.6728e-01    -4.2001e+00i
```

系数 $\{c_k\}$ 可以很容易地由 $Y/\text{length}(Y)$ 给出, 这里 length 函数计算的是向量的维数(本例中为 10)。

注意函数 ifft 虽然可以对 n 的任意值进行计算, 但当 n 为 2 的幂数时, 能够取得最大的效率。

interpft 函数提供了数据集的三角插值。它需要输入一个整数 N 和一个向量值, 该向量是函数(以 p 为周期的周期函数)在点 $x_i = ip/M$ (其中 $i=1, \dots, M-1$) 处取值的集合。 interpft 返回了三角插值的 N 个值, 这些值是通过在节点 $t_i = ip/N$, $i=0, \dots, N-1$ 处进行傅里叶变换得到的。例如, 在 $[0, 2\pi]$ 上重新考虑例 3.4 的函数, 并在 10 个平均分布的节点 $x_i = i\pi/5$, $i=0, \dots, 9$ 上取值。则可以得到三角插值在 100 个平均分布的节点 $t_i = i\pi/100$, $i=0, \dots, 99$ 处的值(见图 3.7):

```
>>x=pi/5*[0:9];y=x.*(x-2*pi).*exp(-x);z=interpft(y,100);
```

在一些情况下, 三角插值的精确度可能会极大地降低, 正如将要在下例中所见到的那样。

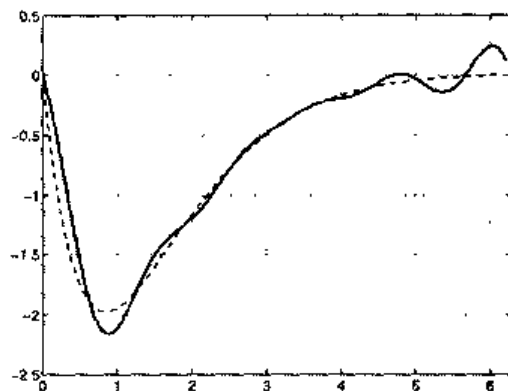


图 3.7 函数 $f(x)=x(x-2\pi)e^{-x}$ (虚线)和相对于 10 个平均分布的节点的相应的三角插值(连续线)

例 3.5 逼近函数 $f(x)=f_1(x)+f_2(x)$, 其中 $f_1(x)=\sin(x)$, $f_2(x)=\sin(5x)$, 在区间 $[0, 2\pi]$ 上利用 9 个平均分布的节点。得到的结果如图 3.8 所示。注意三角逼近在有些区间上对于函数 f 甚至会出现相位倒置的情况。

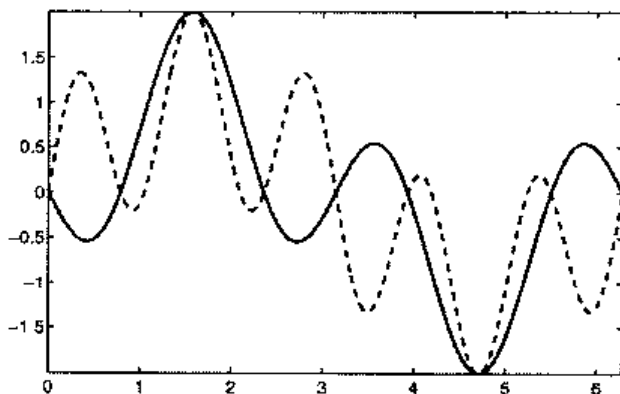


图 3.8 混淆效应:函数 $f(x)=\sin(x)+\sin(5x)$ (实线)与其 $M=3$ 的三角插值(3.9)(虚线)曲线之间的对比情况

产生不准确结果的原因可以解释如下:在上面所选择的节点上,函数 f_2 与较低频率的 $f_3(x)=-\sin(3x)$ (参见图 3.9)是难以区分的。所以实际逼近的函数是 $F(x)=f_1(x)+f_3(x)$ 而不是 $f(x)$ (实际上,图 3.8 的虚线与 F 是不一致的)。

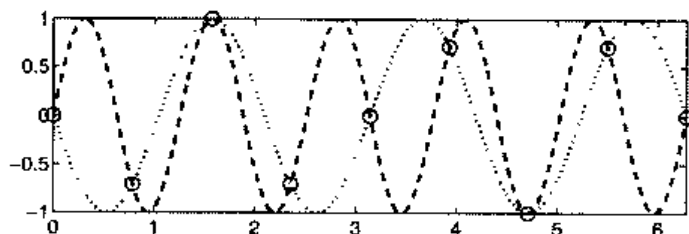


图 3.9 混淆现象:函数 $\sin(5x)$ (虚线)和 $-\sin(3x)$ (点线)在插值点取相同的值。这种情况解释了图 3.8 所示的精确度严重降低的原因

这种现象叫做混淆效应,它出现在被逼近函数是几个不同频率函数之和的情况下。只要节点的数目比较少而不足以解析最高频率,则高频分量就会与低频分量形成干涉,从而使得差值的精确度降低。为了更好地对高频函数进行逼近,必须要增加插值的节点数目。

混淆现象的一个实际例子是有辐车轮的旋转具有明显的倒转的感觉。一旦达到了某个临界的速度,人脑便不再能够精确地采样出运动的图像,结果产生了歪曲的图像。

小结

1. 在区间 $[a, b]$ 上,要逼近一个数据集或者一个函数 f 需要找到一个能够以

足够精度表示它们的合适的函数 $\tilde{f}$;

2. 插值过程就是确定一个函数 $\tilde{f}$, 使得 $\tilde{f}(x_i) = y_i$, 这里 $\{x_i\}$ 是给定的节点, $\{y_i\}$ 是 $\{f(x_i)\}$ 的值, 或是预先给定的一个数值集合;

3. 如果 $n+1$ 个节点 $\{x_i\}$ 互不相同, 就存在惟一的阶数小于或等于 n 的多项式, 该多项式是通过在节点 $\{x_i\}$ 上, 利用预先给定的数值集合 $\{y_i\}$ 进行插值得到的;

4. 对于在 $[a, b]$ 上均匀分布的节点, $[a, b]$ 上任何一点的插值误差, 在 n 趋于无穷时, 并不一定是趋于 0 的。但是也存在特殊分布的节点, 如 Chebyshev 节点, 对于这种节点, 上述收敛性质对于所有的连续函数都是成立的。

5. 三角插值非常适合于逼近周期函数, 它利用正弦和余弦函数的线性组合 $\tilde{f}$ 来作为插值函数。FFT 是一种非常有效的算法, 它能够从节点的值出发计算出一个三角插值的傅立叶系数, 并且还存在着与 FFT 类似的快速逆变换形式 IFFT。

3.2 分段线性插值

Chebyshev 插值提供了一种对已知表达式的平滑函数 f 的精确逼近的方法。在有些情况下, 即 f 是非平滑的或者只给出 f 在一些特定点的值(与 Chebyshev 节点不一致), 就要使用一种不同的插值方法, 叫做线性复合插值。

具体来讲, 给定一组节点 $x_0 < x_1 < \dots < x_n$, 用 I_i 表示区间 $[x_i, x_{i+1}]$ 。用一个连续函数来逼近 f , 此连续函数在每个小区间上由连接 $(x_i, f(x_i))$ 和 $(x_{i+1}, f(x_{i+1}))$ 两点的线段给出(见图 3.10)。该函数用 $\prod_1^H f$ 表示, 称为分段线性插值多项式, 其表达式为:

$$\prod_1^H f(x) = f(x_i) + \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i} (x - x_i), \quad x \in I_i$$

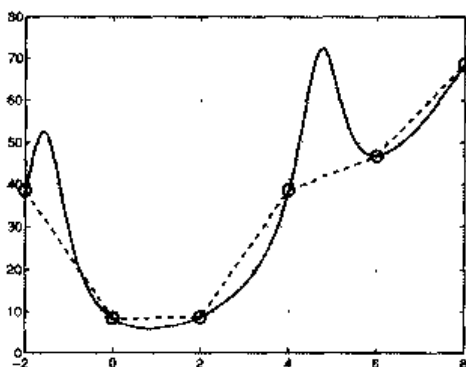


图 3.10 函数 $f(x) = x^2 + 10/(\sin(x) + 1.2)$ (实线) 和它的分段线性插值多项式 $\prod_1^H f$ (虚线)

上角标 H 表示区间 I_i 的最大长度。从式(3.7)可推出下列结果, 设 $n=1$ 和 $h=H$:

命题 3.3 如果 $f \in C^2(I)$, 其中 $I = [x_0, x_n]$, 则

$$\max_{x \in I} \left| f(x) - \prod_1^H f(x) \right| \leq \max_{x \in I} |f''(x)|$$

因此, 对于所有在插值区间的 x , 只要 f 是足够平滑的, 当 $H \rightarrow 0$ 时, $\prod_1^H f(x)$ 将趋于 $f(x)$ 。

利用指令 $s1 = \text{interp1}(x, y, z)$, 可以计算出分段线性多项式在任意点的值, 这些值存储于向量 z 中, 而该多项式是通过在节点 $x(i)$ (其中 $i = 1, \dots, n+1$) 上对 $y(i)$ 进行插值得到的。注意 z 的维数可以是任意的。如果节点是按增序排列的 (即 $x(i+1) > x(i)$, 其中 $i = 1, \dots, n$), 则可以利用加快指令 $\text{interp1q}(q)$ (表示快) 来计算。

值得一提的是命令 fplot , 该命令用于显示函数 f 在给定的区间 $[a, b]$ 上的图形, 其实它就是用分段线性插值来代替函数。插值节点由函数自动产生, 产生的标准是在 f 剧烈变化处插值点密集。我们称这种类型的程序为自适应的。

3.3 样条函数逼近

分段线性插值的主要缺点是 $\prod_1^H f$ 仅仅是一个全局的连续函数。但在一些实际应用中, 如计算机图形学, 希望逼近函数至少是具有连续一阶导数的平滑函数。

于是, 需要构造一个具有下列性质的函数 s_3 :

1. 在每个小区间 $I_i = [x_i, x_{i+1}]$ 中, 其中 $i = 0, \dots, n-1$, s_3 是一个 3 次多项式, 它在多项式中的插值为 $(x_i, f(x_i))$;
2. s_3 在 $[x_0, x_n]$ 上有连续的一、二阶导数。

为了完全确定该多项式, 在每个小区间上需要写出 4 个条件, 因此总共产生 $4n$ 个方程, 它们可以按下面的思路给出: 在节点 x_i 的插值要求写出 $n+1$ 个条件; 在内部节点 $x_1, \dots, x_{n-1}$ 上为了保证多项式的连续性, 进一步产生 $n-1$ 个方程; 在内部节点上为了保证一、二阶导数的连续性, 还可写出 $2(n-1)$ 个新方程。最后还需两个方程, 举例来说可以按下列方式选择:

$$s_3''(x_0) = 0, \quad s_3''(x_n) = 0 \quad (3.17)$$

通过这种方法得到的函数 s_3 , 叫做自然插值三次样条。通过选择合适的未知量 (参见 [QSS00], 8.6.1 节) 来表示 s_3 , 可得到一个具有三角矩阵形式的 $(n+1) \times (n+1)$ 系统, 并且求解该系统所需的运算次数正比于 n 。

式 (3.17) 并不是描述此方程系统的惟一形式, 还可以选择其他的形式。其中之一是在节点 x_1 和 x_{n-1} 上通过保证 s_3 的二阶导数的连续性来代替式 (3.17)。MATLAB

的 `spline`(参见工具箱 `splines`)函数使用的正是这种方法。此时输入参数是向量 x 和 y , z , 向量 z 中包含的是所选择的节点, 而所求结果正是 s_3 在这些节点上的取值。

例 3.6 考虑表 3.1 中对应于列 $K=0.67$ 中的数据, 计算与这些数据相对应的三次样条值 s_3 。将不同的纬度值选作节点 x_i , $i=0, \dots, 12$ 。如果要计算 $s_3(z_i)$ 的值, 其中 $z_i = -55 + i$, $i=0, \dots, 120$, 可以通过下列指令来实现:

```
>>x=[-55:10:65];
>>y=[-3.25 -3.37 -3.35 -3.2 -3.12 -3.02 -3.02 ...
    -3.07 -3.17 -3.32 -3.3 -3.22 -3.1];
>>z=[-55:1:65];
>>s=spline(x,y,z);
```

正如图 3.11 所示的那样, s_3 的图形看起来比 Lagrange 在相同点的插值更加合理。

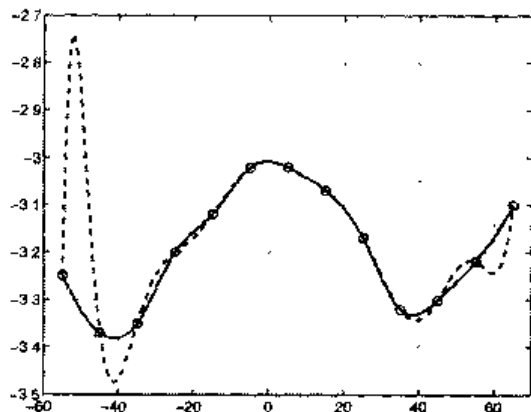


图 3.11 例 3.6 中, 三次样条和 Lagrange 插值间的比较

通过自然三次样条来逼近函数 f (f 是四阶连续可微的) 时产生的误差满足下列不等式:

$$\max_{x \in I} |f^{(r)}(x) - s_3^{(r)}(x)| \leq c_r H^{4-r} \max_{x \in I} |f^{(4)}(x)|, \quad r = 0, 1, 2, 3$$

其中 $I = [x_0, x_n]$, $H = \max_{i=0, \dots, n-1} (x_{i+1} - x_i)$, 常量 C_r 的取值与 r 有关, 与 H 无关。由此可见, 当 H 趋于 0 时, 利用 s_3 逼近 f 及其一阶, 二阶和三阶导数都可以得到很精确的结果。

注 3.2 通常三次样条在相邻的节点上并不保证具有单调性。例如, 利用点 $(x_k = \sin(k\pi/6), y_k = \cos(k\pi/6))$, 其中 $k=0, \dots, 3$, 来逼近单位圆的前四分之一圆周, 能够得到一个振荡的样条(参见图 3.12)。在这样的情况下, 其他的逼近方法可以得到更好的结论。例如, MATLAB 命令 `pchip` 给出了 Hermite 分段三次样条并且保证了插值的局部单调性(参见图 3.12)。Hermite 插值可通过下列指令得到:

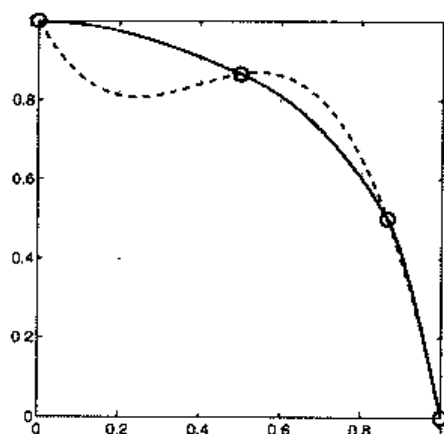


图 3.12 仅使用 4 个节点对单位圆的前四分之一圆周的逼近。虚线是三次样条，而连续的线是分段三次 Hermite 插值

```
>>t=linspace(0,pi/2,4)
>>x=cos(t); y=sin(t);
>>xx=linspace(0,1,40);
>>plot(x,y,'s',xx,[pchip(x,y,xx);spline(x,y,xx)])
```

参见习题 3.5~3.8。

3.4 最小平方方法

由前述内容可以注意到，Lagrange 插值并不能保证当增加多项式的阶数时，能够对给定函数进行更好的逼近。这个问题可通过复合插值来解决(如分段线性插值或者样条)。但它们都不适用于从已知数据中外推其他信息，即在所给插值节点的区间之外的其他节点处生成新值。

例 3.7 在图 3.1 所给出的数据的基础上，预测在未来的几天内股票价格的变化情况。Lagrange 多项式插值在此并不适用，因为它需要计算一个 719 次(剧烈振荡)的多项式，并且只能给出一个完全无用的预测。而分段线性多项式，正如图 3.1 中所示的那样，仅仅利用最近两天的值，就提供了外推的结果，但这样一来就完全忽略了先前的数据。为了得到一个更好的结果，可以避开插值条件，调用下面将要描述的最小平方逼近方法。

假设已知数对 $\{(x_i, f(x_i)), i=0, \dots, n\}$ 。要找出一个阶数至多为 $m \geq 1$ 的多项式 $\tilde{f}$ ，使得对于每个阶数至多为 m 的多项式 p_m ，均满足下列不等式成立：

$$\sum_{i=0}^n [f(x_i) - \tilde{f}(x_i)]^2 \leq \sum_{i=0}^n [f(x_i) - p_m(x_i)]^2 \quad (3.18)$$

如果上式存在， $\tilde{f}$ 就称为 f 的最小平方逼近。对于 m 和 n 的任意值，并不能保证对

于所有的 $i=0, \dots, n$, 都有 $f(x_i) = \tilde{f}(x_i)$ 。

设

$$\tilde{f}(x) = a_0 + a_1x + \dots + a_mx^m \quad (3.19)$$

其中系数 $a_0, \dots, a_m$ 是未知的, 问题(3.18)可重新叙述如下:

$$\Phi(a_0, a_1, \dots, a_m) = \min_{\{b_i, i=0, \dots, m\}} \Phi(b_0, b_1, \dots, b_m)$$

其中

$$\Phi(b_0, b_1, \dots, b_m) = \sum_{i=0}^n \left[f(x_i) - (b_0 + b_1x_i + \dots + b_mx_i^m) \right]^2$$

下面在 $m=1$ 的特殊情况下求解此题。由于

$$\Phi(b_0, b_1) = \sum_{i=0}^n \left[f(x_i)^2 + b_0^2 + b_1^2 x_i^2 + 2b_0b_1x_i - 2b_0f(x_i) - 2b_1x_if(x_i) \right]$$

Φ 的图形是一个凸状抛物面。使得 Φ 取最小值的点 (a_0, a_1) 满足下列条件:

$$\frac{\partial \Phi}{\partial b_0}(a_0, a_1) = 0, \quad \frac{\partial \Phi}{\partial b_1}(a_0, a_1) = 0$$

这里符号 $\partial \Phi / \partial b_j$ 表示在其他变量保持不变的情况下, Φ 对于 b_j 进行求偏导数运算 (即变化率)(参见定义 8.3)。

进行求偏导以后可得:

$$\sum_{i=0}^n [a_0 + a_1x_i - f(x_i)] = 0, \quad \sum_{i=0}^n [a_0x_i + a_1x_i^2 - x_if(x_i)] = 0$$

上式是含有两个变量 a_0 和 a_1 的两个方程系统:

$$\begin{aligned} a_0(n+1) + a_1 \sum_{i=0}^n x_i &= \sum_{i=0}^n f(x_i) \\ a_0 \sum_{i=0}^n x_i + a_1 \sum_{i=0}^n x_i^2 &= \sum_{i=0}^n f(x_i)x_i \end{aligned} \quad (3.20)$$

设 $D = (n+1)\sum_{i=0}^n x_i^2 - (\sum_{i=0}^n x_i)^2$, 则解为:

$$\begin{aligned}
 a_0 &= \frac{1}{D} \left[\sum_{i=0}^n f(x_i) \sum_{j=0}^n x_j^2 - \sum_{j=0}^n x_j \sum_{i=0}^n x_i f(x_i) \right] \\
 a_1 &= \frac{1}{D} \left[(n+1) \sum_{i=0}^n x_i f(x_i) - \sum_{j=0}^n x_j \sum_{i=0}^n f(x_i) \right]
 \end{aligned} \quad (3.21)$$

相应的多项式 $\tilde{f}$ 称为最小平方直线或回归线。

前述方法可以推广到 m 为任意值的情形。相应的对称 $(m+1) \times (m+1)$ 线性系统形式如下：

$$\begin{cases}
 a_0(n+1) + a_1 \sum_{i=0}^n x_i + \cdots + a_m \sum_{i=0}^n x_i^m &= \sum_{i=0}^n f(x_i) \\
 a_0 \sum_{i=0}^n x_i + a_1 \sum_{i=0}^n x_i^2 + \cdots + a_m \sum_{i=0}^n x_i^{m+1} &= \sum_{i=0}^n x_i f(x_i) \\
 \vdots &\vdots \\
 a_0 \sum_{i=0}^n x_i^m + a_1 \sum_{i=0}^n x_i^{m+1} + \cdots + a_m \sum_{i=0}^n x_i^{2m} &= \sum_{i=0}^n x_i^m f(x_i)
 \end{cases}$$

当 $m=n$ 时，最小平方多项式必定与 Lagrange 插值多项式 $\prod_n f$ 一致(参见习题 3.9)。

MATLAB 命令 $c = \text{polyfit}(x, y, m)$ 的默认设置就是计算 m 阶多项式的系数，该多项式利用最小平方法对 $n+1$ 个数对 $(x(i), y(i))$ 进行逼近。正如在 3.1.1 节中所看到的那样，当 m 等于 n 时，它返回的就是插值多项式。

例 3.8 图 3.13 中给出了逼近图 3.1 中的数据所使用的 1、2 和 4 阶最小平方多项式的曲线。4 阶多项式在要考虑的时间区间内生成了相当合理的股票价格变化曲线，从中可以看出不久股市将会上涨。

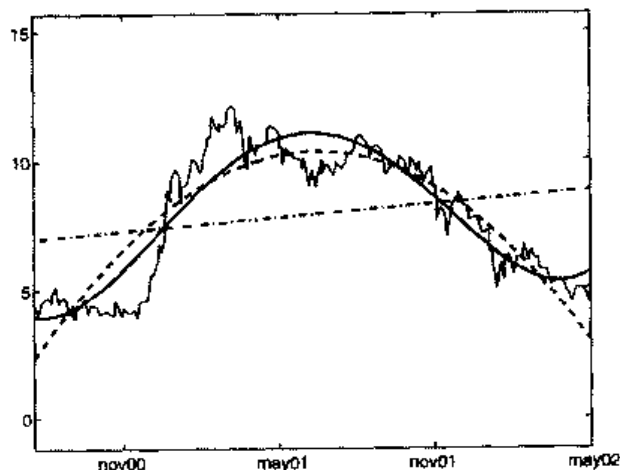


图 3.13 题 3.2 中数据的 1 阶(点划线)、2 阶(虚线)和 4 阶(粗实线)的最小平方逼近曲线。图中细实线表示准确结果

小结

1. 一个函数 f 的复合分段线性插值是一个分段连续线性函数 $\tilde{f}$, 该函数在给定节点 $\{x_i\}$ 上对 f 进行插值。利用这种逼近可以避免当节点的数目增加时产生的 Runge 现象;

2. 三次样条插值就是利用分段三次函数 $\tilde{f}$ 对 f 进行逼近, 其中 $\tilde{f}$ 及其一、二阶导数都是连续的;

3. 在最小平方逼近中, 找到了一个近似的 $\tilde{f}$, 它是一个 m 次(一般, $m < n$)多项式, 该多项式能使均方误差 $\sum_{i=0}^n [f(x_i) - \tilde{f}(x_i)]^2$ 达到最小。

参见习题 3.9~3.13。

3.5 补充说明

对于插值和逼近理论的更一般性的介绍, 读者可以参见 [Dav63], [Mei67] 和 [Gau97]。

多项式插值也可用来逼近多维数据和函数。特别地, 在如下情形: 即区域 Ω 为二维区域, 被分割成多边形(三角形或四边形); 或者是 Ω 为三维区域, 被分割成多面体(四面体或棱柱)时, 非常适合利用复合插值法来求解, 无论该方法是基于分段线性函数还是基于样条函数。

当 Ω 是矩形或是平行六面体时, 情况比较特殊。此时可以很方便地利用命令 $zi = \text{interp2}(x, y, z, xi, yi)$ 或者 $vi = \text{interp3}(x, y, z, v, xi, yi, zi)$ 来实现, 其中 $x, \dots, v$ 是内插值, 而 $xi, \dots, zi$ 是计算插值多项式所选取的节点。

例如, 在正方形 $[0, 1]^2$ 上, 一个有 36 个节点的均匀网格上, 利用三次样条法来逼近函数 $f(x, y) = \sin(2\pi x)\cos(2\pi y)$, 可利用下列指令:

```
>>[x,y]=meshgrid(0:0.2:1,0:0.2:1); z=sin(2*pi*x).*cos(2*pi*y);
```

那么, 在有 441 个节点(x 和 y 方向上都是 21 个)的均匀网格上计算得到的三次样条插值结果如下:

```
>>xi =[0:0.05:1]; yi=[0:0.05:1];
>>[xf,yf]=meshgrid(0:0.05:1,0:0.05:1);
pi3=interp2(x,y,z,xf,yf,'spline');
```

函数 `meshgrid` 将由向量 x 和 y 指定的域转换成矩阵 xf 和 yf , 该矩阵可用于计算含有两个变量的函数并且能够绘制三维 MATLAB 平面图。矩阵 xf 的行等于向量 xi , 矩阵 yf 的列等于向量 yi 。另外, 也可利用函数 `griddata` 来进行拟合(对于三维的数据还可以使用 `griddata3`, 对于 n 维的超曲面也可以利用 `griddatan`)。

当 Ω 是一个任意形状的二维域时, 可以利用图形界面 `pdetool` 将其分割成多个三角形。

对于样条函数的一般介绍可参见[Die93]和[PBP02]。MATLAB 工具箱 `splines` 可以研究样条函数的几种应用。特别指出, 利用 `spdemo` 命令可以查看比较重要的样条函数的性质。有理数样条, 即两个样条函数的比值, 可以通过命令 `rpmak` 和 `rsmak` 得到。还有一个特殊的例子是所谓的 NURBS 样条, 它通常应用于 CAGD(计算机辅助图形设计)之中。

在介绍傅里叶逼近的章节中, 曾提到过基于小波的逼近。这种类型的逼近广泛应用于图像重构和压缩以及信号分析领域中(详细介绍可参见[DL92], [Urb02])。关于小波(及其应用)的详细介绍可参见 MATLAB 工具箱 `wavelet`。

3.6 习题

习题 3.1 证明不等式(3.6)。

习题 3.2 给出下列函数的 Lagrange 插值误差的上界:

$$\begin{aligned} f_1(x) &= \cosh(x), & f_2(x) &= \sinh(x), & x_k &= -1 + 0.5k, & k &= 0, \dots, 4k \\ f_3(x) &= \cos(x) + \sin(x), & & & x_k &= -\pi/21 + \pi k/4, & k &= 0, \dots, 4 \end{aligned}$$

习题 3.3 下列数据是欧洲两个地区公民的平均寿命:

	1975	1980	1985	1990
西欧	72.8	74.2	75.2	76.4
东欧	70.2	70.2	70.3	71.2

使用 3 次插值多项式来估计 1970、1983 和 1988 年的平均寿命。然后外推出 1995 年的值。已知 1970 年, 西欧公民的平均寿命是 71.8 年, 东欧公民的平均寿命是 69.6 年。利用所有已知数据, 能否估计出所预计的 1995 年的平均寿命的精确度?

习题 3.4 已知一种杂志的价格(以欧元为单位)变化如下:

1987 年	1988 年	1990 年	1993 年	1995 年	1996 年	1996 年	2000 年
11 月	12 月	11 月	1 月	1 月	1 月	11 月	11 月
4.5	5.0	6.0	6.5	7.0	7.5	8.0	8.0

通过外推这些数据来估计 2002 年 11 月份的价格。

习题 3.5 重复计算习题 3.3, 现在通过函数 `spline` 利用三次插值样条来进行计算。并对两种方法的结果进行比较。

习题 3.6 下表中给出了在不同温度 T (单位 $^{\circ}\text{C}$) 值下海水密度 ρ (单位 kg/m^3) 的不同取值:

$T/^{\circ}\text{C}$	4	8	12	16	20
$\rho/(\text{kg}\cdot\text{m}^{-3})$	1000.7794	1000.6427	1000.2805	999.7165	998.9700

计算在温度区间 $[4, 20]$ 内的 4 个子区间上的相应的三次样条插值。然后与使用下列数据(对应于其他的 T 值)的样条插值得出的结果进行比较:

$T/^{\circ}\text{C}$	4	10	14	18
$\rho/(\text{kg}\cdot\text{m}^{-3})$	1000.74008	1000.4882	1000.0224	999.3650

习题 3.7 意大利柑橘的产量变化如下:

年份	1965	1970	1980	1985	1990	1991
产量($\times 10^5\text{Kg}$)	17 769	24 001	25 961	34 336	29 036	33 417

使用三次插值样条来估计 1962、1977 和 1992 年的产量。将这些结果与相对应的实际值进行比较, 实际值分别为: 12 380, 27 403 和 32 059kg。再利用 Lagrange 插值多项式重新计算。

习题 3.8 在区间 $[-1, 1]$ 上, 在 21 个平均分布的节点上对函数 $f(x) = \sin(2\pi x)$ 进行估计。计算 Lagrange 插值多项式和三次插值样条。并在给定的区间上将两个函数的曲线与 f 进行比较。使用下列干扰数据 $f(x_i) = (-1)^{i+1} 10^{-4}$ 来重复进行计算, 注意观察对于小扰动, Lagrange 插值多项式比三次样条更敏感。

习题 3.9 验证如果 $m=n$, 在相同节点上, 函数 f 在节点 $x_0, \dots, x_n$ 上的最小平方多项式与插值多项式 $\Pi_n f$ 一致。

习题 3.10 计算逼近表 3.1 中的第二、第三、第四列中的数据的数据的 4 次最小平方多项式。

习题 3.11 利用三次最小平方逼近方法, 重复习题 3.7 的计算。

习题 3.12 对数据集 $\{x_i, i=0, \dots, n\}$, 利用均值 $M = \frac{1}{(N+1)} \sum_{i=0}^n x_i$ 和方差

$v = \frac{1}{(N+1)} \sum_{i=0}^n (x_i - M)^2$ 表示出系统(3.20)的系数。

习题 3.13 验证回归线通过一点, 该点的横坐标是 $\{x_i\}$ 的平均值, 纵坐标是 $\{f(x_i)\}$ 的平均值。

第 4 章 数值微分与数值积分

本章将给出用数值计算思想来求函数的导数和积分的近似值的方法。对一个普通函数求积分时，并非总能得到原函数的显式解。有时即使求出了显式解，也难以计算其数值的大小，正如下例所示：

$$\frac{1}{\pi} \int_0^{\pi} \cos(4x) \cos(3 \sin(x)) \, dx = \left(\frac{3}{2}\right)^4 \sum_{k=0}^{\infty} \frac{(-9/4)^k}{k!(k+4)!}$$

可见在这个例子中，积分计算问题转化成了同样复杂的序列求和问题。有必要指出，在计算一个函数的积分或微分时，有时仅仅知道函数在一些离散点上的取值（例如通过实验测量得到这些数值），这与第 3 章中所讨论的函数逼近问题所面对的情形是完全一致的。

在上述所有情况下，有必要找出这样一种数值计算方法：它可以给出需要求解的量的近似值，并且它的复杂度与函数本身进行积分或微分运算的复杂程度无关。

问题 4.1 (水力学问题) 设一个圆孔半径为 $r=10\text{cm}$ 的圆锥漏斗中液面的高度为 $q(t)$ (单位为 m)，每隔一定的时间测量一次液面高度，得到下列数值：

t	0.0	5	10	15	20
$q(t)$	1.0	0.8811	0.7366	0.5430	0.1698

计算排空速率 $q'(t)$ 的近似值，把它与解析方法得到的值进行比较，解析方法得到的值为 $q'(t) = -0.6\pi r^2 \sqrt{19.6q(t)} / A(t)$ ，其中 $A(t)$ 是在液面高度为 $q(t)$ 时液体水平面的面积。关于这一问题的解答可参见例 4.1。

问题 4.2 (光学问题) 为了设计一个红外射线照射室，需要计算出黑体在波长为 $3\mu\text{m}$ 和 $4\mu\text{m}$ 之间的红外频谱范围内辐射的能量(黑体就是可以在所有的频谱上向外界辐射能量的物体)。这个问题的解可以通过求解下列积分得到：

$$E(T) = 2.39 \times 10^{-11} \int_{3 \times 10^{-4}}^{14 \times 10^{-4}} \frac{dx}{x^5 (e^{1.432/(Tx)} - 1)} \quad (4.1)$$

这就是关于能量 $E(T)$ 的 Planck 方程，其中 x 是波长(cm)， T 是黑体的绝对温度(K)。

关于上述积分的解法可以参见习题 4.17。

问题 4.3 (电磁学问题) 考虑一个半径 r 和传导率 σ 的金属球体, 计算其电流密度 $\mathbf{j}$ (它是 r 和时间 t 的函数) 的分布, 已知初始电流密度分布为 $\rho(r)$ 。这个问题可以利用电流密度, 电场和电荷密度之间的关系来求得。可以看出, 由问题的对称性, $\mathbf{j}(r, t) = j(r, t)\mathbf{r}/|\mathbf{r}|$, 其中 $j = |\mathbf{j}|$, 可以得到:

$$j(r, t) = j(r)e^{-\sigma t/\epsilon_0}, \quad j(r) = \frac{\sigma}{\epsilon_0 r^2} \int_0^r \rho(\xi)\xi^2 d\xi \quad (4.2)$$

其中 $\epsilon_0 = 8.859 \times 10^{-12} \text{ F/m}$ 是真空中介质的常数。

关于 $j(r)$ 的解法可参见习题 4.16。

4.1 函数导数的逼近

考虑在 $[a, b]$ 区间上连续可微的函数 $f: [a, b] \rightarrow \mathbf{R}$ 。现在来求函数 f 在区间 (a, b) 内任意一点 $\bar{x}$ 处的一阶导数。

按照定义式(1.9), 对于足够小的正数 h , 可认为

$$(\delta + f)(\bar{x}) = \frac{f(\bar{x} + h) - f(\bar{x})}{h} \quad (4.3)$$

是 $f'(\bar{x})$ 的一个近似值, 并称之为前向有限差分。估计此时的误差, 只需把 f 展开成 Taylor 级数的形式, 即

$$f(\bar{x} + h) = f(\bar{x}) + hf'(\bar{x}) + \frac{h^2}{2} f''(\xi) \quad (4.4)$$

其中, ξ 是区间 $(\bar{x}, \bar{x} + h)$ 内的一个特定点。因此有

$$(\delta + f)(\bar{x}) = f'(\bar{x}) + \frac{h}{2} f''(\xi) \quad (4.5)$$

和 $(\delta + f)(\bar{x})$ 是 $f'(\bar{x})$ 的关于步长 h 的一阶近似值。类似地, 可以写出 Taylor 展开式:

$$f(\bar{x} - h) = f(\bar{x}) - hf'(\bar{x}) + \frac{h^2}{2} f''(\eta) \quad (4.6)$$

其中, $\eta \in (\bar{x} - h, \bar{x})$, 因此, 后向有限差分

$$(\delta_- f)(\bar{x}) = \frac{f(\bar{x}) - f(\bar{x} - h)}{h} \quad (4.7)$$

也是一阶近似的。注意式(4.3)和(4.7)也可以通过对 f 的插值多项式分别在点 $\{\bar{x}, \bar{x}+h\}$ 和 $\{\bar{x}-h, \bar{x}\}$ 求微分得到。实际上, 从几何观点来看, 上述方法就是利用穿过 $(\bar{x}, f(\bar{x}))$ 和 $(\bar{x}+h, f(\bar{x}+h))$ 两点, 或者 $(\bar{x}-h, f(\bar{x}-h))$ 和 $(\bar{x}, f(\bar{x}))$ 两点之间直线的斜率来作为 $f'(\bar{x})$ 的近似值的(见图 4.1)。

最后介绍中心有限差分公式:

$$(\delta f)(\bar{x}) = \frac{f(\bar{x}+h) - f(\bar{x}-h)}{2h} \quad (4.8)$$

它是 $f'(\bar{x})$ 的关于步长 h 的二阶近似值。实际上, 可以得出:

$$f(\bar{x}) - (\delta f)(\bar{x}) = \frac{h^2}{12} [f'''(\xi) + f'''(\eta)] \quad (4.9)$$

其中, η 和 ξ 分别是区间 $(\bar{x}-h, \bar{x})$ 和 $(\bar{x}, \bar{x}+h)$ 内的特定点(参见习题 4.2)。

而式(4.8)是利用穿过点 $(\bar{x}-h, f(\bar{x}-h))$ 和 $(\bar{x}+h, f(\bar{x}+h))$ 的直线的斜率来作为 $f'(\bar{x})$ 的近似值的。

有限差分近似求法见图 4.1。

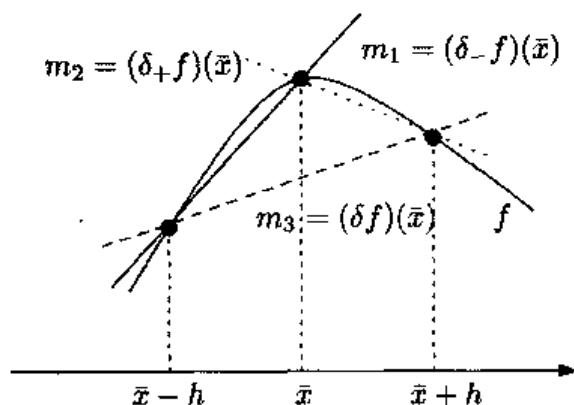


图 4.1 $f'(\bar{x})$ 的有限差分近似求法: 实线表示后向差分, 点线表示前向差分, 虚线表示中心差分, 其中 m_1 , m_2 和 m_3 分别代表三条直线的斜率

例 4.1 对于问题 4.1, 利用式(4.3)、(4.7)和(4.8)来求出 $q'(t)$ 的近似值。得到下列结果:

表 4-1 结果列表

t	0.0	5	10	15	20
$q'(t)$	-0.0220	-0.0259	-0.0326	-0.0473	-0.1504
$\delta+q$	-0.0238	-0.0289	-0.0387	-0.0746	--

(续表)

t	0.0	5	10	15	20
δ_{-q}	--	-0.0238	-0.0289	-0.0387	-0.0746
δ_q	--	-0.0263	-0.0338	-0.0567	--

把有限差分公式计算出来的值与导数的实际值进行比较可以看出, 在 $h=5$ 时, 采用式(4.8)得到的结果要优于式(4.3)和(4.7)得到的结果。

一般来说, 可以假设函数 f 在等间隔的 $n+1$ 个点 $x_i = x_0 + ih$ (其中 $h>0$) 处的值是存在的。此时利用数值方法求导数 $f'(x_i)$ 时, 可以从前面提到的式(4.3)、(4.7)和(4.8)中任选一个, 此时取 $\bar{x} = x_i$ 。

注意中心差分公式(4.8)只能应用于区间的内点, 如 $x_1, \dots, x_{n-1}$, 而不能应用于端点 x_0 和 x_n 。对于端点的情况, 可以采用

$$\begin{aligned} \text{在 } x_0 \text{ 处: } & \frac{1}{2h} [-3f(x_0) + 4f(x_1) - f(x_2)] \\ \text{在 } x_n \text{ 处: } & \frac{1}{2h} [3f(x_n) - 4f(x_{n-1}) + f(x_{n-2})] \end{aligned} \quad (4.10)$$

该方法具有关于步长 h 的二阶精确度。它们是通过在 x_0 点(相应地, 在 x_n 点)处计算在节点 x_0, x_1, x_2 (相应地, x_{n-2}, x_{n-1}, x_n) 生成的二阶插值多项式 f 的一阶导数的值得到的。

参见习题 4.1~4.4。

4.2 数值积分

本节讨论计算下列积分的数值方法:

$$I(f) = \int_a^b f(x) dx$$

式中, f 是区间 $[a, b]$ 上的任意连续函数。下面首先介绍一些简单的公式, 它们实际上也是 Newton-Cotes 公式的几个特例。

4.2.1 中点公式

求解 $I(f)$ 的近似值, 一种简单的方法是把区间 $[a, b]$ 分为区间段 $I_k = [x_{k-1}, x_k]$, $k=1, \dots, M$, 其中 $x_k = a + kH$, $k=0, \dots, M$, 且 $H=(b-a)/M$ 。因为有

$$I(f) = \sum_{k=1}^M \int_{I_k} f(x) dx \quad (4.11)$$

所以可在每个小区间 I_k 上, 用函数 f 在子区间 I_k 上的插值多项式 $\bar{f}$ 代替 f , 然后把 $\bar{f}$ 在小区间 I_k 的积分值作为所求积分的近似值. 选择 $\bar{f}$ 最简单的方法就是在 I_k 的中点处选取:

$$\bar{x}_k = \frac{x_{k-1} + x_k}{2}$$

由此可以得到复合中点求积公式:

$$I_{mp}^c(f) = H \sum_{k=1}^M f(\bar{x}_k) \quad (4.12)$$

符号 mp 代表中点, c 代表复合, 该公式有关于步长 h 的二阶精确度. 如果函数 f 是二阶连续可微的, 则有

$$I(f) - I_{mp}^c(f) = \frac{h^2}{24} H^2 f''(\xi) \quad (4.13)$$

其中, ξ 是区间 (a, b) 内的一个特定点(参见习题 4.6), 式(4.12)也称为复合矩形求积公式, 这是从几何意义上来解释的, 从图 4.2 中可以很明显的看出其含义.

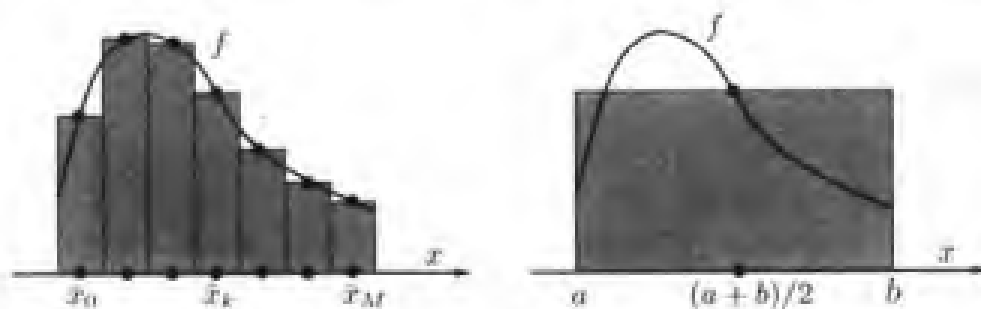


图 4.2 复合中点求积公式(左图); 中点公式(右图)

经典的中点公式(或称矩形公式)是通过在式(4.12)中取 $M=1$ 得到的. 也就是把该公式直接应用于区间 (a, b) 上:

$$I_{mp}(f) = (b-a) f[(a+b)/2]$$

此时的误差为:

$$I(f) - I_{mp}(f) = \frac{(b-a)^3}{24} f''(\xi) \quad (4.14)$$

其中, ξ 是区间 (a, b) 中的一个特定点。式(4.14)既可以看作是式(4.13)的一个特例, 又可以通过运算直接得到。实际上, 设 $\bar{x} = (a+b)/2$, 则有:

$$\begin{aligned} I(f) - I_{mp}(f) &= \int_a^b [f(x) - f(\bar{x})] dx \\ &= \int_a^b f''(\eta(x))(x - \bar{x})^2 dx + \frac{1}{2} \int_a^b f''(\eta(x))(x - \bar{x})^2 dx \end{aligned}$$

其中, $\eta(x)$ 是以 x 和 $\bar{x}$ 为端点构成的区间内的特定点。因为 $\int_a^b (x - \bar{x}) dx = 0$, 并且由中值定理知, $\exists \xi \in (a, b)$ 使得

$$\frac{1}{2} \int_a^b f''(\eta(x))(x - \bar{x})^2 dx = \frac{1}{2} f''(\xi) \int_a^b (x - \bar{x})^2 dx = \frac{(b-a)^3}{24} f''(\xi)$$

成立, 由此式(4.14)得证。

一种求积公式的精确度定义为: 利用该公式求解一个任意 r 阶多项式的积分值时, 在满足求得的近似值等于实际的积分值这一条件下, 能够求解的多项式的最大阶数就称为求积公式的精确度。因此, 中点公式的精确度是 1, 它只能求出阶数是 0 或 1 的多项式(而不能求出阶数为 2 的多项式的)的积分值。

程序 3 是复合中点求积公式的一个实现。输入参数有: 积分区间的端点 a 和 b , 子区间划分的数目 M 和定义 f 函数使用的字符 f 。特别地, 如果函数 f 是一个常函数, 此时为了保证程序能够正确执行, 字符 f 必须要以 ' $c+0.*x$ ' 的形式给出, 其中 c 是常函数 f 的值。

程序 3—midpointc: 复合中点求积公式

```
function lmp=midpointc(a,b,M,f)
%MIDPOINTC Composite midpoint numerical integration.
% IMP=MIDPOINTC(A,B,M,FUN) computes an approximation of the integral
% of the function Fun via the midpoint method (with Mequispaced intervals).
% FUN accepts real scalar input x and returns a real scalar value.
% FUN can also be an inline object.
H=(b-a)/M;
x=linspace(a+H/2,b-H/2,M);
fmp=feval(f,x);
lmp=H*sum(fmp);
```

参见习题 4.5~4.8。

4.2.2 梯形公式

另一个求积公式是在小区间 I_k 内, 利用在节点 x_{k-1} 和 x_k 处生成的插值多项式 f 来代替函数 f (这与 3.2 节中所提到的在整个区间 (a, b) 内用 $\prod_1^n f$ 来代替 f 是等价的), 这时有

$$\begin{aligned} I_k^*(f) &= \frac{H}{2} \sum_{k=1}^M [f(x_k) + f(x_{k-1})] \\ &= \frac{H}{2} [f(a) + f(b)] + H \sum_{k=1}^{M-1} f(x_k) \end{aligned} \quad (4.15)$$

该公式称为复合梯形公式, 它具有关于 H 的二阶精确度。参见图 4.3。

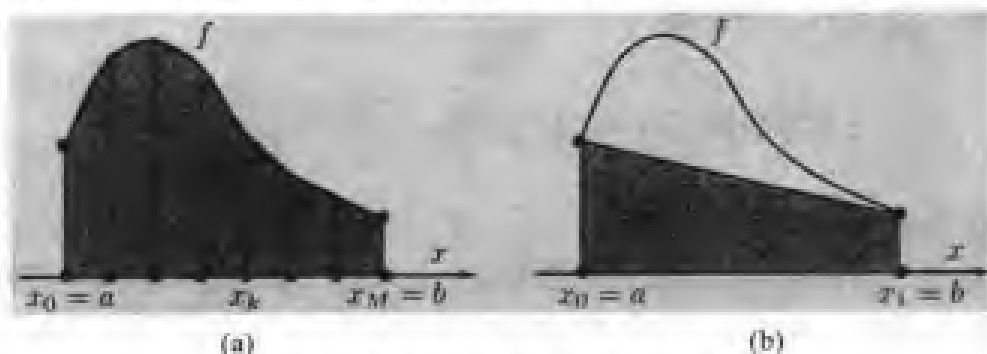


图 4.3 复合梯形公式(a); 梯形公式(b)

实际上, 可以由下式

$$I(f) - I_k^*(f) = -\frac{b-a}{12} H^2 f''(\xi) \quad (4.16)$$

得到求积误差, 式中 $\xi \in (a, b)$, 并假设 $f \in C^2(a, b)$ 。在式(4.15)中取 $M=1$, 得到

$$I_1(f) = \frac{b-a}{2} [f(a) + f(b)] \quad (4.17)$$

它称为梯形公式, 这也是由几何观点定义的。产生的误差可由下式给出:

$$I(f) - I_1(f) = -\frac{(b-a)^2}{12} f''(\xi) \quad (4.18)$$

其中, ξ 是区间 (a, b) 内的一个特定点, 由此可以看出式(4.17)的复杂度等于 1, 与中值公式相同。

把这种方法作简单的改进，就可以得到一个精度更高的公式。此时仍然采用某个多项式来作为 f 的近似值，但是利用 Gauss 节点作为插入点：

$$\begin{aligned}\gamma_{k-1} &= \frac{x_{k-1} + x_k}{2} - \frac{1}{\sqrt{3}} \left(\frac{x_k - x_{k-1}}{2} \right) = x_{k-1} + \left(1 - \frac{1}{\sqrt{3}} \right) \frac{H}{2} \\ \gamma_k &= \frac{x_{k-1} + x_k}{2} + \frac{1}{\sqrt{3}} \left(\frac{x_k - x_{k-1}}{2} \right) = x_{k-1} + \left(1 + \frac{1}{\sqrt{3}} \right) \frac{H}{2}\end{aligned}$$

选择了这些节点以后，便可以得到 Gauss 求积公式：

$$I_{\text{Gauss}}^c(f) = \frac{H}{2} \sum_{k=0}^M [f(\gamma_{k-1}) + f(\gamma_k)] \quad (4.19)$$

这种方法关于 H 具有四阶精确度。这可由下式

$$I(f) - I_{\text{Gauss}}^c(f) = \frac{b-a}{4320} H^4 f^{(4)}(\xi)$$

看出， ξ 是区间 (a, b) 内的一个特定点(参见 [RR85])。如果把式(4.19)限制在一个区间内，则有

$$I_{\text{Gauss}}(f) = \frac{(b-a)}{2} [f(\gamma_1) + f(\gamma_0)] \quad (4.20)$$

其中，

$$\gamma_0 = \frac{a+b}{2} - \frac{1}{\sqrt{3}} \left(\frac{b-a}{2} \right), \quad \gamma_1 = \frac{a+b}{2} + \frac{1}{\sqrt{3}} \left(\frac{b-a}{2} \right)$$

相应的误差为

$$I(f) - I_{\text{Gauss}}(f) = \frac{(b-a)^5}{4320} f^{(4)}(\eta)$$

其中， $\eta \in (a, b)$ 。特别地，还可得出该公式的精确度等于 3，即它只能精确计算阶数小于或等于 3 的多项式的积分值，而在四阶以上时就不再适用了。

程序 4 是梯形公式(4.15)和(4.19)的一个实例。这里采用的选择标准是：当 choice=1 时，选择梯形公式，choice=2 时，选择 Gauss 公式。MATLAB 的 trapz 函数实现式(4.17)，而 cumtrapz 函数实现复合式(4.15)。

程序 4—trapc: 复合梯形公式和复合 Gauss 公式

```

function [Itpc]=trapc(a, b, M,f, choice,varargin)
%TRAPC Composite two-points numerical integration.
%ITPC = TRAPC(A,B,M,FUN,CHOICE) computes an approximation of the
%integral of the function FUN via the trapezoidal method (using Mequispaced
%intervals) if CHOICE=1, via the Gauss composite formula if CHOICE=2.
%FUN accepts real scalar input x and returns a real scalar value.
%FUN can also be an inline object.
H=(b-a)/M;
switch choice
case 1,
    x=linspace(a, b, M + 1);
    fpm=feval(f,x,varargin{:});
    fpm(2:end-1)=2*fpm(2:end-1);
    Itpc=0.5*H*sum(fpm);
case 2,
    z=linspace(a, b, M + 1);
    x=z(1:end-1)+H/2*(1-1/sqrt(3));
    x=[x, x+H/sqrt(3)];
    fpm=feval(f,x,varargin{:});
    Itpc=-0.5*H*sum(fpm);
otherwise
    disp('Unknown formula');
end

```

例 4.2 分别利用复合中点公式、梯形公式和 Gauss 公式计算积分

$I(f) = \int_0^{2\pi} x e^{-x} \cos(2x) dx = -0.122\ 122\ 260\ 461\ 896\ 8$ 的近似值, 比较这三种方法得到的

值。在图 4.4 中按对数坐标给出了误差相对于 H 的曲线, 正如在 1.5 节中所提到的那样, 此时曲线的斜率越大, 则相应的公式收敛阶数就越高。实际结果也正如理论分析的那样, 中点公式和梯形公式有二阶的精确度, 而 Gauss 公式有四阶的精确度。

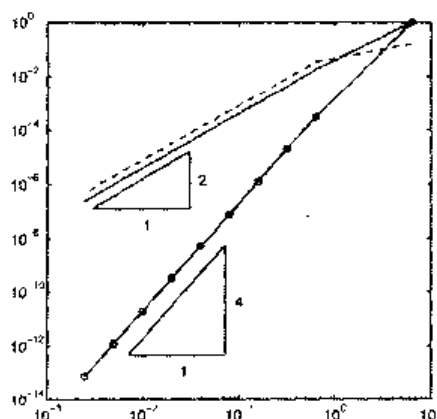


图 4.4 在对数坐标系内三种方法的计算误差相对于 H 的曲线(带圆圈的直线表示 Gauss 复合求积公式, 实线表示复合中点公式, 虚线表示复合梯形公式)

参见习题 4.9~4.11。

4.2.3 Simpson 公式

Simpson 公式计算积分的方法如下：在每个小区间 I_k 内利用在节点 x_{k-1} ， $\bar{x}_k = (x_{k-1} + x_k)/2$ 和 x_k 处的 f 的二阶插值多项式的积分值来代替 f 的积分值，即

$$\begin{aligned} \Pi_2 f(x) = & \frac{2(x - \bar{x}_k)(x - x_k)}{H^2} f(x_{k-1}) + \\ & \frac{4(x_{k-1} - x_k)(x - x_k)}{H^2} f(\bar{x}_k) + \frac{2(x - \bar{x}_k)(x - x_{k-1})}{H^2} f(x_k) \end{aligned}$$

得到的公式便称为复合 Simpson 求积公式，形式为

$$I_s^c(f) = \frac{H}{6} \sum_{k=1}^M [f(x_{k-1}) + 4f(\bar{x}_k) + f(x_k)] \quad (4.21)$$

可以证明它的误差为

$$I(f) - I_s^c(f) = -\frac{b-a}{180} \frac{H^4}{16} f^{(4)}(\xi) \quad (4.22)$$

其中， ξ 是区间 (a, b) 内的特定一点，这里假设 $f \in C^4(a, b)$ 。因此它具有关于 H 的四阶精确度。当式(4.21)只应用于单一区间 (a, b) 时，可以得到所谓的 Simpson 求积公式

$$I_s(f) = \frac{(b-a)}{6} [f(a) + 4f((a+b)/2) + f(b)] \quad (4.23)$$

此时的误差由下式给出：

$$I(f) - I_s(f) = -\frac{1}{16} \frac{(b-a)^5}{180} f^{(4)}(\xi) \quad (4.24)$$

其中， $\xi \in (a, b)$ 。它的精确度等于 3。

程序 5 是复合 Simpson 公式的一个应用实例。

程序 5—simpsonc: 复合 Simpson 求积公式

```
function [Isic]=simpsonc(a,b,M,f,varargin)
%SIMPSONC Composite Simpson numerical integration.
% ISIC = SIMPSONC(A,B,M,FUN) computes an approximation of the
% integral of the function FUN via the Simpson method (using M equispaced
% intervals).
```

```
% FUN accepts real scalar input x and returns a real scalar value.
% FUN can also be an inline object.
H=(b-a)/M;
x= linspace(a, b, M+1);
fpm=feval(f,x,varargin{:});
fpm(2:end-1)= 2*fpm(2:end-1);
Isic= H*sum(fpm)/6;
x= linspace(a + H/2, b- H/2,M );
fpm=feval(f,x,varargin{:});
Isic =Isic+2*H*sum(fpm)/3;
```

注 4.1(插值计算积分) 所有上述介绍的求积公式都是下面这个公式的特例:

$$I_{\text{appr}}(f) = \sum_{j=1}^J \alpha_j f(y_j) \quad (4.25)$$

实数 α_j 是求积权值, 点 y_j 是求积节点。一般来说, 式(4.25)首先要保证能够准确地计算常函数的积分值, 此时要求 $\sum_{j=1}^J \alpha_j = (b-a)$ 。

通过选择合适的权值和节点, 利用式(4.25)可以实现任意大小的精确度。例如, MATLAB 中的函数 `quadl(fun, a, b)`(或 `quad8`), 就是利用高阶 Gauss-Lobatto 求积公式来计算积分值的。函数 `fun` 可作为内嵌函数使用。例如, 如果要计算 $f(x)=1/x$ 在 $[1, 2]$ 内的积分值, 首先必须对函数 `fun` 进行如下定义:

```
fun=inline('1./x','x');
```

这样便可以调用 `quadl(fun, 1, 2)` 函数了。注意在函数 f 的定义中用到了一个分式结构, 它是在给出表达式时引入的(实际上 MATLAB 会求出这个表达式在各个求积节点处的值)。

子区间的数目并没有必要特别指定, 计算机将把求积误差控制在默认的误差容限 10^{-3} 之内, 并自动计算出此时的子区间数目。用户也可以通过 `quadl(fun, a, b, tol)` 函数来给出一个指定的公差。在 4.3 节会介绍一种给出求积误差估计量的方法, 并可以自适应地改变 H 的值。

小结

1. 求积公式是用来求出连续函数积分近似值的公式;
2. 它通常表示为在某些特定的点处(称为节点)的函数值的线性组合的形式, 其中加权系数称为权值;
3. 求积公式的精确度就是利用这一公式所能精确计算的多项式的最高阶数。中点公式和梯形公式的精确度为 1, 复合 Gauss 公式和 Simpson 公式的精确度为 3。
4. 复合求积公式的精确度就是该公式关于子区间长度 H 的阶数。复合中点公

式和复合梯形公式的精确度是 2, 而复合 Gauss 公式和复合 Simpson 公式的精确度是 4。

参见习题 4.12~4.18。

4.3 Simpson 自适应算法

求积公式的积分步长 H 可通过指定的误差容限 $\varepsilon > 0$ 计算出来。比如在利用 Simpson 复合求积公式时, 可以通过式(4.22)来实现, 如果要求

$$\frac{b-a}{180} \frac{H^4}{16} \max_{x \in [a,b]} |f^{(4)}(x)| < \varepsilon \quad (4.26)$$

其中, $f^{(4)}$ 代表 f 的四阶导数。但会出现一个问题, 如果 $f^{(4)}$ 的绝对值只在某个小区间段内很大, 则满足式(4.26)就要求 H 的最大值要非常小, 以致它可能小于计算机所能实现的最短步长。而自适应 Simpson 求积算法在指定误差容限 ε 下计算 $I(f)$ 的近似值时, 在区间 $[a, b]$ 内分配的步长大小并不惟一。此时可以保证精确度与复合 Simpson 公式相同, 但求积节点的数量减少了, 相应地, 关于 f 函数的计算也得到了简化。

为此, 有必要在指定的误差容限下找出一个误差估计量, 同时给出自动修改积分步长 H 的一般步骤。下面首先对修改 H 的一般步骤进行分析, 它与具体采用何种公式来计算积分是无关的。

自适应算法的第一步就是计算出 $I(f) = \int_a^b f(x) dx$ 的估计值 $I_s(f)$ 。设 $H = b - a$, 估计此时的积分误差。如果误差小于指定的误差容限, 则自适应算法终止; 否则步长 H 除以 2, 继续这一过程直到积分 $\int_a^{a+H} f(x) dx$ 满足指定的误差容限为止。这个过程结束之后, 再考虑区间 $(a+H, b)$, 此时初始步长选择为 $b - (a+H)$, 重复前述的算法。

这里会用到下列一些符号:

1. A : 有效积分区间, 即在这些区间内进行积分运算;
2. S : 已经验证过的积分区间, 在这些区间内误差小于指定的误差容限;
3. N : 尚未被验证的积分区间。

在积分运算开始之初, $N = [a, b]$, $A = N$, $S = \emptyset$, 一般的运算过程如图 4.5 所示。

设 $J_s(f)$ 代表 $\int_a^a f(x) dx$ 的估计值, 它的初始值 $J_s(f) = 0$; 在计算过程结束时, 最后得到的 $J_s(f)$ 就是 $I(f)$ 的估计值。另外设 $J_{(\alpha, \beta)}(f)$ 代表函数 f 在小区间 $[\alpha, \beta]$ 内的积分的近似值, 在图 4.5 中这一区间用灰色的线条表示。自适应积分算法的一般步骤可以表示如下:

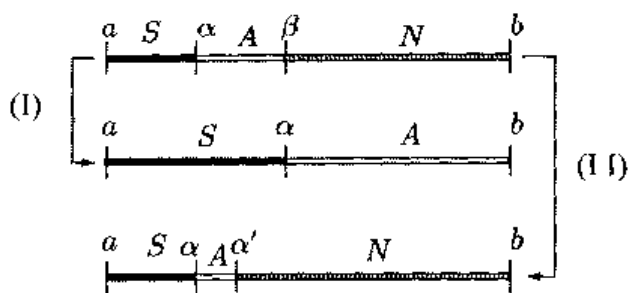


图 4.5 自适应算法中积分间隔的分配和积分坐标的更新

1. 如果误差的估计值在指定的误差容限之内, 则:

- (i) $J_S(f)$ 增加一个增量 $J_{(\alpha, \beta)}(f)$, 即 $J_S(f) \leftarrow J_S(f) + J_{(\alpha, \beta)}(f)$;
- (ii) $S \leftarrow S \cup A$, $A = N$ (与图 4.5 中的路径 I 相对应) 和 $\alpha \leftarrow \beta$, $\beta \leftarrow b$;

2. 如果误差的估计值超出了给定的误差容限, 则:

- (j) A 除以 2, 新的有效区间设为 $A = [\alpha, \alpha']$, 其中 $\alpha' = (\alpha + \beta)/2$ (对应于图 4.5 中路径 II 的情况);
- (jj) $N \leftarrow N \cup [\alpha', \beta]$, $\beta \leftarrow \alpha'$;
- (jjj) 给出新的误差估计值。

为了防止计算过程中出现过小的步长, 需要监测 A 的长度, 以便在步长过度减小以致可能导致积分函数出现异常时, 给出警告信息。

下面的问题是找到一个合适的误差估计量。为此先考虑一般的子区间 $[\alpha, \beta]$ (即计算 $I_s(f)$ 所使用的子区间), 如果在这个区间内误差小于 $\varepsilon(\beta - \alpha)/(b - a)$, 则在区间 $[a, b]$ 内的误差将小于指定的误差 ε 。由 (4.24) 式可以得到:

$$\int_a^b f(x) dx - I_s(f) = -\frac{(\beta - \alpha)^5}{2880} f^{(4)}(\xi) = E_s(f; \alpha, \beta)$$

为了保证误差容限的要求, 需要满足 $E_s(f; \alpha, \beta) < \varepsilon(\beta - \alpha)/(b - a)$ 。但在实际的计算中, 这一算法是难以实现的, 因为点 $\xi \in [\alpha, \beta]$ 的值是未知的。

为了在 $f^{(4)}(\xi)$ 的确切值未知的情况下也能估计出误差的大小, 可以再次利用复合 Simpson 公式来计算 $\int_a^b f(x) dx$, 但此时步长应选为 $(\beta - \alpha)/2$ 。在式 (4.22) 中取 $a = \alpha$, $b = \beta$, 可以得出:

$$\int_a^b f(x) dx - I_s^c(f) = -\frac{(\beta - \alpha)^5}{2880} f^{(4)}(\eta) \quad (4.27)$$

其中, η 是与 ξ 不同的点。合并后两个方程, 可得

$$\Delta I = I_5^*(f) - I_5(f) = -\frac{(\beta - \alpha)^5}{2880} f^{(4)}(\xi) + \frac{(\beta - \alpha)^5}{46080} f^{(4)}(\eta) \quad (4.28)$$

现在假设 $f^{(4)}(x)$ 在子区间 $[\alpha, \beta]$ 内的值是近似不变的。此时, $f^{(4)}(\xi) \approx f^{(4)}(\eta)$ 。可由式(4.28)计算出 $f^{(4)}(\eta)$ 的值, 将其带入式(4.27), 可以得到下列误差估计值:

$$\int_a^\beta f(x) dx - I_5^*(f) \approx \frac{1}{15} \Delta I$$

如果满足 $|\Delta I|/15 < \varepsilon(\beta - \alpha)/[2(b - a)]$, 则认为此时选取的步长 $(\beta - \alpha)/2$ (即在计算 $I_5^*(f)$ 时所采用的步长)是合适的。将这一准则应用于前述自适应算法的求积公式就称为自适应 Simpson 公式。程序 6 是这种算法的一个实现。在输入参数中, f 是定义函数 f 所使用的符号, a 和 b 是积分区间的端点, tol 是指定的误差容限, $h_{\min}$ 是允许采用的积分步长的最小值(以便保证自适应算法在任何情况下都能够正常终止)。

程序 6—simpadpt. 自适应 Simpson 算法

```
function [JSf, nodes]=simpadpt(f,a,b,tol,hmin)
%SIMPADPT Numerically evaluate integral, adaptive simpson quadrature.
% JSF=SIMPADPT(FUN,A,B,TOL,HMIN) tries to approximate the
% integral of function FUN from A to B within an error of TOL using
% recursive adaptive Simpson quadrature. The inline function Y=FUN(V)
% should accept a vector argument V and return a vector result Y, the
% integrand evaluated at each element of X.
%
% [JSF,NODES]=SIMPADPT(...) returns the distribution of nodes.
A=[a,b]; N=[]; S=[]; JSf=0; ba=b-a; nodes=[];
while ~isempty(A),
    [delta,Isc]=caldelta(A,f);
    if abs(delta)<=15*tol*(A(2)-A(1))/ba;
        JSf=JSf+Isc;
        S=union(S,A);
        Nodes=[Nodes,A(1) (A(1)+A(2))*0.5(2)]
        S=[S(1),S(end)]; A=N; N=[];
    elseif A(2)-A(1)<hmin
        JSf=JSf+Isc;
        S=union(S,A);
        S=[S(1),S(end)]; A=N; N=[];
        Warning('Too small step-length');
    else
        Am=(A(1)+A(2))*0.5;
        A=[A(1) Am];
        N=[Am,b];
    end
end
```

```

end
nodes=unique(nodes);
return

function [deltal,ISc]=caldeltai(A,f)
L=A(2)-A(1);

t=[0; 0.25; 0.5; 0.5; 0.75; 1];
x=L*t+A(1);
L=L/6;
w=[1; 4; 1];
fx=feval(f,x);
IS=L*sum(fx([1 3 6])).*w);
ISc=0.5*L *sum(fx.* [w ;w]);
deltal=IS-ISc;
return

```

例 4.3 利用自适应 Simpson 算法来计算积分 $I(f) = \int_{-1}^1 e^{-10(x-1)^2} dx$ 。当选用参数值为 $\text{tol}=10^{-4}$, $\text{hmin}=10^{-3}$ 时, 运行程序 6 得到的结果为 0.280 247 658 847 08, 而真实值是 0.280 249 560 819 90。此时的误差小于指定的误差容限 $\text{tol}=10^{-5}$ 。此时得到这一结果只需要划分成 10 个不相等的子区间, 但如果采用等步长的复合积分公式, 则保证同样的精度需划分出 22 个子区间。

4.4 补充说明

实际上, 中点公式、梯形公式和 Simpson 公式只是 Newton-Cotes 这一求积公式族中的几个特例。关于这个公式的介绍, 可参见 [QSS00] 的第 10 章。类似地, Gauss 公式(4.20)也是 Gauss 求积公式族的特例。在给定的求积节点数目下, 它们可具有最佳的精确度。在 MATLAB 6 中的 `quadl` 函数就是其中一个公式的具体实现。关于 Gauss 公式族的介绍可以参见 [QSS00] 的第 10 章或 [RR85]。有关数值积分的最新成果可以参见 [DR75] 和 [PdDKÜK83]。

数值积分还也用于计算无穷区间上的积分值。例如在计算 $\int_0^\infty f(x) dx$ 时, 一种方法是找到一个点 α , 此时可以用 $\int_0^\alpha f(x) dx$ 来近似代替 $\int_\alpha^\infty f(x) dx$, 然后再利用求积公式计算这个有限区域内的积分值。另一种方法是直接从 Gauss 公式族中选择用于计算无穷边界积分的公式来进行计算(参见 [QSS00] 第 10 章)。

最后, 数值积分方法还可以计算多重积分。特别地, 利用 MATLAB 中的 `dblquad(f', xmin, xmax, ymin, ymax)` 函数可以计算出包含在 MATLAB 的 `f.m` 文件中的函数在矩形区域 $[xmin, xmax] \times [ymin, ymax]$ 内的积分值。注意, 此时函数 f 必须要

有两个输入变量, 它们分别与积分变量 x 和 y 相对应。

4.5 习题

习题 4.1 对于在 x_0 的邻域 I_0 (相应地, x_n 的邻域 I_n) 内的函数 $f \in C^3$, 证明式 (4.10) 的计算误差等于 $-\frac{1}{3}f'''(\xi_0)h^2$ (相应地, 误差为 $-\frac{1}{3}f'''(\xi_n)h^2$), 其中 ξ_0 和 ξ_n 分别是属于 I_0 和 I_n 领域内的点。

习题 4.2 对于在 $\bar{x}$ 的邻域内的函数 $f \in C^3$, 分别利用式 (4.8) 和 (4.9) 来计算其积分, 证明这两种情况下的求积误差是相等的。

习题 4.3 采用数值方法利用下列公式计算 $f'(x_i)$ 的近似值, 并指出该方法关于 h 的精确度是多少。

- $$\frac{-11f(x_i) + 18f(x_{i+1}) - 9f(x_{i+2}) + 2f(x_{i+3})}{6h}$$
- $$\frac{f(x_{i-2}) - 6f(x_{i-1}) + 3f(x_i) + 2f(x_{i+1})}{6h}$$
- $$\frac{f(x_{i-2}) - 8f(x_i) + 8f(x_{i+1}) - f(x_{i+2})}{12h}$$

习题 4.4 下列数值表示某个给定人群的人口数量与时间的变化关系, 已知出生率为常数 ($b=2$), 死亡率为 $d(t) = 0.01n(t)$ 。

$t/\text{月}$	0	0.5	1	1.5	2	2.5	3
n	100	147	178	192	197	199	200

利用这些数据来尽可能准确地估计出人口数量的变化率, 将得到的结果与实际变化率 $n'(t) = 2n(t) - 0.01n^2(t)$ 相比较。

习题 4.5 使用复合中点公式计算下列函数积分的近似值, 并指出在绝对误差小于 10^{-4} 时应该划分的子区间最小数目 M ,

$$f_1(x) = \frac{1}{1+(x-\pi)^2} \quad \text{在区间 } (0, 5), \quad f_2(x) = e^x \cos(x) \quad \text{在区间 } (0, \pi),$$

$$f_3(x) = \frac{1}{\sqrt{x(1-x)}} \quad \text{在区间 } (0, 1)$$

以此来验证程序 3 得到的结果。

习题 4.6 证明式(4.14), 进而证明式(4.13) (式(4.13)是式(4.14)的一般情况)。

习题 4.7 中点公式应用于复合方式时, 收敛阶数减少了一阶, 解释这一现象出现的原因。

习题 4.8 如果 f 是次数少于或等于 1 的多项式, 证明: $I_{mp}(f) = I(f)$, 即此时中点公式的精确度为 1。

习题 4.9 对于习题 4.5 中的函数 f_1 , 在利用复合梯形公式和 Gauss 公式采用数值方法求积分时, 求出误差容限为 10^{-4} 时这两种方法的 M 值。

习题 4.10 利用复合梯形公式对积分 $I(f) = \int_a^b f(x) dx$ 进行求解, 设 I_1 和 I_2 分别是在不同步长 H_1 和 H_2 时得到的近似值。证明, 如果在区间 (a, b) 上 $f^{(2)}$ 的值变化比较缓慢, 则

$$I_R = I_1 + (I_1 - I_2)/(H_2^2/H_1^2 - 1) \quad (4.29)$$

是优于 I_1 和 I_2 的 $I(f)$ 近似值。该方法称为 Richardson 插值法。

习题 4.11 对于所有以 $I_{app}(f) = \alpha f(\bar{x}) + \beta f(\bar{z})$ 表示的公式, 其中 $\bar{x}, \bar{z} \in [a, b]$ 是两个未知节点, α 和 β 是两个未定的权值, 证明利用式(4.20)得到的近似值的精确度最大。

习题 4.12 对于习题 4.5 中列出的前两个函数, 利用复合 Simpson 公式计算它们的积分, 并求出指定的误差容限为 10^{-4} 时应该划分的最小区间数目。

习题 4.13 分别利用 Simpson 公式(4.23)和 Gauss 公式(4.20)计算 $\int_0^2 e^{-x^2/2} dx$ 的值, 比较这两个结果。

习题 4.14 计算积分 $I_k = \int_0^1 x^k e^{x-1} dx, k=1, 2, \dots$, 时可以利用下列递归公式: $I_k = 1 - kI_{k-1}$, 其中 $I_1 = 1/e$ 。利用复合 Simpson 公式计算 I_{20} 的值, 并保证求积误差小于 10^{-3} 。把利用 Simpson 方法得到的结果与利用递归公式得到的结果进行比较。

习题 4.15 利用 Richardson 插值公式(4.29)计算积分 $I(f) = \int_0^2 e^{-x^2/2} dx$ 的近似值, 且 $H_1=1, H_2=0.5$, 再分别利用 Simpson 公式(4.23)和 Gauss 公式(4.20)进行计算, 以此来验证 I_R 要比 I_1 和 I_2 更为精确。

习题 4.16 利用复合 Simpson 公式计算式(4.2)中定义的函数 $j(r)$ 的值。其中 $r = k/10m, k=1, \dots, 9, \rho(\xi) = e^\xi, \sigma = 0.36 W/(m \cdot K)$, 并保证此时积分误差小于 10^{-10} (注: m 表示“米”, W 表示“瓦特”, K 表示绝对温标“开尔文”)。

习题 4.17 利用复合 Simpson 公式和复合 Gauss 公式计算式(4.1)中定义的函数

$E(T)$, 式中 T 等于 213K, 至少保留 10 位有效数字。

习题 4.18 利用复合 Simpson 公式计算积分 $I(f) = \int_1^0 |x^2 - 0.25| dx$, 并保证求积误差小于 10^{-2} 。

第 5 章 线 性 系 统

在应用科学中，解决复杂问题时经常会遇到下列形式的线性系统：

$$Ax=b \quad (5.1)$$

其中， A 是一个 $n \times n$ 维方阵，其中的元素 a_{ij} 可以是实数也可以是复数，而 x 和 b 是 n 维列向量， x 表示未知的解， b 是给定的向量。将(5.1)式展开可以写成如下形式：

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\ \vdots & \quad \quad \quad \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n \end{aligned}$$

下面给出三个例子以加深理解线性系统。

问题 5.1(水力学) 考虑图 5.1 中所示的由 10 个管道组成的水力网络，该网络由一个常压 $p_r=10\text{bar}$ 的蓄水池提供输入。本题中压力值指真实压力与大气压力之间的差值。对第 j 个管道，流速 Q_j (单位 m^3/s)和管道端口处的压力差 Δp_j 满足下列关系：

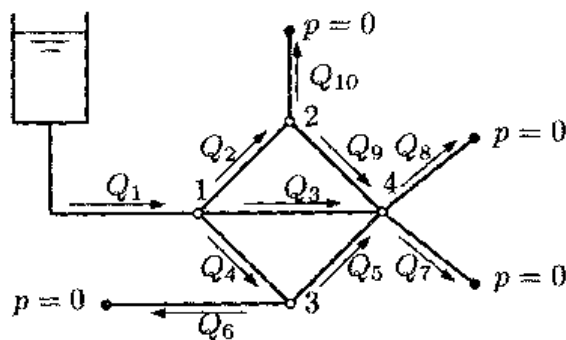


图 5.1 问题 5.1 中的管道网络

$$Q_j = kL\Delta p_j \quad (5.2)$$

其中， k 是水压阻力(单位 $\text{m}^2/(\text{bar s})$)， L 是管道的长度(单位 m)。假设水以大气压力从出口(以黑点表示)流出，为了符合以前的习惯，设大气压为 0bar 。

于是主要问题是求出在每一个内在的节点 1、2、3、4 处的压力值。为此可以对

式(5.2)增加下列补充条件: 对于每一个 $j=1, 2, 3, 4$, 第 j 个节点处流速的代数和一定是零(负值表示出现渗漏)。

设 $p=(p_1, p_2, p_3, p_4)^T$ 表示内部节点的压力向量, 则此时可以得到一个形式为 $Ap=b$ 的 4×4 系统。

表 5.1 总结了不同管道的相关特征。

表 5.1 不同管道的特征

管道	k	L	管道	k	L	管道	k	L
1	0.01	20	2	0.005	10	3	0.005	14
4	0.005	10	5	0.005	10	6	0.002	8
7	0.002	8	8	0.002	8	9	0.005	10
10	0.002	8						

相应地, A 和 b 取下列值(只保留 4 位有效数字):

$$A = \begin{bmatrix} -0.370 & 0.050 & 0.050 & 0.070 \\ 0.050 & -0.116 & 0 & 0.050 \\ 0.050 & 0 & -0.116 & 0.050 \\ 0.070 & 0.050 & 0.050 & -0.202 \end{bmatrix}, \quad b = \begin{bmatrix} -2 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

在例 5.4 中将给出这个系统求解过程。

问题 5.2(光谱测量学) 考虑 n 种不同气体的混合物, 并且气体之间并不发生相互作用。通过质谱仪利用低能电子轰击该混合物: 用检流计来分析产生的混合离子, 检流计可以显示出特定质荷比的峰值。这里只考虑 n 个最相关的峰值。有人推测第 i 个峰值的高度 h_i 是 $\{p_j, j=1, \dots, n\}$ 的线性组合, p_j 是第 j 种气体元素的偏压(即只由混合气体中的一种气体时所产生的气压), 得到

$$\sum_{j=1}^n s_{ij} p_j = h_i, \quad i=1, \dots, n$$

这里, s_{ij} 是灵敏系数。因此要计算偏压的大小, 就要求解线性系统。

问题 5.3(经济问题: 供需平衡分析) 要求出某种商品的供需平衡条件。具体来说, 考虑生产 n 种不同商品的 n 个工厂。每个工厂的产品一部分用于市场消费, 一部分用于满足其他工厂的需要。设 $x_i, i=1, \dots, n$, 为第 i 个工厂生产第 i 种产品的数量, 设 $b_i, i=1, \dots, n$, 为市场消耗第 i 种产品的数量。最后设 c_{ij} 为第 j 个工厂生产第 i 个产品所需消耗的产品; 这个数量是总量 x_j 的一部分(参见图 5.2)。根据 Leontief 模型, 假设相关于各种问题的变换函数是线性的, 则当总产量 x 等于总体需求时, 就达到了平衡, 即 $x=Cx+b$, 其中 $C=(c_{ij}), b=(b_i)$ 。这样, 在这个模型中,

总产量满足下列线性系统:

$$Ax=b, \text{ 其中 } A=I-C \quad (5.3)$$

关于这一问题的解可参见习题 5.17。

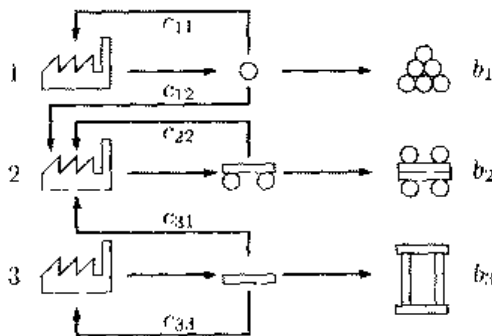


图 5.2 3 个工厂及市场的交互作用方案

当且仅当 A 是非奇异矩阵时, 系统(5.1)的解是存在的。在理论上可以通过 Cramer 法则来求解:

$$x_i = \frac{\det(A_i)}{\det(A)}, \quad i=1, \dots, n$$

其中, A_i 是用 b 来替换 A 的第 i 列而得到的矩阵, $\det(A)$ 表示 A 的行列式。如果这 $n+1$ 个行列式是通过 Laplace 展开式计算的(参见习题 5.1), 将总共需要进行 $2(n+1)!$ 次运算。通常进行的操作包括加减乘除运算。例如, 一个能够执行 10^9 flops(即每秒运算 1G 次)的计算机, 求解维数 $n=15$ 的系统需要 12 小时, 如果 $n=20$ 需要 3240 年, 如果 $n=100$, 则需要 10^{143} 年。如果这 $n+1$ 个行列式用例 1.3 中所介绍的算法计算, 则计算代价会显著减少至 $n^{3.8}$ ops。但由于在实际中经常会遇到较大的 n 值, 这种算法的代价仍然太高。

因此需要寻找另一种途径。注意通常一个系统不能通过少于 n^2 ops 的运算解出。实际上, 如果方程是完全相关的, 可以预料在每次操作中可以对 n^2 个矩阵系数至少进行一次运算。

5.1 LU 因式分解法

设 A 是一个 n 阶方阵。假设存在两个合适的矩阵 L 和 U , 它们分别为下三角阵和上三角阵, 使得

$$A=LU \quad (5.4)$$

则称式(5.4)为 A 的 LU -因式分解。如果 A 是非奇异阵, 则 L 和 U 都是非奇异阵, 它们对角线上的元素是非零的(如 1.3 节中所示)。

此时求解 $Ax=b$ 可以转化为求解两个三角系统:

$$\boxed{Ly=b, \quad Ux=y} \quad (5.5)$$

这两个系统都很容易求解。实际上, L 是下三角阵, $Ly=b$ 系统的第一行有以下形式:

$$l_{11}y_1=b_1$$

上式中, 由于 $l_{11} \neq 0$, 可以求出 y_1 的值。在随后的 $n-1$ 个方程中利用 y_1 的这个值来替换, 可以得到一个新系统, 此时未知量是 $y_2, \dots, y_n$, 接下来可以用相似的方式处理。采用前向处理, 逐个方程进行运算, 可以利用下列前向替换算法计算出所有的未知量:

$$\boxed{\begin{aligned} y_1 &= \frac{1}{l_{11}} b_1 \\ y_i &= \frac{1}{l_{ii}} \left(b_i - \sum_{j=1}^{i-1} l_{ij} y_j \right), \quad i=2, \dots, n \end{aligned}} \quad (5.6)$$

来分析一下式(5.6)所需的运算次数。计算 y_i 需要进行 $i-1$ 次求和运算, $i-1$ 次乘法运算和一次除法运算, 所需要的操作总数是

$$\sum_{i=1}^n 1 + 2 \sum_{i=1}^n (i-1) = 2 \sum_{i=1}^n i - n = n^2$$

系统 $Ux=y$ 可以用相似的处理方法来解。此时第一个要计算的未知量是 x_n , 然后采用后向处理, 可以计算出其余未知量 x_i , 从 $i=n-1$ 开始直到 $i=1$:

$$\boxed{\begin{aligned} x_n &= \frac{1}{u_{nn}} y_n \\ x_i &= \frac{1}{u_{ii}} \left(y_i - \sum_{j=i+1}^n u_{ij} x_j \right), \quad i=n-1, \dots, 1 \end{aligned}} \quad (5.7)$$

该法称为后向替换算法, 它需要的运算次数为 $n^2 \text{ ops}$ 。于是需要找到一种算法, 它能够有效地计算矩阵 A 的因式 L 和 U 。下面两个例子中将给出一个通用的算法。

例 5.1 对于一般化矩阵 $A \in \mathbf{R}^{2 \times 2}$, 按照关系式(5.4)写成下列形式:

$$\begin{bmatrix} l_{11} & 0 \\ l_{21} & l_{22} \end{bmatrix} \begin{bmatrix} u_{11} & u_{22} \\ 0 & u_{22} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$$

L 和 U 的 6 个未知元素一定满足下列(非线性)方程:

$$\begin{aligned} (e_1) \quad l_{11}u_{11} &= a_{11} & (e_2) \quad l_{11}u_{12} &= a_{12} \\ (e_3) \quad l_{21}u_{11} &= a_{21} & (e_4) \quad l_{21}u_{12} + l_{22}u_{22} &= a_{22} \end{aligned} \quad (5.8)$$

系统(5.8)不能完全确定, 因为它的方程数目少于未知量的数目。我们可以通过随机指定 L 的对角元素来确定下来, 例如设 $l_{11}=1$ 和 $l_{22}=1$ 。现在系统(5.8)可以按如下方式求解: 利用 (e_1) 和 (e_2) 来决定 U 的第一行元素 u_{11} 和 u_{12} 。如果 u_{11} 非零, 则从 (e_3) 可推出 l_{21} (即 L 的第一列, 因为此时 l_{11} 已知)。现在由 (e_4) 可得到 U 的第二行的惟一非零元素 u_{22} 。

例 5.2 在 3×3 矩阵的情况下, 重复相同的计算。关于 L 和 U 的 12 个未知系数, 有下列 9 个方程:

$$\begin{aligned} (e_1) \quad l_{11}u_{11} &= a_{11} & (e_2) \quad l_{11}u_{12} &= a_{12} & (e_3) \quad l_{11}u_{13} &= a_{13} \\ (e_4) \quad l_{21}u_{11} &= a_{21} & (e_5) \quad l_{21}u_{12} + l_{22}u_{22} &= a_{22} & (e_6) \quad l_{21}u_{13} + l_{22}u_{23} &= a_{23} \\ (e_7) \quad l_{31}u_{11} &= a_{31} & (e_8) \quad l_{31}u_{12} + l_{32}u_{22} &= a_{32} & (e_9) \quad l_{31}u_{13} + l_{32}u_{23} + l_{33}u_{33} &= a_{33} \end{aligned}$$

为了完成系统的描述, 设 $l_{ii}=1$, 其中 $i=1, 2, 3$ 。于是 U 的第一行系数可以利用 (e_1) 、 (e_2) 、 (e_3) 来得到。接下来, 利用 (e_4) 和 (e_7) , 可以决定 L 的第一列中的 l_{21} 和 l_{31} 的系数。利用 (e_5) 和 (e_6) , 可以计算 U 的第二行中的元素 u_{22} 和 u_{23} 。然后, 利用 (e_8) , 可求出 L 的第二列的 l_{32} 。最后, U 的最后一行(仅包含元素 u_{33})可通过解 (e_9) 来决定。

对于一个任意 n 维的矩阵, 可以如下处理:

1. L 和 U 中的元素满足非线性方程系统

$$\sum_{r=1}^{\min(i,j)} l_{ir}u_{rj} = a_{ij} \quad i, j=1, \dots, n \quad (5.9)$$

2. 系统(5.9)未完全确定; 实际上, 有 n^2 个方程和 n^2+n 个未知数, 这样因式 LU 就不是惟一的;

3. 通过强制设 L 的 n 个对角元素等于 1, 式(5.9)变成一个可确定系统, 可通过

下列 Gauss 算法来解: 设 $A^{(1)}=A$, 即 $a_{ij}^{(1)}=a_{ij}$, 其中 $i, j=1, \dots, n$; 对于 $k=1, \dots, n-1$ 进行下列计算:

$$\begin{array}{l}
 \text{对于 } i=k+1, \dots, n \\
 l_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}} \\
 \text{对于 } i=k+1, \dots, n \\
 a_{ij}^{(k+1)} = a_{ij}^{(k)} - l_{ik} a_{kj}^{(k)}
 \end{array} \quad (5.10)$$

元素 $a_{kk}^{(k)}$ 必定全不为零, 它们称为主元素。对于每个 $k=1, \dots, n-1$, 矩阵 $A_{ij}^{(k+1)} = (a_{ij}^{(k+1)})$ 有 $n-k$ 个行和列。

算法的最后, 对于 $i=1, \dots, n$ 和 $j=i, \dots, n$, 上三角阵 U 的元素通过 $u_{ij} = a_{ij}^{(i)}$ 给出, 而 L 中的元素由算法中得到的系数 l_{ij} 给出。在(5.10)中, 没有计算 L 的对角元素, 因为我们已经知道它们的值等于 1。

这个因式称为 Gauss 因式分解; 计算因式 L 和 U 中元素需要的运算次数约为 $2n^3/3$ ops(参见习题 5.4)。

例 5.3 考虑下列 Vandermonde 矩阵

$$A = (a_{ij}), \quad a_{ij} = x_i^{(n-j)}, \quad i, j=1, \dots, n \quad (5.11)$$

其中, x_i 是 n 个不同的横坐标。它可利用 MATLAB 的 vander 函数建立。在图 5.3 中, 描述了计算 A 的 Gauss 因式分解需要进行的浮点运算次数。在横坐标上给出了几个 n 值, 与其对应的运算次数用圆圈表示。它们可以利用下列命令来实现:

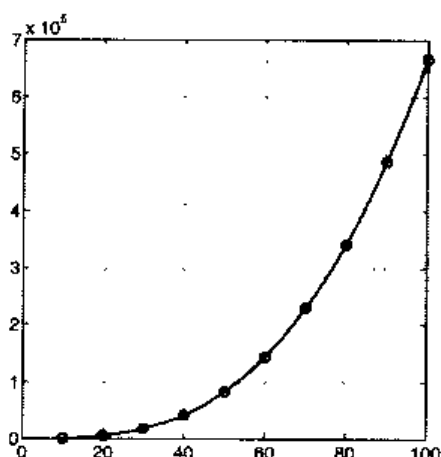


图 5.3 求解 Vandermonde 矩阵的 Gauss 因式分解 LU 时所需的运算次数(ops 的数目)与 n 之间的关系。该函数是一个二次多项式值, 它是利用最小平方的方法对 $n=10, 20, \dots, 100$ 的取值(用圆圈表示)进行逼近而得到的

```
>>ops=[]; for n=[10:10:100]
    A=vander(linspace(1,2,n));
```

```
flops(0), A=lu_gauss(A);
ops=[ops,flops];
end
```

图中所描述的曲线是一个关于 n 的三阶多项式, 它表示的是对上述数据的最小平方逼近, 它们可由以下命令得到:

```
>> c3=polyfit([10:10:100],ops,3)
    0.6667    -0.0000    0.3333    -0.0000
```

n^3 项的系数准确值为 $2/3$ 。

并没有必要把矩阵 $\{A^{(k)}\}$ 全部存储; 实际上, 可以将 $A^{(k+1)}$ 的 $(n-k) \times (n-k)$ 个元素与对应的原始矩阵 A 的最后 $(n-k) \times (n-k)$ 个元素交迭。而且由于在第 k 步, 第 k 列的子对角元素对求解 U 没有任何作用, 因此可以利用 L 的第 k 列元素来替换它们, 如程序 7 所示。于是当算法进行到第 k 步时, 在原来矩阵 A 中存储的元素为:

$$\begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & \cdots & \cdots & a_{1n}^{(1)} \\ l_{21} & a_{22}^{(2)} & & & & a_{2n}^{(2)} \\ \vdots & \ddots & \ddots & & & \vdots \\ l_{k1} & \cdots & l_{k,k-1} & \boxed{\begin{matrix} a_{kk}^{(k)} & \cdots & a_{kn}^{(k)} \\ \vdots & & \vdots \\ a_{kk}^{(k)} & \cdots & a_{kn}^{(k)} \end{matrix}} & & \\ \vdots & & \vdots & & & \vdots \\ l_{n1} & \cdots & l_{n,k-1} & & & \end{bmatrix}$$

这里方框中的子矩阵是 $A^{(k)}$ 。Gauss 因式分解是许多 MATLAB 命令的基础:

$[L, U]=lu(A)$ 它的用法将在 5.2 节中讨论;

inv 对一个矩阵进行求逆运算

$\backslash$ 通过此符号, 可以方便地求解由矩阵 A 和向量 b 所构成的线性系统, 用法为 $A \backslash b$ 。

注 5.1 (行列式的计算) 通过 LU 因式分解的方式可以计算 A 的行列式, 其计算代价为 $O(n^3)$ 次操作, 注意(参见 1.3 节):

$$\det(A) = \det(L) \det(U) = \prod_{k=1}^n u_{kk}$$

实际上, 该程序就是基于 MATLAB 中的 `det` 函数进行运算的。

程序 7 是算法(5.10)的一个实现。因式 L (严格地)存储在 A 的下三角部分, U 存储在 A 的上三角部分(为节省存储空间)。执行程序后, 两个因式可以利用简单的指令 $L=\text{eye}(n)+\text{tril}(A, -1)$ 和 $U=\text{triu}(A)$ 恢复出来, 其中 n 为 A 的大小。

程序 7-lu_gauss: Gauss 因式分解

```

function A=lu_gauss(A)
%LU_GAUSS LU factorization without pivoting.
% A=LU_GAUSS(A) stores an upper triangular matrix in the upper triangular
% part of A and a lower triangular matrix in the strictly lower part of A
% (the diagonal elements of L are 1).
[n,m]=size(A);
if n~=m; error('A is not a square matrix'); else
for k = 1: n-1
for i = k+1:n
A(i,k)= A(i,k)/A(k,k);
if A(k,k) == 0, error('Null diagonal element'); end
for j = k+1: n
A(i,j) = A(i,j) - A(i,k)*A(k,j);
end
end
end
end
end

```

例 5.4 求解问题 5.1 中的系统, 首先进行 LU 因式分解, 然后利用后向和前向替代算法求解。需要计算矩阵 A 及其右侧向量 b , 执行下列指令:

```

>>A=lu_gauss(A);
>>y(1)=b(1);for i=2:4; y=[y;b(i)-A(i,1:i-1)*y(1:i-1)];end
>>x(4)=y(4)/A(4,4);
>>for i=3:-1:1;x(i)=(y(i)-A(i,i+1:4)*x(i+1:4))/A(i,i); end

```

结果是 $p=(8.1172, 5.9893, 5.9893, 5.7779)^T$ 。

例 5.5 利用下列已知条件求解 $Ax=b$

$$A = \begin{bmatrix} 1 & 1-\varepsilon & 3 \\ 2 & 2 & 2 \\ 3 & 6 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5-\varepsilon \\ 6 \\ 13 \end{bmatrix}, \quad \varepsilon \in \mathbf{R} \quad (5.12)$$

其解是 $x=(1, 1, 1)^T$ (与 ε 值无关)。

设 $\varepsilon=1$ 。 A 的 Gauss 因式分解可通过程序 7 得到, 结果如下:

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 3 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 2 & -4 \\ 0 & 0 & 7 \end{bmatrix}$$

如果设 $\varepsilon=0$, 尽管 A 是非奇异的, 但由于算法(5.10)中会出现除以 0 的运算, 所以此时无法进行 Gauss 因式分解。

于是由上例可见, 对于非奇异阵 A , 其 Gauss 因式分解 $A=LU$ 并不是必然存在

的, 关于这一点可以证明存在下列结论:

命题 5.1 对于给定矩阵 $A \in \mathbf{R}^{n \times n}$, 当且仅当 A 的 $i=1, \dots, n-1$ 阶主子矩阵 A_i (即通过仅取 A 的前 i 行和前 i 列得到的矩阵) 是非奇异阵时, 其 Gauss 因式分解是存在的。如果 A 是非奇异阵, 则因式分解形式是惟一的。

再看例 5.5, 注意当 $\varepsilon=0$ 时, 矩阵 A 的二阶主子阵 A_2 是奇异的。

根据命题 5.1 的假定能否成立, 我们可以将矩阵分成不同的类别。特别应提及的有:

1. 对称正定矩阵。如果满足下列条件, 则矩阵 $A \in \mathbf{R}^{n \times n}$ 是正定的

$$\forall x \in \mathbf{R}^n, x \neq 0, x^T A x > 0$$

2. 对角支配矩阵。如果满足下列条件, 称矩阵是行对角支配的:

$$|a_{ii}| \geq \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}|, \quad i=1, \dots, n$$

如果满足下列条件, 则称矩阵是列对角支配的:

$$|a_{ii}| \geq \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ji}|, \quad i=1, \dots, n$$

当把前述不等式中的 $\geq$ 换成 $>$, 会出现特殊的情况。此时矩阵 A 称为严格对角支配矩阵(分别对行或对列)。

如果 A 是对称正定矩阵, 则此时可以构造出一个特别形式的因式分解:

$$A = H H^T \quad (5.13)$$

其中, H 是下三角矩阵, 其对角元素为正。这就是所谓的 Cholesky 因式分解, 它需要经过大约 $n^3/3$ 次运算得到(只是 Gauss LU 因式分解所需要的运算次数的一半)。更进一步, 注意由于存在对称性, 对于矩阵 A 只存储其下三角部分, 于是 H 可存储在相同的区域。

H 的元素可通过下列算法得到: 对于 $i=1, \dots, n$,

$$h_{ij} = \frac{1}{h_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} h_{ik} h_{jk} \right), \quad j=1, \dots, i-1$$

$$h_{ii} = \sqrt{a_{ii} - \sum_{k=1}^{i-1} h_{ik}^2}$$

Cholesky 因式分解在 MATLAB 中可以通过函数 $H=\text{chol}(A)$ 得到。

参见习题 5.1~5.5。

5.2 主元素技术

下面介绍一种特殊的方法，这种方法即使在命题 5.1 的假定条件不成立的情况下，也能够对任意的非奇异矩阵进行 LU 因式分解。

回到例 5.5 所描述的情况，取 $\varepsilon=0$ 。在执行完程序的第一步($k=1$)后，设 $A^{(1)}=A$ ，则 A 的新元素是

$$\begin{bmatrix} 1 & 1 & 3 \\ 2 & 0 & -4 \\ 3 & 3 & -5 \end{bmatrix} \quad (5.14)$$

由于主元素 a_{22} 等于零，此程序不能继续下去了。但是如果预先交换第二行和第三行，可得到矩阵

$$\begin{bmatrix} 1 & 1 & 3 \\ 3 & 3 & -5 \\ 2 & 0 & -4 \end{bmatrix}$$

这样就可以执行因式分解了，而避免了被 0 除的情况。

我们可以将原来矩阵 A 的行进行一种适当的排列，使之在命题 5.1 的假定条件不满足的情况下，仍然可以进行整个因式分解。但是我们并不知道应该改变哪些行的排列。但是我们知道，当运算进行到第 k 步，有一个零对角元素 $a_{kk}^{(k)}$ 产生的时候，便可以决定那一行需要改变了。

回到(5.14)中的矩阵 $A^{(2)}$ ：由于 $a_{22}^{(2)}$ 为零，将 $A^{(2)}$ 的第二行和第三行互换，并检查一下产生的新 $a_{22}^{(2)}$ 是否仍然是零。通过执行因式分解程序的第二步，会发现矩阵 A 的相同两行经过 priori 变换后得到的是相同的矩阵。

因此可以在必要时执行行变换，而不必对 A 作任何的 priori 变换。由于行变换需要改变主元素，这种方法就称为行主元素法。用这种方法得到的因式分解返回一个行变换的原始矩阵。具体说来，得到的结果为：

$$PA=LU$$

P 是合适的变换矩阵。初始时，设它等于单位阵，如果在运算中将 A 的 r 和 s 行进行了互换，则在 P 矩阵上也要进行相应的行互换。相应地，需要求解下列三角矩阵系统：

$$Ly = Pb, \quad Ux = y \quad (5.15)$$

从(5.10)的第二个方程可以看出, 不仅仅零主元素 $a_{kk}^{(k)}$ 运算复杂, 那些小数值元素的运算也很复杂。实际上, 当 $a_{kk}^{(k)}$ 接近于零时, 可能产生的舍入误差对系数 $a_{kj}^{(k)}$ 的影响也会显著增大。

例 5.6 考虑非奇异矩阵

$$A = \begin{bmatrix} 1 & 1+0.5 \times 10^{-15} & 3 \\ 2 & 2 & 20 \\ 3 & 6 & 4 \end{bmatrix}$$

在利用程序 7 进行因式分解的过程中, 没有出现零主元素。然而, 因式 L 和 U 的结果并不准确, 这可以通过计算残留矩阵 $A - LU$ (如果所有计算都是精确的, 则该矩阵应为零矩阵) 来得到:

$$A - LU = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

于是应该在所有主元素 $a_{kk}^{(k)}$ 中查找模值最大的那个元素, 其中 $i=k, \dots, n$, 从而在因式分解的每一步运算中都执行主元素技术。在每一步都利用行主元素法之后, 算法(5.10)变为下列形式, 其中 $k=1, \dots, n$

$$\begin{array}{l} \text{对于 } i = k+1, \dots, n \\ \text{查找 } \bar{r} \text{ 使得 } |a_{rk}^{(k)}| = \max_{i=k+1, \dots, n} |a_{ik}^{(k)}| \\ \text{交换 } k \text{ 行和 } \bar{r} \text{ 行,} \\ l_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}} \\ \text{对于 } j = k+1, \dots, n \\ a_{ij}^{(k-1)} = a_{ij}^{(k)} - l_{ik} a_{kj}^{(k)} \end{array} \quad (5.16)$$

前边提到过的 MATLAB 程序 lu 就是利用主元素技术来计算 Gauss 因式分解的。其完整形式是 $[L, U, P] = \text{lu}(A)$, P 是变换矩阵。简化写法为 $[L, U] = \text{lu}(A)$, 矩阵 L 等于 P^*M , 其中 M 是下三角矩阵, P 是行主元素技术生成的变换矩阵。当计算中产生一个零(或非常小的)主元素时, 程序 lu 会自动调用行主元素技术。

参见习题 5.6~5.8。

5.3 LU 因式分解的精确度

在例 5.6 中已经注意到, 由于舍入误差的影响, LU 的乘积并不能精确重现出矩阵 A 。虽然主元素技术可以减少误差, 但有时还是不能得到满意的结论。

例 5.7 考虑线性系统 $A_n x_n = b_n$, 其中 $A_n \in \mathbf{R}^{n \times n}$ 是所谓的 Hilbert 矩阵, 其元素是

$$a_{ij} = 1/(i+j-1), \quad i, j=1, \dots, n$$

选择 b_n 使得精确解是 $x_n = (1, 1, \dots, 1)^T$ 。矩阵 A_n 显然是对称的, 并且可以证明它也是正定矩阵。

对于不同的 n 值, 通过 MATLAB 的 `lu` 函数利用行主元素法求得 A_n 的 Gauss 因式分解。然后求解相应的线性系统(5.15), 用 $\hat{x}_n$ 表示计算结果。在图 5.4 中, (在对数坐标系中)描述了相对误差:

$$E_n = \|x_n - \hat{x}_n\| / \|x_n\| \quad (5.17)$$

在 1.3.1 节已介绍过用 $\|\cdot\|$ 表示 Euclidean 范数。如果 $n \geq 13$, 有 $E_n \geq 10$ (即此时求解的相对误差竟高于 1000%!), 但对于任意 n 值, $R_n = L_n U_n - P_n A_n$ 是零矩阵(视机器精度)。

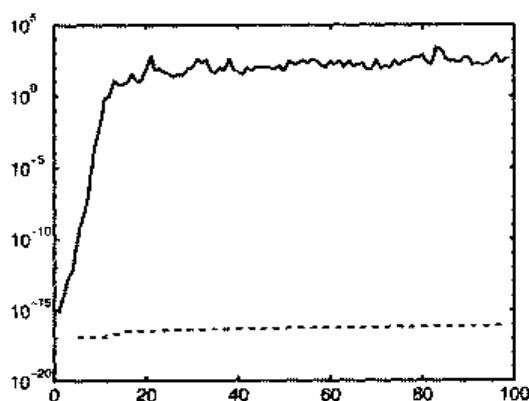


图 5.4 对于例 5.7 的 Hilbert 系统, 在对数坐标系中 E_n (实线)相对于 n 的变化曲线和 $\max_{i,j=1, \dots, n} |r_{ij}|$ (虚线)相对于 n 的变化曲线。 (r_{ij}) 是矩阵 R 的元素。

基于前面的注解注释可以得到如下思路: 当利用数值方法求解一个线性系统时, 实际上是在求解如下扰动系统的精确解 $\hat{x}$

$$(A + \delta A) \hat{x} = b + \delta b \quad (5.18)$$

这里, δA 和 δb 分别是矩阵和向量, 它们与所选用的具体的数值方法有关。首先考虑 $\delta A=0$ 和 $\delta b \neq 0$ 的情形, 此时运算要比大多数情况简单。而且为了简化问题, 仍假设 A 是正定对称矩阵。

比较(5.1)和(5.18)可以看出 $x - \hat{x} = -A^{-1}\delta b$, 于是

$$\|x - \hat{x}\| = \|A^{-1}\delta b\| \quad (5.19)$$

为了给(5.9)式的等号右侧找到一个上界, 需要做如下处理。由于 A 是对称正定阵, 它的特征向量集 $\{v_i\}_{i=1}^n$ 是 $\mathbf{R}^n$ 空间的一组正交基(参见[QSS00], 第五章)。这表示

$$\begin{aligned} Av_i &= \lambda_i v_i & i=1, \dots, n \\ v_i^T v_j &= \delta_{ij} & i, j=1, \dots, n \end{aligned}$$

其中, λ_i 是矩阵 A 与 v_i 相应的特征值, δ_{ij} 是 Kronecker 符号。因此, 一个一般向量 $w \in \mathbf{R}^n$ 可以写成下列形式:

$$w = \sum_{i=1}^n w_i v_i$$

其中 $w_i \in \mathbf{R}$ 是一组合适的系数(且是惟一的)。我们有

$$\begin{aligned} \|Aw\|^2 &= (Aw)^T (Aw) \\ &= \left[w_1 (Av_1)^T + \dots + w_n (Av_n)^T \right] [w_1 Av_1 + \dots + w_n Av_n] \\ &= (\lambda_1 w_1 v_1^T + \dots + \lambda_n w_n v_n^T) (\lambda_1 w_1 v_1 + \dots + \lambda_n w_n v_n) \\ &= \sum_{i=1}^n \lambda_i^2 w_i^2 \end{aligned}$$

用 $\lambda_{\max}$ 表示 A 的最大特征值。由于 $\|w\|^2 = \sum_{i=1}^n w_i^2$, 可得出结论

$$\|Aw\| \leq \lambda_{\max} \|w\| \quad \forall w \in \mathbf{R}^n \quad (5.20)$$

通过类似的方法, 可得

$$\|A^{-1}w\| \leq \frac{1}{\lambda_{\min}} \|w\|$$

这里重新强调一下, A^{-1} 的特征值是 A 的特征值的倒数。该不等式与(5.19)式联立

可得:

$$\frac{\|x - \hat{x}\|}{\|x\|} \leq \frac{1}{\lambda_{\min}} \frac{\|\delta b\|}{\|x\|} \quad (5.21)$$

再利用式(5.20)与 $Ax=b$ 联立, 可以求出:

$$\frac{\|x - \hat{x}\|}{\|x\|} \leq \frac{\lambda_{\max}}{\lambda_{\min}} \frac{\|\delta b\|}{\|b\|} \quad (5.22)$$

可得出结论, 即解的相对误差与通过下列常数(≥ 1)计算得到的数值的相对误差有关:

$$K(A) = \frac{\lambda_{\max}}{\lambda_{\min}} \quad (5.23)$$

上式称为矩阵 A 的谱条件数。 $K(A)$ 可在 MATLAB 中利用命令 `cond(A)`来求得。关于非对称矩阵的谱条件数的其他的定义可以参见[QSS00], 第 3 章。

注 5.2 MATLAB 命令 `cond(A)`可以计算任意类型矩阵 A 的条件数, 此时包括 A 是非对称和非正定的情况。特别地, 命令 `condest(A)`可以用来近似计算 A 的条件数, `rcond(A)`可以用来计算它的倒数, 它们可以节省很多浮点运算。如果矩阵 A 不满足条件(即 $K(A) \gg 1$), 则此时求得的条件数将很不准确。例如考虑不同 n 值的三对角矩阵 $A_n = \text{tridiag}(-1, 2, -1)$, A_n 是对称正定阵, 它的特征值是 $\lambda_j = 2 - 2\cos(j\theta)$, $j=1, \dots, n$, $\theta = \pi/(n+1)$, 因此可以准确地求得 $K(A_n)$ 的值。在图 5.5 中, 描述了误差值 $E_K(n) = |K(A_n) - \text{cond}(A_n)|/K(A_n)$ 。注意当 n 增加时, $E_K(n)$ 也随之增加。

进一步的证明可以得出下列更一般的结论, 当 δA 是任意对称正定矩阵, 且足够小到满足 $\lambda_{\max}(\delta A) < \lambda_{\min}(A)$ 时:

$$\frac{\|x - \hat{x}\|}{\|x\|} \leq \frac{K(A)}{1 - \lambda_{\max}(\delta A)/\lambda_{\min}} \left(\frac{\lambda_{\max}(\delta A)}{\lambda_{\max}} + \frac{\|\delta b\|}{\|b\|} \right)$$

如果 $K(A)$ 很“小”, 即阶数是一个单位时, 则 A 称为充分条件矩阵。此时, 数据上的小误差会导致运算结果也产生相同数量级的误差。对于非充分条件的矩阵没有此结论。

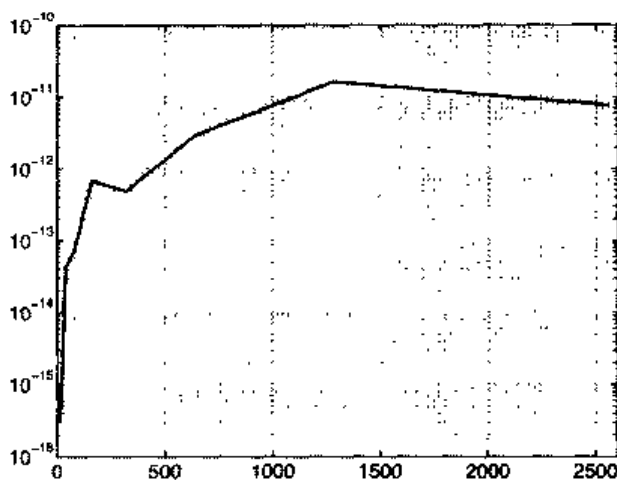


图 5.5 $E_K(n)$ 作为 n 的函数的变化(在对数范围内)

例 5.8 对于例 5.7 中介绍的 Hilbert 矩阵, $K(A_n)$ 是随 n 迅速增长的函数。如果 $n > 13$, 就有 $K(A_n) > 15000$, 由于此时条件数非常大, MATLAB 将会给出警告信息, 提示此矩阵“接近奇异”。实际上, $K(A_n)$ 是以指数规律增长的: $K(A_n) \approx e^{3.5n}$ (参见 [Hig96])。这就间接解释了例 5.7 中得到错误结果的原因。

利用残量 r , 不等式(5.22)可写成如下形式:

$$r = b - Ax \quad (5.24)$$

如果 $\hat{x}$ 是精确解, 残量就应是零向量。这样, 一般来说, r 可以认为是 $x - \hat{x}$ 的误差估计。残量对于误差估计的好坏程度取决于 A 的条件数的大小。实际上, 因为 $\delta b = A(\hat{x} - x) = A\hat{x} - b = -r$, 由式(5.22)可得:

$$\frac{\|x - \bar{x}\|}{\|x\|} \leq K(A) \frac{\|r\|}{\|b\|} \quad (5.25)$$

这样如果 $K(A)$ 很“小”, 可以确定残量很小时实际的误差也很小, 而当 $K(A)$ 很“大”时, 此结论未必成立。

例 5.9 求解例 5.7 中的线性系统时, 残量非常小(它们的范数介于 10^{-16} 和 10^{-11} 之间); 但此时计算结果与精确解之间差异很大。

参见习题 5.9~5.10。

5.4 三对角系统的解法

在很多应用中(例如见第 8 章), 需要求解下列形式的矩阵系统:

$$A = \begin{bmatrix} a_1 & c_1 & & 0 \\ e_2 & a_2 & \ddots & \\ & \ddots & & c_{n-1} \\ 0 & & e_n & a_n \end{bmatrix}$$

由于非零元素只在主对角线和第一上下子对角线上, 这种矩阵称为三对角矩阵。

设矩阵元素是实数, 那么如果 A 的 Gauss LU 因式分解存在, 则 L 和 U 必定是双对角线的(分别在上面和下面), 具体来讲:

$$L = \begin{bmatrix} 1 & & & 0 \\ \beta_2 & 1 & & \\ & \ddots & \ddots & \\ 0 & & \beta_n & 1 \end{bmatrix}, \quad U = \begin{bmatrix} \alpha_1 & c_1 & & 0 \\ & \alpha_2 & \ddots & \\ & & \ddots & c_{n-1} \\ 0 & & & \alpha_n \end{bmatrix}$$

未知系数 α_i 和 β_i 可通过确保等式 $LU=A$ 成立来确定。这就产生了下列计算因式 L 和 U 的递归关系:

$$\alpha_1 = a_1, \quad \beta_i = \frac{e_i}{\alpha_{i-1}}, \quad \alpha_i = a_i - \beta_i c_{i-1}, \quad i = 2, \dots, n \quad (5.26)$$

利用式(5.26), 可以方便地解出两个二对角线系统 $Ly=b$ 和 $Ux=y$, 得到下列公式:

$$(Ly=b) \quad y_1 = b_1, \quad y_i = b_i - \beta_i y_{i-1}, \quad i = 2, \dots, n \quad (5.27)$$

$$(Ux=y) \quad x_n = \frac{y_n}{\alpha_n}, \quad x_i = (y_i - c_i x_{i+1}) / \alpha_i, \quad i = n-1, \dots, 1 \quad (5.28)$$

该方法称为 Thomas 算法, 其计算代价的阶数为 n 。

MATLAB 命令 `spdiags` 可以用于构造一个三对角矩阵。例如, 命令

```
>> b=ones(10,1); a=2*b; c=3*b;
>> T=spdiags([b a c],-1:1,10,10);
```

可以得到三对角矩阵 $T \in \mathbf{R}^{10 \times 10}$, 该矩阵主对角元素等于 2, 下副对角元素等于 1, 上副对角元素等于 3。

注意 T 是以稀疏模式存储的, 这种模式仅对非零元素进行存储。当一个系统通过调用命令 `\` 来求解时, MATLAB 可以识别出矩阵的类型(尤其是可以判断出矩阵是否为稀疏模式)从而选择最合适的求解算法。特别地, 当 A 是一个由稀疏模式产生的三对角矩阵时, 则选择的算法为 Thomas 算法。

例 5.10 解线性系统 $Tx=b$, 这里 T 是由前述方法构造出的矩阵大小 n 可变的

三对角矩阵, 且选择 b 使精确解是 $x=(1, \dots, 1)^T$ 。利用下列指令, 对于增加的 n 值来求解这个系统(通过调用 `\` 命令), 并且计算 ops 除以 n 的结果。当 n 值较大时, 这个比率趋于一个常数, 由此正如所期望的那样, 可以证明出操作数目与 n 成线性关系。

```
>> ratio=[];
>> for k=50:25:200
    b=ones(k,1); a=2*b;c=3*b;
    T=spdiags([b a c],[-1:1,k,k]);
    rhs=T*ones(k,1);
    flops(0);x=T\rhs;ratio=[ratio,flops/k];
end
>> ratio=
Columns 1 through 7
24.5000 24.6133 24.7300 24.7520 24.8067 24.7771 24.8150
```

小结

1. A 的 LU 因式分解即计算一个下三角阵 L 和上三角阵 U 使得 $A=LU$;
2. LU 因式分解(假设存在)并不是惟一的。但通过添加额外的条件如令 L 的对角元素等于 1, 则 LU 分解就可惟一确定下来。这种情况称为 Gauss 因式分解。
3. 当且仅当 A 的 1, 2, $\dots$, $(n-1)$ 阶主子阵是非奇异的(否则将至少有一个主元素是零)时, Gauss 因式分解是存在的; 更进一步, 如果 $\det(A) \neq 0$, 则 Gauss 因式分解是惟一的。
4. 如果计算中出现一个零主元素, 可以通过适当方式交换系统的两行(或列)来获得新的主元素, 这就是主元素方法;
5. Gauss 因式分解的计算大约需要 $2n^3/3$ 次运算, 三对角系统需要的运算次数的阶数为 n 。
6. 对于对称正定矩阵, 可以利用 Cholesky 因式分解 $A=HH^T$, 其中 H 为一个下三角矩阵, 此时计算代价数量级为 $n^3/3$ 。
7. 数据扰动对结果的影响大小取决于系统矩阵的条件数; 具体说来, 对于病态矩阵, 计算结果的精确度会降低。

5.5 迭代方法

求解线性系统(5.1), 如果利用迭代方法, 需要建立向量序列 $x^{(k)} \in \mathbf{R}^n$, 并使该序列收敛到精确解 x , 即

$$\lim_{k \rightarrow \infty} x^{(k)} = x \quad (5.29)$$

其中, 初始向量 $x^{(0)} \in \mathbb{R}^n$ 可以是任意给定的。进一步理解这个过程可以通过下列递归定义式实现:

$$x^{(k+1)} = Bx^{(k)} + g, \quad k \geq 0 \quad (5.30)$$

其中, B 是合适的矩阵(与矩阵 A 有关), g 为合适的向量(取决于 A 和 b), 它们必须满足下列关系:

$$x = Bx + g \quad (5.31)$$

由于 $x = A^{-1}b$, 所以有 $g = (I - B)A^{-1}b$ 。

将 $e^{(k)} = x - x^{(k)}$ 定义为第 k 步的误差。用(5.31)式减去(5.30)式可得

$$e^{(k+1)} = Be^{(k)}$$

正是由于这个原因, B 称为(5.30)的迭代矩阵。如果 B 是对称正定阵, 由(5.20)可得:

$$\|e^{(k+1)}\| = \|Be^{(k)}\| \leq \rho(B)\|e^{(k)}\|, \quad \forall k \geq 0$$

用 $\rho(B)$ 表示 B 的谱半径, 即 B 的特征值的最大模值。通过后向迭代相同的不等式, 可得

$$\|e^{(k+1)}\| \leq [\rho(B)]^k \|e^{(0)}\|, \quad k \geq 0 \quad (5.32)$$

这样只要 $\rho(B) < 1$, 对于任意的 $e^{(0)}$ (和 $x^{(0)}$), 当 $k \rightarrow \infty$ 时 $e^{(k)} \rightarrow 0$ 。实际上, 这条性质对于收敛性也是必要的。

如果通过某种方法可以得到 $\rho(B)$ 的近似值, 由(5.32)就可以推断出为了使初始误差控制在因子 ε 范围内所需的最少迭代数目 $k_{\min}$ 。实际上, $k_{\min}$ 应该是满足 $[\rho(B)]^{k_{\min}} \leq \varepsilon$ 的最小正整数。

综上所述, 可以得到下面的结论:

命题 5.2 求解形式为(5.30)并且迭代矩阵满足(5.31)的系统所采用的迭代方法, 当且仅当 $\rho(B) < 1$ 时, 对于任意的 $x^{(0)}$ 收敛。而且 $\rho(B)$ 越小, 通过给定因式降低初始误差时所需的迭代次数就越少。

5.5.1 如何构造迭代方法

设计一种迭代方法的基础是对矩阵 A 进行分解, $A=P-(P-A)$, P 为适当的非奇异矩阵(称为 A 的预处理阵)。则有

$$x = P^{-1}(P-A)x + P^{-1}b$$

只要设 $B=P^{-1}(P-A)=I-P^{-1}A$ 和 $g=P^{-1}b$, 则上式就具有(5.31)的形式。因此, 可定义下列迭代方法:

$$P(x^{(k+1)} - x^{(k)}) = r^{(k)}, \quad k \geq 0$$

其中, $r^{(k)}=b-Ax^{(k)}$ 表示第 k 次迭代的残留向量。这种迭代方法的一般表述形式如下:

$$P(x^{(k+1)} - x^{(k)}) = \alpha_k r^{(k)}, \quad k \geq 0 \quad (5.33)$$

其中, 参数 $\alpha_k \neq 0$ 的值在每一步迭代都将改变。

方法(5.33)在每一步运算中都需要求解下列线性系统:

$$Pz^{(k)} = r^{(k)} \quad (5.34)$$

然后再通过 $x^{(k+1)} = x^{(k)} + \alpha_k z^{(k)}$ 来定义新的迭代。因此矩阵 P 的选择标准是使求解(5.34)的计算代价比较小(当 P 是对角阵、三角阵或三对角阵时均满足此条件)。现在考虑形式为(5.33)的迭代方法的特殊情形。

Jacobi 方法

如果 A 的对角元素是非零的, 可设 $P=D$, 这里 D 是由 A 的对角元素构成的对角阵。Jacobi 方法对应于对所有 k 取 $\alpha_k=1$ 。则由(5.33)式可得

$$Dx^{(k+1)} = b - (A-D)x^{(k)}, \quad k \geq 0$$

或展开写成

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1, j \neq i}^n a_{ij} x_j^{(k)} \right), \quad i=1, \dots, n \quad (5.35)$$

其中, $k \geq 0$, $x^{(0)} = (x_1^{(0)}, x_2^{(0)}, \dots, x_n^{(0)})^T$ 是初始向量。

于是迭代矩阵为:

$$B = D^{-1}(D - A) = \begin{bmatrix} 0 & -a_{12}/a_{11} & \cdots & -a_{1n}/a_{11} \\ -a_{21}/a_{22} & 0 & & -a_{2n}/a_{22} \\ \vdots & & \ddots & \vdots \\ -a_{n1}/a_{nn} & -a_{n2}/a_{nn} & \cdots & 0 \end{bmatrix} \quad (5.36)$$

下面的结论证明了命题 5.2, 而没有计算 $\rho(B)$:

命题 5.3 如果矩阵是严格行对角支配的, 则 Jacobi 法收敛。

实际上, 可以验证 $\rho(B) < 1$, 其中 B 在(5.36)中已经给定。首先注意由于矩阵是严格对角支配的, 所以 A 的对角元素非零。设 λ 是 B 的广义特征值, x 是相应的特征向量。则

$$\sum_{j=1}^n b_{ij} x_j = \lambda x_i, \quad i=1, \dots, n$$

为考虑方便, 假设 $\max_{k=1, \dots, n} |x_k| = 1$ (这并非是限制条件, 因为特征向量乘以一个常数仍为特征向量), 设 x_i 是模为 1 的特征向量, 则

$$|\lambda| = \left| \sum_{j=1}^n b_{ij} x_j \right| = \left| \sum_{j=1, j \neq i}^n b_{ij} x_j \right| \leq \sum_{j=1, j \neq i}^n \left| \frac{a_{ij}}{a_{ii}} \right|$$

注意 B 仅有零对角元素。因此利用矩阵 A 的假设条件, 可得 $|\lambda| < 1$ 。

程序 8 是 Jacobi 法的一个实现, 在输入参数中设 $P='J'$ 。输入参数是: 系统矩阵 A , 右侧向量 b , 初始向量 x_0 和指定的最大迭代数目 $nmax$ 。只要当前残量和初始残量的 Euclidean 范数的比值小于预先给定的容限 tol , 则迭代程序就将终止(关于该终止标准的讨论可以参见 5.6 节)。

程序 8—itermeth: 一般迭代方法

```
function [x, iter]=itermeth(A,b,x0,nmax,tol,P)
%ITERMETH General iterative method
% X = ITERMETH(A,B,X0,NMAX,TOL,P) attempts to solve the system of
% linear equations A*X=B for X. The N-by-N coefficient matrix A
% must be not singular and the right hand side column vector B must
% have length N. If P='J' the Jacobi method is used, if P='G' the
% Gauss-Seidel method is selected. Otherwise, P is a N-by-N matrix
% that play the role of a preconditioner. TOL specifies the tolerance
% of the method. NMAX specifies the maximum number of iterations.
[n,n]=size(A);
if nargin == 6
    if ischar(P)==1
```

```

if P=='J'
    L = diag(diag(A));
    U = eye(n); beta= 1; alpha= 1;
elseif P == 'G'
    L= tril(A);
    U = eye(n); beta = 1; alpha = 1;
end
else
    [L,U]=lu(P); beta = 0;
end
else
    L = eye(n); U = L; beta = 0;
end
iter = 0;
r=b-A*x0;
r0= norm(r);
err = norm (r); x = x0;
while err > tol & iter < nmax
    iter = iter + 1;
    z = L\r;
    z = U\z;
    if beta == 0
        alpha = z'*r/(z'*A*z);
    end
    x = x + alpha*z;
    r=b-A*x;
    err= norm (r) / r0;
end

```

Gauss-Seidel 方法

当应用 Jacobi 方法时, 新向量中的每个元素, 即 $x_i^{(k+1)}$, 的计算都是与其他分量独立进行的。这表明如果利用已得到的新分量 $x_j^{(k+1)}$, $j=1, \dots, i-1$, 和原来的分量 $x_j^{(k)}$, $j \geq i$, 联合起来计算 $x_i^{(k+1)}$, 就很有可能得到更快速的收敛。此时需要将(5.35)作如下修改: 对于 $k \geq 0$ (仍假设 $a_{ii} \neq 0$, 其中 $i=1, \dots, n$)

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right), \quad i=1, \dots, n \quad (5.37)$$

元素的更新是按顺序进行的, 而且在原始的 Jacobi 方法中, 更新是同时进行的(或称为并行的)。新方法, 称之为 Gauss-Seidel 方法, 选择 $P=D-E$ 且 $\alpha_k=1$, $k \geq 0$, 在(5.33)中, 这里 E 是下三角矩阵, 其非零元素是 $e_{ij}=-a_{ij}$, $i=2, \dots, n$, $j=1, \dots, i-1$ 。则相应的迭代矩阵是

$$B = (D - E)^{-1}(P - A)$$

这种方法的一种推广情形称为松弛法, 在此法中 $P = \frac{1}{\omega}D - E$, 其中 ω 是松弛参数, $\alpha_k = 1$, $k \geq 0$ (参见习题 5.13)。

对 Gauss-Seidel 方法也存在特殊的矩阵 A , 其相关的迭代矩阵满足命题 5.2 的假设(这些条件保证收敛)。下面介绍其中的几种:

1. 矩阵是严格行对角支配的;
2. 矩阵是对称的和正定的。

程序 8 是 Gauss-Seidel 方法的一个实现, 设输入参数 P 等于 'G'。

没有一般性结论表明 Gauss-Seidel 方法比 Jacobi 方法收敛得快。但在一些特殊情况下这个结论是成立的, 此时存在下列命题:

命题 5.4 设 A 是 $n \times n$ 三对角非奇异矩阵, 其对角元素全部非零。则 Jacobi 方法和 Gauss-Seidel 方法或者全部发散或者全部收敛。当全部收敛时, Gauss-Seidel 方法比 Jacobi 方法收敛得快; 另外, Gauss-Seidel 方法迭代矩阵的谱半径等于 Jacobi 迭代矩阵谱半径的平方。

例 5.11 考虑线性系统 $Ax=b$, 其中选择 b 使得解为单位向量 $(1, 1, \dots, 1)^T$, A 是 10×10 的三对角矩阵, 其主对角元素均等于 3, 下副对角元素均为 -2, 上副对角元素均为 -1。由于其迭代矩阵的谱半径都严格地小于 1, 所以 Jacobi 方法和 Gauss-Seidel 方法都收敛。举例来说, 从零初始向量开始, 设 $\text{tol} = 10^{-12}$, 利用 Jacobi 法在 277 次迭代后收敛, 而 Gauss-Seidel 法仅需 143 次迭代。利用下列指令可以得到这一结果:

```
>>n=10;A=3*eye(n)-2*diag(ones(n-1,1),1)-diag(ones(n-1,1),-1);
>>b=A*ones(n,1);
>>[x,iter]=itermeth(A,b,zeros(n,1),400,1.e-12,'J');iter
iter=
    277
>>[x,iter]=itermeth(A,b,zeros(n,1),400,1.e-12,'G');iter
iter=143
```

参见习题 5.11~5.14。

5.6 迭代法的终止条件

理论上, 迭代法需要无限次迭代才能收敛到精确解。实际上, 这既不合理也不必要。其实并不需要求得精确解, 而只要能得到误差低于一定的容限值 ε 的近似解

$x^{(k)}$ 即可。在另一方面,由于误差本身未知(其大小与精确解有关),我们需要一个适当的后验误差估计,从而能够预测中间结果的计算误差。

第一种类型的估计由如下定义的残量表示

$$r^{(k)} = b - Ax^{(k)}$$

$x^{(k)}$ 是第 k 步迭代解的近似值。

更确切地说,可以在第一次迭代到 $k_{\min}$ 步时停止迭代,此时 $k_{\min}$ 应满足下式:

$$\|r^{(k_{\min})}\| \leq \varepsilon \|b\|$$

设式(5.25)中 $\hat{x} = x^{(k_{\min})}$ 且 $r = r^{(k_{\min})}$, 则有:

$$\frac{\|e^{(k_{\min})}\|}{\|x\|} \leq \varepsilon K(A)$$

上式是对相对误差的估计。可推断出关于残量的控制只对那些条件数较小的矩阵有意义。

例 5.12 考虑线性系统(5.1), 其中 $A=A_{20}$ 是例 5.7 中介绍的 20 维的 Hilbert 矩阵, 并且构造 b 使精确解是 $x=(1, 1, \dots, 1)^T$ 。由于 A 是对称正定的, Gauss-Seidel 方法必定收敛。利用程序 8 求解这个系统, 设 x_0 是零初始向量, 误差容限为 10^{-5} 。程序在 472 次迭代后收敛; 但是相对误差较大, 等于 0.26。这是由于矩阵 A 是病态矩阵造成的, 此时有 $K(A) \approx 10^{17}$ 。图 5.6 中分别给出了随着迭代数目的增加, 残量的变化(标准化到初始值)和其误差的变化曲线。

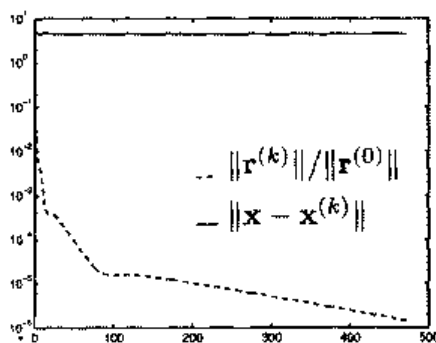


图 5.6 例 5.12 中的系统利用 Gauss-Seidel 方法的残量(虚线)和误差(实线)的变化曲线

另一种方法是基于增量估计的, 即设增量为 $\delta^{(k_{\min})} = x^{(k_{\min}+1)} - x^{(k_{\min})}$ 。具体说来, 当迭代进行到 $k_{\min}$ 步时停止, 此时 $k_{\min}$ 应满足

$$\|\delta^{(k_{\min})}\| \leq \varepsilon \|b\|$$

在特殊情况下, 即 B 是对称和正定的, 有

$$\|e^{(k)}\| = \|e^{(k+1)} - \delta^{(k)}\| \leq \|e^{(k)}\| = \|e^{(k+1)} - \delta^{(k)}\|$$

为使该方法收敛, 则 $\rho(B)$ 应小于 1, 则有

$$\|e^{(k)}\| \leq \frac{1}{1-\rho(B)} \|\delta^{(k)}\| \quad (5.38)$$

从最后一个不等式, 可以看到只有当 $\rho(B)$ 比 1 小的多时, 对增量的控制才有意义, 这是因为此时误差和增量的大小是相当的。

实际上, 即使 B 不是对称和正定的, 也有相同的结论成立(如在 Jacobi 和 Gauss-Seidel 方法中的那样); 但在那种情况下(5.38)不再成立。

例 5.13 考虑一个系统, 其矩阵 $A \in \mathbf{R}^{50 \times 50}$ 是一个三对角对称阵, 其主对角线等于 2.001, 两个副对角线都等于 1。和通常一样, 选择右侧的 b 使其精确解是单位向量 $(1, \dots, 1)^T$ 。由于 A 是严格对角支配的三对角阵, Gauss-Seidel 方法会比 Jacobi 方法的收敛快一倍(由命题 5.4)。利用程序 8 来解此系统, 在该程序中把残量终止标准替换成增量终止标准。设初始向量为零, 误差容限 $\text{tol}=10^{-5}$, 在 1604 次迭代后, 程序返回了一个解, 其误差 0.0029 是非常大的。原因是迭代矩阵的谱半径等于 0.9952, 非常接近于 1。如果设对角元素等于 3, 则只需 17 次迭代就可以实现误差等于 10^{-5} 。实际上, 此时迭代矩阵的谱半径等于 0.428。

5.7 Richardson 方法

考虑方法(5.33), 其加速参数 α_k 是非零的。对于任意 $k \geq 0$, $\alpha_k = \alpha$ (给定的常数) 情况称为静态的。而在迭代期间 α_k 变化的情况, 称为动态的。在此方法中, 非奇异矩阵 P 仍然称为 A 的预处理矩阵。

于是一个重要的问题是选择参数的方法。在这方面, 有下列结论成立(可参见 [QV94, 第 2 章], [Axe94])。

命题 5.5 如果 P 和 A 都是对称正定矩阵, 当且仅当 $0 < \alpha < 2/\lambda_{\max}$ 时, 静态 Richardson 方法对于任意的 $x^{(0)}$ 收敛, 其中 $\lambda_{\max} (> 0)$ 是 $P^{-1}A$ 的最大特征值。并且, 迭代矩阵 $B_\alpha = I - \alpha P^{-1}A$ 的谱半径 $\rho(B_\alpha)$ 在 $\alpha = \alpha_{\text{opt}}$ 时最小, 其中

$$\alpha_{\text{opt}} = \frac{2}{\lambda_{\min} + \lambda_{\max}} \quad (5.39)$$

$\lambda_{\min}$ 是 $P^{-1}A$ 的最小特征值。

在对矩阵 P 和 A 给出同样的假设条件下, 如果按下述方法选择 α_k , 则动态 Richardson 方法收敛,

$$\alpha_k = \frac{\left(z^{(k)}\right)^T r^{(k)}}{\left(z^{(k)}\right)^T A z^{(k)}} \quad \forall k \geq 0$$

其中, $z^{(k)} = P^{-1}r^{(k)}$ 是预处理残量。选择 α_k 的方法(5.33)称为预处理梯度法, 或者, 当预处理矩阵 P 是单位阵时简称为梯度法。

在以上两种情况下, 均有下列收敛估计成立:

$$\|e^{(k)}\|_A \leq \left(\frac{K(P^{-1}A) - 1}{K(P^{-1}A) + 1} \right)^k \|e^{(0)}\|_A, \quad k \geq 0 \quad (5.40)$$

其中, $\|v\|_A = \sqrt{v^T A v}$, $\forall v \in \mathbf{R}^n$, 称为矩阵 A 的能量范数。

因此, 动态的方法要优于静态的方法, 因为它不需要 $P^{-1}A$ 的特征值极值的知识。而且参数 α_k 是由前面迭代中求出的已知量决定的。

可以通过下列算法(其推导过程作为练习)提高预处理梯度方法的效率: 给定 $x^{(0)}$, 对于任意 $k \geq 0$, 计算

$$\begin{aligned} p_z^{(k)} &= r^{(k)} \\ \alpha_k &= \frac{\left(z^{(k)}\right)^T r^{(k)}}{\left(z^{(k)}\right)^T A z^{(k)}} \\ x^{(k+1)} &= x^{(k)} + \alpha_k z^{(k)} \\ r^{(k+1)} &= r^{(k)} - \alpha_k A z^{(k)} \end{aligned} \quad (5.41)$$

只需将 α_k 用常数值 α 来代替, 就可以利用上述算法实现静态 Richardson 方法。

从(5.40)可推断出如果 $P^{-1}A$ 是病态的, 则即使 $\alpha = \alpha_{\text{opt}}$ (在 $\rho(B_{\text{aopt}}) \approx 1$ 的情况下), 其收敛速率仍然很低。只要能够选择一个合适的 P , 就可避免这种情况。这也是 P 称为预处理阵或预处理矩阵的原因。

如果 A 是一个一般矩阵, 将很难找到一个预处理矩阵, 使得在抑制条件数目和求解系统(5.34)的较低的计算代价之间作到最好的折衷。

程序 8 是动态 Richardson 方法的一个实现, 其中输入参数 P 表示预处理矩阵(当没有预先指出时, 程序将设 $P=I$, 此时执行未经预处理的方法)。

例 5.14 本例题只从理论角度出发, 目的是比较 Jacobi, Gauss-Seidel 和梯度

法来解下列(简单)线性问题的收敛特性:

$$2x_1 + x_2 = 1, \quad x_1 + 3x_2 = 0 \quad (5.42)$$

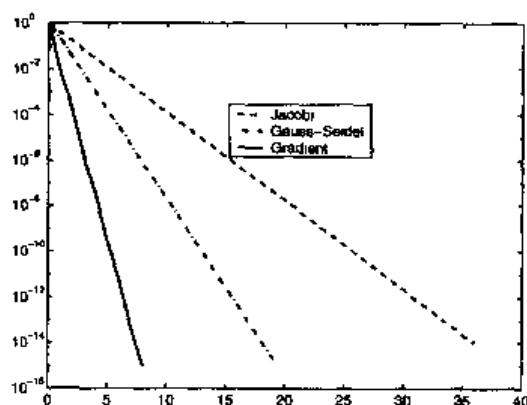


图 5.7 分别利用 Jacobi、Gauss-Seidel 和梯度法求解系统(5.42)的收敛特性曲线

设初始向量 $\mathbf{x}^{(0)} = (1, 1/2)^T$ 。注意系统矩阵是对称正定的, 精确解是 $\mathbf{x} = (3/5, -1/5)^T$ 。图 5.7 给出了以上三种方法的相对残量 $E^{(k)} = \|\mathbf{r}^{(k)}\| / \|\mathbf{r}^{(0)}\|$ 的变化曲线。当迭代进行到第 $k_{\min}$ 步时, 此时 $k_{\min}$ 满足 $E^{(k_{\min})} \leq 10^{-14}$, 迭代终止。从图中可见梯度法收敛速率最快。

例 5.15 考虑系统 $A\mathbf{x} = \mathbf{b}$, 其中 $A \in \mathbf{R}^{100 \times 100}$ 是五对角阵, 其主对角线的所有元素等于 4, 第一和第三的下和上的副对角线的元素都等于 -1。按通常作法, 取 $\mathbf{b}$ 使该系统的精确解为 $\mathbf{x} = (1, \dots, 1)^T$ 。利用预处理 Richardson 方法的程序 8 来解此系统。设 $\text{tol} = 1.e-05$, $n_{\max} = 5000$, $\mathbf{x}_0 = \text{zeros}(100, 1)$ 且预处理阵 P 是三对角阵, 其主对角元素全等于 2, 而上下副对角线的元素全等于 -1。 A 和 P 都是对称正定阵。该方法在 18 次迭代后收敛, 而程序 8(采用 Gauss-Seidel 方法)基于同样的终止标准要经过 2421 次迭代。

例 5.16 (直接迭代法) 回到例 5.7 的 Hilbert 矩阵, 用预处理梯度法解此系统(对于 n 的不同值), 利用 Hilbert 阵的对角元素来构造预处理阵 D 。我们也给出利用更有效的迭代方法, 即共轭梯度法(CG)得出的结果。第 5.8 节会提到 CG 法。设 $\mathbf{x}^{(0)}$ 是零向量, 迭代终止条件为相对残量小于 10^{-6} 。在表 5.2 中, 描述了利用上述迭代方法和利用 GaussLU 因式分解法得到的绝对误差(相对于精确解的)。当 n 变大且利用直接方法时, 误差将会减小。另一方面可以看出如果选用一种更为合适的迭代方法, 如 CG 法, 将有利于减少迭代步数。

表 5.2 利用预处理梯度法(PG), 预处理共轭梯度法(PCG)和 Guauss LU 因式分解法求解 Hilbert 系统的计算误差

N	$K(A)$	LU		=PG	PCG	迭代次数
		误差	误差	迭代次数	误差	
4	1.55e+04	3.90e-13	1.74e-02	995	2.24e-02	3
6	1.50e+07	2.62e-10	8.80e-03	1813	9.50e-03	9
8	1.53e+10	1.34e-07	1.78e-02	1089	2.13e-02	4
10	1.60e+13	7.60e-04	2.52e-03	875	6.98e-03	5
12	1.67e+16	4.97e-01	1.76e-02	1355	1.12e-02	5
14	3.04e+17	6.65e+00	1.46e-02	1379	1.61e-02	5

参见习题 5.15~5.17。

小结

1. 用迭代法求解一个线性系统的步骤为: 从给定初始向量 $\mathbf{x}^{(0)}$ 开始, 建立一个向量序列 $\mathbf{x}^{(k)}$, 要求该序列在 $k \rightarrow \infty$ 时, 收敛到精确解;
2. 对于任意初始向量 $\mathbf{x}^{(0)}$, 当且仅当迭代矩阵的谱半径严格小于 1 时, 迭代法是收敛的;
3. Jacobi 和 Gauss-Seidel 是典型的迭代法。对于迭代收敛的充分条件是系统矩阵可以进行严格行对角化(或在 Gauss-Seidel 的情况下, 是对称正定的);
4. 在 Richardson 方法中, 引入了一个参数和(可能的)一个适当的预处理矩阵, 可以使收敛加速;
5. 对于迭代方法有两种可能的停止标准: 控制残量或控制增量。前者适用于系统矩阵条件充分时的情况, 后者适用于迭代矩阵的谱半径不接近于 1 时的情况。

5.8 补充说明

Guauss LU 因式分解中的一些有效变量可以适用于维数较大的系统。在许多有效的方法中, 特别介绍一下所谓的多前沿方法(multifrontal method), 该方法对未知系统进行适当的重新排序, 从而使因式 L 和 U 尽可能地稀疏。详细介绍可以参见 [GL89], [DD95]和[Iro70]。

关于迭代方法, 存在一族广泛的现代方法, 即所谓的 Krylov 方法, 该方法的效率要高于上面给出的方法。其中一些算法具有有限终止的优良性质, 即在有限的迭代次数中, 利用精确的算法可以给出精确解。值得一提的是共轭梯度法(它可应用于系统矩阵是对称正定的情况)和 GMRES(广义最小残量)。详细介绍可以参见[Axe94]

和[Saa96]。也可以在 MATLAB 工具箱 `sparfun` 中找到它们,名称分别为 `pcg` 和 `gmres`。

正如已经提到的那样,如果系统矩阵是严重病态的,则迭代方法收敛缓慢。关于这方面,已经开发了几种预处理方案(参见[dV89]),其中一些是纯粹的代数方法,即基于不完全的(或者不精确的)给定系统矩阵的因式分解,并且它们已经在 MATLAB 函数 `luinc` 和 `cholinc` 中进行了应用。另外还特别开发了其他方案,该方案描述了目前生成线性系统问题的意义和结构,并据此进行运算。

最后介绍一下多栅算法,该算法利用了与原始系统相似的可变维系统,这种方法可以更有效地减少误差(参见[Hac85],[Wes91]和[Hac94])。

5.9 习题

习题 5.1 对于给定的矩阵 $A \in \mathbf{R}^{n \times n}$,找出通过递归公式(1.8)来计算其行列式的所需的操作数(结果应为 n 的函数)。

习题 5.2 利用 MATLAB 命令 `magic(n)`, $n=3, 4, \dots, 500$, 来构造 n 阶魔方,即那些行、列、对角线上的元素之和都相等的矩阵。通过 1.3 节介绍的命令 `det` 来计算它们的行列式,利用命令 `cputime` 来计算所需 CPU 时间。最后,通过最小平方法来逼近这些数据,并推断出 CPU 时间大致为 n^3 数量级。

习题 5.3 当 ε 取何值时,(5.12)所定义的矩阵不满足命题(5.1)的假定条件。当 ε 取何值时,此矩阵是奇异的?此时能否计算出矩阵的 LU 因式分解?

习题 5.4 验证计算 n 维方阵 A 的 LU 因式分解所需的操作数约为 $2n^3/3$ 。

习题 5.5 说明 A 的 LU 因式分解可以用来计算逆矩阵 A^{-1} (注意 A^{-1} 的第 j 列向量满足线性系统 $Ay_j=e_j$, e_j 是第 j 个元素为 1,其他元素均为零的向量)。

习题 5.6 计算例 5.6 中矩阵的因式 L 和 U ,并验证此时的 LU 因式分解是不准确的。

习题 5.7 说明为什么偏行主元素法不能适用于对称矩阵。

习题 5.8 考虑线性系统 $Ax=b$, 其中

$$A = \begin{bmatrix} 2 & -2 & 0 \\ \varepsilon - 2 & 2 & 0 \\ 0 & -1 & 3 \end{bmatrix}$$

选择向量 b 使得对应解是 $x=(1, 1, 1)^T$, ε 是正实数。计算 A 的 Gauss 因式分解,并注意当 $\varepsilon \rightarrow 0$ 时, $l_{32} \rightarrow \infty$ 。验证此时计算出的解是精确的。

习题 5.9 考虑线性系统 $A_i x_i = b_i$, $i=1, 2, 3$, 其中

$$A_1 = \begin{bmatrix} 15 & 6 & 8 & 11 \\ 6 & 6 & 5 & 3 \\ 8 & 5 & 7 & 6 \\ 11 & 3 & 6 & 9 \end{bmatrix}, \quad A_i = (A_1)^i, \quad i=2, 3$$

选取合适的 b_i 使得解为 $x_i = (1, 1, 1, 1)^T$ 。利用偏行主元素法通过 Gauss 因式分解解此系统，并分析所得结论。

习题 5.10 说明对于对称正定矩阵 A ，有 $K(A^2) = (K(A))^2$ 。

习题 5.11 求解矩阵为下列形式的线性系统的解，分析 Jacobi 和 Gauss-Seidel 方法的收敛性质。

$$A = \begin{bmatrix} \alpha & 0 & 1 \\ 0 & \alpha & 0 \\ 1 & 0 & \alpha \end{bmatrix}, \quad \alpha \in \mathbf{R}$$

习题 5.12 给出关于 β 的充分条件，使对下列矩阵的系统求解时，Jacobi 和 Gauss-Seidel 方法都收敛

$$A = \begin{bmatrix} -10 & 2 \\ \beta & 5 \end{bmatrix} \quad (5.43)$$

习题 5.13 求解线性系统 $Ax=b$ ，其中 $A \in \mathbf{R}^{n \times n}$ ，利用松弛方法，并设 $x^{(0)} = (x_1^{(0)}, \dots, x_n^{(0)})^T$ ， $k=0, 1, \dots$ ，计算

$$r_i^{(k)} = b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)}, \quad x_i^{(k+1)} = (1-\omega) x_i^{(k)} + \omega \frac{r_i^{(k)}}{a_{ii}}$$

对于 $i=1, \dots, n$ ，其中 ω 是一个实参数。找出对应迭代矩阵的显式形式，然后验证 $0 < \omega < 2$ 是该方法收敛的必要条件。注意如果 $\omega=1$ ，则该方法变为 Gauss-Seidel 方法。如果 $1 < \omega < 2$ ，则该方法就是 SOR(successive over-relaxation 过松弛继承)方法。

习题 5.14 考虑线性系统 $Ax=b$ ，其中 $A = \begin{bmatrix} 3 & 2 \\ 2 & 6 \end{bmatrix}$ ，并在不直接求出迭代矩阵谱

半径的情况下，指出 Gauss-Seidel 方法是否收敛。

习题 5.15 求解系统(5.42)的解，计算 Jacobi、Gauss-Seidel 和预处理梯度法的第一步迭代(通过 A 的对角化来给定预处理矩阵)。

习题 5.16 证明(5.39)，然后证明

$$\rho(B_{\text{opt}}) = \frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{K(P^{-1}A) - 1}{K(P^{-1}A) + 1} \quad (5.44)$$

习题 5.17 考虑一个生产 $n=20$ 种不同商品的 20 个 L 厂的问题。参考问题 5.3 中引入的 Leontief 模型，设矩阵 C 有下列整数元素： $c_{ij}=i+j-1$ ，其中 $i, j=1, \dots, n$ ，而 $b_i=i$ ，其中 $i=1, \dots, 20$ 。能否通过梯度法解此系统？提出一种应用梯度法的方法，注意如果 A 是非奇异的，则矩阵 $A^T A$ 是对称正定的。

第 6 章 特征值和特征向量

考虑下列问题, 对于方阵 $A \in \mathbb{R}^{n \times n}$, 如果存在数 λ (实数或复数) 和一个非零向量 x , 满足

$$Ax = \lambda x \quad (6.1)$$

则称 λ 是方阵 A 的一个特征值, 而 x 是相应的特征向量。特征向量并不惟一, 实际上所有的乘积 αx (α 为不为 0 的实数或复数) 均是与 λ 相应的特征向量。如果 x 已知, 则 λ 可以由 Rayleigh 商 $x^* Ax / \|x\|^2$ 求出, 其中向量 x^* 的第 i 个元素等于 $\bar{x}_i$ 。

如果 λ 是下列 n 阶多项式 (称为 A 的特征多项式) 的一个根, 则 λ 是 A 的一个特征值:

$$p_A(\lambda) = \det(A - \lambda I)$$

因此, 一个 n 维方阵恰好有 n 个特征值, 特征值既可以是实数, 也可以是复数, 并且特征值之间可以取值相同。如果 A 的元素为实数, 同时 $p_A(\lambda)$ 的系数也为实数, 则 A 的复特征值将以共轭复数对的形式出现。

问题 6.1 (动力学问题) 考虑如图 6.1 所示的系统, 两个质量为 m 的质点 P_1 和 P_2 由两个弹簧相连, 两质点可以沿直线自由滑动, 设 $x_i(t)$ 代表质点 P_i 在时刻 t 时所处的位置, 其中 $i=1, 2$ 。则由动力学第二定律可得:

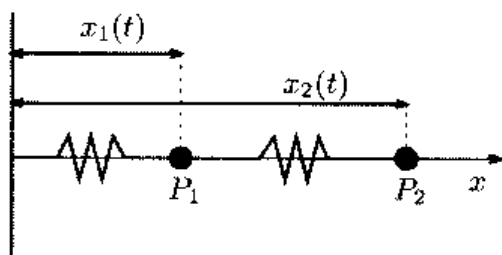


图 6.1 由弹簧相连的两个质量相等的质点所构成的系统

$$m\ddot{x}_1 = k(x_2 - x_1) - Kx_1, \quad m\ddot{x}_2 = K(x_1 - x_2)$$

这是一个受迫振动问题, 相应的解是 $x_i = a_i \sin(\omega t + \phi)$, 其中 $i=1, 2$, $a_i \neq 0$, 因此有:

$$\begin{aligned} -ma_1\omega^2 &= K(a_2 - a_1) - Ka_1 \\ -ma_2\omega^2 &= K(a_1 - a_2) \end{aligned} \quad (6.2)$$

这是一个 2×2 的齐次系统, 它有非零解 a_1, a_2 的充分必要条件是: $\lambda = m\omega^2/K$ 是下列矩阵 A 的一个特征值.

$$A = \begin{bmatrix} 2 & -1 \\ -1 & -1 \end{bmatrix}$$

有了 λ 的定义, 式(6.2)可化为 $Aa = \lambda a$. 由 $p_A(\lambda) = (2-\lambda)(1-\lambda)-1$, 可求出两个特征值分别为 $\lambda_1 \approx 2.618$ 和 $\lambda_2 \approx 0.382$, 它们分别对应于系统的固有振荡频率 $\omega_i = \sqrt{K\lambda_i/m}$.

问题 6.2(人口统计学问题) 为了预测一些特定物种(包括人类或者动物)的进化情况, 人们提出了几种数学模型. 最简单的人口数量模型是在 1920 年由 Lotka 引入并由 Leslie 在 20 年后正式提出的, 它基于不同年龄区间的出生率和死亡率进行计算. 设 $i=0, \dots, n$, $x_i^{(t)}$ 代表 t 时刻年龄处于第 i 个区间的女性个体的数目, 且初始值 $x_i^{(0)}$ 给定. 再设 s_i 代表第 i 个区间的女性个体的成活率, m_i 表示第 i 个年龄区间内的每个女性个体繁衍出来的新的女性个体数目的平均值.

由 Lotka 和 Leslie 提出的模型可用下列方程组来描述:

$$\begin{aligned} x_{i+1}^{(t+1)} &= x_i^{(t)} s_i \quad i=0, \dots, n-1 \\ x_0^{(t+1)} &= \sum_{i=0}^n x_i^{(t)} m_i \end{aligned}$$

前 n 个方程描述了人口的发展, 最后一个方程代表人口的繁殖. 以矩阵形式可以表述如下:

$$x^{(t+1)} = Ax^{(t)}$$

其中, $x^{(t)} = (x_0^{(t)}, \dots, x_n^{(t)})^T$, A 是 Leslie 矩阵:

$$A = \begin{bmatrix} m_0 & m_1 & \dots & \dots & m_n \\ s_0 & 0 & \dots & \dots & 0 \\ 0 & s_1 & \ddots & & \vdots \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & 0 & s_{n-1} & 0 \end{bmatrix}$$

在 6.1 节中会看到人口数量的动态特性是由矩阵 A 的按模最大的特征值 λ_1 来决定的,而在不同年龄区间内,个体的分布(针对整个人口群体而言)是通过计算 $t \rightarrow \infty$ 时 $x^{(t)}$ 的极限而得到的,同时满足 $Ax = \lambda_1 x$ 。这个问题的解答可参见习题 6.2。

问题 6.3(交通有效性问题) 考虑 n 个城市之间的道路连接问题,设 A 是一个矩阵,如果第 i 个城市与第 j 个城市是直接相连的,则矩阵 A 的元素 a_{ij} 为 1,否则为 0。可见与按模最大的特征值相对应的特征向量 x (单位长度)描述的就是每个城市的可达性(即交通的方便程度)。例如考虑意大利北部 Lombardy 地区连接 11 个主要城市的铁路网,如图 6.2 所示。在这个例子中向量 x 的元素模分别是:

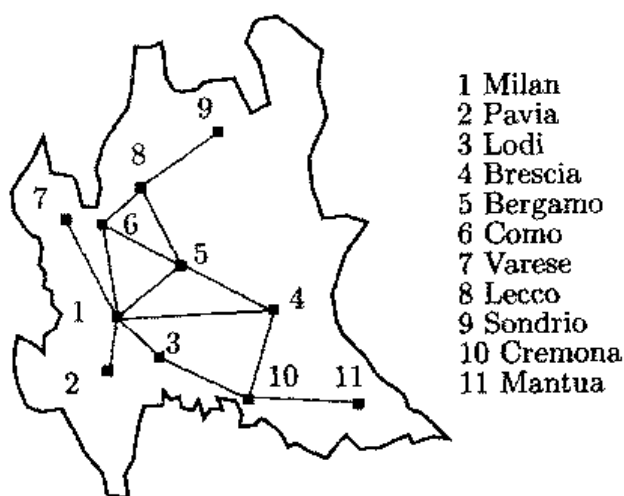


图 6.2 连接 Lombardy 地区主要城市的铁路网示意图

0.5271 (Milan)	0.1590 (Pavia)
0.2165 (Lodi)	0.3579 (Brescia)
0.4690 (Bergamo)	0.3861 (Como)
0.1590 (Varese)	0.2837 (Lecco)
0.0856 (Sondrio)	0.1906 (cremona)
0.0575 (Mantua)	

从中可以看出可达性最好的城市分别是(按降序排列)Milan、Bergamo、Como 和 Brescia,而 Mantua 是可达性最差的一个。值得注意的是,尽管 Pavia, Varese, Mantua 和 Sondrio 进入城市的道路数目相同(均为 1 个),但它们的可达率是不同的。显然,这种分析方法主要考虑的是连接的频繁程度,而不是城市之间实际存在的路径多少。

在特殊情况下,如 A 是对角阵或三角阵,它的特征值就是对角线上的元素。如果 A 是一个普通矩阵,并且维数 n 很大时,要找出 $p_A(\lambda)$ 的零点并不是非常容易,而特别算法则适合于解这类问题,下节中将会介绍其中的一种方法。

6.1 幂法

从问题 6.2 和 6.3 中可以看出, 我们不需知道 A 的整个频谱(即它的所有特征值序列), 而通常只需知道两端的特征值, 也就是模值最大或最小的特征值。

例如, 假设 A 是一个元素均为实数的 n 维方阵, 设它的特征值可以排列成下列形式:

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n| \quad (6.3)$$

由此可将 λ_1 与 A 的其他特征值区别开。再设 x_1 代表与 λ_1 相应的特征向量(单位长度)。如果 A 的特征向量是线性无关的, 则 λ_1 和 x_1 可以利用下列迭代公式来计算, 这种方法通常称为幂法。

任给一个初始向量 $x^{(0)}$, 设 $y^{(0)} = x^{(0)} / \|x^{(0)}\|$, $k \geq 1$ 时计算:

$$x^{(k)} = Ay^{(k-1)}, \quad y^{(k)} = \frac{x^{(k)}}{\|x^{(k)}\|}, \quad \lambda^{(k)} = (y^{(k)})^T Ay^{(k)} \quad (6.4)$$

注意, 利用递归式可得 $y^{(k)} = \beta^{(k)} A^k y^{(0)}$, 其中 $\beta^{(k)} = (\prod_{i=1}^k \|x^{(i)}\|)^{-1}$, $k \geq 1$, 幂法这一名称即来源于式中 A 的幂方项的存在。

正如将要在第 7 章中提到的, 当 $k \rightarrow \infty$ 时, 幂法生成的单位长度的向量 $\{y^{(k)}\}$ 将对准特征向量 x_1 的方向。对于普通矩阵, 误差 $\|y^{(k)} - x_1\|$ 与比值 $|\lambda_2/\lambda_1|$ 成比例, 而当 A 是对称阵时, 则与 $|\lambda_2/\lambda_1|^2$ 成比例。于是可得当 $k \rightarrow \infty$ 时, 有 $\lambda^{(k)} \rightarrow \lambda_1$ 。

程序 9 是幂法的一个实现。如果经过 k 次迭代后满足下列条件, 则迭代程序将终止:

$$|\lambda^{(k)} - \lambda^{(k-1)}| < \varepsilon |\lambda^{(k)}|$$

其中, ε 是期望方差。输入参数有: 矩阵 A , 初始向量 x_0 , 终止误差容限 tol 和最大允许的迭代次数 $n_{\max}$ 。输出参数有: 按模最大的特征值 lambda , 相应的特征向量, 以及实际进行的迭代运算次数。

程序 9 幂法

```
function [lambda,x,iter]=eigpower(A,tol, nmax,x0)
%EIGPOWER Numerically evaluate one eigenvalue of a matrix.
```

```

% LAMBDA = EIGPOWER(A) compute with the power method the
% eigenvalue of A of maximum modulus from an initial guess which by default
% is an all one vector.
% LAMBDA = EIGPOWER(A,TOL,NMAX,X0) uses an absolute error
% tolerance TOL (the default is 1.e-6) and a maximum number of iterations
% NMAX (the default is 100), starting from the initial vector X0.
% [LAMBDA,V,ITER] = EIGPOWER(A,TOL,NMAX,X0) also returns the eigenvector
% V such that A*V=LAMBDA*V and the iteration number at which V was computed.
[n,m] = size(A);
if n ~= m, error('Only for square matrices. ');end
if nargin==1
    tol=1.e-06;
    x0 = ones(n,1);
    nmax= 100;
end
x0= x0/norm(x0);
pro = A*x0;
lambda = x0'* pro;
err = tol*abs(lambda) + 1;
iter=0
while err > tol*abs(lambda) & abs(lambda) ~= 0 & iter <= nmax
    x= pro; x=x/norm(x);
    pro= A*x; lambdanew=x'*pro;
    err= abs(lambdanew-lambda);
    lambda =lambdanew;
iter=iter+1;
end

```

例 6.1 考虑矩阵族

$$A(\alpha) = \begin{bmatrix} \alpha & 2 & 3 & 13 \\ 5 & 11 & 10 & 8 \\ 9 & 7 & 6 & 12 \\ 4 & 14 & 15 & 1 \end{bmatrix}, \quad \alpha \in \mathbf{R}$$

利用幂法求出按模最大的特征值的近似解。取 $\alpha = 30$ ，则矩阵的特征值分别为： $\lambda_1 = 39.396$ ， $\lambda_2 = 17.8208$ ， $\lambda_3 = -9.5022$ ， $\lambda_4 = 0.2854$ (这里只给出了前四位小数)。它经过 22 次迭代运算可给出 λ_1 的估计值，误差容限为 $\varepsilon = 10^{-10}$ ，并且 $\mathbf{x}^{(0)} = \mathbf{1}^T$ 。但如果取 $\alpha = -30$ ，则需要 708 次迭代运算。

后一种情况下迭代次数增加了，这是因为此时得到的特征值分别为 $\lambda_1 = -30.643$ ， $\lambda_2 = 29.7359$ ， $\lambda_3 = -11.6806$ 和 $\lambda_4 = 0.5878$ ，可见这时 $|\lambda_2|/|\lambda_1| = 0.9704$ 是接近于 1 的。

收敛性分析

前面已假设 A 的特征向量 $x_1, \dots, x_n$ 之间是线性无关的, 于是这样一组特征向量可以看作空间 C^n 的一个基。所以向量 $x^{(0)}$ 和 $y^{(0)}$ 可以写成下列形式:

$$x^{(0)} = \sum_{i=1}^n \alpha_i x_i, \quad y^{(0)} = \beta^{(0)} \sum_{i=1}^n \alpha_i x_i, \quad \text{其中 } \beta^{(0)} = 1/\|x^{(0)}\|$$

采用幂法进行的第一次迭代为:

$$x^{(1)} = Ay^{(0)} = \beta^{(0)} A \sum_{i=1}^n \alpha_i x_i = \beta^{(0)} \sum_{i=1}^n \alpha_i \lambda_i x_i$$

类似地,

$$y^{(1)} = \beta^{(1)} \sum_{i=1}^n \alpha_i \lambda_i x_i, \quad \beta^{(1)} = \frac{1}{\|x^{(0)}\| \|x^{(1)}\|}$$

在第 k 步时可以得到:

$$y^{(k)} = \beta^{(k)} \sum_{i=1}^n \alpha_i \lambda_i^k x_i, \quad \beta^{(k)} = \frac{1}{\|x^{(0)}\| \cdots \|x^{(k)}\|}$$

于是

$$y^{(k)} = \lambda_1^k \beta^{(k)} \left(\alpha_1 x_1 + \sum_{i=2}^n \alpha_i \frac{\lambda_i^k}{\lambda_1^k} x_i \right)$$

由于 $|\lambda_i/\lambda_1| < 1 \quad \forall i=2, \dots, n$, 则当 k 趋于 $+\infty$ 时, 向量 $y^{(k)}$ 将与特征向量 x_1 的方向趋于一致, 假设 $\alpha_1 \neq 0$ 。 $\alpha_1 \neq 0$ 这一条件并不是必需的, 在实际中, 由于 x_1 未知, 所以实际上这一条件也很难保证。舍入误差的影响使得沿 x_1 方向有非零元素存在(初始向量 $x^{(0)}$ 例外), 可以说这是一个由舍入误差造成有益影响的为数不多的示例之一。

例 6.2 考虑例 6.1 中的矩阵 $A(\alpha)$, 取 $\alpha=16$ 。与 λ_1 相应的单位长度的特征向量 x_1 为 $(1/2, 1/2, 1/2, 1/2)^T$ 。选择一个特定的初始向量 $(2, -2, 3, -3)^T$ 与 x_1 正交。图 6.3 表示 $y^{(k)}$ 和 x_1 之间夹角的余弦值。可以看到利用幂法进行 30 次迭代运算以后, 余弦函数的值趋于 -1 , 而夹角趋于 π , 而 $\lambda^{(k)}$ 序列趋近于 $\lambda_1=34$ 。得益于舍入误差的影响, 幂法运算生成的向量组 $y^{(k)}$ 中, 所有指向 x_1 方向的元素都相应

地增加。

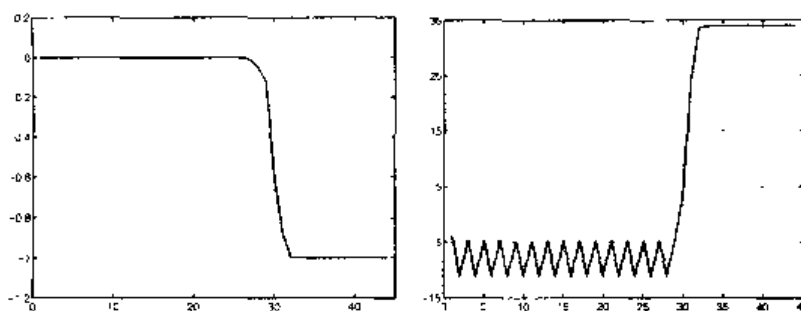


图 6.3 向量 $y^{(k)}$ 和 x_1 之间夹角的余弦(左图); $\lambda^{(k)}$ 的值(右图)其中 $k=1, \dots, 44$

注 6.1 可以证明即使 λ_1 是 $p_A(\lambda)$ 的一个复根, 幂法也是收敛的。反之当存在两个不同的按模最大的特征值时, 幂法就不再收敛了。在这种情况下, 序列 $\lambda^{(k)}$ 不收敛于任何极限, 在两个值之间振荡。

参见习题 6.1~6.3。

6.2 幂法的变形

幂法的第一种变形是在把幂法应用于求 A 的逆矩阵时(假设 A 是非奇异的)提出的。由于 A^{-1} 的特征值与 A 的特征值互逆, 则此时可以求出矩阵 A 的按模最小的特征值, 由此得到了反幂法:

对于任何给定的初始向量 $x^{(0)}$, 设 $y^{(0)} = x^{(0)} / \|x^{(0)}\|$, 且 $k \geq 1$, 计算下列式子:

$$x^{(k)} = A^{-1} y^{(k-1)}, \quad y^{(k)} = \frac{x^{(k)}}{\|x^{(k)}\|}, \quad \mu^{(k)} = (y^{(k)})^T A^{-1} y^{(k)}$$

如果 A 的特征向量之间是线性无关的, 并且按模最小的特征值 λ_n 互不相同, 则

$$\lim_{k \rightarrow \infty} \mu^{(k)} = 1/\lambda_n$$

(即当 $k \rightarrow \infty$ 时, $(\mu^{(k)})^{-1}$ 趋于 λ_n)。

每进行一次迭代运算都需要解一个形式为 $Ax^{(k)} = y^{(k-1)}$ 的线性系统。因此如果把 A 进行 LU 分解可使运算大大简化(或者当 A 是正定对称阵时, 进行 Cholesky 分解), 这时在每次迭代运算时只需处理两个三角阵描述的系统。

例 6.3 当把反幂法应用于例 6.1 中的矩阵 $A(30)$ 时, 经过 7 次迭代运算得到的值是 3.5037。这样 $A(30)$ 的按模最小的特征值近似等于 $1/3.5037 \approx 0.2854$ 。

幂法的另一种变形是由下列问题引出的。设 λ_μ 表示 A 的特征值(未知的)中与给定的数 μ (实数或复数)距离最近的一个特征值。为了求出 λ_μ 的近似值,首先可以近似求出转换矩阵 $A_\mu = A - \mu I$ 的按模最小的特征值 $\lambda_{\min}(A_\mu)$,再设 $\lambda_\mu = \lambda_{\min}(A_\mu) + \mu$ 。于是可以对 A_μ 应用反幂法,得到 $\lambda_{\min}(A_\mu)$ 的近似值。这种方法称为原点移位法,数 μ 称为移位量。

程序 10 给出了移位反幂法的一个实现。当取 $\mu=0$ 时,原点移位法就转化为反幂法。前 4 个输入参数与程序 9 相同, μ 是移位量。输出参数有: A 的特征值 λ_μ , 相关特征向量 x 和实际进行的迭代运算次数。

程序 10 移位反幂法

```
function [lambda,x,iter]=invshift(A,mu,tol,nmax,x0)
%INVSHIFT Numerically evaluate one eigenvalue of a matrix.
% LAMBDA = INVSHIFT(A) compute with the inverse power method the
% eigenvalue of A of minimum modulus.
% LAMBDA = INVSHIFT(A,MU) compute the eigenvalue
% of A closest to the given number (real or complex) MU
% LAMBDA = INVSHIFT(A,MU,TOL,NMAX,X0) uses an absolute error
% tolerance TOL (the default is 1.e-6) and a maximum number of iterations
% NMAX (the default is 100), starting from the initial vector X0
% [LAMBDA,V,ITER] = INVSHIFT(A,MU,TOL,NMAX,X0) also returns the eigenvector
% V such that A*V=LAMBDA*V and the iteration number at which V was computed.
[n,m]=size(A);
if n ~= m, error('Only for square matrices'); end
if nargin ==1
    x0 = rand(n,1); nmax = 100; tol= 1.e-06; mu = 0;
elseif nargin == 2
    x0= rand(n,1); nmax= 100; tol= 1.e-06;
end
[L, U]=lu (A-mu*eye(n));
if norm(x0) ==0
    x0= rand(n,1);
end
x0==x0/norm(x0);
z0=L\x0;
pro=U\z0;
lambda=x0'*pro;
err=tol*abs(lambda)+ 1; iter=0;
while err > tol*abs(lambda) & abs(lambda) ~= 0 & iter <= nmax
    x = pro; x= x/norm(x);
    z=L\x; pro=U\z;
    lambdanew = x*pro;
    err = abs(lambdanew- lambda);
    lambda =lambdanew;
    iter= iter + 1.
```

```
end
lambda= 1/lambda + mu;
```

例 6.4 对于例 6.1 中给出的矩阵 $A(30)$, 求出与数值 17 距离最近的特征值。为此, 程序 10 中选取参数为 $\mu=17$, $\text{tol}=10^{-10}$, $x_0=[1; 1; 1; 1]$ 。程序经过 8 次迭代运算返回值为 $\text{lambda}=1.2183$ 。如果移位量取值不准确, 则所需的迭代次数将大为增加。如取 $\mu=13$ 时, 程序经过 19 次迭代运算的返回值为 17.820 8。

在迭代运算过程中, 移位量的数值可以通过设定 $\mu=\lambda^{(k)}$ 来修改, 这将使收敛加快, 但由于每次迭代时矩阵 A_μ 的值都要改变, 所以实际所需的运算量也要增加。

参见习题 6.4~6.6。

6.3 计算移位量的方法

要想成功应用原点移位法, 需要在复平面上定位出(精确的或大致的定位均可) A 的特征值, 为此来介绍下面的定义:

设 A 是一个 n 维方阵, 则分别与第 i 行和第 i 列相对应的 Gershgorin 环 $C_i^{(r)}$ 和 $C_i^{(c)}$ 定义为:

$$C_i^{(r)} = \left\{ z \in \mathbb{C} : |z - a_{ii}| \leq \sum_{j=1, j \neq i}^n |a_{ij}| \right\}$$

$$C_i^{(c)} = \left\{ z \in \mathbb{C} : |z - a_{ii}| \leq \sum_{j=1, j \neq i}^n |a_{ji}| \right\}$$

$C_i^{(r)}$ 称为第 i 行环, $C_i^{(c)}$ 称为第 i 列环。

通过程序 11 可分别在两个窗口中观察到同一矩阵的行环和列环。hold on 函数可以实现多个图形的迭加(本例中即是把不同的环迭加在一起), 该命令可由 hold off 函数来取消。

patch 函数可用来给圆环标上颜色, axis 函数可以在图形中给出背景坐标。

程序 11 Gershgorin 环

```
function gershcircles(A)
%GERSHCIRCLES plots the Gershgorin circles
% GERSHGOIRINCIRCLES(A) plots the Gershgorin circles for
% the square matrix A and its transpose.
Abs = abs(A); n = max(size(A)); radii = sum(Abs,2)-diag(Abs);
xcenter = real(diag(A)); ycenter = imag(diag(A));
theta = [0:pi/100:2*pi];
```



```

    costheta = cos(theta); sintheta = sin(theta);
x=[]; y=[]; figure(1); clf; axis equal; hold on;
for i = 1: n
x=[x; radii(i)*cos(theta)+xcenter(i)];y=[y; radii(i)*sin(theta)+ycenter
(i)];
    patch(x(i,:),y(i,:), 'red');
end
for i= 1: n, plot(x(i,:),y(i,:), 'k',xcenter(i),ycenter(i), 'xk'); end
xmax= max(max(x)); ymax=max(max(y));
xmin = min(min(x)); ymin=min(min(y));
hold off; figure(2); clf; axis equal; hold on; radii=sum(Abs)-(diag
(Abs))';
x=[]; y=[]; clf; axis equal; hold on;
for i = 1: n
x=[x; radii(i)*cos(theta)+xcenter(i)];y=[y; radii(i)*sin(theta)+ycenter
(i)];
    patch(x(i,:),y(i,:), 'green')
end
for i= 1: n, plot(x(i,:),y(i,:), 'k', xcenter(i),ycenter(i), 'xk'); end
xmax= max(max(max(x)),xmax); ymax=max(max(max(y)),ymax);
xmin = min(min(min(x)),xmin); ymin=min(min(min(y)),ymin); hold off;
axis([xmin xmax ymin ymax]); figure(1); axis([xmin xmax ymin ymax]);

```

例 6.5 图 6.4 中给出了下列矩阵 A 的 Gershgorin 环

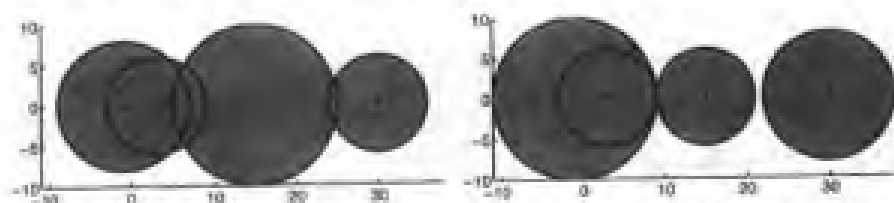


图 6.4 例 6.5 中矩阵的行环(左图)和列环(右图)

环的中心用圆点做了标记。

$$A = \begin{bmatrix} 30 & 1 & 2 & 3 \\ 4 & 15 & -4 & -2 \\ -1 & 0 & 3 & 5 \\ -3 & 5 & 0 & -1 \end{bmatrix}$$

正如前面所提到的, Gershgorin 环可用来定位出矩阵的特征值, 正如在下列命题中所表述的那样。

命题 6.1 一个给定矩阵 $A \in \mathbb{C}^{n \times n}$ 的所有特征值均落在复平面内由行环集合和列环集合所构成的交集区域内。

而且如果 m 个行环(或列环)(其中 $1 \leq m \leq n$)与余下的 $n-m$ 个连在一起的环是相分离的, 则由 $n-m$ 个相连的环覆盖的区域恰好包含 m 个特征值。

一个环只有在与其他的环相分离的情况下, 这个环内才一定包含特征值。以上的结论可用于对移位量进行初步估计, 正如下例所示:

例 6.6 分析例 6.1 中的矩阵 $A(30)$ 的行环, 可以得出 A 的特征值的实部介于 $-32 \sim 48$ 之间。通过程序 10 来计算按模最大的特征值, 取移位量为 $\mu = 48$, 经过 16 次迭代运算可以得到收敛值。而在相同的初始条件 $x_0 = [1; 1; 1; 1]$ 和同样的误差容限 $\text{tol} = 1.e-10$ 下, 利用幂法需要经过 24 次迭代运算才可以得到收敛值。

参见习题 6.7~6.8。

6.4 计算全部特征值的方法

两个同维矩阵 A 和 B 如果满足下列条件: 存在非奇异矩阵 P 使得

$$P^{-1}AP = B$$

成立, 则称矩阵 A 和 B 是相似矩阵。

相似矩阵有着相同的特征值。实际上, 如果 λ 是矩阵 A 的一个特征值, $x \neq 0$ 是相关的特征向量, 则有:

$$BP^{-1}x = P^{-1}Ax = \lambda P^{-1}x$$

即 λ 也是 B 的特征值, 其对应的特征向量是 $y = P^{-1}x$ 。

计算 A 的全部特征值采用下列方法: 将 A 经过有限次运算转换为对角或三角形相似矩阵, 它们主对角线上的元素就是特征值。

实现这一算法有多种途径, 我们介绍其中的 QR 分解方法, MATLAB 中的 eig 函数就是用这一方法实现的。具体说来, $D = \text{eig}(A)$ 函数返回的向量 D 就包含了 A 的所有特征值。通过函数 $[X, D] = \text{eig}(A)$ 可以得到两个矩阵: 由 A 的特征值所构成的对角阵 D 和由 A 的特征向量作为列向量的矩阵 X , 于是 $A * X = X * D$ 。当 A 是稀疏矩阵时, 利用函数 $\text{eigs}(A, k)$ 可以计算出 A 的前 k 个按模最大的特征值。

如果矩阵 A 的所有特征值都是互不相同的, 用 QR 方法得到的特征值序列将收敛于 A 的相似上三角阵, 而当 A 为对称阵时, 则收敛于一个对角阵。基于 QR 分解的有关算法在习题 6.9 和 6.10 中有所涉及, 而在 [QSS00] 的第 5 章中则有更为详尽的介绍。

小结

1. 幂法是求解一个给定矩阵的按模最大的特征值时采用的迭代算法;
2. 反幂法可以求解按模最小的特征值; 它需要将给定的矩阵进行分解;
3. 原点移位法可以计算出与给定的数值最接近的特征值; 应用这一方法时需要

预先知道矩阵特征值位置的信息，这些知识可以由 Gershgorin 环得到：

4. QR 方法可以求出一个给定矩阵的所有特征值的近似解。

参见习题 6.9~6.11。

6.5 补充说明

以上分析还没有涉及到关于特征值的条件数问题，条件数问题描述的是特征值相对于矩阵元素变化的敏感程度。有兴趣的读者可以参考 [Wil65]、[GL89] 和 [QSS00] 的第 5 章。

需要指出的是，即使一个矩阵的条件数很大，特征值计算问题并不一定就是“病态”的。举例来说，尽管 Hilbert 矩阵(见例 5.8)的条件数非常大，但由于它是对称正定阵，所以 Hilbert 矩阵特征值的计算仍是一个良态的问题。

除了 QR 方法，如果需要同时计算全部特征值，也可以使用 Jacobi 法，它通过相似变换，每一步消掉一个对角线以外的元素，最后把一个对称矩阵转换成一个对角阵。这种方法并不一定经过有限次的运算就终止，原因在于一个新的元素设为零以后，那些经过运算的元素可能又变为非零元素了。

其他的方法还包括 Lanczos 方法和利用 Sturm 序列来求解的方法。关于这些方法的介绍可以参见 [Saa92]。

MATLAB 的程序库 ARPACKC(可以通过 arpackc 函数来得到)可以用来计算大矩阵的特征值，也可以从下列网址下载：

<http://www.caam.rice.edu/software/ARPACK/>

可以使用 eigs 函数来调用这个库。

最后简单介绍一下压缩方法，这种方法通过消去已经计算出来的特征值来减少运算。压缩方法可以加快前述算法的收敛速度，从而降低运算复杂度。

6.6 习题

习题 6.1 设误差容限为 $\varepsilon=10^{-10}$ ，利用幂法近似求解下列矩阵的按模最大的特征值，设初始向量 $\mathbf{x}^{(0)}=(1, 2, 3)^T$ ：

$$A_1 = \begin{bmatrix} 1 & 2 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 0.1 & 3.8 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \quad A_3 = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

并对三种情况下的收敛性进行比较。

习题 6.2 正如在问题 6.2 中提到的那样, 设下列 Leslie 矩阵描述的是鱼群的特征:

年龄段/月	$x^{(0)}$	m_i	s_i
0~3	6	0	0.2
3~6	12	0.5	0.4
6~9	8	0.8	0.8
9~12	4	0.3	-

根据在问题 6.2 中所叙述的方法, 找出描述不同年龄段的鱼群的规范化分布的向量 x 。

习题 6.3 设某矩阵的按模最大的特征值 $\lambda_1 = \gamma e^{i\theta}$, 另一个特征值是 $\lambda_2 = \gamma e^{-i\theta}$, 其中 $i = \sqrt{-1}$, $\gamma, \theta \in \mathbf{R}$ 。证明利用幂法来对该矩阵进行求解时并不收敛。

习题 6.4 证明 A^{-1} 的特征值与 A 的特征值是倒数关系。

习题 6.5 证明幂法不能用于计算下列矩阵的按模最大的特征值, 并解释原因。

$$A = \begin{bmatrix} \frac{1}{3} & \frac{2}{3} & 2 & 3 \\ 3 & 3 & & \\ 1 & 0 & -1 & 2 \\ 0 & 0 & -\frac{5}{3} & -\frac{2}{3} \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

习题 6.6 利用原点移位法, 计算下列矩阵最大的正特征值和绝对值最大的负特征值:

$$A = \begin{bmatrix} 3 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 2 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 2 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 3 \end{bmatrix}$$

A 称为 Wilkinson 矩阵, 它可由命令 `wilkinson(7)` 得到。

习题 6.7 利用 Gershgorin 环, 近似求出下列矩阵的复特征值的最大个数。

$$A = \begin{bmatrix} 2 & -\frac{1}{2} & 0 & -\frac{1}{2} \\ 0 & 4 & 0 & 2 \\ -\frac{1}{2} & 0 & 6 & \frac{1}{2} \\ 0 & 0 & 1 & 9 \end{bmatrix}, \quad B = \begin{bmatrix} -5 & 0 & 1/2 & 1/2 \\ 1/2 & 2 & 1/2 & 0 \\ 0 & 1 & 0 & 1/2 \\ 0 & 1/4 & 1/2 & 3 \end{bmatrix}$$

习题 6.8 使用命题 6.1 的结论, 找到一个最合适的移位量, 用于计算下列矩阵的按模最大的特征值:

$$A = \begin{bmatrix} 5 & 0 & 1 & -1 \\ 0 & 2 & 0 & -\frac{1}{2} \\ 0 & 1 & -1 & 1 \\ -1 & -1 & 0 & 0 \end{bmatrix}$$

在同等误差容限 10^{-14} 下, 比较采用移位和没有移位这两种情况下的迭代运算次数和幂法的计算代价。

习题 6.9 矩阵 $A \in \mathbb{R}^{n \times n}$ 在满足下列条件时可以分解为 QR 因式: 存在矩阵 $Q \in \mathbb{R}^{n \times n}$ 满足正交性 $Q^T Q = I$, 并存在上三角阵 $R \in \mathbb{R}^{n \times n}$, 这时有 $A = QR$ 。这个因式可以通过函数 $[Q, R] = \text{qr}(A)$ 得到。使用这个函数来求出习题 6.7 中的矩阵 A 的 QR 因式。并证明此时矩阵 $C = RQ$ 相似于 A 。

习题 6.10 下列算法为 QR 方法奠定了基础, QR 方法可用来求解矩阵 A 的全部特征值。

设 $A^{(0)} = A$;

对于 $k=0, 1, \dots$, 计算 $Q^{(k)}$ 和 $R^{(k)}$, 于是 $A^{(k)} = Q^{(k)} R^{(k)}$;

再设 $A^{(k+1)} = R^{(k)} Q^{(k)}$ 。

编写 MATLAB 程序实现这一方法。并对习题 6.7 中的矩阵 A 进行一定次数的迭代运算, 并求出矩阵序列 $A^{(k)}$ 的极限。

习题 6.11 利用 eig 函数计算习题 6.7 中所给的两个矩阵的全部特征值, 并检验通过命题 6.1 得出的结论的准确程度。

第 7 章 常微分方程

微分方程是含有某个未知函数的一个或多个导数的方程。如果所有的导数都是对某个独立的自变量求解的, 则称这种微分方程为常微分方程; 如果在微分方程里包含有偏导数, 则称之为偏微分方程。

如果微分方程(常微分方程或者偏微分方程)的变量的最高阶次是 p , 称之为 p 阶微分方程。第 8 章将主要讨论一种特殊的二阶偏微分方程, 即椭圆方程。而本章则主要介绍一阶常微分方程。

常微分方程能够描述各种不同领域的很多现象的演化, 从下面的三个例子中可以看出这一点。

问题 7.1(热力学) 将一个内部温度为 T 的物体置于常温为 T_e 的环境中。假定它的质量 m 集中于一点, 即质点上。则物体与外部环境之间的热传递可以用史蒂芬-玻尔兹曼定律来描述:

$$v(t) = \epsilon \gamma S (T^4(t) - T_e^4)$$

其中, t 是时间变量, ϵ 是玻尔兹曼常数(等于 $5.6 \times 10^{-8} \text{J/m}^2 \text{K}^4 \text{s}$, 其中 J 代表焦耳, K 代表开尔文, m 代表米, s 代表秒), γ 是物体的辐射系数, S 是物体的表面积, v 是热传递的速率。能量的变化率 $E(t) = mCT(t)$ (其中 C 表示组成物体材料的特定热量值), 在绝对值上等于速率 v 。通常设 $T(0) = T_0$, 则计算 $T(t)$ 需要求解下列常微分方程:

$$\frac{dT}{dt} = -\frac{v(t)}{mC} \quad (7.1)$$

问题 7.2(生物学) 考虑限定环境下的细菌群体问题, 在该环境中共存的个体数目不能超过 B 。假定在初始时刻, 个体的数量等于 $y_0 < B$, 而细菌的增长率是一个正常数 C 。在这种情况下, 细菌群体的变化率正比于存活细菌的数目, 而总数不能超过 B 。由微分方程表达如下

$$\frac{dy}{dt} = Cy \left(1 - \frac{y}{B} \right) \quad (7.2)$$

其中, $y=y(t)$ 表示细菌在 t 时刻的数量。

假定存在两个竞争的群体 y_1 和 y_2 , 则此时的方程将不再是(7.2)式, 而是

$$\begin{aligned}\frac{dy_1}{dt} &= C_1 y_1 (1 - b_1 y_1 - d_2 y_2) \\ \frac{dy_2}{dt} &= -C_2 y_2 (1 - b_2 y_2 - d_1 y_1)\end{aligned}\quad (7.3)$$

其中, C_1 和 C_2 分别代表两个群体的增长率。系数 d_1 和 d_2 决定了两个群体相互作用的类型, 而 b_1 和 b_2 则与获得的营养数量相关。上述等式(7.3)叫做洛特卡-沃尔特方程, 此方程在许多实际情况中都得到了应用。关于它们的数值解法, 可以参考例 7.6。

问题 7.3 (电力学) 考虑图 7.1 所示的电路图。计算电容 C 两端的电压降 $v(t)$, 将开关 I 断开的时刻记为初始时刻 $t=0$ 。假定电感 L 能够由电流强度 i 表示成显函数, 即 $L=L(i)$ 。由欧姆定律

$$e - \frac{d(iL(i))}{dt} = i_1 R_1 + v$$

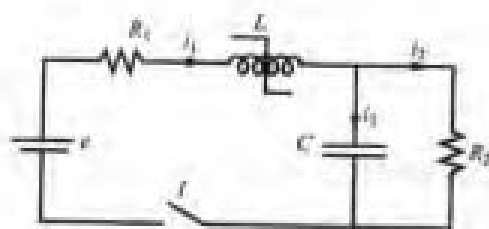


图 7.1 问题 7.3 中的电路图

其中 R_1 表示电阻。假定电流方向如图 7.1 所示, 将基尔霍夫方程 $i_1 = i_2 + i_3$ 的等号两端同时对 t 求导数, 并注意 $i_3 = C dv/dt$ 和 $i_2 = v/R_2$, 则有

$$\frac{di_1}{dt} = C \frac{d^2 v}{dt^2} + \frac{1}{R_2} \frac{dv}{dt}$$

于是最后得到的是由两个微分方程组成的系统, 并且微分方程的解中包含两个与时间 t 有关的变量 i_1 和 v 。还有第二个方程是二阶的, 有关的解法可以参见例 7.7。

7.1 柯西问题

因为一个 $p(p>1)$ 阶微分方程总能化成 p 个一阶微分方程, 因此只需研究一阶微分方程的问题。关于一阶系统的问题将在 7.8 节介绍。

一般来说, 常微分方程有无穷多个解。为了求出一个固定的解, 就需要给出边界条件, 即给出积分区间内某个给定点的值。例如, 方程 7.2 的解为 $y(t) = B\varphi(t)/(1+\varphi(t))$, 其中 $\varphi(t) = e^{Ct+K}$, K 是任意常数。如果给定边界条件 $y(0)=1$, 就

能得出相应的惟一值 $K=\ln[1/(B-1)]$ 。

因此下面将讨论所谓的柯西问题的解法, 这种问题形式如下:

求解函数 $y: I \rightarrow R$, 使得

$$\begin{cases} y'(t) = f(t, y(t)) & \forall t \in I \\ y(t_0) = y_0 \end{cases} \quad (7.4)$$

其中, I 是 R 中的一段区间, $f: I \times R \rightarrow R$ 为给定的函数, y' 是 y 对于 t 的导数。 t_0 是 I 中的一点, y_0 称为初始数值。

在下面的命题中给出了分析的典型结果。

命题 7.1 假定函数 $f(t, y)$ 满足:

1. 对两个变量都是连续的;
2. 对第二个变量是 Lipschitz 连续, 即存在一个正常数 L , 使得下式成立:

$$|f(t, y_1) - f(t, y_2)| \leq L|y_1 - y_2|, \quad \forall t \in I, \quad \forall y_1, y_2 \in R$$

则柯西问题(7.4)的解存在且惟一。

但是一般只对特殊类型的常微分方程才有显式解。而在其他情况情况下, 通常只有隐式解。举例来说, 方程 $y' = (y-t)/(y+t)$ 的解满足下列隐式关系:

$$\frac{1}{2} \ln(t^2 + y^2) + \arctg \frac{y}{t} = C$$

其中, C 是任意常数。在有些情况下, 方程的解甚至不能用隐式表示出来, 例如方程 $y' = e^{-t}$ 的解只能用一个级数展开式来表示。

基于上述原因, 对确实存在解的每一族常微分方程, 需要找到求近似解的数值方法。

对于所有数值方法, 通常的做法是把积分区间 $I=[t_0, T]$, $T < +\infty$, 划分成 N_h 个长度为 $h=(T-t_0)/N_h$ 的小区间; h 称为离散化步长。接下来, 在每一个节点 $t_n(0 \leq n \leq N_h-1)$ 处, 求出 $y_n=y(t_n)$ 的近似值 u_n 。于是 $\{u_0=y_0, u_1, \dots, u_{N_h}\}$ 就是求得的数值解法。

7.2 欧拉方法

前向欧拉法是一种经典方法, 利用它得到的数值解的形式如下:

$$u_{n+1} = u_n + hf_n, \quad n = 0, \dots, N_h - 1 \quad (7.5)$$

这里采用了符号的简写形式 $f_n = f(t_n, u_n)$ 。考察微分方程(7.4)的每一个节点 t_n , $n=1, \dots, N_h$, 用增量比率(4.3)的方法替代导数的精确值 $y'(t_n)$, 这样便得到了欧拉方法。

类似地, 用增量比率(4.7)的方法去近似 $y'(t_{n+1})$, 可得到后向欧拉法

$$u_{n+1} = u_n + hf_{n+1}, \quad n = 0, \dots, N_h - 1 \quad (7.6)$$

两种欧拉法都是“一步法”的实例, 这是由于在计算节点 t_{n+1} 的数值解 u_{n+1} 时, 只需已知前一个节点 t_n 的值。

具体说来, 在前向欧拉方法中, u_{n+1} 仅与先前计算出来的 u_n 的值有关, 而在后向欧拉法中, u_{n+1} 还与 f_{n+1} 本身的值有关。因此, 第一种方法称为显式欧拉方法, 第二种方法称为隐式欧拉方法。

例如, 利用前向欧拉方法对(7.2)式离散化, 每步只需经过简单的计算

$$u_{n+1} = u_n + hCu_n(1 - u_n/B)$$

而采用后向欧拉方法, 则必须解非线性方程

$$u_{n+1} = u_n + hCu_{n+1}(1 - u_{n+1}/B)$$

这样, 由于在每个时刻 t_{n+1} 必须解非线性方程来求 u_{n+1} , 所以隐式方法要比显式方法代价大。但隐式法比显式方法有更好的稳定性。

程序 12 是前向欧拉法的一个实现; 积分区间是 $tspan=[t0, tfinal]$, 字符串 `odefun` 表示包含变量 t 和 y 的函数 $f(t, y(t))$, 或者称为由前两个变量 t 和 y 构成的内嵌函数。

程序 12-feuler: 前向欧拉法

```
function [t,y]=feuler(odefun,tspan,y, Nh,varargin)
%FEULER Solve differential equations using the forward Euler method.
% [T,Y] = FEULER(ODEFUN,TSPAN,Y0,NH) with TSPAN =[TO TFINAL]
% integrates the system of differential equations y' = f(t,y) from
% time TO to TFINAL with initial conditions Y0 using the forward Euler
% method on an equispaced grid of NH intervals.
% Function ODEFUN(T,Y) must return a column vector
% corresponding to f(t,y). Each row in the solution array Y corresponds to
% a time returned in the column vector T.
% [T,Y] = FEULER(ODEFUN,TSPAN,Y0,NH,PI,P2,...) passes the additional
% parameters PI,P2,... to the function ODEFUN as ODEFUN(T,Y,PI,P2,...).
```

```

h=( tspan(2)-tspan(1))/Nh;
tt=linspace(tspan(1),tspan(2),Nh+1);
for t = tt(1:end-1)
    y= [y; y(end,:) + h*feval(odefun,t,y(end,:),varargin{:})];
end
t=tt;

```

程序 13 是后向欧拉法的一个实现。注意在每一步运算中是采用 `fzero` 函数来求解非线性问题的。并且利用上一个数值计算结果作为 `fzero` 函数的初始数据。注意其中 `num2str(num, s)` 命令的作用是把存储于字符串 `num` 中的数字进行保留 16 位有效数字的转换。

程序 13-beuler: 后向欧拉法

```

function [t,u]=beuler(odefun,tspan,y,Nh,varargin)
%BEULER Solve differential equations using the backward Euler method.
% [T,Y] = BEULER(ODEFUN,TSPAN,Y0,NH) with TSPAN =[TO TFINAL]
% integrates the system of differential equations y' = f(t,y) from
% time TO to TFINAL with initial conditions Y0 using the backward Euler
% method on an equispaced grid of NH intervals.
% Function ODEFUN(T,Y) must return a column vector
% corresponding to f(t,y). Each row in the solution array Y correspondsto
% a time returned in the column vector T.
% [T,Y] = BEULER(ODEFUN,TSPAN,Y0,NH,P1,P2,...) passes the additional
% parameters P1,P2,... to the function ODEFUN as ODEFUN(T,Y,P1,P2,...).
h=(tspan(2)-tspan(1))/Nh;
tt=linspace(tspan(1),tspan(2), Nh+1);
u(1)=y;
syms x;
y=x;
for t=tt(2:end)
    fun=inline(['x-',num2str(h,16),'*(',char(feval(odefun,t,y,varargin{
:})),...
    ' )-', num2str(u(end),16)], 'x');
    u =[u, fzero(fun,u(end))];
end
t=tt;

```

收敛性分析

如果满足下列条件，数值方法就是收敛的

$$\forall_n = 0, \dots, N_h, \quad |u_n - y_n| \leq C(h)$$

这里， $C(h)$ 是 h 趋于零时，相对于 h 的无穷小量。如果在 $p > 0$ 时有 $C(h) = O(h^p)$ ，那么认为此方法 p 阶收敛。为了确定前向欧拉法的收敛性，将误差写成下列形式：

$$e_n = u_n - y_n = (u_n - u_n^*) + (u_n^* - y_n) \quad (7.7)$$

其中, $y_n = y(t_n)$, 并且

$$u_n^* = y_{n-1} + hf(t_{n-1}, y_{n-1})$$

也就是 u_n^* 表示在时间 t_n 处的数值解, 可以从开始时间 t_{n-1} 的精确解得到; 参见图 7.2。

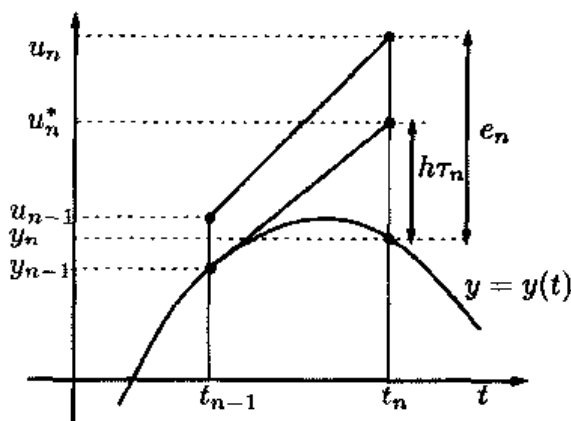


图 7.2 前向欧拉法的单步几何表示

式(7.7)的项 $u_n^* - y_n$ 表示前向欧拉法单步产生的误差, 而 $u_n - u_n^*$ 表示在先前的时间点 t_{n-1} 时刻的积累误差从 t_{n-1} 到 t_n 的过程中的增量。如这两项在 $h \rightarrow 0$ 时都趋向于零, 该方法就是收敛的。如果 y 的二阶导数存在且连续, 利用式(4.5)可得到

$$u_n^* - y_n = \frac{h^2}{2} y''(\xi_n), \text{ 对于适当的 } \xi_n \in (t_{n-1}, t_n) \quad (7.8)$$

因此当 $h \rightarrow 0$ 时, $u_n^* - y_n$ 趋向于零。

式 $\tau_n(h) = (u_n^* - y_n)/h$ 的量值表示精确解与数值解之间的误差, 因此该误差称为局部截断误差。(整体)截断误差定义为:

$$\tau(h) = \max_{n=0, \dots, N_h} |\tau_n(h)|$$

注意对于局部和整体截断误差的定义不仅仅适用于欧拉法, 也适用于大多数的数值方法。

由式(7.8), 前向欧拉法的截断误差有如下形式:

$$\tau(h) = Mh/2 \quad (7.9)$$

这里, $M = \max_{t \in [t_0, T]} |f''(t, y(t))|$ 。

由式(7.8)可推出 $\lim_{h \rightarrow 0} \tau(h) = 0$, 如果满足这个条件, 则称该方法是一致的。更进一步, 如果对于合适的整数 $p \geq 1$, 有 $\tau(h) = O(h^p)$, 则称为 p 阶一致。

考察式(7.7)中的另一项。有

$$u_n^* - u_n = e_{n-1} + h[f(t_{n-1}, y_{n-1}) - f(t_{n-1}, u_{n-1})] \quad (7.10)$$

由于 f 对于第二自变量是 Lipschitz 连续的, 可得

$$|u_n^* - u_n| \leq (1 + hL)|e_{n-1}|$$

如果 $e_0 = 0$, 由前述关系产生

$$\begin{aligned} |e_n| &\leq |u_n - u_n^*| + |u_n^* - y_n| \\ &\leq (1 + hL)|e_{n-1}| + h\tau_n(h) \\ &\leq [1 + (1 + hL) + \cdots + (1 + hL)^{n-1}] h\tau(h) \\ &= \frac{(1 + hL)^n - 1}{L} \tau(h) \leq \frac{e^{L(t_n - t_0)} - 1}{L} \tau(h) \end{aligned}$$

上述推导中使用了恒等式

$$\sum_{k=0}^{n-1} (1 + hL)^k = [(1 + hL)^n - 1] / hL$$

不等式 $1 + hL \leq e^{hL}$, 注意 $nh = t_n - t_0$ 。因此可得到

$$|e_n| \leq \frac{e^{L(t_n - t_0)} - 1}{L} \frac{M}{2} h, \quad \forall n = 0, \dots, N_h \quad (7.11)$$

由此可得出结论, 前向欧拉法是一阶收敛的。注意这种方法的阶数与它的局部截断误差的阶数是一致的。很多由数值方法计算出来的常微分方程的数值解都具有这种性质。

注 7.1 收敛估计(7.11)式仅要求 f 是 Lipschitz 连续的。如果 f 满足进一步的条件: 即对所有的 $t \in [t_0, T]$ 及所有的 $-\infty < y < \infty$, 都有 $\partial f(t, y) / \partial y \leq 0$, 则可以得到更准确的估计, 结果为 $|e_n| \leq Mh(t_n - t_0)/2$ 。实际上, 此时利用 Taylor 展开式, 由式(7.10)可得 $u_n^* - u_n = (1 + h\partial f / \partial y(t_{n-1}, y_{n-1}))e_{n-1}$, 则有 $|u_n^* - u_n| \leq |e_{n-1}|$, 此时假设稳定性不等式 $h < 2 / \max_t |\partial f / \partial y(t, y(t))|$ 成立。则有 $|e_n| \leq |u_n^* - u_n| + |e_{n-1}| \leq |e_0| + nh\tau(h)$, 以上推导中使用了式(7.9)。

注 7.2 (一致性) 一致性是收敛的必要条件。实际上, 如果不满足一致性, 数

值方法在每一步会产生一个对于 h 不是无穷小量的误差。则当 $h \rightarrow 0$ 时, 由于以前误差的积累就会阻止整体误差收敛到零。

用相似的方法, 可证明后向欧拉法也是关于 h 一阶收敛的。

例 7.1 看下面的柯西问题

$$\begin{cases} y'(t) = \cos(2y(t)) & t \in (0, 1] \\ y(0) = 0 \end{cases} \quad (7.12)$$

其解是 $y(t) = \frac{1}{2} \arcsin((e^{4t} - 1)/(e^{4t} + 1))$ 。用前向欧拉法(程序 12)和后向欧拉法(程序 13)来求解。通过以下命令, 利用不同的 h 值: $1/2, 1/4, 1/8, \dots, 1/512$:

```
>> tspan=[0,1]; y0=0; f=inline('cos(2*y)','t','y');
>> u=inline('0.5*asin((exp(4*t)-1)/(exp(4*t)+1))','t');
>> Nh=2;
for k=1:10
    [t, ufe]=feuler(f,tspan,y0,Nh); fe(k)=abs(ufe(end)-feval(u,t(end)));
    [t, ube]=feuler(f,tspan,y0,Nh); be(k)=abs(ube(end)-feval(u,t(end)));
    Nh = 2*Nh;
end
```

在点 $t=1$, 误差分别存贮在变量 fe (前向欧拉)和变量 be (后向欧拉)中。再利用公式(1.11)来估计收敛的阶数。利用下列命令

```
>> p=log(abs(fe(1:end-1)./fe(2:end)))/log(2); p(1:2:end)
    1.2898    1.0349    1.0080    1.0019    1.0005
1>> p=log(abs(be(1:end-1)./be(2:end)))/log(2); p(1:2:end)
    0.9070    0.9720    0.9925    0.9981    0.9995
```

可以确定, 两种方法都是一阶收敛的。

注 7.3 误差估计公式(7.11)的前提假设是数值解 $\{u_n\}$ 是通过精确计算得到的。如果考虑(不可避免的)舍入误差的影响, 当 h 以 $O(1/h)$ 趋近于零时, 误差可能溢出(参见 [Atk89])。该结论表明在实际的计算中, 为了确保计算的合理性, h 不能低于特定的门限值 h^* (实际上是极其小的值)。

参见习题 7.1~7.3。

7.3 Crank-Nicolson 方法

将前向和后向欧拉法的一般步骤结合起来, 可得到所谓的 Crank-Nicolson 方法。

$$u_{n+1} = u_n + \frac{h}{2} [f_n + f_{n+1}] \quad (7.13)$$

此种方法是单步隐式法，程序 14 是该方法的一个实现。其输入输出参数与欧拉法相同。

程序 14—cranknic: Crank-Nicolson 方法

```
function [t,u]=cranknic(odefun,tspan,y, Nh,varargin)
%CRANKNIC Solve differential equations using the Crank-Nicolson method.
% [T,Y] = CRANKNIC(ODEFUN,TSPAN,Y0,NH) with TSPAN = [TO TFINAL]
% integrates the system of differential equations y' = f(t,y) from
% time T0 to TFINAL with initial conditions Y0 using the Crank-Nicolson
% method on an equispaced grid of NH intervals.
% Function ODEFUN(T,Y) must return a column vector
% corresponding to f(t,y). Each row in the solution array Y corresponds to
% a time returned in the column vector T.
% [T,Y] = CRANKNIC(ODEFUN,TSPAN,Y0,NH,P1,P2,...) passes the additional
% parameters P1,P2,... to the function ODEFUN as ODEFUN(T,Y,P1,P2,...).
h=(tspan(2)-tspan(1))/Nh;
tt=cinspace(tspan(1),tspan(2),Nh+1);
u(1)=y;
fold = feval(odefun,tspan(1),y,varargin{:});
syms x;
y=x;
for t=tt(2:end)
    y=x;
    fun=inline(['x-',num2str(h,16),'*0.5*(',char(feval(odefun,t,y,vara
        rgin{:})),')+('
    num2str(fold,16),')')~(' ',num2str(u(end),16),')'], 'x');
    u =[u, fzero(fun,u(end))];
    fold = feval(odefun,t,u(end),varargin{:});
end
t=tt;
```

Crank-Nicolson 方法的局部截断误差满足

$$\begin{aligned} \tau_n(h) &= y(t_n) - y(t_{n-1}) - \frac{h}{2} [f(t_n, y(t_n)) + f(t_{n-1}, y(t_{n-1}))] \\ &= \int_{t_{n-1}}^{t_n} f(t, y(t)) dt - \frac{h}{2} [f(t_n, y(t_n)) + f(t_{n-1}, y(t_{n-1}))] \end{aligned}$$

最后一步等式使用了积分基本定理(在 1.4.3 节复述过)。第二项表示与数值积分(4.17)的梯形法相关联的误差。如果假定 $y \in C^3$ 并且由式(4.18)，可推断出

$$\tau_n(h) = -\frac{h^2}{12} y'''(\xi_n) \text{ 对于适当的 } \xi_n \in (t_{n-1}, t_n) \quad (7.14)$$

这样, Crank-Nicolson 方法是 2 阶一致的。即它的局部截断误差是按照 h^2 的特性趋于零的。用与前向欧拉法相似的方法, 可得出 Crank-Nicolson 方法对于 h 是 2 阶收敛的。

例 7.2 选取与例 7.1 相同的 h 值, 利用 Crank-Nicolson 方法求解柯西问题(7.12)。由结果可以看出误差是 2 阶趋于零的。

```
>> y0=0; tspan=[0 1]; N=2; f=inline('cos(2*y)', 't', 'y');
>> y='0.5*asin((exp(4*t)-1)./(exp(4*t)+1))';
>> for k=1:10
    [tt, u]=cranknic(f, tspan, y0, N);
    t=tt(end); e(k)=abs(u(end)-eval(y)); N=2*N; end
>> p=log(abs(e(1:end-1)./e(2:end)))/log(2); p(1:2:end)
    1.7940  1.9944  1.9997  2.0000  2.0000
```

7.4 零稳定性

在稳定性的概念中, 有一种叫做零稳定性, 这种稳定性确保在固定的边界区间中, 当 $h \rightarrow 0$ 时, 数据上的小扰动只会引起数值解在一定范围内的扰动。

更精确地说, 使用数值方法求解问题(7.4), 这里 $I=[t_0, T]$, 如对于所有 $\delta > 0$ 及任何的足够小的 h , 存在 $C > 0$ 满足下式成立, 则称该法是零稳定的

$$|z_n - u_n| \leq C\delta, \quad 0 \leq n \leq N_h \quad (7.15)$$

其中, C 为常数, 它取决于积分区间 I 的长度。 z_n 是使用数值方法解决扰动问题所得到的解, δ 表示扰动的最大值。显然, δ 一定要足够小, 以确保扰动问题在积分区间内有惟一解。

例如, 在前向欧拉法的情况下, z_n 满足

$$\begin{cases} z_{n+1} = z_n + h[f(t_n, z_n) + \rho_{n+1}] \\ z_0 = y_0 + \rho_0 \end{cases} \quad (7.16)$$

其中, $0 \leq n \leq N_h - 1$, 并且假设 $|\rho_n| \leq \delta$, $0 \leq n \leq N_h$ 。

对于一致的单步法, 可以证明零稳定性是函数 f 对于第二个变量是 Lipschitz 连续的结果(参见 [QSS00]), 此时 C 取决于函数 $\exp((T-t_0)L)$, 其中 L 是 Lipschitz 常数。但对于其他类型的方法未必正确。例如假设数值方法可以写成一般形式

$$u_{n+1} = \sum_{j=0}^p a_j u_{n-j} + h \sum_{j=0}^p b_j f_{n-j} + h b_{-1} f_{n+1}, \quad n = p, p+1, \dots \quad (7.17)$$

其中, $\{a_k\}$ 和 $\{b_k\}$ 为合适的相关系数, 且 $p > 0$ 。这是一种线性多步方法, 且 $p+1$ 表示步数。7.6 节有一些多步方法的例子。多项式

$$\pi(r) = r^{p+1} - \sum_{j=0}^p a_j r^{p-j}$$

称为与数值方法(7.17)相关的第一特征多项式, 用 $r_j (j=0, \dots, p)$ 表示它的根。当且仅当满足下列根的条件时, 方法(7.17)是零稳定的:

$$\begin{cases} |r_j| \leq 1 \text{ 对于所有 } j=0, \dots, p \\ \text{进一步 } \pi'(r_j) \neq 0 \text{ 对于 } j, \text{ 使得 } |r_j| = 1 \end{cases} \quad (7.18)$$

例如, 对于前向欧拉法有 $p=0$, $a_0=1$, $b_{-1}=0$, $b_0=1$ 。对于后向欧拉法有 $p=0$, $a_0=1$, $b_{-1}=1$, $b_0=0$ 。对于 Crank-Nicolson 法, 有 $p=0$, $a_0=1$, $b_{-1}=1/2$, $b_0=1/2$ 。在上述情况下, $\pi(r)$ 只有一个根等于 1, 因此所有的这些方法是都零稳定的。

下列性质称为 Lax-Ritchmyer 等式定理, 是数值方法理论中非常重要的一个定理(参见[IK66]), 为了说明零稳定性的电要作用, 下面特别给出这一定理。

当且仅当算法为零稳定时, 一致的算法才是收敛的

参见练习 7.4~7.5。

7.5 无边界区间上的稳定性

在前几节中, 考察了有边界区间的柯西问题的解。在那些节中, 子区间 N_h 的数目取决于 h 且在 h 趋于零时, 趋于无穷大。

但另一方面, 有时希望在非常大(虚无穷)的时间区间对柯西问题积分。在这种情况下, 即使 h 是固定的, N_h 也趋向于无穷大, 则当不等式的右边含有一个无边界的量时, 则结果(7.11)将不再有意义。所以需要寻找一种数值方法, 使得对于任意长的时间区间, 即使是单步的大小相对“非常大”时, 仍然能够逼近解。

但实现方法比较简单的前向欧拉法并不具有这种性质。为了说明这一点, 考虑下列模型问题

$$\begin{cases} y'(t) = \lambda y(t), & t \in (0, \infty) \\ y(0) = 1 \end{cases} \quad (7.19)$$

其中, λ 是负实数。上式的精确解是 $y(t) = e^{\lambda t}$, 当 t 趋于无穷大时, 它趋向于零。对式(7.19)使用前向欧拉法, 可以看到

$$u_0 = 1, \quad u_{n+1} = u_n(1 + \lambda h) = (1 + \lambda h)^{n+1}, \quad n \geq 0 \quad (7.20)$$

则当且仅当

$$-1 < 1 + h\lambda < 1, \quad \text{即} \quad h < 2/|\lambda| \quad (7.21)$$

时, $\lim_{n \rightarrow \infty} u_n = 0$ 。

上述结论给出了实现下列结果所需的条件, 即对于固定的 h , 当 t_n 趋向于无穷大时, 数值方法应该能够重新产生精确解。如果 $h > 2/|\lambda|$, 则 $\lim_{n \rightarrow \infty} |u_n| = +\infty$; 这样式(7.21)是一个稳定的条件。性质 $\lim_{n \rightarrow \infty} u_n = 0$ 叫做绝对稳定。

例 7.3 使用前向欧拉法来求解问题(7.19), 其中 $\lambda = -1$ 。此时要保证绝对稳定, 必须使 $h < 2$ 。在图 7.3 中, 表示出了在区间 $[0, 30]$ 中, 用三个不同的 h 值获得的解, 三个值分别为 $h = 30/14 (> 2)$, $h = 30/16 (< 2)$ 及 $h = 1/2$ 。可以看出在前两种情况下, 数值解是振荡的。而且只有在第一种情况下(此时不满足稳定条件), 数值解的绝对值在无穷远处不趋于零(实际上它是发散的)。

在(7.19)中, 当 λ 是关于 t 的负函数时, 也存在类似的结论。但此时在稳定性的条件中, $|\lambda|$ 必须用 $\max_{t \in [0, \infty]} |\lambda(t)|$ 代替。但这一条件也可放宽到并不十分严格的可变步长 h_n 的情况, 可变步长 h_n 描述的是 $|\lambda(t)|$ 在每个区间 (t_n, t_{n+1}) 的局部特性。

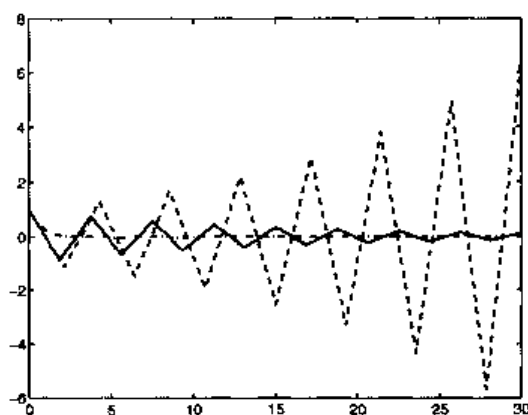


图 7.3 利用前向欧拉法求得的问题(7.19)的解, 其中 $\lambda = -1$, 其中的曲线分别对应于 $h = 30/14 (> 2)$ (虚线), $h = 30/16 (< 2)$ (实线) 及 $h = 1/2$ (点划线)

特殊情况下, 可以使用下列自适应前向欧拉法:

选择 $u_0=y_0$, $h_0=2\alpha/|\lambda(t_0)|$; 则

对于 $n=0, 1, \dots$, 计算

$$\begin{aligned} t_{n+1} &= t_n + h_n \\ u_{n+1} &= u_n + h_n \lambda(t_n) u_n \\ h_{n+1} &= 2\alpha / |\lambda(t_{n+1})| \end{aligned} \quad (7.22)$$

其中, α 为一个常数, 为了保证算法的绝对稳定, α 必须小于 1。

例如, 考虑下列问题

$$y'(t) = -(10e^{-t} + 1)y(t), \quad t \in (0, 10)$$

设 $y(0)=1$ 。由于 $|\lambda(t)|$ 是下降函数, 对于前向欧拉法, 保证绝对稳定的最严格条件是 $h < h_0 = 2/|\lambda(0)| = 2/11$ 。在图 7.4 的左图中比较了在三个不同的 α 值下, 前向欧拉法和自适应前向欧拉法计算结果之间的比较。注意, 尽管只需 $\alpha < 1$ 就可以确保算法是稳定的, 但是为了得到更精确的解需要选择足够小的 α 。在图 7.4 的右图中, 描绘了 h_n 对应于相同 α 值的变化曲线。从图中可见序列 $\{h_n\}$ 随着 n 单调增长。

与前向欧拉法相比, 后向欧拉法和 Crank-Nicolson 法的绝对稳定都不需要对 h 进行限制。实际上, 由后向欧拉法可得 $u_{n+1} = u_n + \lambda h u_{n+1}$, 于是有

$$u_{n+1} = \left(\frac{1}{1 - \lambda h} \right)^{n+1}$$

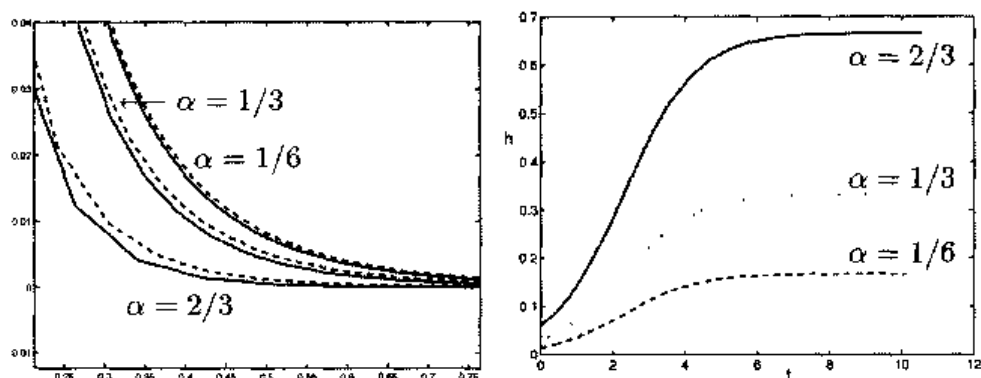


图 7.4 左侧: 在区间 $(0.2, 0.8)$ 上, 对于三个不同 α 值, 设 $h=ah_0$ 时, 利用前向欧拉法(虚线)与自适应可变步长前向欧拉法(7.22)(实线)得到的数值解。右侧: 适应法(7.22)中可变步长 h 的变化

对所有的 $h>0$, 当 $n \rightarrow \infty$ 时, 上式趋向于零。类似地, 由 Crank-Nicolson 法可得

$$u_{n+1} = \left[\left(1 + \frac{h\lambda}{2} \right) / \left(1 - \frac{h\lambda}{2} \right) \right]^{n+1}$$

对于所有可能的 $h > 0$, 当 $n \rightarrow \infty$ 时, 上式趋向于零。可得出结论, 前向欧拉法是条件绝对稳定的, 而后向欧拉法和 Crank-Nicolson 法都是无条件绝对稳定的。无条件绝对稳定的方法对于式(7.19)中有负实部的所有复数 λ 也称作 A 稳定。欧拉法和 Crank-Nicolson 法都是 A 稳定的。需要指出的是只有隐式解才能是 A 稳定的。这种性质使隐式法更具实用性, 尽管在计算上隐式法仍要比显示法花费更大的代价。

绝对稳定控制扰动

考虑下列一般模型问题

$$\begin{cases} y'(t) = \lambda(t)y(t) + r(t), & t \in (0, +\infty) \\ y(0) = 1 \end{cases} \quad (7.23)$$

这里, λ 和 r 是两个连续的函数, $-\lambda_{\max} \leq \lambda(t) \leq -\lambda_{\min}$, 其中, $0 < \lambda_{\min} \leq \lambda_{\max} < +\infty$ 。在这种情况下, 在 t 趋向于无穷时精确解并不一定是趋向于零的; 例如如果 r 和 λ 均为常数, 则有

$$y(t) = \left(1 + \frac{r}{\lambda} \right) e^{\lambda t} - \frac{r}{\lambda}$$

当 t 趋向于无穷时, 上式的极限是 $-r/\lambda$ 。于是总的说来, 求解问题(7.23)时, 便不再需要一种绝对稳定的数值方法。下面将要讨论的一种数值方法对于(7.19)类的模型问题是绝对稳定的, 而对于形式为(7.23)的一般化问题, 则保证当 t 趋向于无穷时(时间步长 h 的选择满足适当的约束条件), 扰动是在控制范围之内的。

为简化起见, 下面只对前向欧拉法进行分析; 利用该方法求解(7.23)时, 该式变为

$$\begin{cases} u_{n+1} = u_n + h(\lambda_n u_n + r_n), & n \geq 0 \\ u_0 = 1 \end{cases}$$

且它的解是(参见习题 7.10)

$$u_n = u_0 \prod_{k=0}^{n-1} (1 + h\lambda_k) + h \sum_{k=0}^{n-1} r_k \prod_{j=k+1}^{n-1} (1 + h\lambda_j) \quad (7.24)$$

这里, $\lambda_k = \lambda(t_k)$ 且 $r_k = r(t_k)$ 。考虑下列“扰动”方法

$$\begin{cases} z_{n+1} = z_n + h(\lambda_n z_n + r_n + \rho_{n+1}), & n \geq 0 \\ z_0 = u_0 + \rho_0 \end{cases} \quad (7.25)$$

这里, $\rho_0, \rho_1, \dots$ 是在每个给定时间步长内引入的扰动。这是一个简单的模型, 其中 ρ_0 和 ρ_{n+1} 分别考虑到了 u_0 和 r_n 都不能精确确定的问题(如果考虑到在每一步引入的舍入误差, 那么扰动模型就会变得非常棘手, 很难分析)。如果将模型看作用 z_k 代替了 u_k , 用 $r_k + \rho_{k+1}$ 代替了 r_k , 对于所有的 $k=0, \dots, n-1$, 则(7.25)的解与(7.24)的解将是十分相似的, 此时有

$$z_n - u_n = \rho_0 \prod_{k=0}^{n-1} (1 + h\lambda_k) + h \sum_{k=0}^{n-1} \rho_{k+1} \prod_{j=k+1}^{n-1} (1 + h\lambda_j) \quad (7.26)$$

值得注意的是, 这个量值并不取决于函数 $r(t)$ 。

i. 为说明原因, 考虑第一种特殊的情况, 设 λ_k 和 ρ_k 分别等于常数 λ 和 ρ 。假设 $h < h_0(\lambda) = 2/|\lambda|$, 该条件是为了保证利用前向欧拉法求解问题(7.19)时能够绝对稳定而设定的。则利用下列几何平均恒等式

$$\sum_{k=0}^{n-1} a^k = \frac{1-a^n}{1-a}, \quad \text{如果 } |a| < 1 \quad (7.27)$$

可得

$$z_n - u_n = \rho \left\{ (1 + h\lambda)^n \left(1 + \frac{1}{\lambda} \right) - \frac{1}{\lambda} \right\} \quad (7.28)$$

扰动误差 $|z_n - u_n|$ 应该满足(参见习题 7.11)

$$|z_n - u_n| \leq \varphi(\lambda) |\rho| \quad (7.29)$$

如 $\lambda \leq -1$, 则 $\varphi(\lambda) = 1$, 而如果 $-1 \leq \lambda < 0$, 则 $\varphi(\lambda) = |1 + 2/\lambda|$, 并且

$$\lim_{n \rightarrow \infty} |z_n - u_n| = \frac{\rho}{|\lambda|}$$

例如, 图 7.5 表示了 $\rho = 0.1$, $\lambda = -2$ (左侧)和 $\lambda = -0.5$ (右侧)相应的图形。在两种情况下, 均设 $h = h_0(\lambda) - 0.01$ 。此时得到的结论是扰动误差的边界值是 $|\rho|$ 乘以一个与 n 和 h 无关的常数。显然, 如果不满足稳定极限 $h < h_0$, 则当 n 增长时扰动误差会出现溢出。

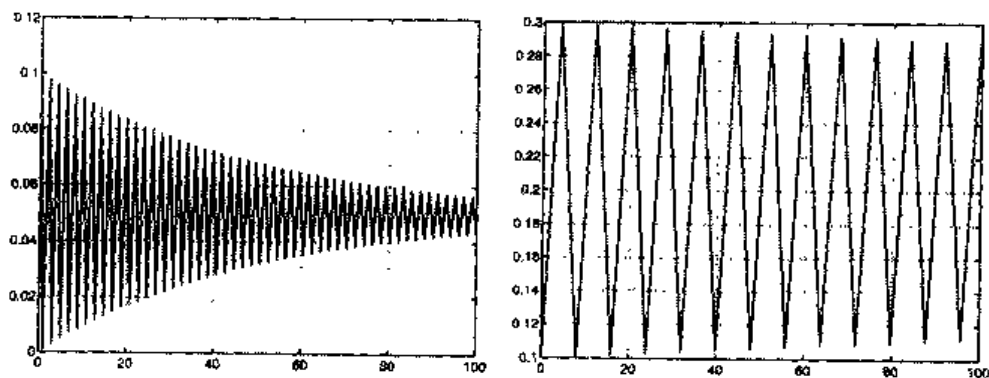


图 7.5 当 $\rho=0.1$, $\lambda=-2$ (左侧)和 $\lambda=-0.5$ (右侧)时的扰动误差。两种情况下均有 $h=h_0(\lambda)\approx 0.01$

ii. 一般情况下, 当 λ 和 r 不是常数时, 需要 h 满足限制条件 $h < h_0(\lambda)$, 其中 $h_0(\lambda)=2/\lambda_{\max}$ 。则

$$|1+h\lambda_k| \leq a = a(h, \lambda_{\min}, \lambda_{\max}) = \max\{|1-h\lambda_{\min}|, |1-h\lambda_{\max}|\}$$

由于 $a < 1$, 仍能在(7.26)中使用恒等式(7.27), 可得(参见习题 7.11)

$$|z_n - u_n| \leq \rho_{\max} \left(a^n + h \frac{1 - \delta(h)^n}{1 - \delta(h)} \right) \quad (7.30)$$

这里, $\rho_{\max} = \max|\rho_k|$, 如果 $h \leq h^*$, 则 $\delta(h) = |1 - h\lambda_{\min}|$, 如果 $h^* \leq h \leq 2/h_0(\lambda)$, 则 $\delta(h) = |1 - h\lambda_{\max}|$, 这里已经设 $h^* = 2/(\lambda_{\min} + \lambda_{\max})$ 。当 $h \leq h^*$ 时, 则得

$$|z_n - u_n| \leq \rho_{\max} (a^n + 1/\lambda_{\min}) \quad (7.31)$$

尤其是:

$$\lim_{n \rightarrow \infty} |z_n - u_n| \leq \rho_{\max} / \lambda_{\min} \quad (7.32)$$

从上式, 仍可得出结论, 即扰动误差的边界是 $\rho_{\max}$ 乘以一个与 n 和 h 无关的常数(尽管此时振荡并不像前一种情况那样是衰减的)。

实际上, 当 $h^* \leq h \leq 2/h_0(\lambda)$ 时可以得到相似的结论, 此时的结论并不是根据上边界条件(7.13)得到的, 因为此时的上边界条件并不适用于此时的情况。在图 7.6 中, 描述了求解问题(7.23)得到的扰动误差, 这里 $\lambda(t) = -2 - \sin(t)$, $\rho_k = \rho(t_k)$, $\rho(t) = 0.1 \sin(t)$, 其中 $h < h^*$ (左侧)和 $h^* \leq h < h_0(\lambda)$ (右侧)。

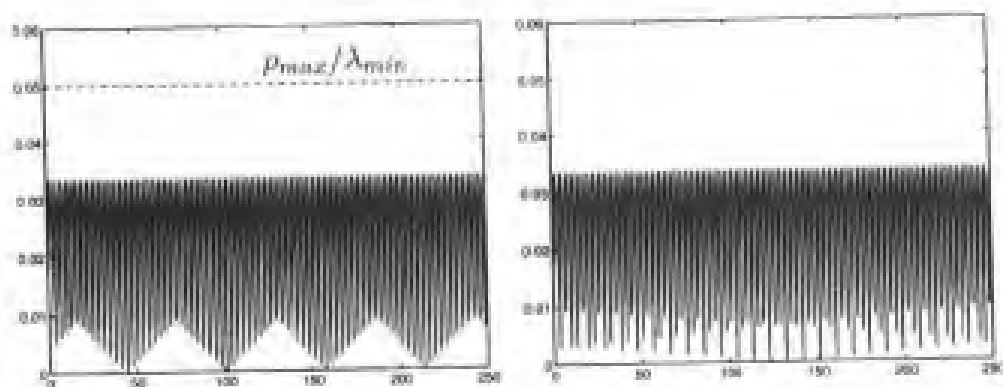


图 7.6 当 $\rho(t)=0.1\sin(t)$, $\lambda(t)=-2-\sin(t)$ 时的扰动误差, 其中步长分别为 $h=h^*-0.01=0.39$ (左侧)和 $h=h^*+0.01$ (右侧)

iii. 现在考虑一般柯西问题(7.4)。这个问题与一般模型问题(7.23)相关, 在如下的条件下, 对于合适的 $\lambda_{\min}$ 值, $\lambda_{\max} \in (0, +\infty)$:

$$-\lambda_{\max} < \partial f / \partial y(t, y) < -\lambda_{\min} \quad \forall t \geq 0, \quad \forall y \in (-\infty, \infty)$$

对于一般区间 (t_n, t_{n+1}) 内的每个 t , 从式(7.16)中减去式(7.5)可得到下列关于扰动误差的方程:

$$z_n - u_n = (z_{n-1} - u_{n-1}) + h \{ f(t_{n-1}, z_{n-1}) - f(t_{n-1}, u_{n-1}) \} + h \rho_n$$

应用平均值理论可得

$$f(t_{n-1}, z_{n-1}) - f(t_{n-1}, u_{n-1}) = \lambda_{n-1}(z_{n-1} - u_{n-1})$$

这里, $\lambda_{n-1} = f_y(t_{n-1}, \xi_{n-1})$, 并且定义 $f_y = \partial f / \partial y$, ξ_{n-1} 是区间内的一个合适的点, 区间端点为 u_{n-1} 和 z_{n-1} 。则

$$z_n - u_n = (1 + h\lambda_{n-1})(z_{n-1} - u_{n-1}) + h\rho_n$$

通过这个公式的递归使用, 可得恒等式(7.26), 从中可得出与 ii. 相同的结论, 假设此时满足稳定性限制条件 $0 < h < 2/\lambda_{\max}$ 。

例 7.4 考虑柯西问题

$$y'(t) = \arctan(3y) - 3y + t, \quad t > 0 \quad (7.33)$$

设 $y(0)=1$ 。由于 $f_y=3/(1+9y^2)-3$ 是负的, 可选择 $\lambda_{\max}=\max|f_y|=3$, 设 $h<2/3$ 。于是因为 $h<2/3$, 可以保证前向欧拉法的扰动在控制范围之内。图 7.7 所述的结果可证实这一点。注意在这个例子中, 设 $h=2/3+0.01$ (这样便不再满足前面的稳定条件), 则扰

动误差随着 t 的增长将会溢出。

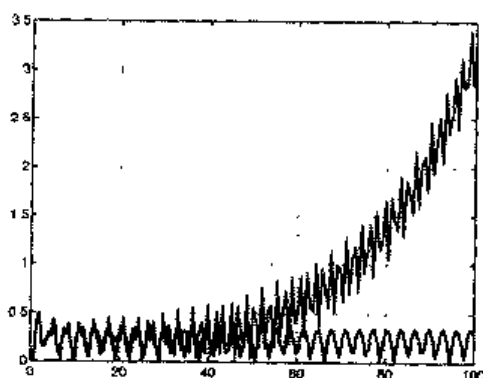


图 7.7 当 $\rho(t)=\sin(t)$ 时, 求解柯西问题(7.33)的扰动误差曲线, 分别取 $h=2\lambda_{\max}-0.01$ (粗线)和 $h=2\lambda_{\max}+0.01$ (细线)

例 7.5 找出关于 h 的极限, 以保证利用前向欧拉法求解下列柯西问题时是稳定的

$$y' = 1 - y^2, \quad t > 0 \quad (7.34)$$

设 $y(0) = \frac{e-1}{e+1}$ 。其精确解是 $y(t) = (e^{2t+1} - 1) / (e^{2t+1} + 1)$ 且 $f_y = -2y$ 。由于对所有 $t > 0$ 有

$f_y \in (-2, -0.9)$, 则可使 h 小于 $h_0 = 1$ 。在图 7.8 的左侧, 描述了在区间 $(0, 35)$, 分别取 $h=0.95$ (粗线)和 $h=1.05$ (细线)时得到的解。两种情况下的解都是振荡的, 但也都是有界的。而且由于第一种情况满足稳定条件, 随着 t 的增加, 振荡是衰减的, 从而数值解趋向于精确解。在图 7.8 的右侧, 描述了 $\rho(t)=\sin(t)$ 的相应误差, 此时分别取 $h=0.95$ (粗线)和 $h=h^*+0.1$ (细线)。两种情况下, 扰动误差均是有界的; 而且在前一种情况下满足上界(7.32)。

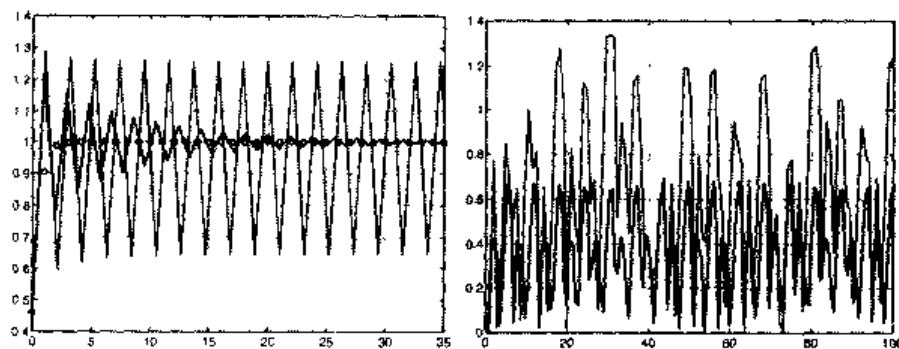


图 7.8 左侧: 利用前向欧拉法求解问题(7.34)得到的数值解, 其中 $h=20/19$ (细线)和 $h=20/21$ (粗线)。精确解的数值用圆圈表示。右侧: 求解 $\rho(t)=\sin(t)$ 时相应的扰动误差, 其中 $h=0.95$ (粗线)和 $h=h^*$ (细线)

在上述情况下, 如果不知道 y 的信息, 则很难找出 $\lambda_{\max} = \max |f_y|$ 的值。此时可以利用可变步长算法来实现。具体说来, 可以设 $t_{n+1} = t_n + h_n$, 这里

$$h_n < 2 \frac{\alpha}{|f_y(t_n, u_n)|}$$

其中, α 为适当的严格小于 1 的值。注意分母与 u_n 的值有关(该值是已知)。图 7.9 中描述了例 7.5 中的扰动误差, 图中曲线分别与两个不同 α 值相对应

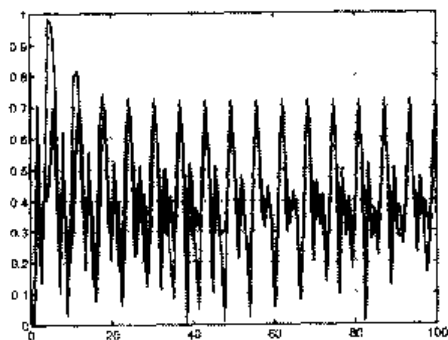


图 7.9 用自适应方法求解例 7.5 时的扰动误差, 此时 $\rho(t) = \sin(t)$, 分别取 $\alpha=0.8$ (粗线) 和 $\alpha=0.9$ (细线)

注 7.4 前述的分析也可应用到其他种类的单步方法上, 尤其是对于后向欧拉法和 Crank-Nicolson 法。对于这些 A 稳定的方法, 并不需要给出关于时间步长的限制条件, 就可以得出相同的关于扰动误差的结论。实际上, 在前边的分析中, 利用后向欧拉法时, 只需用 $(1 - h\lambda_k)^{-1}$ 来代替 $1 + h\lambda_k$; 当利用 Crank-Nicolson 法时, 只需用 $(1 + h\lambda_k/2)/(1 - h\lambda_k/2)$ 代替 $1 + h\lambda_k$ 即可。

小结

1. 利用绝对稳定算法求解模型问题(7.19)时, 可以得到解 u_n , 并且当 t_n 趋向于无穷大时, u_n 趋向于零;
2. 所谓 A 稳定的方法就是无论时间步长 h 怎样选取, 算法都是绝对稳定的(否则叫条件稳定, h 应小于一个由 λ 决定的常数);
3. 当利用绝对稳定算法求解一般化模型问题(如(7.23))时, 扰动误差(扰动解与非扰动解之差的绝对值)是一致有界的(相对于 h)。简而言之, 绝对稳定算法可以使扰动误差是可控的。
4. 线性模型问题的绝对稳定性, 可以通过寻找关于时间步长的稳定条件来分析, 在考虑非线性柯西问题(7.4)时是通过寻找满足 $\partial f / \partial y < 0$ 的函数 f 来实现的。在那种情况下, 稳定性条件要求选择一个 $\partial f / \partial y$ 的函数来作为步长。具体说来, 选择

积分区间 $[t_n, t_{n+1}]$ 的标准是： $h_n = t_{n+1} - t_n$ 满足条件 $h_n < 2/|\partial f(t_n, u_n)/\partial y|$ 。

参见习题 7.6~7.14。

7.6 高阶方法

到目前为止，所介绍的方法基本都是单步长法。利用更为复杂的方法可以得到阶数更高的精确性，如 Runge-Kutta 法和多步法。Runge-Kutta 法仍然是单步的方法；但这种方法在每一段区间 $[t_n, t_{n+1}]$ 内有多函数 $f(t, y)$ 的近似值。Runge-Kutta 法中最有效的一种是：

$$u_{n+1} = u_n + \frac{h}{6}(K_1 + 2K_2 + 2K_3 + K_4)$$

这里，

$$\begin{aligned} K_1 &= f_n \\ K_2 &= f\left(t_n + \frac{h}{2}, u_n + \frac{h}{2}K_1\right) \\ K_3 &= f\left(t_n + \frac{h}{2}, u_n + \frac{h}{2}K_2\right) \\ K_4 &= f(t_{n+1}, u_n + hK_3) \end{aligned}$$

该法是一种关于 h 的四阶显式方法；在每个时间步长都包含了对函数 f 的四种新的估计。其他的 Runge-Kutta 法，无论是显式的还是隐式的，都可以构成任意阶形式。

这些方法都基于一个 MATLAB 程序族，这些程序的名字是由字根 ode 加上数字或字母构成的。特别指出，ode45 是分别基于一对 4 和 5 阶的显式 Runge-Kutta 法(所谓的 Dormand-Prince 对)的程序。ode23 采用的算法是另一对显式 Runge-Kutta 法(Bogacki 和 Shampine 对)。在这些方法中，积分步长是变化的，以保证误差低于给定的容限(在这些程序中标量相对误差容限 RelTol 的默认设置为 10^{-3})。程序 ode23b 采用的是一种隐式 Runge-Kutta 公式，其中第一段采用的是梯形准则，第二段采用的是二阶后向差分公式(将在后面介绍)。

多步法(参见(7.17))由于在计算 u_{n+1} 时利用了 $u_n, u_{n-1}, \dots$ ，等多个数据的信息，所以可以获得更高阶的精度。举例来说，对于柯西问题可以应用积分基本定理(在 1.4.3 节中曾经介绍过)来实现该算法，即

$$y_{n+1} = y_n + \int_{t_n}^{t_{n+1}} f(t, y(t)) dt, \quad (7.35)$$

再通过求积公式计算积分, 该公式中利用了函数 f 在积分区域内适当点处的内插值。多步法中, 值得介绍的一种是三步三阶(显式)Adams-Bashforth 公式(AB3)

$$u_{n+1} = u_n + \frac{h}{12}(23f_n - 16f_{n-1} + 5f_{n-2}) \quad (7.36)$$

上式是通过在(7.35)中利用节点 t_{n-2} 、 t_{n-1} 、 t_n 处的内插 2 次多项式来替代 f 而得到的。另一个重要的例子是三步, 四阶(隐式)Adams-Moulton 公式(AM4)

$$u_{n+1} = u_n + \frac{h}{24}(9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}) \quad (7.37)$$

上式是将式(7.35)中的 f 替换为节点 t_{n-2} 、 t_{n-1} 、 t_n 、 t_{n+1} 的内插三次多项式而得到的。

多步法的另一个分支实现方法是: 写出在时间 t_{n+1} 的微分方程, 并且用高阶的单侧增长率来替代 $y'(t_{n+1})$ 。下列公式给出了这种方法的一个实例

$$u_{n+1} = \frac{18}{11}u_n - \frac{9}{11}u_{n-1} + \frac{2}{11}u_{n-2} + \frac{6h}{11}f_{n+1} \quad (7.38)$$

上式称为三步, 三阶(隐式)后向差分公式(BDF3)。

所有这些方法都是一致和零稳定的。事实上, (7.36)和(7.37)的第一特征多项式都是 $\pi(r) = r^3 - r^2$, 并且根都是 $r_0 = 1$, $r_1 = r_2 = 0$, 而(7.38)的第一特征多项式是 $\pi(r) = r^3 - 18/11r^2 + 9/11r - 2/11$, 它的根是 $r_0 = 1$, $r_1 = 0.3182 + 0.2539i$, $r_2 = 0.3182 - 0.2539i$, 这里 i 是虚部单位。在所有情况下, 都满足根的条件(7.18)。

而且, 当应用到模型问题(7.19)时, 如果 $h < 0.54/|\lambda|$, AB3 是绝对稳定的, 而如果 $h < 3/|\lambda|$, AM4 是绝对稳定的。方法 BDF3 对所有负实数 λ 是无条件绝对稳定的。但是, 当 $\lambda \in C$ 的实部为负时, 上述命题不再成立。换言之, BDF3 不是 A 稳定的。更一般地说, 当严格大于二阶时, 不存在 A 稳定的多步法。

在许多 MATLAB 程序中都有多步法的实现, 例如函数 `ode15s`。

7.7 预测纠正法

在 7.2 节中已经指出对于未知值的 u_{n+1} , 隐式方法在每一步都需要求解一个非线性问题。求解方法可以使用第 2 章介绍的方法之一, 也可以利用函数 `fzero`, 正如

程序 13 和 14 中所做的那样。

另外，可以在每一个时间段内进行固定点迭代。例如，对于 Crank-Nicolson 法 (7.13)， $k=0, 1, \dots$ ，计算下式直到收敛为止

$$u_{n+1}^{(k+1)} = u_n + \frac{h}{2} \left[f_n + f(t_{n+1}, u_{n+1}^{(k)}) \right]$$

可证明，如果初始猜测值 $u_{n+1}^{(0)}$ 选择合适，迭代一次就可得到 $u_{n+1}^{(1)}$ 的数值解，它的精度与原始隐式方法得到的解 u_{n+1} 的精度同阶。更精确地说，如果原始隐式方法是 p 阶的，那么初始猜测 $u_{n+1}^{(0)}$ 必定由一个(至少) $p-1$ 阶的显式方法产生。

例如，如果使用一阶(显式)前向欧拉法来初始化 Crank-Nicolson 法，得到 Heun 法(也叫作改进欧拉法)，它是二阶显式 Runge-Kutta 法：

$$\begin{cases} u_{n+1}^* = u_n + hf_n \\ u_{n+1} = u_n + \frac{h}{2} \left[f_n + f(t_{n+1}, u_{n+1}^*) \right] \end{cases} \quad (7.39)$$

显式的步骤叫做预测，而隐式的步骤叫做纠正。另一个例子综合了(AB3)法(7.36)(具有预测功能)和(AM4)法(7.37)(具有纠正功能)二者的特点，因此该方法称为预测纠正法。它们的精确度与纠正法相同。但由于是显式形式，所以它们也必须满足与预测法相同的关于稳定性的限制条件。因此，它们就不能够在一个无限区域里对一个柯西问题进行积分。

程序 15 是通常预测纠正法的一个实现。字符串 predictor 和 corrector 确定所选方法的类型。例如，如果使用程序 16 中所定义的函数 eeonestep 和 cnonestep，可以按如下方法调用 predcor 函数并得到 Heun 方法：

```
>>[t,u]=predcor(t0,y0,T,N,f,'eeonestep','cnonestep');
```

程序 15—predcor: 预测纠正法

```
function [t,u]=predcor(odefun,tspan,y,Nh,pred,corr,varargin)
%PREDCOR Solve differential equations using a predictor-corrector method
% [T,Y]=PREDCOR(ODEFUN,TSPAN,Y0,NH,PRED,CORR)with TSPAN=
% [TO TFINAL] integrates the system of differential equations y' = f(t,y)
% from time To to TFINAL with initial conditions Y0 using a general predictor
% corrector
% method on an equispaced grid of NH intervals. Function ODEFUN(T,Y)
% must return a column vector corresponding to f(t,y). Each row in the
% solution array Y corresponds to a time returned in the column vector T.
% Functions PRED and CORR identify the type of method that is chosen.
% [T,Y]= PREDCOR(ODEFUN,TSPAN,Y0,NH,PRED,CORR,P1,P2,...) passes
% the additional parameters P1,P2,... to the functions ODEFUN, PRED and
% CORR as ODEFUN(T,Y,P1,P2,...), PRED(T,Y,P1,P2,...), CORR(T,Y,P1,P2,...)
```

```

h=(tspan(2)-tspan(1))/Nh; tt= [tspan(1):h:tspan(2)];
u=y; [n,m]=size(u); if n ~= m, u=u'; end
for t=tt(1:end-1)
    y = u(:,end); fn = feval(odefun,t,y,varargin{:});
    upre= feval(predictor,t,y,h,fn);
    ucor = feval(corrector,t+h,y,upre,h,odefun,fn,varargin{:});
    u=[u,ucor],
end
t = tt;

```

程序 16—onestep: 前向欧拉的一步运算(eeonestep), 后向欧拉法的一步运算(eionestep), Crank-Nicolson 的一步运算(cnonestep)

```

function [u]=feonestep(t,y, h,f)
u = y + h*f;
return

function [u]=beonestep(t, u,y,h,f, fn,varargin)
u = u + h*feval(f,t,y,varargin{:});
return

function [u]=cnonestep(t,u,y,h,f, fn,varargin)
u = u + 0.5*h*(feval(f,t,y,varargin{:})+fn);
return

```

MATLAB 程序 ode113 用可变步长实现了一个 Adams Moulton Bashforth 方法。参见习题 7.15~7.18。

7.8 微分方程系统

考虑下列一阶常微分方程系统, 其未知量为 $y_1(t), \dots, y_m(t)$:

$$\begin{aligned}
 y_1' &= f_1(t, y_1, \dots, y_m) \\
 &\vdots \\
 y_m' &= f_m(t, y_1, \dots, y_m)
 \end{aligned}$$

这里, $t \in (t_0, T]$, 初始条件

$$y_1(t_0) = y_{0,1}, \dots, y_m(t_0) = y_{0,m}$$

求解该系统可以对每个单个的方程使用先前介绍过的应用于标量问题的求解方法中的一种。例如, 前向欧拉法的第 n 步应是

$$\begin{cases} u_{n+1,1} = u_{n,1} + hf_1(t_n, u_{n,1}, \dots, u_{n,m}) \\ \vdots \\ u_{n+1,m} = u_{n,m} + hf_m(t_n, u_{n,1}, \dots, u_{n,m}) \end{cases}$$

将符号用简化形式表示, 以向量 $y'(t)=F(t, y(t))$ 的形式写出该系统, 这样可以很自然地将该算法从单个方程推广到向量运算中。例如, 方法

$$u_{n+1} = \vartheta F(t_{n+1}, u_{n+1}) + (1-\vartheta)F(t_n, u_n), \quad n \geq 0$$

使 $u_0=y_0$, $0 \leq \vartheta \leq 1$, 如果 $\vartheta=0$, 则是前向欧拉法的向量形式; 如果 $\vartheta=1$, 则是后向欧拉法的向量形式; 如果 $\vartheta=1/2$, 则是 Crank-Nicolson 法的向量形式。

例 7.6 使用前向欧拉法解 Lotka-Volterra 方程(7.3), 设 $C_1=C_2=1$, $b_1=b_2=0$ 且 $d_1=d_2=1$ 。为了利用程序 12 来求解一个常微分方程系统, 构造一个函数 f , 使其包含向量函数 F 中的元素, 将其保存为文件 $f.m$ 。则对于该系统有:

```
C1=1;C2=1;d1=1;d2=1;b1=0;b2=0;
yy(1)=C1*y(1)*(1-b1*y(1)-d2*y(2)); %first equation
y(2)=-C2*y(2)*(1-b2*y(2)-d1*y(1)); %second equation
y(1)=yy(1);
```

现在该用下列指令来执行程序 12

```
[t,u]=feuler('f',[0,10],[2 2],2000);
plot(t,u);figure(2);plot(u(:,1),u(:,2));
```

它们对应于时间步长为 $h=0.005$, 时间区间为 $[0, 10]$ 时, Lotka-Volterra 系统问题的求解。

图 7.10 的曲线, 左侧, 表现了解的两个元素随时间的变化。注意它们是以 2π 为周期的。图 7.10 的第二个图, 右侧, 给出了所谓的由相平面(即笛卡尔平面, 其坐标轴为 y_1 和 y_2)上的初始值而展开的轨迹。轨迹限制在 (y_1, y_2) 平面的有界区域内。如果从点 $(1.2, 1.2)$ 开始, 轨迹将会在围绕点 $(1, 1)$ 的一个更小的区域。这种现象可以解释如下, 微分系统允许在 $y_1'=0$ 和 $y_2'=0$ 处是两点平衡的, 其中一点是 $(1, 1)$ (另一个是 $(0, 0)$)。实际上, 它们是通过求解下列非线性系统而得到的:

$$\begin{aligned} y_1' &= y_1 - y_1 y_2 = 0 \\ y_2' &= -y_2 + y_2 y_1 = 0 \end{aligned}$$

如果初始数据与其中的某个点中是一致的, 则解在时间上是一个常数。并且 $(0, 0)$ 是一个不稳定平衡点, $(1, 1)$ 是稳定平衡点, 即所有从靠近 $(1, 1)$ 的一个点展开的轨

迹在相平面上均是有界的。

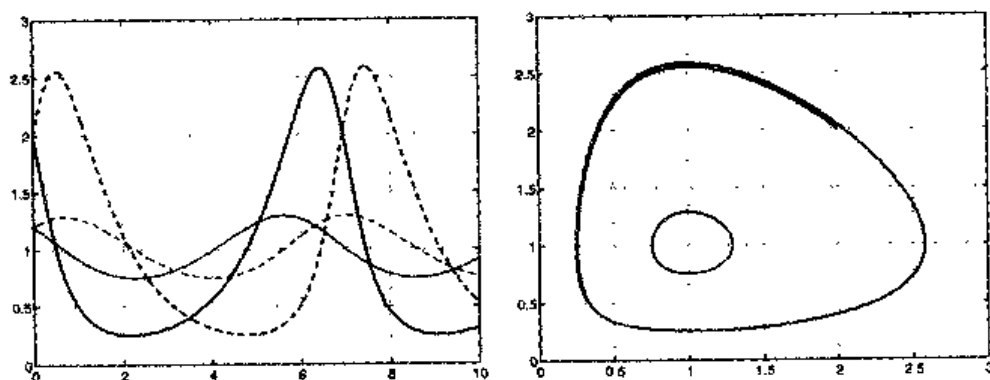


图 7.10 系统(7.3)的数值解。左侧表示在时间区间(0, 10)内 y_1 和 y_2 的关系, 实线表示 y_1 , 虚线表示 y_2 。其中分别选取了两个不同初始值: (2, 2)(粗线)和(1.2, 1.2)(细线)。右侧给出了相平面上相应的轨迹

当用显式的方法时, 步长 h 应该满足与 7.5 节中类似的稳定条件限制。在下列情况下, 即当 F 的雅可比行列式 $A(t) = [\partial F / \partial y](t, y)$ 的特征值 λ_k 的实部均为负值时, 可设 $\lambda = -\max_k \rho(A(t))$, 这里 $\rho(A(t))$ 是 $A(t)$ 的谱半径, 然后用这个 λ 值替换出现在标量柯西问题稳定条件中的 λ (参见式(7.2))。

注 7.5 前面提到一些 MATLAB 程序(ode23, ode45, ...)也可以用来求解常微分方程系统。应用形式是 `odeXX('f', [t0 tf], y0)`, 这里 y_0 是初始条件的向量, f 是由用户指定的函数, `odeXX` 是 MATLAB 提供的方法。

现在考虑 m 阶常微分方程的情形

$$y^{(m)}(t) = f(t, y, y', \dots, y^{(m-1)}) \quad (7.40)$$

其中, $t \in (t_0, T)$, 解的形式 (如果存在) 为一族函数, 其中最多含有 m 个任意常数。它们可以通过给出 m 初始条件来确定

$$y(t_0) = y_0, \quad y'(t_0) = y_1, \quad \dots, \quad y^{(m-1)}(t_0) = y_m$$

设

$$w_1(t) = y(t), \quad w_2(t) = y'(t), \quad \dots, \quad w_m(t) = y^{(m-1)}(t)$$

方程(7.40)可变形为 m 个一阶微分方程构成的系统

$$\begin{aligned}
 w'_1 &= w_2 \\
 w'_2 &= w_3 \\
 &\vdots \\
 w'_{m-1} &= w_m \\
 w'_m &= f(t, w_1, \dots, w_m)
 \end{aligned}$$

初始条件

$$w_1(t_0) = y_0, \quad w_2(t_0) = y_1, \quad \dots, w_m(t_0) = y_{m-1}$$

这样可以将求解一个 $m > 1$ 阶微分方程转化为求解 m 个一阶方程等价问题，然后再对这个系统进行离散化。

例 7.7 考虑问题 7.3 的电路，设 $L(i_1) = L$ 是常数， $R_1 = R_2 = R$ 。此时可通过求解下列两个微分方程系统而得到 v 的值：

$$\begin{cases} v' = w \\ w' = -\frac{L + RC^2}{LCR}w - \frac{2}{LC}v + \frac{e}{LC} \end{cases} \quad (7.41)$$

初始值为 $v(0) = 0$ ， $w(0) = 0$ 。上述系统是由下列二阶微分方程得到的

$$LC \frac{d^2 v}{dt^2} + \left(\frac{L}{R^2} + R_1 C \right) \frac{dv}{dt} + \left(\frac{R_1}{R_2} + 1 \right) v = e$$

设 $L = 0.1 \text{ H}$ ， $C = 10^{-3} \text{ F}$ ， $R = 10 \Omega$ 和 $e = 5 \text{ V}$ ，这里 H 、 F 、 Ω 和 V 分别是电感、电容、电阻和电压的单位。现在使用前向欧拉法，在时间区域 $[0, 0.1]$ 内，取步长 $h = 0.001$ 秒，通过程序 12：

```
>>[t,u]=feuler('fsys',[0,0.1],[0 0],1000);
```

这里 `fsy` 包含在文件 `fsy.m` 中

```
function y=fsys(t,y)
L=0.1; C=1.e-03;R=10;e=5;
yy(1)=y(2);
y(2)=- (L/R+R*C)/(L*C)*y(2)/(L*C)*y(1)+e/(L*C);
y(1)=yy(1);
```

在图 7.11 中，描述了 v 和 w 的近似值。正如所期望的那样，对于很大的 t ， $v(t)$ 趋向

于 $e/2=2.5\text{V}$ 。在这种情况下, $A(t)=[\partial F/\partial y](t, y)$ 的特征值的实部为负, 且 λ 可设成 -141.4214 。则绝对稳定的一个条件是使 $h < 2/|\lambda| = 0.0282$ 。

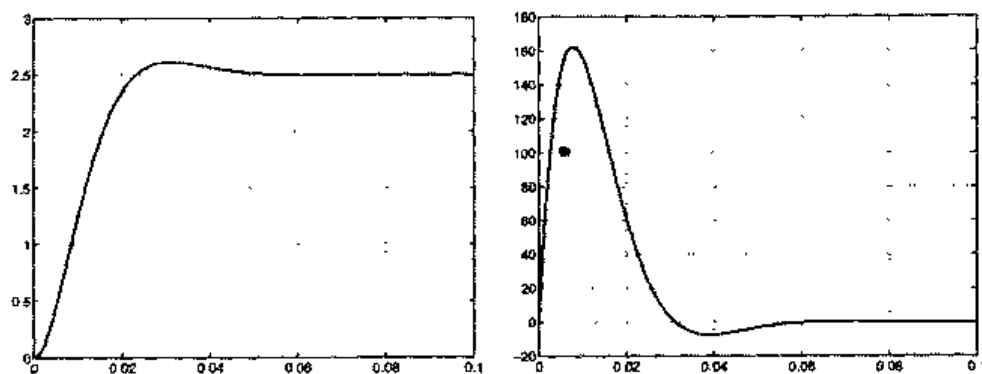


图 7.11 系统(7.41)的数值解。左侧是电压降 $v(t)$, 右侧是它的导数 w

参见练习 7.19~7.20。

7.9 补充说明

对于 Runge-Kutta 法这族算法的详细介绍可以参考 [But87], [Lam91] 和 [QSS00], 第 11 章。

对于多步法的进一步介绍可以参考 [Arn73] 和 [Lam91]。

前面没有讨论所谓的 stiff 问题, 即当其中 F 的雅克比行列式为病态时的与微分系统相关的问题。它们的数值解需要利用特别的隐式方法, 否则需要选取的时间步长将会很小, 以致难以实现(参见 [QSS00], 11 章, [Lam91] 和 [DB02])。MATLAB 中的程序 ode15s、ode23s 和 ode23tb 适合于求解 stiff 问题。例如考虑下列 Van Der Pol 问题

$$\begin{cases} y_1' = y_2 \\ y_2' = 1000(1 - y_1^2)y_2 - y_1 \\ y_1(0) = 2, \quad y_2(0) = 0 \end{cases} \quad (7.42)$$

在区间 $(0, 3000)$ 上, 如果使用程序 ode45, 所需的 CPU 运算时间将会很长, 因为此时为了保证稳定性需要选取极小的步长。而如果利用 ode15s, 则“只”需要经过 592 个时间步长(参见图 7.12)就可以得到一个精确的解。

注意采用了可变步长, 既保证了稳定性又保证了期望的精度。它的选择要基于具体情况下的误差的表达形式而定(参见 [Lam91]、[SR97] 和 [DB02])。

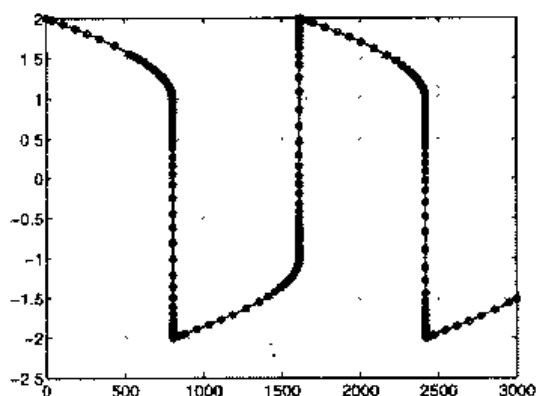


图 7.12 使用程序 `ode15s` 求出的系统(7.42)的解中的元素 y_1 。注意其中积分节点的分布是非一致的

7.10 习题

习题 7.1 使用后向欧拉法和前向欧拉法求解柯西问题

$$y' = \sin(t) + y, \quad t \in (0, 1], \quad \text{其中 } y(0) = 0 \quad (7.43)$$

并验证这两种方法都是一阶收敛的。

习题 7.2 考虑柯西问题

$$y' = -te^y, \quad t \in (0, 1], \quad \text{其中 } y(0) = 0 \quad (7.44)$$

使用前向欧拉法, 取 $h=1/100$, 估计在 $t=1$ 时的近似解的有效数字的位数(已知此时的精确解介于 -1 和 0 之间)。

习题 7.3 用后向欧拉法求解问题(7.44)时, 在每步运算中都需要求解非线性方程: $u_{n+1} = u_n - h t_{n+1} e^{-u_{n+1}} = \phi(u_{n+1})$ 。解 u_{n+1} 可通过固定点的迭代法来得到: 对于 $k=0, 1, \dots$, 计算 $u_{n+1}^{(k+1)} = \phi(u_{n+1}^{(k)})$ 的值, 其中 $u_{n+1}^{(0)} = u_n$ 。找出迭代收敛时关于 h 的限制条件。

习题 7.4 用 Crank-Nicolson 法重复计算习题 7.1。

习题 7.5 验证 Crank-Nicolson 法可由柯西问题(7.4)的下列积分形式推出

$$y(t) - y_0 = \int_0^t f(\tau, y(\tau)) d\tau$$

假设积分是利用梯形公式(4.17)来近似求解的。

习题 7.6 考虑问题(7.19), 其中 λ 是一个有负实部的复数。注意即使在这种情况下, 当 t 趋向于无穷时, $u(t)$ 仍趋向于 0。定义数值方法的绝对稳定域为使得该方法是绝对稳定的复平面上点的集合, 以 $z=h\lambda$ 表示。求出前向欧拉法的绝对稳定域。

习题 7.7 求解问题(7.19), 利用前向欧拉法, 取 $\lambda=-1+i$, 并确定使算法为绝对稳定时的 h 值。

习题 7.8 说明由(7.39)定义的 Heun 法是一致的。编写一个 MATLAB 程序求解柯西问题(7.43), 并且利用实验方法证明该法对于 h 是 2 阶收敛的。

习题 7.9 证明当 $-2 \leq h\lambda \leq 0$ 时, 其中 h 是负实数, Heun 法(7.39)是绝对稳定的。

习题 7.10 证明公式(7.24)。

习题 7.11 证明不等式(7.29)。

习题 7.12 证明不等式(7.30)。

习题 7.13 验证下列方法(3 阶 Runge-Kutta 法的显式形式)的一致性

$$\begin{aligned} u_{n+1} &= u_n + \frac{1}{6}(k_1 + 4k_2 + k_3), \quad k_1 = hf(t_n, u_n) \\ k_2 &= hf\left(t_n + \frac{h}{2}, u_n + \frac{k_1}{2}\right), \quad k_3 = hf(t_{n+1}, u_n + 2k_2 - k_1) \end{aligned} \quad (7.45)$$

编写 MATLAB 程序求解柯西问题(7.43), 并且用实验方法证明该法对于 h 是 3 阶收敛的。方法(7.39)和(7.45)都是基于 MATLAB 程序 ode23 来求解常微分方程的。

习题 7.14 证明如果 $-2.5 \leq h\lambda \leq 0$, 这里 h 是负实数, 则方法(7.45)是绝对稳定的。

习题 7.15 修正欧拉法定义如下:

$$u_{n+1}^* = u_n + hf(t_n, u_n), \quad u_{n+1} = u_n + hf(t_{n+1}, u_{n+1}^*) \quad (7.46)$$

说明当 h 满足什么条件时, 该算法是绝对稳定的。

习题 7.16 用 Crank-Nicolson 法和 Heun 法求解方程(7.1), 题目中的物体是边长为 1m, 质量为 1kg 的立方体。假设 $T_0=180\text{K}$, $T_c=200\text{K}$, $\gamma=0.5$, $C=100\text{J}/(\text{kg}/\text{K})$ 。求 t 从 0 变化到 200 秒时, 分别取 $h=20$ 和 $h=10$ 进行计算, 并比较这两种情况下得到的结果。

习题 7.17 利用 MATLAB 计算 Heun 法的绝对稳定域。

习题 7.18 用 Heun 法求解柯西问题(7.12)并确定它的阶数。

习题 7.19 一个由给定重量的物体和弹簧组成的振动系统, 其中弹簧的弹力与速度成比例, 位移量 $x(t)$ 满足二阶微分方程 $x'' + 5x' + 6x = 0$ 。利用 Heun 法来求解该系统, 设 $x(0)=1$ 和 $x'(0)=0$, $t \in [0, 5]$ 。

习题 7.20 无摩擦 Foucault 摆的运动可以由下列两个方程来描述:

$$x'' - 2\omega \sin(\Psi)y' + k^2x = 0, \quad y'' + 2\omega \cos(\Psi)x' + k^2y = 0$$

这里, Ψ 是摆所处位置的纬度, $\omega = 7.29 \times 10^{-5} \text{s}^{-1}$ 是地球自转的角速度, $k = \sqrt{g/l}$, $g = 9.8 \text{m/s}^2$, l 为摆长。利用前向欧拉法计算 $x=x(t)$ 和 $y=y(t)$, 其中 t 的范围为 0 到 300 秒, $\Psi = \pi/4$ 。

第 8 章 边值问题数值方法

边值问题是在一维区间 (a, b) 或多维开区域 $\Omega \in \mathbf{R}^d (d=2, 3)$ 上的微分问题, 它在区间的端点 a 和 b 或者在多维区域的边界 $\partial\Omega$ 上给定已知条件, 这些条件可能是函数本身或它的导数。

在多维情形下, 微分方程的已知条件还包括方程的精确解相对于空间各维坐标的偏微分。下面给出几个边值问题的例子:

1. Poisson 方程:

$$-u''(x) = f(x), \quad x \in (a, b) \quad (8.1)$$

或(在多维情况下)

$$-\Delta u(x) = f(x), \quad x \in \Omega \quad (8.2)$$

其中, f 是一个给定的函数, Δ 称为 Laplace 算子:

$$\Delta u = \sum_{i=1}^d \frac{\partial^2 u}{\partial x_i^2}$$

符号 $\partial \cdot / \partial x_i$ 表示对于变量 x_i 求偏微分, 对于每个点 x^0

$$\frac{\partial u}{\partial x_i}(x^0) = \lim_{h \rightarrow 0} \frac{u(x^0 + h e_i) - u(x^0)}{h} \quad (8.3)$$

其中, e_i 是 $\mathbf{R}^d$ 空间的第 i 个单位向量。

2. 热传导方程:

$$\frac{\partial u(x, t)}{\partial t} - \mu \frac{\partial^2 u(x, t)}{\partial x^2} = f(x, t), \quad x \in (a, b), \quad t > 0$$

或(在多维情况下)

$$\frac{\partial u(x, t)}{\partial t} - \mu \Delta u(x, t) = f(x, t), \quad x \in \Omega, \quad t > 0$$

其中, t 是时间变量, $\mu > 0$ 是一个给定的系数, 代表热导率, f 是一个给定的函数。

3. 波动方程:

$$\frac{\partial^2 u(x, t)}{\partial t^2} - c \frac{\partial^2 u(x, t)}{\partial x^2} = 0, \quad x \in (a, b), \quad t > 0$$

或(在多维情况下)

$$\frac{\partial^2 u(x, t)}{\partial t^2} - c \Delta u(x, t) = 0, \quad x \in \Omega, \quad t > 0$$

其中, c 是一个大于 0 的常数。

本章将主要讨论形式为(8.1)和(8.2)的方程。如果方程是依赖时间的, 如热力学方程和波动方程, 则称之为初始边值问题。对于这类问题, 需要给出在 $t=0$ 时的初始条件 $u=u_0$ 。关于初始边值问题和偏微分方程的更一般介绍, 读者可以参考 [QV94], [EEHJ96] 或 [Lan03]。

问题 8.1 (热力学问题) 计算一个边长为 L 的正方形区域 Ω 内的温度分布情况, 可以通过计算边长为 $l \ll L$ 的无穷小的正方形区域内的净能量在各个方向上的变化率来实现。即

$$J(x) - J(x + le_i) = -le_i \frac{\partial J}{\partial x_i}(x)$$

其中, $J(x)$ 代表单位时间内的能量转移。Fourier 定理表明 J 与温度 T 的变化率成比例, 于是

$$J(x) - J(x + le_i) = -le_i \frac{\partial}{\partial x_i} \left(kl \frac{\partial T}{\partial x_i} \right) = kl^2 \frac{\partial^2 T}{\partial x_i^2}$$

其中, k 是一个大于 0 的常数, 表示热导系数。在达到热平衡时, 总的能量变化之和必然为 0, 即

$$\Delta T(x) = \sum_{i=1}^d \frac{\partial^2 T}{\partial x_i^2}(x) = 0 \quad x \in \Omega$$

上式即为 Poisson 问题在 $f=0$ 时的情形。

问题 8.2 (水文地质学问题) 在一些情况下, 地下水的渗透问题就可以用形式为(8.2)的方程来描述。考虑一块由渗透介质(比如土层或粘土)填充的区域 Ω , 根据 Darcy 定律, 水的渗透速度 $\mathbf{q}=(q_1, q_2, q_3)^T$ 就等于介质上方水平面高度 ϕ 的变化率, 即

$$\mathbf{q} = -K(\partial\phi/\partial x_1, \partial\phi/\partial x_2, \partial\phi/\partial x_3)^T \quad (8.4)$$

其中, K 为常数, 表示介质的渗透系数。设液体密度为常量, 由质量守恒定律可以得到方程 $\operatorname{div} \mathbf{q} = 0$, 其中 $\operatorname{div} \mathbf{q}$ 是向量 $\mathbf{q}$ 的散度, 它定义为:

$$\operatorname{div} \mathbf{q} = \sum_{i=1}^3 \frac{\partial q_i}{\partial x_i}$$

由式(8.4)可以推出 ϕ 满足 Poisson 方程 $\Delta\phi = 0$ (参见习题 8.9)。

前述 Poisson 方程(8.2)有无穷多个解。为了得到惟一解, 需要在区域 Ω 的边界 $\partial\Omega$ 上给出适当的限制条件。其中一种形式是在 $\partial\Omega$ 上给定 u 的值, 即

$$u(x) = g(x) \quad x \in \partial\Omega \quad (8.5)$$

其中, g 是一个给定的函数。

如果边界条件是以(8.5)的形式给出的, 则问题(8.2)便成为 Dirichlet 边值问题, 这也是下一节主要讨论的问题。如果在(8.5)中给出的条件是在边界 $\partial\Omega$ 的方向上 u 的偏微分, 则此时得到的是 Neumann 边值问题。

可以证明, 如果 f 和 g 是两个连续函数, Ω 是足够规则的区域, 则 Dirichlet 边值问题有惟一的解(而 Neumann 边值问题的任意的两个解之间的差均为常数)。

在一维情况下, Dirichlet 边值问题形式如下: 对于两个给定的常量 α 、 β 和函数 $f = f(x)$, 找到一个函数 $u = u(x)$, 使之满足

$$\begin{cases} -u''(x) = f(x), & x \in (a, b) \\ u(a) = \alpha, & u(b) = \beta \end{cases} \quad (8.6)$$

采用二重积分很容易看出如果 $f \in C^0([a, b])$, 则解 u 存在且惟一, 并且它属于空间 $C^2([a, b])$ 。

尽管式(8.6)也是一个常微分问题, 但它却不能按照描述常微分方程通常采用的 Cauchy 问题的形式来对待, 因为这里 u 的值是在两个不同的点给出的。

这一问题的数值解法与多维边值问题数值解法应用的基本原理是一样的, 因此在 8.1 节中将对问题(8.6)的数值解法进行分析。

8.1 边值问题逼近

将区间 $[a, b]$ 划分成多个小区间 $I_j = [x_j, x_{j+1}]$, $j=0, \dots, N$, 其中 $x_0 = a$, $x_{N+1} = b$ 。为了分析方便, 设所有的小区间都有相等的长度 h 。

8.1.1 有限差分逼近

如果对于区间 (a, b) 内的任意一点 x_j (以下称为节点), 均满足下列方程成立:

$$-u''(x_j) = f(x_j), \quad j=1, \dots, N$$

于是可以通过下列方法来计算这 N 个方程, 用一个适当的有限差分来代替二阶导数, 正如在第 4 章中用类似的方法代替一阶导数那样。特别地, 如果 $u: [a, b] \rightarrow \mathbf{R}$ 在特定点 $\bar{x} \in (a, b)$ 的一个邻域内是光滑函数, 则

$$\delta^2 u(\bar{x}) = \frac{u(\bar{x}+h) - 2u(\bar{x}) + u(\bar{x}-h)}{h^2} \quad (8.7)$$

可以作为 $u''(\bar{x})$ 的一个相对于 h 具有二阶精确度的近似值(参见习题 8.3)。于是可以使用下式来逼近问题(8.6): 找出 $\{u_j\}_{j=1}^N$, 使得

$$-\frac{u_{j+1} - 2u_j + u_{j-1}}{h^2} = f(x_j), \quad j=1, \dots, N \quad (8.8)$$

其中, $u_0 = \alpha$, $u_{N+1} = \beta$ 。方程(8.8)描述一个线性系统:

$$Au_h = h^2 f \quad (8.9)$$

其中 $u_h = (u_1, \dots, u_N)^\top$ 是未知向量,

$f = (f(x_1) + \alpha/h^2, f(x_2), \dots, f(x_{N-1}), f(x_N) + \beta/h^2)^\top$, A 是一个三对角矩阵:

$$\begin{bmatrix} 2 & -1 & 0 & \dots & 0 \\ -1 & 2 & \ddots & & \vdots \\ 0 & \ddots & \ddots & -1 & 0 \\ \vdots & & -1 & 2 & -1 \\ 0 & \dots & 0 & -1 & 2 \end{bmatrix} \quad (8.10)$$

A 是正定对称阵, 所以系统的解是惟一的(参见习题 8.1)。又因为 A 是一个三对角矩阵, 所以系统可以采用 5.4 节中讲到的 Thomas 算法求解。这里需要指出, 当 h 的值很小(或 N 的值很大)时, 矩阵 A 是病态的。实际上, $K(A) = \lambda_{\max}(A)/\lambda_{\min}(A) = Ch^{-2}$, 其中 C 是与 h 无关的常量(参见习题 8.2)。因此, 在利用数值方法求解系统(8.9)时, 无论采用直接法还是迭代法, 都需要特别注意这一点。尤其在使用迭代法时, 还需要进行适当的预处理。

可以证明(参见 [QSS00], 第 12 章), 如果 $f \in C^2([a, b])$, 则

$$\max_{j=0, \dots, N+1} |u(x_j) - u_j| \leq \frac{h^2}{96} \max_{x \in [a, b]} |f''(x)|$$

即, 有限差分方法(8.8)相对于 h 是二阶收敛的。

程序 17 中给出了下列边值问题的解法:

$$\begin{cases} -u''(x) + \delta u'(x) + \gamma u(x) = f(x), & x \in (a, b) \\ u(a) = \alpha, & u(b) = \beta \end{cases} \quad (8.11)$$

它是问题(8.6)的推广。而解决这一问题采用的差分方法也是式(8.8)经过推广以后得到的:

$$\begin{cases} -\frac{u_{j+1} - 2u_j + u_{j-1}}{h^2} + \delta \frac{u_{j+1} - u_{j-1}}{2h} + \gamma u_j = f(x_j), & j = 1, \dots, N \\ u_0 = \alpha, & u_{N+1} = \beta \end{cases}$$

程序 17 的输入参数有: 区间的端点 a 和 b , 区间内节点的个数 N , 常系数 δ 和 γ , 以及字符串 `bvpfun`。还有, ua 和 ub 分别代表求出的解在 $x=a$ 和 $x=b$ 处的取值。输出参数是节点向量 x , 和计算结果 uh 。

程序 17-bvp: 利用有限差分法求解 2 点边值问题的近似解

```
function [x,uh]=bvp(a,b,N,delta,gamma,bvpfun,ua,ub,varargin)
%BVP Solve two-point boundary value problems.
```



```
% [X,UH]=BVP(A,B,N,DELTA,GAMMA,BVPFUN,UA,UB) solves with the
% centered finite difference method the boundary-value
% problem
% -D(DU/DX)/DX+DELTA*DU/DX+GAMMA*U=BVPFUN
% on the interval (A,B) with boundary conditions U(A)=UA
% and U(B)=UB.
% BVPFUN can be an inline function.
h=(b-a)/(N+1);
z=linspace(a,b,N+2);
e=ones(N,1);
A=spdiags([-e-0.5*h*delta 2*e+gamma*h^2-e+0.5*h*delta],-1:1,N,N);
x=z(2:end-1);
f=h^2*feval(bvpfun,x,varargin{~});
f=f'; f(1)=f(1)+ua; f(end)=f(end)+ub;
uh=A\f;
uh=[ua;uh;ub];
x=z;
```

8.1.2 有限元法逼近

求解边值问题，除了前述的有限差分法，还可以采用有限元法。有限元法源自问题(8.6)的一个变形。

在式(8.6)两端同乘以一个函数 v ，然后同时对方程两端在区间 (a, b) 上取积分，得到：

$$-\int_a^b u''(x)v(x)dx = \int_a^b f(x)v(x)dx \quad (8.12)$$

如果 $v \in C^1([a, b])$ ，利用分部积分得到：

$$\int_a^b u'(x)v'(x)dx - [u'(x)v(x)]_a^b = \int_a^b f(x)v(x)dx$$

如果进一步假设在端点 $x=a$ 和 $x=b$ 处 v 取值为零，则问题(8.6)变成：找出函数 u ，使得 $u(a)=\alpha$ ， $u(b)=\beta$ 并且

$$\int_a^b u'(x)v'(x)dx = \int_a^b f(x)v(x)dx \quad (8.13)$$

对于每个 $v \in C^1([a, b])$ ，有 $v(a)=v(b)=0$ 。上式称为问题(8.6)的弱形式(实际上，函数 v 并非一定要在空间 $C^1([a, b])$ 内，这一要求还可以再减弱，参见 [QSS00]。

而解这一问题的有限元法定义如下：

$$\left. \begin{array}{l} \text{找到 } u_h \in V_h \text{ 使得 } u_h(a) = \alpha, \quad u_h(b) = \beta, \text{ 且} \\ \sum_{j=0}^N \int_{x_j}^{x_{j+1}} u'_h(x) v'_h(x) \, dx = \int_a^b f(x) v_h(x) \, dx, \quad \forall v_h \in V_h^0 \end{array} \right\} \quad (8.14)$$

其中

$$V_h = \{v_h \in C^0([a, b]): v_{h|I_j} \in P_1, \quad j=0, \dots, N\}$$

即 V_h 是由 (a, b) 区间上的所有连续函数构成的空间, 对于这些函数的限制条件是在每个小区间 I_j 上它们都可以写成线性多项式的形式。而且 V_h 的子空间 V_h^0 表示由端点 a 和 b 处取值为零的函数整体所构成的空间。 V_h 称为 1 维有限元空间。

空间 V_h^0 中的函数是分段线性多项式(参见图 8.1 的左图)。

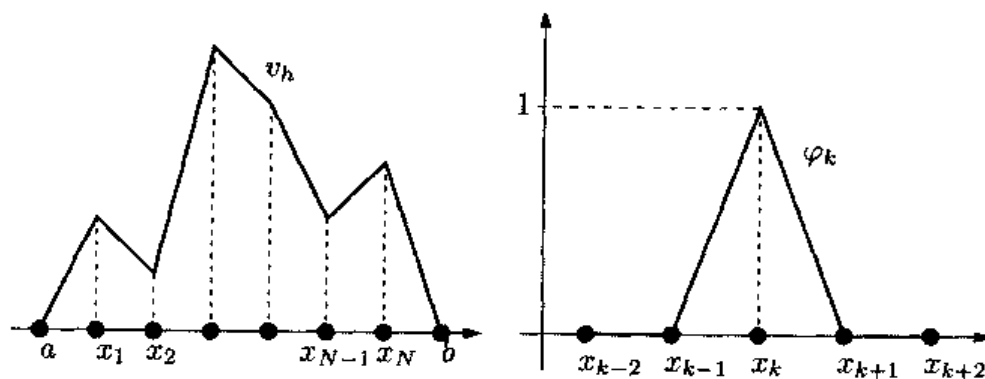


图 8.1 左图: 某一函数 $v_h \in V_h^0$; 右图: V_h^0 空间在第 k 个节点的基函数

特别地, 空间 V_h^0 中的任意函数 v_h 可以表示成:

$$v_h(x) = \sum_{j=1}^N v_h(x_j) \varphi_j(x)$$

其中

$$\varphi_j(x) = \begin{cases} \frac{x - x_{j-1}}{x_j - x_{j-1}} & \text{如果 } x \in I_{j-1} \\ \frac{x - x_{j+1}}{x_j - x_{j+1}} & \text{如果 } x \in I_j \\ 0 & \text{其他情况} \end{cases}$$

其中 $j=1, \dots, N$ 。 φ_k 在节点 x_k 处取值为 1, 即 $\varphi_k(x_k)=1$, 而在除 x_k 以外的其他节点 x_j 处为零(如图 8.1 的右图所示)。函数 $\varphi_j, j=1, \dots, N$ 称为形函数, 它们可以作为向量空间 V_h^0 的一个基。

因此可以只在形函数 $\varphi_j, j=1, \dots, N$ 处求解式(8.14)。由于 φ_j 在小区间 I_{j-1} 和 I_j 之外取值均为零, 由式(8.14)可得:

$$\int_{I_{j-1} \cup I_j} u'_h(x) \varphi'_j(x) dx = \int_{I_{j-1} \cup I_j} f(x) \varphi_j(x) dx, \quad \forall j=1, \dots, N \quad (8.15)$$

另外, 还可得出 $u_h(x) = \sum_{j=1}^N u_j \varphi_j(x) + \alpha \varphi_0(x) + \beta \varphi_{N+1}(x)$, 其中 $u_j = u_h(x_j)$, $\varphi_0(x) = (a+h-x)/h$, 且 $a \leq x \leq a+h$, $\varphi_{N+1}(x) = (x-b+h)/h$, $b-h \leq x \leq b$, 而其余点处 $\varphi_0(x)$ 和 $\varphi_{N+1}(x)$ 均为零。把上式带入式(8.15)中可以得到, 对于所有的 $j=1, \dots, N$

$$\begin{aligned} & u_{j-1} \int_{I_{j-1}} \varphi'_{j-1}(x) \varphi'_j(x) dx + u_j \int_{I_{j-1} \cup I_j} \varphi'_j(x) \varphi'_j(x) dx \\ & + u_{j+1} \int_{I_j} \varphi'_{j+1}(x) \varphi'_j(x) dx = \int_{I_{j-1} \cup I_j} f(x) \varphi_j(x) dx + B_{1,j} + B_{N,j} \end{aligned}$$

其中,

$$B_{1,j} = \begin{cases} -a \int_{I_0} \varphi'_0(x) \varphi'_1(x) dx = -\frac{\alpha}{x_1 - a}, & \text{如果 } j=1 \\ 0 & , \text{其他情况} \end{cases}$$

而

$$B_{N,j} = \begin{cases} -\beta \int_{I_N} \varphi'_{N+1}(x) \varphi'_j(x) dx = -\frac{\beta}{b - x_N} & \text{如果 } j=N \\ 0 & , \text{其他情况} \end{cases}$$

在特殊情况下, 比如所有的区间都有相等的长度 h , 则在区间 I_{j-1} 内 $\varphi'_{j-1} = -1/h$, 区间 I_{j-1} 内 $\varphi'_j = 1/h$, 在区间 I_j 内 $\varphi'_j = -1/h$, 区间 I_j 内 $\varphi'_{j+1} = 1/h$ 。因此, 对于 $j=1, \dots, N$ 可得:

$$-u_{j-1} + 2u_j - u_{j+1} = h \int_{I_{j-1} \cup I_j} f(x) \varphi_j(x) dx + A_{0,j} + B_{0,j}$$

该线性系统的矩阵形式与有限差分系统(8.9)相同, 但方程右边的形式不同(因此尽管

此时表示形式一致也会得到不同的解)。在计算节点最大误差时,利用有限差分法和有限元法得到的解相对于 h 的精确度是相同的。

显然,有限元逼近法可以推广应用于形式为(8.11)的问题(也可应用于 δ 和 γ 与 x 有关的情况)。进一步还可以利用阶数大于 1 的分段多项式来近似,并在高阶时收敛。在这种情况下有限元矩阵与有限差分矩阵不再一致,并且收敛阶数要高于线性多项式的情况。

参见习题 8.1~8.8。

8.2 二维有限差分

在二维区域 Ω 内考虑一个偏微分方程,比如方程(8.2)。

有限差分的基本思想是把出现在偏微分方程中的偏导数用适当的网格(称为计算网格)内的增量比来近似代替,这些网格是由有限数量的节点构成的。于是在这些节点处偏微分方程的解 u 可以近似求出。

因此首先需要引入计算网格。为了分析方便,区域 Ω 选为矩形 $(a, b) \times (c, d)$ 。把区间 $[a, b]$ 划分成多个小区间 (x_k, x_{k+1}) , 其中 $k=0, \dots, N_x$, 且 $x_0=a, x_{N_x+1}=b$ 。用 $\Delta_x = \{x_0, \dots, x_{N_x+1}\}$ 来代表由这些小区间的端点所构成的序列, 用 $h_x = \max_{k=0, \dots, N_x} (x_{k+1} - x_k)$ 来表示小区间段长度的最大值。

采用类似的方法,可以在 y 轴上取增量为 $\Delta_y = \{y_0, \dots, y_{N_y+1}\}$, 其中 $y_0=c, y_{N_y+1}=d$ 。Cartesian 积 $\Delta_h = \Delta_x \times \Delta_y$ 就表示在 Ω 上的计算网格(如图 8.2 所示), $h = \max\{h_x, h_y\}$ 是网格大小的一个量度。下一步要找出 $u(x_i, y_j)$ 的近似值 $u_{i,j}$ 。为了分析方便,选取节点为均匀分布,即 $x_i = x_0 + ih_x, i=0, \dots, N_x+1, y_j = y_0 + jh_y, j=0, \dots, N_y+1$ 。

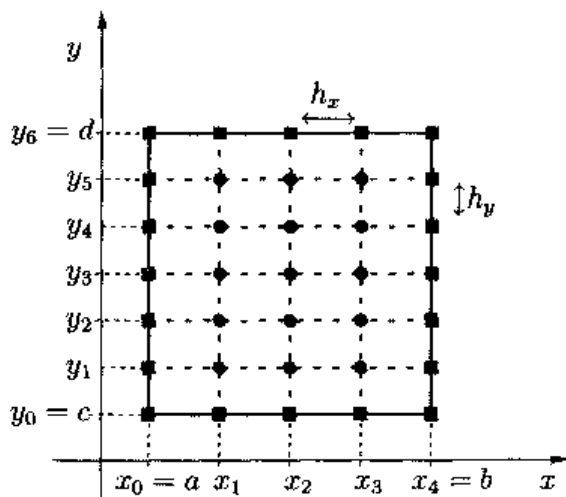


图 8.2 在矩形区域内由 15 个内部节点构成的计算网格 Δ_h

与计算普通导数时采用的方法一样,函数的二阶偏导数也可以利用一个增量比来作为近似。对于含有两个变量的函数,定义下列增量:

$$\delta_x^2 u_{i,j} = \frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h_x^2}, \quad \delta_y^2 u_{i,j} = \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{h_y^2} \quad (8.16)$$

它们在节点 (x_i, y_j) 处分别作为 $\partial^2 u / \partial x^2$ 和 $\partial^2 u / \partial y^2$ 的近似值时有关于 h_x 和 h_y 的二阶精确度。如果采用式(8.16)来代替 u 的二阶偏微分,为使偏微分方程在所有的内部节点 Δ_h 处都成立,有下列方程:

$$-(\delta_x^2 u_{i,j} + \delta_y^2 u_{i,j}) = f_{i,j}, \quad i=1, \dots, N_x, \quad j=1, \dots, N_y \quad (8.17)$$

其中,设 $f_{i,j} = f(x_i, y_j)$ 。还必须再加一个方程,给出在边界上的 Dirichlet 条件,即

$$u_{i,j} = g_{i,j} \quad \forall i, j \text{ 使得 } (x_i, y_j) \in \partial\Delta_h \quad (8.18)$$

其中, $\partial\Delta_h$ 代表在 Ω 边界 $\partial\Omega$ 上的节点序列。这些节点在图 8.2 中是以小方块作为标志的。如果进一步假设计算网格在每个 Cartesian 方向上都是均匀分布的,即 $h_x = h_y = h$, 则会得到与式(8.17)不同的形式:

$$-\frac{1}{h^2}(u_{i-1,j} + u_{i,j-1} - 4u_{i,j} + u_{i,j+1} + u_{i+1,j}) = f_{i,j} \quad (8.19)$$

$$i=1, \dots, N_x, \quad j=1, \dots, N_y$$

对于由式(8.19)(或式(8.17))和式(8.18)的形式给出的系统可以计算出在所有的节点 Δ_h 处的值 $u_{i,j}$ 。正如图 8.3 所示的那样,对于每个固定的下标对 i 和 j , 方程(8.19)都包含 5 个未知的节点值。于是这种有限差分方法便称为 Laplace 算子的五点差分格式。特别指出,与边界节点相关的未知数可以利用式(8.18)(或式(8.17))消去,因此式(8.19)只含有 $N=N_x N_y$ 个未知数。

如果采用 lexicographic 方法,则系统可写成更简明的表达形式,此时所有节点(连同未知元素)是按照从左到右,从上到下的顺序进行编号的。得到一个形式为(8.9)的系统,它的矩阵 $A \in \mathbf{R}^{N \times N}$ 形式为分块三对角阵:

$$A = \begin{bmatrix} T & D & 0 & \dots & 0 \\ D & T & \ddots & & \vdots \\ 0 & \ddots & \ddots & D & 0 \\ \vdots & & D & T & D \\ 0 & \dots & 0 & D & T \end{bmatrix}$$

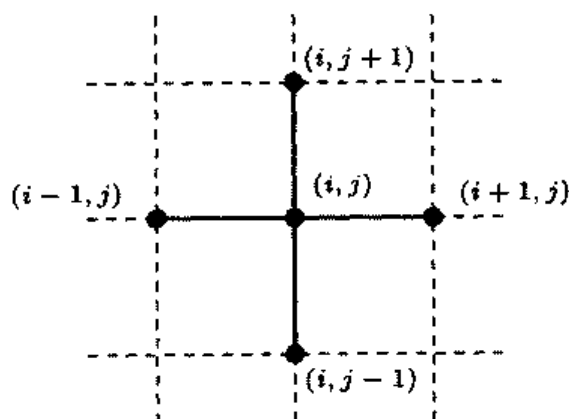


图 8.3 Laplace 算子的五点差分格式示意图

该矩阵有 N_y 行, N_y 列, 每个元素(采用大写字母表示)是一个 $N_x \times N_x$ 矩阵。特别地, $D \in \mathbf{R}^{N_x \times N_x}$ 是一个对角阵, 对角线上的元素是 $-1/h_y^2$, 而 $T \in \mathbf{R}^{N_x \times N_x}$ 是一个对称的三对角阵:

$$T = \begin{bmatrix} \frac{2}{h_x^2} + \frac{2}{h_y^2} & -\frac{1}{h_x^2} & 0 & \dots & 0 \\ -\frac{1}{h_x^2} & \frac{2}{h_x^2} + \frac{2}{h_y^2} & \ddots & & \vdots \\ 0 & \ddots & \ddots & -\frac{1}{h_x^2} & 0 \\ \vdots & & -\frac{1}{h_x^2} & \frac{2}{h_x^2} + \frac{2}{h_y^2} & -\frac{1}{h_x^2} \\ 0 & \dots & 0 & -\frac{1}{h_x^2} & \frac{2}{h_x^2} + \frac{2}{h_y^2} \end{bmatrix}$$

由于所有的对角块都是对称的, 所以 A 也是对称矩阵。同时 A 是正定的, 即 $v^T A v > 0$, $\forall v \in \mathbf{R}^N$, $v \neq 0$ 。实际上, 把 v 分成 N_y 个长度为 N_x 的向量 v_i 可得:

$$v^T A v = \sum_{k=1}^{N_y} v_k^T T v_k - \frac{2}{h_y^2} \sum_{k=1}^{N_y} v_k^T v_{k+1} \quad (8.20)$$

于是有 $T=2/h_y^2 I + 1/h_x^2 K$, 式中 K 是由式(8.10)给出的矩阵(对称正定阵)。因此(8.20)变成:

$$\left(v_1^T K v_1 + v_2^T K v_2 + \cdots + v_{N_y}^T K v_{N_y} \right) / h_x^2$$

它是一个正实数, 因为 K 是正定的, 并且至少有一个非零向量 v_i 。

已经证明如果 A 是一个非奇异矩阵, 则有限差分系统存在惟一解 u_h 。

如果矩阵 A 的非零元素远远少于零元素, 则称 A 是稀疏矩阵(在图 8.4 中给出的矩阵结构在计算时采用的是由 11×11 个均匀分布的节点所构成的网格。该图是利用 `spy(A)` 函数得到的。由图可见在 5 条对角线上只有非零元素)。

稀疏矩阵可以按特殊方式存储, 即只存储非零元素, 并允许对非零元素进行快速访问。程序 18 中用到的 `sparse` 函数可用来实现这一操作。由于 A 是对称正定阵, 所以相应系统可以利用直接法或迭代法求解, 正如在第 5 章中所提到的那样。最后有必要指出, 与一维的情况类似, 矩阵 A 也会出现病态的情况: 实际上, 它的条件数在 h 趋于零时呈 h^{-2} 增长, 其中 $h=\max(h_x, h_y)$ 。

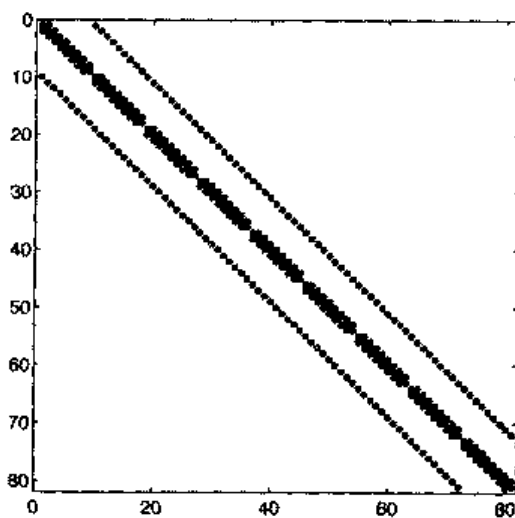


图 8.4 利用五点差分格式按 lexicographic 方式排列未知数时得到的矩阵的图形

在程序 18 中构造并求解了系统(8.17)~(8.18)。输入参数 a, b, c 和 d 表示矩形区域 $\Omega=(a, c) \times (b, d)$ 的四个顶点, 而 nx 和 ny 分别代表 N_x 和 N_y 的值(设 $N_x \neq N_y$)。还有, 字符串 `fun` 和 `bound` 代表方程右边的函数 $f=f(x, y)$ (或者称为源项)和边值条件 $g=g(x, y)$ 。输出参数是二维矩阵 u , 该矩阵的第 i 行第 j 列的元素就是节点值 $u_{i,j}$ 。数值解可以使用函数 `mesh(x, y, u)` 显示出来。如果预先知道方程的精确解是存在的, 则字符串 `uex`(可以任选)就可以代表这个初始问题的精确解(只考虑理论意义)。此时,

输出参数 **error** 表示数值解和真实解之间的节点相对误差, 它的计算公式如下:

$$\text{error} = \max_{i,j} |u(x_i, y_j) - u_{i,j}| / \max_{i,j} |u(x_i, y_j)|$$

程序 18—poissonfd 利用五点有限差分法求解含有 Dirichlet 条件的 Poisson 问题的近似解

```
function [u,x,y,error]=poissonfd (a,c,b,d,nx, ny,fun, bound,
uex,varargin)
%POISSONFD two-dimensional Poisson solver
% [U,X,Y]=POISSONFD(A,C,B,D,NX,NY,FUN,BOUND) solves by
% the 5-points finite difference scheme the problem
% -LAPL(U) = FUN in the rectangle (A,C)X(B,D) with
% Dirichlet boundary conditions U(X,Y)=BOUND(X,Y) for any
% (X,Y) on the boundary of the rectangle.
%
% [U,X,Y,ERROR]=POISSONFD(A,C,B,D,NX,NY, FUN,BOUND,UEX) compu'
% also the maximum nodal error ERROR with respect to the exact
% solution UEX.
% FUN, BOUND and UEX can be online functions.
if nargin ==8
    uex = inline('0','x','y');
end
nx=nx+1;      ny=ny+1;
hx = (b-a)/nx; hy=(d-c)/ny;
nx1 = nx+1;   hx2 = hx^2;
hy2 = hy^2;
kii = 2/hx2+2/hy2; kix =-1/hx2; kiy=-1/hy2;
dim = (nx+1)*(ny+1);
K = speye(dim,dim);
rhs = zeros(dim,1);
y=c;
for m = 2:ny
    x = a; y = y + hy;
    for n = 2:nx
        i= n+(m-1)*(nx+1);
        x = x + hx;
        rhs(i) = feval(fun,x,y,varargin{:});
        K(i,i) = kii;
        K(i,i-1) =kix;
        K(i,i+1) = kix;
        K(i,i+nx1) = kiy;
        K(i,i-nx1) = kiy;
    end
end
rhs1 = zeros(dim,i);
x= [a:hx:b];
rhs1(1:nx1) = feval(bound,x,c,varargin{:});
rhs1(dim-nx:dim) = feval(bound,x,d,varargin{:});
```



```

y = [c:hy:d];
rhs1(1:nx1:dim-nx) = feval(bound,a,y,varargin{:});
rhs1(nx1:nx1:dim) = feval(bound,b,y,varargin{:});
rhs = rhs = K*rhs1;
nbound= [[1:nx1],[dim-nx:dim],[1:nx1:dim-nx],[nx1:nx1:dim]];
ninternal= setdiff([1:dim],nbound);
K = K(ninternal,ninternal);
Rhs= rhs(ninternal);
Utemp= K\Rhs;
uh = rhs1;
uh(ninternal) = utemp;
k= 1;y=c;
for j = 1:ny+1
    x=a;
    for i= 1:nx1
        u(i,j) = uh(k);
        k=k+1;
        ue(i,j) = feval(uex,x,y,varargin{:});
        x = x + hx;
    end
    y = y + hy;
end
x=[a:hx:b];
y = [c:hy:d];
if nargin== 4 & nargin == 8
    warning('Exact solution not available');
    error= [];
else
    error = max(max(abs(u-ue)))/max(max(abs(ue)));
end, end
return

```

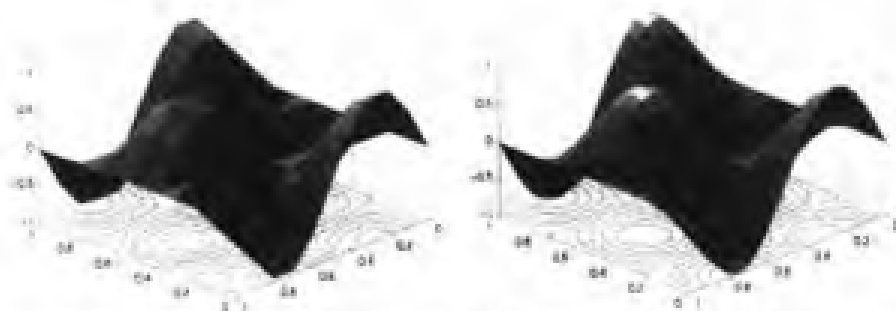


图 8.5 在均匀网格下得到的弹性薄膜的横迁移量。在水平面上标出的是数值解的等值线。
 Ω 区域分成小三角块是为了使结果显示的更直观

例 8.1 在参考平面 $\Omega = (0, 1)^2$ 上强度为 $f(x, y) = 8\pi^2 \sin(2\pi x) \cos(2\pi y)$ 的弹性薄膜的横迁移 u 在区域 Ω 内可以看作是形式为(8.2)的 Poisson 问题。在边界 $\partial\Omega$ 上的 Dirichlet 位移量为: 在 $x=0$ 和 $x=1$ 时, $g=0$; 在 $0 < x < 1$ 时, $g(x, 0) = g(x, 1) = \sin(2\pi x)$ 。

该问题的解的形式如下: $u(x, y) = \sin(2\pi x) \cos(2\pi y)$ 。在图 8.5 中给出了在均匀网格下利用五点差分格式得到的数值解。在两个图中 h 的值分别取 $h=1/10$ (左图) 和 $h=1/20$ (右图)。 h 越小, 则数值解越精确, 实际上, 在 $h=1/10$ 时节点相对误差是 0.029 2, 当 $h=1/20$ 时, 节点的相对误差是 0.008 1。

有限差分法也可推广应用于二维情况。此时问题(8.2)需要写成积分形式, 并且对一维区间 (a, b) 进行的分割改为对区域 Ω 进行多边形分割(一般分为三角形), 这些小多边形称为单元。形函数 φ_k 此时仍然是连续函数, 但要求此时形函数在每个单元上可以写成 1 阶多项式的形式, 并且在第 k 个三角形的顶点(或称节点)处取值为 1, 在其他节点处均为零。上述操作可以通过 MATLAB 工具箱 `pde` 来实现。

一致性和收敛性

由前面的章节我们知道, 有限差分法得到的解是存在并惟一的。下面来讨论一下近似误差问题, 为了分析方便, 设 $h_x = h_y = h$ 。如果

$$\max_{i,j} |u(x_i, y_j) - u_{i,j}| \rightarrow 0 \quad \text{在 } h \rightarrow 0 \text{ 时}$$

则称该方法是收敛的。

正如前面所提到的那样, 一致是收敛的必要条件。如果在 h 趋于 0 时, 计算得到的数值解与精确值的差值也趋于 0, 则称这种方法是一致的。如果考虑五点有限差分格式, 在 Δ_h 的每个内部节点 (x_i, y_j) 处定义:

$$\tau_h(x_i, y_j) = -f(x_i, y_j) - \frac{u(x_{i-1}, y_j) + u(x_i, y_{j-1}) - 4u(x_i, y_j) + u(x_i, y_{j+1}) + u(x_{i+1}, y_j)}{h^2}$$

则称之为在节点 (x_i, y_j) 处的局部舍位误差。由式(8.2)可得:

$$\begin{aligned} \tau_h(x_i, y_j) = & \left\{ \frac{\partial^2 u}{\partial x^2}(x_i, y_j) - \frac{u(x_{i-1}, y_j) - 2u(x_i, y_j) + u(x_{i+1}, y_j))}{h^2} \right\} \\ & + \left\{ \frac{\partial^2 u}{\partial y^2}(x_i, y_j) - \frac{u(x_i, y_{j-1}) - 2u(x_i, y_j) + u(x_i, y_{j+1}))}{h^2} \right\} \end{aligned}$$

利用 8.2 节得到的结论可以看出, 在 h 趋于 0 时, 上式等号右边的两项也趋于 0。

于是

$$\lim_{h \rightarrow 0} \tau_h(x_i, y_j) = 0, \quad \forall (x_i, y_j) \in \Delta_h \setminus \partial \Delta_h$$

即五点差分格式是一致的。并且由下面的命题(关于这一命题的证明可参见[IK66])还可以看出该方法是收敛的。

命题 8.1 如果精确解 $u \in C^4(\bar{\Omega})$, 即 u 的所有四阶偏导数在闭区域 $\bar{\Omega}$ 内连续, 则存在一个大于零的常量 C , 使得

$$\max_{i,j} |u(x_i, y_j) - u_{i,j}| \leq CMh^2$$

成立, 其中 M 是在区域 $\bar{\Omega}$ 内 u 的 4 阶偏导数的绝对值的最大值。

例 8.2 验证: 利用五点差分格式求解例 8.1 中的 Poisson 问题时相对于 h 是 2 阶收敛的。首先选取 $h=1/4$, 然后每次将 h 除以 2, 直到 $h=1/64$, 采用的指令如下:

```
>>a=0;b=1;c=0;d=1;
>>f=inline('8*pi^2*sin(2*pi*x).*cos(2*pi*y)','x','y');
>>g=inline('sin(2*pi*x).*cos(2*pi*y)','x','y');ues=g;nx=4;ny=4;
>>for n=1:5
[u,x,y,error(n)]=poissonfd(a,c,b,d,nx,ny,f,g,ues);nx=2*nx;ny=2*ny;
end
```

则表示误差的向量是:

```
>>format short e; error
1.3565e-01 4.3393e-02 1.2308e-02 3.2775e-03 8.4557e-04
```

通过下列指令

```
>>log(abs(error(1:end-1)./error(2:end)))/log(2)
1.6443e+00 1.8179e+00 1.9089e+00 1.9546e+00
```

可以验证误差是按照 h^2 减小的。

小结

1. 边值问题是在空间区域 $\Omega \in \mathbb{R}^d$ 内(当 $d=1$ 时为一个区间)的微分方程问题, 它需要在区域的边界处给定已知条件;

2. 有限差分逼近是把给定的微分方程在一些特定的点(称为节点)离散化, 然后在节点处用有限差分公式来代替导数的值;

3. 有限差分法生成的节点向量的元素收敛于精确解在相应节点处的值, 精确解是网格大小的二次多项式;

4. 有限元法的两个步骤: 首先将初始微分方程化成积分的形式, 然后再利用假设前提, 即近似解可以表示成分段多项式的形式;

5. 由有限差分 and 有限元逼近生成的矩阵是稀疏的和病态的。

参见习题 8.9~8.10。

8.3 补充说明

偏微分方程求解这一数值分析领域博大精深、包罗万象, 即使只是叙述一下最基本的概念, 也足以写成一本专著(参见 [TW98]、[EEHJ96]), 因此本章所提到的只是一些很浅显的知识。

还需指出, 有限元法是当今在求偏微分方程数值解时应用最广泛的一种方法(参见 [QV94]、[Bra97]、[BS01])。正如前面章节中所提到的, MATLAB 的 pde tool 工具箱就是利用有限元法来求解更一般形式的偏微分方程的。

另外几种经常使用的方法还有谱法(spectral methods)(参见 [CHQZ88]、[Fun92]、[BM92]、[KS99])和有限体积法(finite volume method)(参见 [Krö 98]、[Hir88] 和 [LeV02])。

8.4 习题

习题 8.1 验证矩阵(8.10)是正定的。

习题 8.2 验证矩阵(8.10)的特征值是

$$\lambda_j = 2(1 - \cos(j\theta)), \quad j = 1, \dots, N$$

相应的特征向量是

$$q_j = (\sin(j\theta), \sin(2j\theta), \dots, \sin(nj\theta))^T$$

其中, $\theta = \pi/(n+1)$, 由此得出 $K(A)$ 与 h^{-2} 成比例。

习题 8.3 证明: 式(8.7)作为 $u''(\bar{x})$ 的近似值时具有相对于 h 的二阶准确度。

习题 8.4 求出描述问题(8.11)时采用的数值方法的矩阵形式和方程右边的函数。

习题 8.5 使用有限差分法求解下列边值问题:

$$\begin{cases} -u'' + \frac{k}{T}u = \frac{w}{T} & \text{在}(0, 1)\text{上} \\ u(0) = u(1) = 0 \end{cases}$$

其中, $u = u(x)$ 代表一个长度为 1 的绳子的竖直位移, 绳子单位长度的的横向负载强度为 w , T 是绳子的张力, k 是绳子的弹性系数。取 $w = 1 + \sin(4\pi x)$, $T = 1$, $k = 0.1$, 计算当 $h = 1/i$, $i = 10, 20, 40$ 时的解, 求出这种方法的精确度。

习题 8.6 在区间 $(0, 1)$ 上考虑问题(8.11), 取参数为 $\gamma = 0$, $f = 0$, $\alpha = 0$, $\beta = 1$ 。利用程序 17 找出 h 的最大值 h_{crit} , 当 $\delta = 100$ 时, 数值解关于 h 是单调的(这一点与精确解相问)。再取 $\delta = 1000$, 观察此时得出的结论。构造一个以 δ 为变量的经验公式 $h_{\text{crit}}(\delta)$, 并取几个 δ 值来验证这个公式。

习题 8.7 利用有限差分法求解问题(8.11), 这里在端点给出 Neumann 边值条件:

$$u'(a) = \alpha, \quad u'(b) = \beta$$

利用公式(4.10)对 $u'(a)$ 和 $u'(b)$ 进行离散化。

习题 8.8 验证下列结论: 当使用均匀网格时, 利用中心有限差分格式计算得到的系统方程右侧与利用有限元法得到的系统方程右侧是一致的, 在计算 I_{k-1} 和 I_k 单元上的积分时采用复合梯形公式。

习题 8.9 验证 $\text{div} \nabla \phi = \Delta \phi$, 其中 ∇ 是梯度算子, 对函数 u 进行梯度运算得到的结果是一个以 u 的一阶偏导数作为其元素的向量。

习题 8.10 考虑一个边长为 20cm 的正方形钢板, 它的导热系数是 $k = 0.2 \text{ cal/s} \cdot \text{cm} \cdot ^\circ\text{C}$, 单位面积上的放热率是 $Q = 5 \text{ cal/cm}^2 \cdot \text{s}$ 。钢板的温度 $T = T(x, y)$ 满足方程 $-\Delta T = Q/k$ 。设钢板三个边上 T 值为零, 第四个边上 T 值为 1, 求出钢板中心的温度 T 。

第 9 章 习 题 解 答

9.1 第 1 章

解 1.1 只有形式为 $\pm 0.1a_1 \times 2^t$ 且 $a_1 = 0, 1, t = +2, +1, 0$ 的数值才属于集合 $F(2, 2, -2, 2)$ 。当指数给定时, 这个集合中的元素只能利用 0.10 和 0.11 或它们的相反数来表示。因此, 属于集合 $F(2, 2, -2, 2)$ 的元素数目是 20 个, 并且 $\epsilon_M = 1/2$ 。

解 1.2 当指数给定时, 每个数位 $a_2, \dots, a_t$ 可以取 β 个不同的值, 而 a_1 只能取 $\beta - 1$ 个值, 于是可以表示出 $2(\beta - 1)\beta^{t-1}$ 个不同的数值, 上式乘 2 是因为每个数值可以取正、负两种符号。另外, 由于指数可以取 $U - L + 1$ 个值, 所以集合 $F(\beta, t, L, U)$ 包含的元素个数为 $2(\beta - 1)\beta^{t-1}(U - L + 1)$ 。

解 1.3 可以采用下列指令:

上三角阵: $U = 2 * \text{eye}(10) - 3 * \text{diag}(\text{ones}(8, 1), 2)$

下三角阵: $L = 2 * \text{eye}(10) - 3 * \text{diag}(\text{ones}(8, 1), -2)$

解 1.4 把上题中矩阵的第三行和第七行相交换, 可以采用下列指令实现: $r = [1:10]; r(3) = 7; r(7) = 3; L(r, :)$ 。需要注意的是在 $L(r, :)$ 中的符号 $:$ 表示所有 L 列是按增序进行操作的(从第一列至最后一列)。如果要把第八列与第四列相交换, 可以写成 $c = [1:10]; c(8) = 4; c(4) = 8; L(:, c)$ 。对于上三角阵可以采用类似的指令。

解 1.5 设矩阵 $A = [v1; v2; v3; v4]$, 其中 $v1, v2, v3$ 和 $v4$ 是 4 个给定的行向量。当且仅当 A 的行列式不为 0 时, 这四个向量是线性无关的, 由于本题中 A 的行列式为 0, 所以这四个向量线性相关。

解 1.6 两个函数 f 和 g 都是以符号表达式的形式给出的:

```
>>syms x
>>f=sqrt(x^2+1);pretty(f)
(x^2+1)^{1/2}
>>g=sin(x^3)+cosh(x);pretty(g)
sin(x^3)+cosh(x)
```

`syms x` 函数的作用是: 声明将 x 作为一个符号变量来使用。函数 `pretty(f)` 可以按照标准数学公式的格式显示出 f 的表达式。于是, 函数 f 的一、二阶导数和积分的符号表达式可以通过下列指令得到:

```
>>diff(f,x)
ans=
-1/(x^2+1)^(3/2)*x^2+1/(x^2+1)^(1/2)
>>int(f,x)
ans=
1/2*x*(x^2+1)^(1/2)+1/2*asinh(x)
```

对于函数 g 可以采用类似的指令。

解 1.7 当多项式的阶数增加时, 计算得到的根的精确度逐渐降低。本例表明, 要精确计算一个高阶多项式的根是比较困难的。

解 1.8 计算这个序列可以使用下面的程序:

```
function l=sequence(n)
l=zeros(n+2,1); l(1)=(exp(1)-1/exp(1));
for i=0:n, l(i+2)=1-(i+1)* l(i+1);end
```

利用该程序计算时, 序列并不收敛于零(当 n 增加时), 而是符号交替变换并发散的。

解 1.9 该序列的计算异常现象是由内部操作所产生的舍入误差的累加引起的。特别地, 当 $4^{i-n} z_0^2$ 小于 ε_M 时, 序列的元素等于 0, 此时 $n=27$ 。

解 1.10 本题中给出的方法是 Monte Carlo 方法的一个特例, 它可以通过下列程序实现:

```
function mypi=pimontecarlo(n)
x=rand(n,1);y=rand(n,1);
z=x.^2+y.^2;
v=(z<=1);
m=sum(v);mypi=r*m/n;
```

函数 rand 用于产生一组伪随机数序列。指令 $v = (z \leq 1)$ 的功能是: 判断向量 z 的每个元素是否满足 $z(k) \leq 1$ 。如果向量 z 的第 k 个元素满足该不等式(即点 $(x(k), y(k))$ 属于单位圆内部)则 $v(k)$ 等于 1, 否则为 0。利用函数 sum(v) 可以计算出向量 v 的所有元素之和, 也就是随机点落在单位圆内部的个数。

在不同 n 值下执行函数 $mypi=pimontecarl(n)$, 当 n 增加时, π 的近似值 $mypi$ 精确度提高。比如当 $n=1000$ 时, $mypi=3.1120$, 而当 $n=300000$ 时, $mypi=3.1406$ 。

解 1.11 计算二项式可以采用下列程序(也可以利用 MATLAB 的 nchoosek 函数来计算)

```
function bc=bincoeff(n,k)
k=fix(k); n=fix(n);
if k>n, disp('k must be between 0 and n');break; end
if k>n/2, k=n-k;end
if k<=1, bc=n^k;else
num=(n-k+1):n;den=1:k;el=num./den;bc=prod(el);
end
```

$\text{fix}(k)$ 函数返回的是与 k 距离最近并小于 k 的整数。 $\text{disp}(\text{string})$ 函数的功能是显示出括号内的字符串 string 。 break 函数一般用于终止 for 和 while 循环的执行,但在 if 处是否执行 break 语句则取决于在该点相应条件是否满足。还有, $\text{prod}(\text{el})$ 函数的功能是计算向量 el 的所有元素的乘积。

9.2 第 2 章

解 2.1 利用函数 fplot 可以绘出给定函数 f 在不同的 γ 下的曲线图。当 $\gamma=1$ 时, 相应的函数没有实零点。当 $\gamma=2$ 时只有惟一的零点 $\alpha=0$, 它是四重零点(即 $f(\alpha)=f'(\alpha)=f''(\alpha)=f'''(\alpha)=0$, 但 $f^{(4)}(\alpha)\neq 0$)。还有在 $\gamma=3$ 时, f 有两个不同的零点, 一个在区间 $(-3, -1)$ 中, 另一个在区间 $(1, 3)$ 中。在 $\gamma=2$ 时不能应用二分法, 因为此时找不出一个区间 (a, b) 使得 $f(a)f(b)<0$ 成立。取 $\gamma=3$, 从区间 $[a, b]=[-3, -1]$ 开始, 二分法(程序 1)经过 34 次迭代运算将收敛于 $\alpha=-1.857\ 920\ 829\ 148\ 50$ (此时 $f(\alpha)\approx -3.6\times 10^{-12}$), 使用下列指令:

```
>>f=inline('cosh(x)+cos(x)-3');a=-3;b=-1;tol=1.e-10;nmax=20;
>>[zero,res,niter]=bisection(f,a,b,tol,nmax)
zero=
    -1.8579
res=
    -3.6872e-12
niter=
    34
```

类似地, 在 $\gamma=4$ 时, 二分法经过 34 次迭代运算之后收敛于数值 $\alpha=-2.205\ 621\ 636\ 880\ 10$, 此时 $f(\alpha)\approx -1.8\times 10^{-10}$ 。

解 2.2 本题需要计算出函数 $f(V)=pV+aN^2/V-abN^3/V^2-pNb-kNT$ 的零点。绘出函数 f 的图形, 由图可见函数只在区间 $(0.10, 0.06)$ 内有惟一的零点, 且 $f(0.01)<0$, $f(0.06)>0$ 。可以采用下列二分法来计算这个零点:

```
>>f=inline('350000000*V+401000/V-17122.7/V^2-1494500','V');
>>[zero,res,niter]=bisection(f,0.01,0.06,1.e-12,100)
zero=
    0.0427
zero=
    -6.3814e-05
niter=
    35
```

解 2.3 未知数 ω 的值就是函数 $f(\omega)=s(1, \omega)-1=9.8/2[\sinh(\omega)-\sin(\omega)]/\omega^2-1$ 的零点。由函数 f 的图形可以看出 f 在区间 $(0.5, 1)$ 内有惟一的实零点。从这个区间

开始的二分法程序可以写成下列形式,在给定的误差容限下经过 15 次迭代运算可以得到 $\omega = 0.612\ 144\ 470\ 214\ 84$;

```
>>f=inline('9.8/2*sinh(omega)-sin(omega))./omega.^2-1','omega');
>>[zero,res,niter]=bisection(f,0.5,1,1.e-05,100)
zero=
    6.1214e-01
res=
    3.1051e-06
niter=
    15
```

利用 Newton 法在初始值为 0.5 时,要使得到的结果具有相同的有效数字位数只需经过 3 次迭代运算,并且此时的残差要小得多:

```
>>fx=inline('(49/10*cosh(omega)-49/10*cos(omega))./omega.^2-2*...
    (49/10*sinh(omega)-49/10*sin(omega))./omega.^3','omega')
>>[zero,res,niter]=Newton(f,fx,0.5,1.e-05,100)
zero=
    6.1214e-01
res=
    4.4409e-16
niter=
    3
```

解 2.4 不等式(2.3)可由 $|e^{(k)}| < |I^{(k)}|/2$ 和 $|I^{(k)}| < \frac{1}{2}|I^{(k-1)}| < 2^{-k}(b-a)$ 两式联立推导来得到。因此如果在第 $k_{\min}$ 次迭代的误差小于 ε , 则有 $2^{-k_{\min}-1}(b-a) < \varepsilon$, 即 $2^{-k_{\min}-1} < \varepsilon/(b-a)$, 则式(2.3)得证。

解 2.5 原因是第一个公式对舍入误差不敏感。

解 2.6 在解 2.1 中已经分析了给定函数在 γ 取值不同时的零点分布情况。考虑 $\gamma = 2$ 的情况,由初始推测值 $x^{(0)} = 1$ 开始,Newton 法(程序 2)在第 31 次迭代时收敛于 $\bar{a} = 1.011\ 3e-0.4$,误差容限是 $\text{tol} = 1.e-10$,而 f 零点的准确值是 0。可见此时误差较大,原因是 f 在其零点的邻域附近几乎是一个常数。实际上, MATLAB 计算得到相应的残差为 0。再取 $\gamma = 3$, Newton 法在第 10 次迭代运算时收敛于 1.857 920 829 150 20,误差容限为 $\text{tol} = 1.e-16$;而当 $\gamma = 4$ 时经过 12 次迭代收敛于 2.205 621 636 929 58(在这两种情况下 MATLAB 的残差都是零)。

解 2.7 一个正数 a 的平方根和立方根分别是方程 $x^2 = a$ 和 $x^3 = a$ 的解。相应的算法为:对于给定的 $x^{(0)}$ 计算

$$x^{(k+1)} = \frac{1}{2} \left(x^{(k)} + \frac{a}{x^{(k)}} \right), \quad k \geq 0 \quad \text{得到平方根}$$

$$x^{(k+1)} = \frac{1}{3} \left(2x^{(k)} + \frac{a}{(x^{(k)})^2} \right), \quad k \geq 0 \quad \text{得到立方根}$$

解 2.8 设 $\delta x^{(k)} = x^{(k)} - \alpha$, 由函数 f 的 Taylor 展开式可以得到:

$$0 = f(\alpha) = f(x^{(k)}) + \delta x^{(k)} f'(x^{(k)}) + \frac{1}{2} (\delta x^{(k)})^2 f''(x^{(k)}) + O\left((\delta x^{(k)})^3\right) \quad (9.1)$$

由 Newton 法可得:

$$\delta x^{(k+1)} = \delta x^{(k)} - f(x^{(k)}) / f'(x^{(k)}) \quad (9.2)$$

将(9.1)和(9.2)合并, 可得

$$\delta x^{(k+1)} = \frac{1}{2} (\delta x^{(k)})^2 \frac{f''(x^{(k)})}{f'(x^{(k)})} + O\left((\delta x^{(k)})^3\right)$$

再同除以 $(\delta x^{(k)})^2$, 设 $k \rightarrow \infty$, 即可以得到收敛结果。

解 2.9 对于一些给定的 β 值, 方程(2.2)可以有两个不同的根, 它们分别与杠杆系统的不同结构相对应。给定的两个初始值分别使得 Newton 法收敛于其中的一个根。设给定 β 的值为 $\beta = 2k\pi/300$, $k=0, \dots, 100$, 在此情况下要求出该问题的解, 可以使用下列指令来实现(如图 9.1 所示):

```
>>a=[10 13 8 10];
>>f=inline('a(1)/a(2)*cos(beta)-a(1)/a(4)*cos(alpha)-cos(beta-alpha)+...
    a(1)^2+a(2)^2-a(3)^2+a(4)^2)/(2*a(2)*a(4))','alpha','beta','a');
>>df=inline('a(1)/a(2)*sin(alpha)-sin(beta-alpha)','alpha','beta',
    'a');
>>tol=1.e-05;x0=-0.1;x1=2*pi/3;nmax=200;
>>alpha1=[];alpha2=[];niter1=[];niter2=[];
>>vbeta=linspace(0,2*pi/3,100);
>>for beta=vbeta
    [zero,res,niter]=Newton(f,df,x0,tol,nmax,beta,a);
    alpha1=[alpha1,zero]; niter1=[niter1,niter];
    [zero,res,niter]=Newton(f,df,x1,tol,nmax,beta,a);
    alpha2=[alpha2,zero]; niter2=[niter2,niter];
end
```

向量 **alpha1** 和 **alpha2** 元素的值就是不同 β 下得到的角度值, 而向量 **niter1** 和 **niter2** 分别表示在给定误差容限下, 利用 Newton 法需要进行的迭代运算次数(5 或 6 次)。

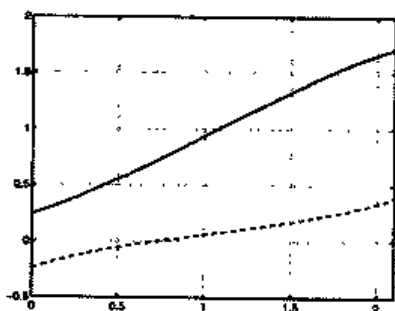


图 9.1 在 $\beta \in [0, 2\pi/3]$ 下, 描述两种不同系统结构的两条曲线

解 2.10 由函数曲线可以看出, f 有两个正实零点 ($\alpha_2 \approx 1.5$ 和 $\alpha_3 \approx 2.5$) 和一个负零点 ($\alpha_1 \approx -0.5$)。Newton 法经过 4 次迭代将收敛于数值 α_1 (设 $x^{(0)} = -0.5$, $\text{tol} = 1.0 \times 10^{-10}$):

```
>>f=inline('exp(x)-2*x^2');df=inline('exp(x)-4*x');x0=-0.5;tol=1.e-10;
>>nmax=100;format long;[zero,res,niter]=Newton(f,df,x0,tol,nmax)
zero=
    -0.53983527690282
res=
    0
niter=
    4
```

给定函数在 $\bar{x} \approx 0.3574$ 处有一个最大值(它可以通过将 Newton 法应用于函数 f' 来得到): 当 $x^{(0)} < \bar{x}$ 时, 该方法收敛于负零。而当 $x^{(0)} = \bar{x}$ 时不能应用 Newton 法, 因为此时 $f'(\bar{x}) = 0$ 。

解 2.11 设 $x^{(0)} = 0$, $\text{tol} = 10^{-17}$ 。Newton 法经过 32 次迭代运算收敛于数值 0.641 182 397 636 49, 将它作为零点 α 的精确解, 可以看出近似误差 $x^{(k)} - \alpha$, $k = 1, 2, \dots, 31$ 在 k 增加时是呈线性减小的。这是因为 α 的阶数是大于 1 的(参见图 9.2)。要得到二阶收敛的方法, 可以通过改进后的 Newton 法来实现。

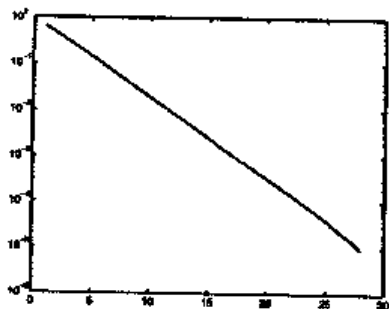


图 9.2 利用 Newton 法计算函数 $f(x) = x^3 - 3x^2 2^{-x} + 3x 4^{-x} - 8^{-x}$ 零点时产生的误差和迭代次数之间的关系

解 2.12 实际上本题就是求解函数 $f(x) = \sin(x) - \sqrt{2gh/v_0^2}$ 的零点。由函数曲线可以看出, f 在区间 $(0, \pi/2)$ 内有一个零点。取 $x^{(0)} = \pi/4$ 和 $\text{tol} = 10^{-10}$, 利用 Newton 法经过 5 次迭代可以得到结果 0.458 628 632 278 59。

解 2.13 利用题设所给条件, 通过下列指令可以求出本题的解:

```
>>f=inline('M-v*(1+l),*((1+l).^n-1)./1','l','M','v','n');
>>df=inline('-v*((1+l).^n-1)./1-n*v*(1+l).^n./1+v*(1+l).*((1+l).^n-1)./1.^2','l','M','v','n');
>>[zero,res,niter]=bisection(f,0.01,01,1.e-12,4,6000,1000,5);
>>[zero,res,niter]=Newton(f,df,zero,1.e-12,100,6000,1000,5);
```

Newton 法经过 3 次迭代可以收敛于期望的结果。

解 2.14 由函数图形可以看出, 区间 $(\pi/6, \pi/4)$ 内有一点 α 满足式(2.15)。由初始值 $x^{(0)} = \pi/4$ 开始, Newton 法经过 5 次迭代运算收敛于近似值 0.596 279 927 465 47。由此得到在走廊中可以穿过的杆的最大长度为 $L = 30.84$ 。

解 2.15 如果 α 是函数 f 的 m 重零点, 则存在函数 h 使得 $h(\alpha) \neq 0$ 和 $f(x) = h(x)(x-\alpha)^m$ 成立。在 Newton 法中用 f 和它的一阶导数的表达式来代替 h 项, 可以得到期望的结果。此时得到的迭代函数形式如下:

$$\phi_N(x) = x - \frac{h(x)(x-\alpha)^m}{h'(x)(x-\alpha)^m + mh(x)(x-\alpha)^{m-1}} = x - \frac{h(x)(x-\alpha)}{h'(x)(x-\alpha) + mh(x)}$$

对上式求导可得:

$$\phi'_N(x) = 1 - \frac{[h'(x)(x-\alpha) + h(x)]D(x) - h(x)(x-\alpha)D'(x)}{[D(x)]^2}$$

其中, $D(x) = h'(x)(x-\alpha) + mh(x)$ 。因此 $\phi'_N(\alpha) = 1 - 1/m$, 于是 Newton 法只有当 $m = 1$ 时才是二阶收敛的。

解 2.16 通过下列指令可以得出函数 f 的曲线:

```
>>f='x^3+4*x^2-10';fplot(f,[-10,10]);grid on;
>>fplot(f,[-5.5]);grid on;
>>fplot(f,[0.5]);grid on
```

可见 f 只有一个实零点, 它的值近似等于 1.36 (参见图 9.3)。

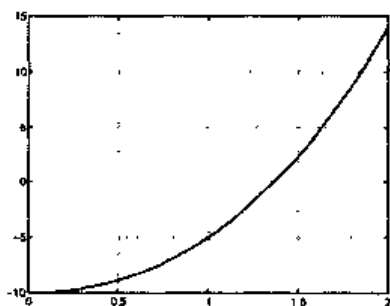


图 9.3 函数 $f(x) = x^3 + 4x^2 - 10$ 在区间 $x \in [0, 2]$ 内的曲线

迭代函数和它的导数分别为:

$$\phi(x) = \frac{2x^3 + 4x^2 + 10}{3x^2 + 8x}$$

$$\phi'(x) = \frac{(6x^2 + 8x)(3x^2 + 8x) - (6x + 8)(2x^3 + 4x^2 + 10)}{(3x^2 + 8x)^2}$$

且 $\phi(\alpha) = \alpha$ 。因为 $\phi'(x) = (6x + 8)f(x)/(3x^2 + 8x)^2$, 所以 $\phi'(\alpha) = 0$ 。因此, 所给方法是二阶收敛的。

解 2.17 因为 $\phi(\alpha) = 0$, 所以题目中给出的方法至少是 2 阶收敛的。

解 2.18 保持所给参数不变, 经过 3 次迭代运算, 该方法收敛于数值 0.641 185 736 496 23。此时得到的结果与先前的计算结果之间的插值小于 10^{-9} 。但是由于函数在 $x=0$ 附近相当平坦, 所以先前计算得到的结果更精确。在图 9.4 中给出了函数 f 在区间 $(0.5, 0.7)$ 内的曲线, 该曲线可通过下列指令得到:

```
>>f='x^3-3*x^2*2^(-x)+3*x*4^(-x)-8^(-x)';
>>fplot(f,[0.5 0.7]); grid on
```

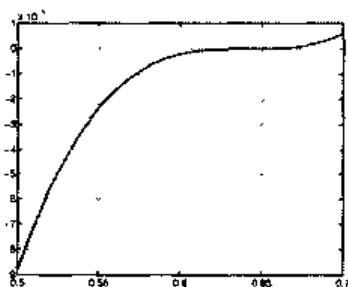


图 9.4 函数 $f(x) = x^3 - 3x^2 2^{-x} + 3x 4^{-x} - 8^{-x}$ 在区间 $x \in [0.5, 0.7]$ 内的曲线

9.3 第 3 章

解 3.1 因为 $x \in (x_0, x_n)$, 所以存在小区间 $I_i = (x_{i-1}, x_i)$ 使得 $x \in I_i$ 。显然,

$\max_{x \in I_i} |(x - x_{i-1})(x - x_i)| = h^2/4$ 。由于 $|x - x_{i+1}|$ 的上限是 $2h$, $|x - x_{i-2}|$ 的上限是 $3h$, 将这两个条件代入即可以得到不等式(3.6)。

解 3.2 这里取 $n=4$, 在给定的区间内可以求出每个函数的 5 阶导数。得到下列结果 $\max_{x \in [-1,1]} |f_1^{(5)}| < 1.76$, $\max_{x \in [-1,1]} |f_2^{(5)}| < 1.55$, $\max_{x \in [-\pi/2, \pi/2]} |f_3^{(5)}| < 1.42$ 。相应的误差上限分别是 0.0028、0.0024 和 0.0212。

解 3.3 利用 `polyfit` 函数可以计算出这两种情况下的 3 阶插值多项式:

```
>>years=[1975 1980 1985 1990];
>>east=[70.2 70.2 70.3 71.2];
>>west=[72.8 74.2 75.2 76.4];
>>ceast=polyfit(years,eor,3);
>>cwest=polyfit(years,eoc,3);
>>esteast=polyval(ceast,[1970 1983 1988 1995])
esteast=
    69.6000    70.2032    70.6992    73.6000
>>estwest=polyval(cwest,[1970 1983 1988 1995])
estwest=
    70.4000    74.80096    75.8576    78.4000
```

可见, 1970 年西欧的平均寿命是 70.4(`estwest(1)`), 与真实值相差 1.4。由插值多项式曲线的对称性可以得出, 1995 年平均寿命的近似值为 78.4, 它比实际值也多出同一个量(实际上, 真实寿命值是 77.5)。西欧的情况与此不同, 1970 年的近似值与真实值完全相等, 而 1995 年的估计值比真实值要大得多(结果为 73.6, 大于真实值 71.2)。

解 3.4 以月为时间单位, 初始时间 $t_0=1$ 对应于 1987 年 11 月, 而 $t_7=157$ 对应于 2000 年 11 月。利用下列指令可以求出所给价格的插值多项式系数:

```
>>time=[1 14 37 63 87 99 109 157];
>>price=[4.5 5 6 6.5 7 7.5 8 8];
>>[c]=polyfit(time,price,7);
```

设 `[price2002]=polyval(c,180)`, 可以得出在 2002 年 11 月该杂志价格的近似值为 10.8 欧元。

解 3.5 在题设给出的特定条件下, 通过 `spline` 函数利用立方插值样条计算得到的结果与插值多项式所得结果是一致的。但这一结论不能推广应用到一般情况。

解 3.6 可以利用下列指令求解:

```
>>T=[4:4:20];
>>rho=[1000.7794,1000.6427,1000.2805,999.7165,998.9700];
>>Tnew=[6:4:18];format long e;
>> rhonew=spline(T,rho,Tnew)
rhonew=
Columns 1 through 3
1.000740787500000e+03    1.000488237500000e+03    1.000022450000000e+03
```

Column 4

9.993649250000000e+02

进一步测量结果表明,此时得到的近似值是相当精确的。值得注意的是,海水(UNESCO, 1980)状态方程中密度关于温度是四阶相关的。而 T 的四次方的数量级是 10^{-9} 。

解 3.7 描绘立方差值样条曲线,可以利用下列指令来实现:

```
>>year=[1965 1970 1980 1990 1991];
>>production=[17769 24001 25961 34336 29036 33417];
>>years=[1962 1977 1992];
>>estimateprod=spline(year,production,years);
```

得到的值如下:在 1962 年产量为 $5146.1 \times 10^5 \text{ kg}$; 在 1977 年为 $22641.7 \times 10^5 \text{ kg}$; 在 1992 年为 $41894.4 \times 10^5 \text{ kg}$ 。与相应的真实值(分别为 12380 、 27403 、 $32059 \times 10^5 \text{ kg}$)进行对比表明:在插值区间之外,利用样条法得到的结果是不准确的(参见图 9.5)。

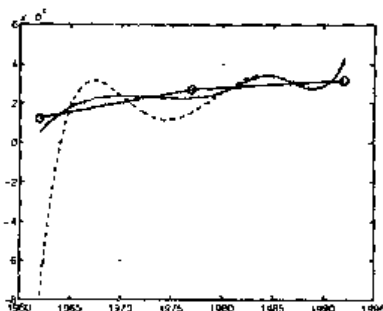


图 9.5 习题 3.7 中的立方样条曲线(实线)和插值多项式曲线(虚线)。圆点表示插值运算中使用的值

但尽管如此,样条曲线在插值区间的端点处仍然可以得到 1965 年的结果。而在这个端点附近插值多项式曲线则发生较大的振荡,在 1962 年的估计值小于真实值 $-77685 \times 10^5 \text{ kg}$ 。

解 3.8 要求出在区间 $[-1, 1]$ 上均匀分布的 21 个节点处的插值多项式 p 和样条 $s3$,可以利用下列指令在 21 个节点处进行运算得到:

(译者注:原书中出现的“201”属印刷错误,应为“21”。)

```
>>pert=1.e-04;
>>x=[-1:2/20:1]; y=sin(2*pi*x)+(-1).^[1:21]*pert; z=[-1:0.01:1];
>>c=polyfit(x,y,20); p=polyval(c,z); s3=spline(x,y,z);
```

在没有使用扰动数据时($pert=0$),函数 p 和 $s3$ 的曲线与所给函数的曲线是一致的。在使用了扰动数据后($pert=1.e-0.4$),曲线会发生较大变化。特别指出,在区间的端点处插值多项式会发生强烈的振荡,而样条曲线则没有发生变化(参见图 9.6)。这个例子表明,利用样条曲线作为近似时总体扰动误差是更稳定的。

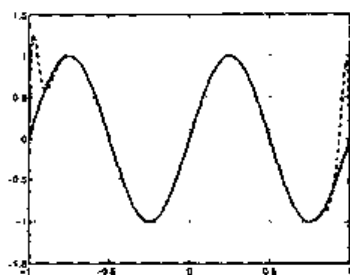


图 9.6 在扰动数据下的插值多项式曲线(虚线)和立方插值样条曲线(实线)。注意图中插值多项式在区间端点的振荡情况

解 3.9 如果 $n=m$, 设 $\tilde{f} = \prod_n f$, 可见(3.18)的第一项为零。则此时 $\prod_n f$ 是最小平方系统的惟一解。

解 3.10 利用 polyfit 函数求得的多项式系数为(只显示出 4 位有效数字):

$$K=0.67, \quad a_4=6.301 \times 10^{-8}, \quad a_3=-8.320 \times 10^{-8}, \quad a_2=-2.850 \times 10^{-4}, \\ a_1=9.718 \times 10^{-4}, \quad a_0=-3.032;$$

$$K=1.5, \quad a_4=-4.225 \times 10^{-8}, \quad a_3=-2.066 \times 10^{-6}, \quad a_2=3.444 \times 10^{-4}, \\ a_1=3.364 \times 10^{-3}, \quad a_0=3.364;$$

$$K=2, \quad a_4=-1.012 \times 10^{-7}, \quad a_3=-1.431 \times 10^{-7}, \quad a_2=6.988 \times 10^{-4}, \\ a_1=-1.060 \times 10^{-4}, \quad a_0=4.927;$$

$$K=3, \quad a_4=-2.323 \times 10^{-7}, \quad a_3=7.980 \times 10^{-7}, \quad a_2=1.420 \times 10^{-3}, \\ a_1=-2.605 \times 10^{-3}, \quad a_0=7.315.$$

在图 9.7 中给出了利用表 3.1 的第一列数据求得的多项式的曲线。

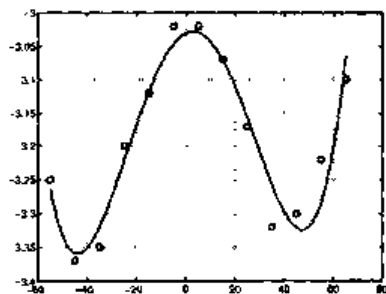


图 9.7 四阶最小平方多项式(实线)与表 3.1 中的第一列数据的对比情况

解 3.11 利用 polyfit 函数, 重复解 3.7 中的前 3 条指令, 可以得到下列数值(单位为 10^5 kg): 在 1962 年为 15280.12, 在 1977 年为 27407.10, 在 1992 年为 32019.01, 可见此时得到的结果更接近真实值(分别为 12380、27403 和 32059)。

解 3.12 注意方差可以表示成 $v = \frac{1}{n+1} \sum_{i=0}^n x_i^2 - M^2$, 据此可以用均值和方差表示出系统(3.20)系数。

解 3.13 题设结论可以由相应系统的第一个方程推导得出, 该系统给出了最小平方直线的系数。

9.4 第4章

解 4.1 在点 x_0 处将函数 f 展成三阶 Taylor 展开式, 可得:

$$\begin{aligned} 4f(x_1) &= 4f(x_0) + 4hf'(x_0) + 2h^2f''(x_0) + \frac{2}{3}h^3f'''(\xi) \\ -f(x_2) &= -f(x_0) - 2hf'(x_0) - 2h^2f''(x_0) - \frac{4}{3}h^3f'''(\eta) \end{aligned}$$

其中, $\xi \in (x_0, x_1)$, $\eta \in (x_0, x_2)$ 是两个特定点。将两个不等式相加得到:

$$-3f(x_0) + 4f(x_1) - f(x_2) = 2hf'(x_0) + \frac{2h^3}{3}(f'''(\xi) - 2f'''(\eta))$$

两边同除以 $2h$, 利用中值定理, 可以得出在 $\xi_0 \in (x_0, x_2)$ 处题设结论是成立的。采用类似的方法, 在 x_n 处使用这一公式也可以得到相应的结论。

解 4.2 由 Taylor 展开式

$$\begin{aligned} f(\bar{x}+h) &= f(\bar{x}) + hf'(\bar{x}) + \frac{h^2}{2}f''(\bar{x}) + \frac{h^3}{6}f'''(\xi) \\ f(\bar{x}-h) &= f(\bar{x}) - hf'(\bar{x}) + \frac{h^2}{2}f''(\bar{x}) - \frac{h^3}{6}f'''(\eta) \end{aligned}$$

其中, ξ 和 η 是两个特定点。将这两个表达式相减, 再将方程两边同除以 $2h$ 可以得到结果(4.9)。

解 4.3 设 $f \in C^4$, 按照解 4.2 中的方法, 可以得到下列误差(ξ_1 、 ξ_2 和 ξ_3 是特定的点):

$$\text{a. } -\frac{1}{4}f^{(4)}(\xi_1)h^3, \quad \text{b. } -\frac{1}{12}f^{(4)}(\xi_2)h^3, \quad \text{c. } \frac{1}{30}f^{(4)}(\xi_3)h^4$$

解 4.4 利用近似式(4.8)计算得到的结果如下:

$t/\text{月}$	0	0.5	1	1.5	2	2.5	3
δ_n	--	78.04	44.95	19.28	7.02	2.39	--
n'	--	77.78	39.09	15.19	5.29	1.77	--

将这一结果与 $n'(t)$ 的精确解进行比较, 可以看出计算得到的值是足够精确的。

解 4.5 求积误差的界限可由下式给出:

$$(b-a)^3 / (24M^2) \max_{x \in [a, b]} |f''(x)|$$

其中, $[a, b]$ 为积分区间, M 是小区间个数(未知的)。

函数 f_1 是无限可微的。由 f_1'' 的图形可以看出在积分区间内 $|f_1''(x)| \leq 2$ 。于是在 f_1 的求积误差小于 10^{-4} 时, 有 $5^3 / (24M^2) 2 < 10^{-4}$, 即 $M > 322$ 。

函数 f_2 也是任意阶可微的。由于 $\max_{x \in [0, \pi]} |f_2''(x)| = \sqrt{2}e^{3/4\pi}$, 在积分误差小于 10^{-4} 时, 有 $M > 439$ 。这些结果实际上对求积误差进行了过高估计。事实也的确如此, 在误差容限 10^{-4} 下, 需要划分区间的最小数目远低于这里得到的结果(比如对于函数 f_1 所需的区间数为 51)。需要注意的是: 函数 f_3 在积分区间内是不可微的, 这种从理论上估计误差的方法不再适用。

解 4.6 在每个小区间 I_k , $k=1, \dots, M$ 上, 产生的误差等于 $H^3/24 f''(\xi_k)$, 其中, $\xi_k \in (x_{k-1}, x_k)$, 因此整体误差为 $H^3/24 \sum_{k=1}^M f''(\xi_k)$ 。由于 f'' 是区间 (a, b) 上的连续函数, 于是存在点 $\xi \in (a, b)$, 使得 $f''(\xi) = \frac{1}{M} \sum_{k=1}^M f''(\xi_k)$ 成立。利用这一结论连同 $MH=b-a$, 可以推出式(4.13)

解 4.7 在每个小区间上的局部误差的积累是产生这一现象的原因。

解 4.8 通过适当构造, 中点公式可以实现对变量求积分运算。要验证它对线性多项式进行的是求积运算, 只需验证 $I(x) = I_{MP}(x)$ 即可。实际上可得下式:

$$I(x) = \int_a^b x \, dx = \frac{b^2 - a^2}{2}, \quad I_{MP}(x) = (b-a) \frac{b+a}{2}$$

(译者注: 此处原书中 $I(x) = I_{PM}(x)$ 属印刷错误, 应改为 $I(x) = I_{MP}(x)$ 。)

解 4.9 对于函数 f_1 利用梯形公式计算时可得 $M=71$, 利用 Gauss 公式计算时 $M=7$ 。可以很明显地看出后者的计算效果更好。

解 4.10 方程(4.16)表明复合梯形公式在 $H=H_1$ 时的求积误差是 CH_1^2 , 其中, $C = -\frac{b-a}{12} f''(\xi)$ 。如果 f'' 变化不是非常显著, 可以认为在 $H=H_2$ 时, 求积误差是 CH_2^2 。于是可得到下列表达式:

$$I(f) \approx I_1 + CH_1^2, \quad I(f) \approx I_2 + CH_2^2 \quad (9.3)$$

联立可得 $C = (I_1 - I_2)/(H_2^2 - H_1^2)$ 。将这个值代入(9.3)中的一个公式中, 可以得到式(4.29), 它是优于 I_1 和 I_2 的 $I(f)$ 近似值。

解 4.11 寻找使得 $I_{\text{approx}}(x^p) = I(x^p)$ 成立的最大正整数 p 。当 $p=0, 1, 2, 3$, 时, 可以得到下列非线性系统, 它含有 4 个方程和 4 个未知数 α 、 β 、 $\bar{x}$ 和 $\bar{z}$:

$$\begin{aligned} p=0 & \rightarrow \alpha + \beta = b - a \\ p=1 & \rightarrow \alpha \bar{x} + \beta \bar{z} = \frac{b^2 - a^2}{2} \\ p=2 & \rightarrow \alpha \bar{x}^2 + \beta \bar{z}^2 = \frac{b^3 - a^3}{3} \\ p=3 & \rightarrow \alpha \bar{x}^3 + \beta \bar{z}^3 = \frac{b^4 - a^4}{4} \end{aligned}$$

由前两个方程可以消去 α 和 $\bar{z}$, 得到一个只含有未知数 β 和 $\bar{x}$ 的系统。特别指出, 此时得到的是一个关于 β 的二阶方程, 可以把 β 作为 $\bar{x}$ 的函数来求解这个方程。另外, 含有 $\bar{x}$ 的非线性方程可以利用 Newton 法来解, 得到两个 $\bar{x}$ 值就是 Gauss 求积点的横坐标。

解 4.12 由下式

$$\begin{aligned} f_1^{(4)}(x) &= \frac{24}{\left(1+(x-\pi)^2\right)^5 (2x-2\pi)^4} - \frac{72}{\left(1+(x-\pi)^2\right)^4 (2x-2\pi)^2} \\ &\quad + \frac{24}{\left(1+(x-\pi)^2\right)^3}, \\ f_2^{(4)}(x) &= -4e^x \cos(x) \end{aligned}$$

可以看出 $|f_1^{(4)}(x)|$ 的最大值的上限是 $M_1 \approx 25$, 而 $|f_2^{(4)}(x)|$ 的最大值的上限是 $M_2 \approx 93$, 因此由(4.22)可得: 对于前一个函数 $H < 0.21$, 对于后一个函数 $H < 0.16$ 。

解 4.13 利用指令 `int('exp(-x^2/2)', 0, 2)` 可得积分值是 1.196 288 013 322 61。

在同一区间内利用 Gauss 公式(4.20)求解得到的值是 1.202 780 276 223 54(此时绝对误差是 $6.4923e-03$), 而 Simpson 公式返回的值是 1.187 152 640 695 72, 此时的误差稍大一些(等于 $9.1354e-03$)。

解 4.14 由于被积函数是非负的, 所以 $I_k > 0 \forall k$ 。因此, 由递归公式计算得到

的所有值都应是而非负的。但递归公式会因舍入误差的积累变得不稳定，使得输出的结果为负：

```
>>l(1)=1/exp(1);for k=2:20,l(k)=1-k*l(k-1);end
>>l(20)
-30.1924
```

利用复合 Simpson 公式，取 $H < 0.25$ ，可以在给定的误差容限下计算出积分值。

解 4.15 利用 Simpson 公式得到：

$$I_1 = 1.19616568040561, \quad I_2 = 1.19628173356793, \quad \Rightarrow I_R = 1.19628947044542$$

I_R 的绝对误差等于 $-1.4571e-06$ (求解 I_1 时得到的是二阶的情况，求解 I_2 时还需再乘一个因子 $1/4$)。利用 Gauss 公式可以得到(误差的大小在括号内给出)：

$$\begin{aligned} I_1 &= 1.19637085545393 \quad (-8.2842e-05), \\ I_2 &= 1.19629221796844 \quad (-4.2046e-06), \\ I_R &= 1.19628697546941 \quad (1.0379e-06). \end{aligned}$$

显然，Richardson 插值法更优。

解 4.16 利用 Simpson 公式计算出 $j(r) = \sigma / (\varepsilon_0 r^2) \int_0^r f(\xi) d\xi$ ，其中， $r = k/10$ ， $k = 1, \dots, 10$ ， $f(\xi) = e^\xi \xi^2$ 。

要求出积分误差，需要计算四阶导数 $f^{(4)}(\xi) = e^\xi (\xi^2 + 8\xi + 12)$ 。因为 $f^{(4)}$ 在积分区间 $(0, r)$ 内是单调递增的，所以导数 $f^{(4)}$ 的最大值在 $\xi = r$ 处取得。于是可以得到下列结果：

```
>>r=[0.1:0.1:1];
>>maxf4=exp(r).*(r.^2+8*r+12);
maxf4=
Columns 1 through 7
    14.1572    16.6599    19.5595    22.9144    26.7917    31.2676    36.4288
Columns 8 through 10
    42.3743    49.2167    57.0839
```

对于给定的 r ，误差小于 10^{-10} 时，有 $H_r^4 < 10^{-10} 2880 / (rf^{(4)}(r))$ 。当 $r = k/10$ ， $k = 1, \dots, 10$ 时，通过下列指令可以计算出满足这一不等式需要划分的小区数最小数目。向量 M 的元素为下列数值：

```
>>x=[0.1:0.1:1];f4=exp(x).*(x.^2+8*x+12);
>>H=(10^(-10)*2880./(x.*f4)).^(1/4); M=fix(x./H)
M=
     4     11     20     30     41     53     67     83    100    118
```

因此， $j(r)$ 的值是：

```

>>sigma=0.36;epxilon0=8.859e-12;
f=inline('exp(x).*x.^2');
for k=1:10
    r=k/10;
    j(k)=simpsonc(0,4,M(k),f);
    j(k)=j(k)*sigma/r*epsilon0;
end

```

解 4.17 利用复合 Simpson 公式计算 $E(213)$ 的值, 在计算中不断增加小区间的个数, 直到两次相邻结果的差值小于 10^{-11} 为止:

```

>>f=inline('2.39e-11./((x.^5).*(exp(1.432./T*x))-1))','x',T');
>>a=3.e-04;b=14.e-04;T=213;
>>i=1;err=1;lold=0;while err>=1.e-11
    l=simpsonc(a,b,i,f,T);
    err=abs(l-lold)/abs(l);
    lold=l;
    i=i+1
end

```

程序返回值为 $i = 59$ 。因此, 使用 58 个均匀分布的子区间计算得到的 $E(213)$ 值含有 10 位有效数字。利用 Gauss 公式经过 53 次迭代运算可以得到同样的结果。值得注意的是, 如果利用复合梯形公式求解, 需要划分 1609 个小区间。

解 4.18 在整个区间内, 给定函数不是规则的, 无法应用理论收敛方法(4.22)来计算。可以将积分区间分成两段(0, 0.5)和(0.5, 1), 在每个区间内函数是规则的(3 阶多项式)。特别指出, 如果在每个区间内分别应用 Simpson 公式, 可以精确地计算出 f 的积分值。

9.5 第 5 章

解 5.1 利用 Laplace 公式(1.8)计算阶数 $k \geq 2$ 的矩阵的行列式的值, 需要进行的代数运算(加法、减法和乘法)次数 r_k 满足下列差分方程:

$$r_k - kr_{k-1} = 2k - 1$$

其中, $r_1=0$ 。方程两边同乘以 $1/k!$, 可得:

$$\frac{r_k}{k!} - \frac{r_{k-1}}{(k-1)!} = \frac{2k-1}{k!}$$

对上式两边从 2 到 n 求和, 可得:

$$r_n = n! \sum_{k=2}^n \frac{2k-1}{k!} = n! \sum_{k=1}^{n-1} \frac{2k+1}{(K+1)!}, \quad n \geq 1$$

解 5.2 利用下列 MATLAB 指令可以计算出行列式的值和相应的 CPU 时间:

```
>>t=[];for i=3:500
    A=magic(i);tt=cputime;d=det(A);t=[t,cputime-tt];
end
```

对数值 $n=[3:500]$ 和 t 进行近似的三次最小平方多项式的系数为:

```
>> format long;c=polyfit(n,t,3)
c=
    0.000000002102187    0.00000171915661   -0.00039318949610
    0.01055682398911
```

第一项的系数(与 n^3 相乘的)比较小,但是与第二项相比还不能忽略。实际上,在计算四阶最小平方多项式时,可以得到下列系数:

```
>>c=polyfit(i,t,4)
c=
Columns 1 through 4
-0.0000000000000051    0.000000002153039    0.00000155418071
-0.00037453657810
Column 5
    0.01006704351509
```

从这个结果可以看出,求一个 n 阶的矩阵的行列式的值所需的操作数近似为 n^3 ops。

解 5.3 首先 $\det A_1=1$, $\det A_2=\varepsilon$, $\det A_3=\det A=2\varepsilon+12$ 。因此如果 $\varepsilon=0$, 则第二个子矩阵是奇异的,命题 5.1 不能适用。如果 $\varepsilon=-6$ 则矩阵 A 是奇异的。此时 Gauss 因式分解得到的值为:

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 1.25 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 1 & 7 & 3 \\ 0 & -12 & -4 \\ 0 & 0 & 0 \end{bmatrix}$$

注意上式中的 U 是奇异矩阵(这可以利用矩阵 A 是奇异矩阵这一条件来得到)。

解 5.4 在第 1 步时,计算元素 l_{ik} , $i=2, \dots, n$ 需要进行 $n-1$ 次除法运算。求解新元素 $a_{ij}^{(2)}$, $j=2, \dots, n$ 时需要进行 $(n-1)^2$ 次乘法和 $(n-1)^2$ 次加法运算。在第 2 步需要进行的除法运算的次数是 $(n-2)$, 而乘法和加法的运算次数是 $(n-2)^2$ 。在最后一步即第 $n-1$ 步时,需要进行 1 次加法、1 次乘法和 1 次除法运算。使用下列等式:

$$\sum_{s=1}^q s = \frac{q(q+1)}{2}, \quad \sum_{s=1}^q s^2 = \frac{q(q+1)(2q+1)}{6}, \quad q \geq 1$$

可以得出进行 Gauss 分解所需的运算次数为 $2(n-1)n(n+1)/3 + n(n-1)$ ops。忽略低次项, 可以得出进行 Gauss 因式分解的计算代价为 $2n^3/3$ ops。

解 5.5 由定义可知, 矩阵 $A \in \mathbf{R}^{n \times n}$ 的逆矩阵 X 满足 $XA = AX = I$ 。因此, 取 $j=1, \dots, n$ 时, X 的列向量 y_j 是线性系统 $Ay_j = e_j$ 的解, 其中 e_j 是 $\mathbf{R}^n$ 空间的规范基的第 j 个向量, 它的第 j 个元素等于 1 而其余元素均为零。将矩阵 A 进行了 LU 分解之后, 求解 A 的逆矩阵只须求解相同矩阵表示的线性系统即可, 只是此时方程右侧的形式是不同的。

解 5.6 利用程序 7 求得的因式为:

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & -3.38 \cdot 10^{15} & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 1 & 1 & 3 \\ 0 & -8.88 \cdot 10^{-16} & 14 \\ 0 & 0 & 4.73 \cdot 10^{-16} \end{bmatrix}$$

计算这两个矩阵的乘积可以得到

```
>>L*U
ans=
    1.0000    1.0000    3.0000
    2.0000    2.0000   20.0000
    3.0000    6.0000   -2.0000
```

它不等于 A , 因为在位置(3, 3)处的元素等于-2, 而相应的 A 中元素等于 4。

解 5.7 通常, 对称矩阵只存储它的三角部分(上三角或下三角)的元素。因此, 如果一种算法没有充分利用矩阵的对称性, 则从存储效率角度来讲就不是最优的。这正是对行主元素进行操作时所发生的情况。我们可以采取的一种办法是对于具有相同下标的元素同时进行行列互换, 以便把主元素的选择限制在对角元素上。更一般地, 行列互换的主元素法称为全主元法(参见 [QSS00], 第 3 章)。

解 5.8 L 和 U 因子分别为:

$$L = \begin{bmatrix} 1 & 0 & 0 \\ (\varepsilon-2)/2 & 1 & 0 \\ 0 & -1/\varepsilon & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 2 & -2 & 0 \\ 0 & \varepsilon & 0 \\ 0 & 0 & 3 \end{bmatrix}$$

当 $\varepsilon \rightarrow 0$ 时, $l_{32} \rightarrow \infty$ 。尽管如此, 当 ε 趋于零时, 系统的解仍是准确的, 这一点可以通过下列指令看出:

```

>>e=1;for k=1:10
    b=[0;e;2];
    L=[1 0 0; (e-2)*0.5 1 0; 0 -1/e 1]; U=[2 -2 0; 0 e 0; 0 0 3];
    Y=L\b; x=U\Y;err(k)=max(abs(x-ones(3,1)));e=e*0.1;
end
>>err
err=
    0    0    0    0    0    0    0    0    0    0

```

解 5.9 当 i 增加时, 计算结果的精确度逐渐下将。实际上, 当 $i=1$ 时误差标准等于 $2.63 \cdot 10^{-14}$, 当 $i=2$ 时等于 $9.89 \cdot 10^{-10}$, 当 $i=3$ 时等于 $2.10 \cdot 10^{-6}$ 。产生这一现象的原因是当 i 增加时, A_i 的条件数也相应地增加。利用 `cond` 函数可以求出, A_i 的条件数在 $i=1$ 时 $\approx 10^3$, 在 $i=2$ 时 $\approx 10^7$, 在 $i=3$ 时 $\approx 10^{11}$ 。

解 5.10 如果 (λ, v) 表示一个矩阵 A 的特征值和特征向量构成的数对, 则 λ^2 是矩阵 A^2 相对于同一特征向量的特征值。实际上, 从 $Av = \lambda v$ 可以得到 $A^2v = \lambda Av = \lambda^2 v$ 。因此如果 A 是正定对称阵, 则有 $K(A^2) = (K(A))^2$ 。

解 5.11 Jacobi 法的迭代矩阵为:

$$B_J = \begin{bmatrix} 0 & 0 & -\alpha^{-1} \\ 0 & 0 & 0 \\ -\alpha^{-1} & 0 & 0 \end{bmatrix}$$

它的特征值为 $\{0, \alpha^{-1}, -\alpha^{-1}\}$ 。则当 $|\alpha| > 1$ 时, 该方法收敛。

Gauss-Seidel 法的迭代矩阵为:

$$B_{GS} = \begin{bmatrix} 0 & 0 & -\alpha^{-1} \\ 0 & 0 & 0 \\ 0 & 0 & \alpha^{-2} \end{bmatrix}$$

它的特征值为 $\{0, 0, \alpha^{-2}\}$ 。因此当 $|\alpha| > 1$ 时, 该方法收敛。需要特别指出的是, 由于 $\rho(B_{GS}) = [\rho(B_J)]^2$, Gauss-Seidel 法的收敛速度要比 Jacobi 法更快一些。

解 5.12 使得 Jacobi 法和 Gauss-Seidel 法同时收敛的充分条件为矩阵 A 是严格主元素占优的。 A 的第 2 行满足主元素占优, 需要 $|\beta| < 5$ 。注意到如果直接要求迭代矩阵的谱半径小于 1 (它是收敛的充分必要条件), 此时这两种方法的 (非严格的) 限制条件是 $|\beta| < 25$ 。

解 5.13 向量形式的松弛法为:

$$(I - \omega D^{-1}E)x^{(k+1)} = [(1-\omega)I + \omega D^{-1}F]x^{(k)} + \omega D^{-1}b$$

其中, $A=D-E-F$, D 为矩阵 A 的对角线矩阵, E 和 F 是矩阵 A 下(上)三角阵。相应的迭代矩阵是:

$$B(\omega) = (I - \omega D^{-1}E)^{-1}[(1-\omega)I + \omega D^{-1}F]$$

如果设 λ_i 表示 $B(\omega)$ 的特征值, 可得:

$$\left| \prod_{i=1}^n \lambda_i \right| = \left| \det[(1-\omega)I + \omega D^{-1}F] \right| = |1-\omega|^n$$

因此, 至少有一个特征值应满足不等式 $|\lambda_i| \geq |1-\omega|$ 。收敛的必要条件之一是 $|1-\omega| < 1$, 即 $0 < \omega < 2$ 。

解 5.14 给定矩阵是对称的, 要验证出它是否为正定阵, 即对于 $\mathbf{R}^2$ 空间中所有的 $z \neq \mathbf{0}$, 是否有 $z^T A z > 0$, 可以利用下列指令实现:

```
>>syms z1 z2 real
>>z=[z1;z2];A=[3 2; 2 6];
>>pos=z'*A*z;simple(pos)
ans
3*z1^2+4*z1*z2+6*z2^2
```

其中, `syms z1 z2 real` 的作用是声明符号变量 $z1$ 和 $z2$ 为实数, 而 `simple(pos)` 函数是将 `pos` 进行代数化简, 并返回最简形式。因为得到的结果形式为 $2*(z1+z2)^2+z1^2+4*z2^2$, 可以看出这一结果是正数。因此, 给定矩阵是正定对称阵, Gauss-Seidel 法是收敛的。

解 5.15 由已知可得:

$$\text{对于 Jacobi 法: } \begin{cases} x_1^{(i)} = \frac{1}{2}(1-x_2^{(i)}) \\ x_2^{(i)} = -\frac{1}{3}(x_1^{(i)}) \end{cases} \Rightarrow \begin{cases} x_1^{(i)} = \frac{1}{4} \frac{\partial^2 \Omega}{\partial v^2} \\ x_2^{(i)} = -\frac{1}{3} \frac{\partial^2 \Omega}{\partial v^2} \end{cases}$$

$$\text{对于 Gauss-Seidel 法: } \begin{cases} x_1^{(i)} = \frac{1}{2}(1-x_2^{(i)}) \\ x_2^{(i)} = -\frac{1}{3}x_1^{(i)} \end{cases} \Rightarrow \begin{cases} x_1^{(i)} = \frac{1}{4} \\ x_2^{(i)} = -\frac{1}{12} \end{cases}$$

对于梯度法, 首先计算初始残量:

$$\mathbf{r}^{(0)} = \mathbf{b} - A\mathbf{x}^{(0)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} - \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \mathbf{x}^{(0)} = \begin{bmatrix} -3/2 \\ -5/2 \end{bmatrix}$$

又因为: $\mathbf{P}^{-1} = \begin{bmatrix} 1/2 & 0 \\ 0 & 1/3 \end{bmatrix}$

可得 $\mathbf{z}^{(0)} = \mathbf{P}^{-1}\mathbf{r}^{(0)} = (-3/4, 5/6)^T$, 因此可得:

$$\alpha_0 = \frac{(\mathbf{z}^{(0)})^T \mathbf{r}^{(0)}}{(\mathbf{z}^{(0)})^T A \mathbf{z}^{(0)}} = \frac{77}{107}$$

和

$$\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \alpha_0 \mathbf{z}^{(0)} = (197/428, -32/321)^T$$

解 5.16 在稳态情况下, $\rho(B_\alpha) = \min_{\lambda} |1 - \alpha\lambda|$, 其中 λ 是 $P^{-1}A$ 的特征值。 α 的最优解可以通过方程 $|1 - \alpha\lambda_{\min}| = |1 - \alpha\lambda_{\max}|$ 来得到, 即 $1 - \alpha\lambda_{\min} = -1 + \alpha\lambda_{\max}$, 于是可以推导出式(5.39)。又因为

$$\rho(B_\alpha) = 1 - \alpha\lambda_{\min} \quad \forall \alpha \leq \alpha_{\text{opt}}$$

当取 $\alpha = \alpha_{\text{opt}}$ 时, 可得式(5.44)。

解 5.17 在已知条件下, 与 Leontieff 模型相关的矩阵不是正定矩阵。实际上, 利用下列指令:

```
>>for i=1:20;for j=1:20;c(i,j)=i+j;end ;end;A=eye(20)-c;
>>min(eig(A))
ans=
    -448.5830
>>max(eig(A))
ans=
    30.5830
```

可以得到, 最小的特征值是负数, 而最大的特征值是正数。因此, 此时不能保证梯形法收敛。但由于 A 是非奇异的, 给定系统就等价于 $A^T A \mathbf{x} = A^T \mathbf{b}$, 其中 $A^T A$ 是对称正定阵。利用梯形法求解后者, 需要残差小于 10^{-10} , 从初始值 $\mathbf{x}^{(0)} = \mathbf{0}^T$ 开始:

```
>>b=[1:20]'; aa=A'*A;b=A'*b;x0=zeros(20,1);
>>[x,iter]=richardson(aa,b,x0,100,1.e-10);
```

经过 15 次迭代运算,可以得到收敛结果。在这一过程中的缺点是矩阵 A^1A 的条件数大于矩阵 A 的条件数。

9.6 第 6 章

解 6.1 对于矩阵 A_1 : 幂法在第 34 次迭代运算时收敛于数值 2.000 000 000 049 89。对于矩阵 A_2 : 从相同初始向量开始运算,幂法需要 457 次迭代运算才收敛于数值 1.999 999 999 906 11。收敛速度变慢的原因是两个最大的特征值距离较近。还有,对于矩阵 A_3 ,幂法是不收敛的,因为矩阵 A_3 在模值最大时有两个不同的特征值(i 和 $-i$)。

解 6.2 与表格中的数值相对应的 Leslie 矩阵为:

$$A = \begin{bmatrix} 0 & 0.5 & 0.8 & 0.3 \\ 0.2 & 0 & 0 & 0 \\ 0 & 0.4 & 0 & 0 \\ 0 & 0 & 0.8 & 0 \end{bmatrix}$$

利用幂法可得 $\lambda_1 \approx 0.5353$ 。不同年龄区段内人口数量的标准分布是通过下列单位向量给出的,即 $x_1 \approx (0.8477, 0.3167, 0.2367, 0.3537)^T$ 。

解 6.3 由初始假设条件:

$$y^{(0)} = \beta^{(0)} \left(\alpha_1 x_1 + \alpha_2 x_2 + \sum_{i=3}^n \alpha_i x_i \right)$$

其中, $\beta^{(0)} = 1/\|x^{(0)}\|$ 。利用与 6.1 节中类似的方法,在第 k 步运算时可得:

$$y^{(k)} = \gamma^k \beta^{(k)} \left(\alpha_1 x_1 e^{ik\theta} + \alpha_2 x_2 e^{-ik\theta} + \sum_{i=3}^n \alpha_i \frac{\lambda_i^k}{\lambda^k} x_i \right)$$

式中前两项没有消掉,元素符号相反, $y^{(k)}$ 序列是振荡的,不会收敛。

解 6.4 由特征值方程 $Ax = \lambda x$, 可得 $A^{-1}Ax = \lambda A^{-1}x$, 于是 $A^{-1}x = (1/\lambda)x$ 。

解 6.5 幂法应用于矩阵 A 时,生成的按模最大特征值序列是振荡的(参见图 9.8)。产生原因是该特征值不惟一。

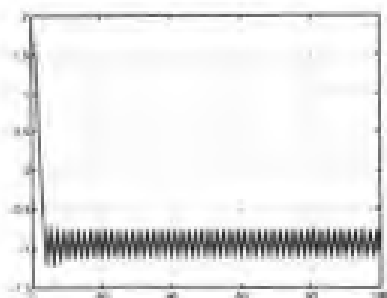


图 9.8 利用幂法得到的解 6.5 中的矩阵的按模最大特征值

解 6.6 利用程序 9 计算按模最大的特征值:

```
>>A=Wilkinson(7);
>>x0=ones(7,1);tol=1.e-15;nmax=100;
>>[lambda,s,iter]=eigpower(A,tol,nmax,x0);
```

经过 35 次迭代得到 $\lambda = 3.76155718183189$ 。要找出 A 的最大的负特征值,可以利用移位幂法,特别地,可以把计算得到的最大正特征值作为幂法的移位置量。于是:

```
>>[lambda2,x,iter]=eigpower(A-lambda*eye(7),tol,nmax,x0);
>>lambda2+lambda
ans=
-1.12488541976457
```

经过 $\text{iter}=33$ 次迭代,得到的结果可以作为矩阵 A 的最大(正、负)特征值的近似值。

解 6.7 因为 A 的特征多项式系数均为实数,特征值将以复数对的形式出现,并且两个成对复特征值同在一个 Gershgorin 环内。矩阵 A 有 2 个列环与其他环分离(参见图 9.9 的左图),这两个列环中各自分别含有一个特征值,且均为实数。于是 A 至少有 2 个实特征值。

再来考虑矩阵 B ,它只有一个独立的环(参见 9.9 的右图)。利用前面的分析结论可知,相应的特征值必然是实数。而对于其余的特征值,有两种情况:或者都是实数,或者 1 个实数 2 个复数。

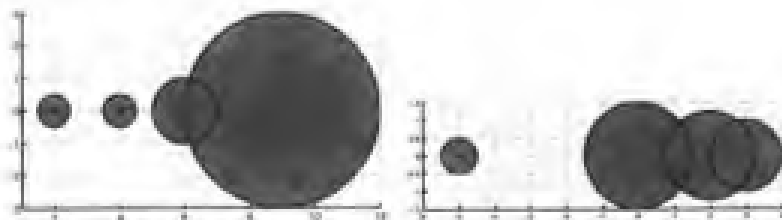


图 9.9 左图:解 6.7 中矩阵 A 的列环。右图:解 6.7 中 B 矩阵的列环

解 6.8 A 的行环中有一个圆心为 5 半径为 2 的独立的环,按模最大的特征值必然在这个环中。因此,可以设移位置量等于 5。在采用移位和没有移位这两种情况

下, 要比较二者的迭代运算次数和幂法的计算代价, 可以通过下列指令实现:

```
>>A=[5 0 -1;0 2 0 -1/2;0 1 -1 1; -1 -1 0 0];
>>flops(0); [lambda,x,iter]=eigpower(A,tol,nmax,x0); flops,iter
ans=
    2204
iter=
    34
>>flops(0);[lambda,x,iter]invshift(A,tol,nmax,5,x0);flops,iter
ans=
    1082
iter=12
```

可见与一般幂法相比, 原点移位法所需的迭代次数较少(1 比 3), 代价只是一般幂法的一半(此时考虑了进行 Gauss 因式分解时所需的额外时间)。

解 6.9 使用 `qr` 函数可以得到:

```
>>A=[2-1/2 0 -1/2; 0 4 0 2; -1/2 0 6 1/2; 0 0 1 9];
>>[Q,R]=qr(A)
Q=
   -0.9701    0.0073   -0.2389   -0.0411
         0   -0.9995   -0.0299   -0.0051
   0.2425    0.0294   -0.9557   -0.1643
         0         0   -0.1694    0.9855
R=
   -2.0616    0.4851    1.4552    0.6063
         0   -4.0018    0.1764   -1.9881
         0         0   -5.9035   -1.9426
         0         0         0    8.7981
```

要证明 RQ 相似于 A , 可以写出

$$Q^T A = Q^T Q R = R$$

上式利用了 Q 的正交性。于是 $C = Q^T A Q = RQ$, 由于 $Q^T = Q^{-1}$, 可以得出 C 相似于 A 的结论。

解 6.10 给定算法的一种简单实现方式如下:

```
function [A]=qrsimple(A,it)
for I=1:it
[Q,R]=qr(A);
A=R*Q;
end
```

其中, it 代表在算法终止时所进行的迭代次数。对于一个给定的矩阵, 经过 10 次迭代, 可以得到 $A^{(10)}$:

```
>>A10=
```

```

9.2090    1.1678    1.5237    0.0705
0.1220    4.9228   -0.5739    0.0461
-0.1732   -1.3095    4.8686   -0.6902
0          0          0.0017    1.9996

```

而经过 50 次迭代运算可以得到矩阵 $A^{(50)}$ ：

```

>>A50=
9.1728   -0.8113    1.8631    0.1059
0.0000    5.8003    0.6208    0.5914
-0.0000   -0.0000    4.0268   -0.3526
0          0          0.0000    2.0000

```

值得注意的是，由于 $A^{(k+1)} = (Q^{(k)})^T A^{(k)} Q^{(k)}$, $k \geq 0$ (参见解 6.9)，所以矩阵 $A^{(k)}$ 之间互为相似矩阵。因此可以推断出当 $k \rightarrow \infty$ 时，矩阵序列趋于一个上三角阵，其元素是矩阵 A 的特征值。

解 6.11 可以利用 eig 函数，写出下列指令： $[X, D] = \text{eig}(A)$ ，其中 X 是以 A 的单位特征向量作为列的矩阵， D 是以 A 的特征值作为对角元素的对角阵。对于习题 6.7 中的矩阵 A 和 B 可以采用下列指令：

```

>>A=[2 -1/2 0 -1/2; 0 4 0 2; -1/2 0 6 1 1/2; 0 0 1 9];
>>eig(A)
ans=
2.0000
5.8003
9.1728
4.0268
>>B=[-5 0 1/2 1/2; 1/2 2 1/2 0; 0 1 0 1/2; 0 1/4 1/2 3];
>>eig(B)
ans=
-4.9921
2.1666
-0.3038
3.1292

```

9.7 第 7 章

解 7.1 取插值为 $h: 1/2, 1/4, 1/8, \dots, 1/512$ ，利用前向欧拉法可以求得柯西问题(7.43)的精确解 $y(t) = \frac{1}{2}[e^t - \sin(t) - \cos(t)]$ 的近似值。相关误差可以通过下列指令来得到：

```

>>y0=0; f=inline('sin(t)+y', 't', 'y');
y='0.5*(exp(t)-sin(t)-cos(t))';
>>tspan=[0 1]; N=2; for k=1:10
[tt,u]=feuler(f,tspan,y0,N);t=tt(end);e(k)=abs(u(end)-eval(y));N=2

```

```
*N;end
>>e
e=
Columns 1 through 7
0.4285    0.2514    0.2379    0.0725    0.0372    0.0189    0.0095
Columns 8 through 10
0.0048    0.0024    0.0012
```

再利用公式(1.11)来估计收敛阶数:

```
>>p=log(abs(e(1:end-1)./e(2:end)))/log(2)
p=
Columns 1 through 7
0.7696    0.8662    0.9273    0.9620    0.9806    0.9902    0.9951
Columns 8 through 7
0.9975    0.9988
```

正如所预料的那样,此时的收敛阶数是 1, 利用同样的指令(用 beuler 函数来代替 feuler 函数), 可以得到后向欧拉法的收敛阶数的近似值:

```
>>p=log(abs(e(1:end-1)./e(2:end)))/log(2)
p=
Columns 1 through 7
1.5199    1.1970    1.0881    1.0418    1.0204    1.0101    1.0050
Columns 8 through 9
1.0025    1.0012
```

解 7.2 利用前向欧拉法求解给定柯西问题的数值解, 可以通过下列指令实现:

```
>>tspan=[0 1];N=100;f=inline('-t*exp(-y) ','t','y');y0=0;
>>[t,u]=feuler(f,tspan,y0,N);
```

要计算有效数字的位数, 首先需要估计出公式(7.11)中的常量 L 和 M 的值。因为在给定区间内 $f(t, y(t)) < 0$, $y(t) = \log(1-t^2/2)$ 是单调递减函数, 且在 $t=0$ 时取值为零。又因为 f 和它的一阶导数均为连续函数, 可以得出 L 的估计值为 $L = \max_{0 \leq t \leq 1} |L(t)|$, 其中, $L(t) = \partial f / \partial y = te^{-y}$ 。同时对于所有的 $t \in (0, 1]$ 有 $L(0) = 0$ 和 $L(t) > 0$ 成立。于是 $L = e$ 。

类似地, 由于在 $t=1$ 时函数有最大值, 可得 $M = \max_{0 \leq t \leq 1} |y''(t)|$, 其中, $y'' = -e^{-y} - t^2 e^{-2y}$, 于是 $M = e + e^2$ 。由式(7.11)可得:

$$|u_{100} - y(1)| \leq \frac{e^L - 1}{L} \frac{M}{200} = 0.26$$

因此, 此时只能保证第一位有效数字是准确的。实际上可得 $u(end) = -0.6785$, 而精确解在 $t=1$ 处的值为 $y(1) = -0.6931$ 。

解 7.3 迭代函数为 $\phi(u) = u_n - ht_{n+1}e^{-u}$, 当 $|\phi'(u)| < 1$ 时固定点迭代是收敛的。

这一性质可以通过条件 $h(t_0 + (n+1)h) < e^u$ 来保证。如果用精确解来代替 u , 则可以得到 h 值的一个先验估计。在 $u = -1$ 时是条件最严格的情况(参见解 7.2)。此时, 不等式 $(n+1)h^2 < e^{-1}$ 的解为 $h < \sqrt{e^{-1}/(n+1)}$ 。

解 7.4 重复执行解 7.1 中指令, 但此时利用 `cranknic` 函数(程序 14)来代替 `feuler` 函数。通过理论分析可以得到下列二阶收敛结果:

```
>>p=log(abs(e(1:end-1)./e(2:end)))/log(2)
p=
Columns 1 through 7
    2.0379    2.0092    2.0023    2.0006    2.0001    2.0000    2.0000
Columns 8 through 9
    2.0000    2.0000
```

解 7.5 在区间 $[t_n, t_{n+1}]$ 内考虑柯西问题(7.4)的积分公式:

$$\begin{aligned} y(t_{n+1}) - y(t_n) &= \int_{t_n}^{t_{n+1}} f(\tau, y(\tau)) d\tau \\ &= \frac{h}{2} [f(t_n, y(t_n)) + f(t_{n+1}, y(t_{n+1}))] \end{aligned}$$

上式中已经利用了梯形公式(4.17)来计算积分的近似值。设 $u_0 = y(t_0)$, 用近似值 u_n 来代替 $y(t_n)$, 用等号 “=” 来代替 “ $\approx$ ”, 可得:

$$u_{n+1} = u_n + \frac{h}{2} [f(t_n, u_n) + f(t_{n+1}, u_{n+1})], \quad \forall n \geq 0$$

上式即为 Crank-Nicolson 法。

解 7.6 设 $h\lambda = x + iy$, 所以 $|1 + h\lambda|^2 = (1+x)^2 + y^2$ 。因此假设条件 $|1 + h\lambda| < 1$ 就等价于 $(1+x)^2 + y^2 < 1$ 。可见在复平面上, 前向欧拉法的绝对稳定区域就是单位圆(参见图 9.11)。

解 7.7 利用解 7.6 中得出的结论可知, 要满足题设条件, 必须有 $|1 - h + ih| < 1$, 即 $0 < h < 1$ 。

解 7.8 将 Heun 法写成下列形式(与 Runge-Kutta 法类似):

$$u_{n+1} = u_n + \frac{1}{2}(k_1 + k_2), \quad k_1 = hf(t_n, u_n), \quad k_2 = hf(t_{n+1}, u_n + k_1) \quad (9.4)$$

于是有 $h\tau_{n+1}(h) = y(t_{n+1}) - y(t_n) - (\hat{k}_1 + \hat{k}_2)/2$, 其中, $\hat{k}_1 = hf(t_n, y(t_n))$, $\hat{k}_2 = hf(t_{n+1}, y(t_n) + \hat{k}_1)$ 。因此有下式成立:

$$\lim_{h \rightarrow 0} \tau_{n+1} = y'(t_n) - \frac{1}{2} [f(t_n, y(t_n)) + f(t_n, y(t_n))] = 0$$

所以该方法是二阶一致的。

程序 19 是 Heun 法的一个实现。利用这一程序求出的收敛阶数与解 7.1 中的结论一致。利用下列指令，可以看出 Heun 法相对于 h 是二阶收敛的：

```
>> p=log(abs(e(1:end-1./e(2:end))))/log(2)
p=
Columns 1 through 7
    1.7642    1.8796    1.9398    1.9700    1.9851    1.9925    1.9963
Columns 8 through 9
    1.9981    1.9991
```

程序 19—rk2: Heun 法

```
function [t,u]=rk2(odefun,tspan,y0,Nh,varargin)
h=(tspan(2)-tspan(1)-t0)/Nh;tt=[tspan(1):h:tspan(2)]; u(1)=y0;
for s=tt(1:end-1)
    t=s; y=u(end);k1=h*(feval(odefun,t,y,varargin{:}));
    t=t+h;y=y+k1;k2=h*(feval(odefun,t,y,varargin{:}));
    u=[u,u(end)+0.5*(k1+k2)];
end
t=tt;
```

解 7.9 把公式(9.4)应用于问题(7.19)时，可以得到 $k_1 = h\lambda u_n$ 和 $k_2 = h\lambda u_n(1+h\lambda)$ 。因此 $u_{n+1} = u_n[1+h\lambda + (h\lambda)^2/2] = u_n p_2(h\lambda)$ 。为保证该方法绝对稳定需要 $|p_2(h\lambda)| < 1$ ，因为 $p_2(h\lambda)$ 为正，所以 $0 < p_2(h\lambda) < 1$ 。解后一个不等式可以得出 $-2 < h\lambda < 0$ ，即 $h < 2/|\lambda|$ 。

解 7.10 因为

$$u_n = u_{n-1}(1 + h\lambda_{n-1}) + hr_{n-1}$$

将上式进行 n 次递归运算，即可得到所证结论。

解 7.11 设

$$\varphi(\lambda) = \left\| 1 + \frac{1}{\lambda} + \frac{1}{\lambda} \right\|$$

则由(7.28)可以很容易地推导出不等式(7.29)。

解 7.12 由式(7.26)可得：

$$|z_n - u_n| \leq \rho_{\max} a^n + h \rho_{\max} \sum_{k=0}^{n-1} \delta(h)^{n-k-1}$$

再利用公式(7.27)即可得出要证明的结论。

解 7.13 首先

$$\begin{aligned} h\tau_{n+1}(h) &= y(t_{n+1}) - y(t_n) - \frac{1}{6}(\hat{k}_1 + 4\hat{k}_2 + \hat{k}_3) \\ \hat{k}_1 &= hf(t_n, y(t_n)), \quad \hat{k}_2 = hf\left(t_n + \frac{h}{2}, y(t_n) + \frac{\hat{k}_1}{2}\right) \\ \hat{k}_3 &= hf\left(t_{n+1}, y(t_n) + 2\hat{k}_2 - \hat{k}_1\right) \end{aligned}$$

又因为

$$\lim_{h \rightarrow 0} \tau_{n+1} = y'(t_n) - \frac{1}{6}[f(t_n, y(t_n)) + 4f(t_n + \frac{h}{2}, y(t_n) + \frac{\hat{k}_1}{2}) + f(t_n + h, y(t_n) + 2\hat{k}_2 - \hat{k}_1)] = 0$$

所以这种方法是一致的。

这种方法就是 3 阶 Runge-Kutta 法的显示形式, 程序 20 是它的一个实现。与解 7.8 相同, 可以利用下列指令来近似求出收敛阶数:

```
>> p=log(abs(e(1:end-1)./e(2:end)))/log(2)
p=
Columns 1 through 7
2.7306 2.8657 2.9330 2.9666 2.9833 2.9916 2.9958
Columns 8 through 9
2.9979 2.9990
```

解 7.14 由解 7.9 可以得到下列关系式:

$$u_{n+1} = u_n \left[1 + h\lambda + \frac{1}{2}(h\lambda)^2 + \frac{1}{6}(h\lambda)^3 \right] = u_n p_3(h\lambda)$$

利用下列指令可以得到 p_3 的曲线

```
>>c=[1/6 1/2 1 1];z=[-3;0.01:1]; p=polyval(c,z); plot(z,abs(p))
```

由曲线可以看出, 当 $-2.5 < h\lambda < 0$ 时, $|p_3(h\lambda)| < 1$ 。

程序 20—rk3: 3 阶显式 Runge-Kutta 法

```
function [t,u]=rk3(odefun,tspan,y0,Nh,varargin)
h=(tspan(2)-tspan(1))/Nh; tt=[tspan(1):h:tspan(2)]; u(1)=y0;
for s=tt(1:end-1)
t=s; y=u(end); k1=h*feval(odefun,ty,y,varargin{:});
t=t+h*0.5;y=y+0.5*k1; k2=h*feval(odefun,t,y,varargin{:});
```

```

t=s+h; y=u(end)+2*k2-k1;k3=h*feval(odefun,t,y,varargin{:});
u=[u,u(end)+(k1+4*k2+k3)/6];
end
t=tt;

```

解 7.15 对于问题(7.19)应用式(7.46)可以得到方程 $u_{n+1} = u_n(1 + h\lambda + (h\lambda)^2)$ 。从 $1 + z + z^2$ (其中 $z = h\lambda$) 的曲线可以得出, 当 $-1 < h\lambda < 0$ 时, 该方法是绝对稳定的。

解 7.16 利用给定的数值求解问题 7.1, 分别取 $N=10$, $N=20$, 重复执行下列指令:

```

>>f=inline('-1.68*10^(-9)*y^4+2.6880','t','y');
>>[t,uc]=cranknic(f,[0,200],180,N);
>>[t,u]=predcor(f,[0 200],180,N,'feonestep','cnonestep');

```

图 9.10 给出了计算得到的曲线。利用 Crank-Nicolson 法得到的结果要比 Heun 法得到的结果更加精确。

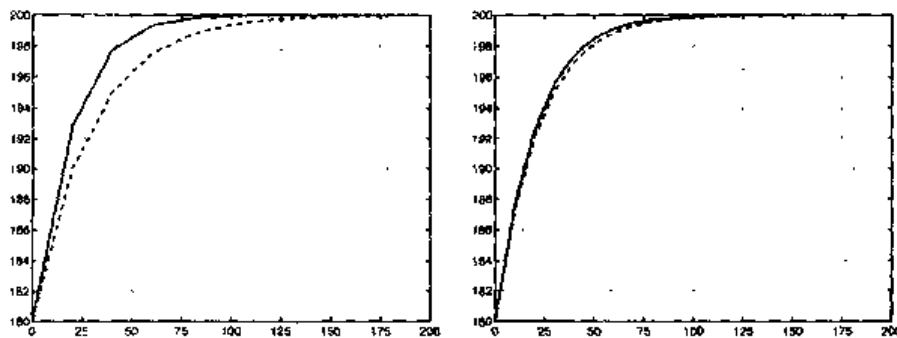


图 9.10 解 7.16 中的柯西问题得到的数值解(左图为 $h=20$ 的情况, 右图为 $h=10$ 的情况; 实线表示 Crank-Nicolson 法得到的结论, 虚线表示 Heun 法得到的结论)

解 7.17 把 Heun 法应用于问题(7.19)中可得:

$$u_{n+1} = u_n \left(1 + h\lambda + \frac{1}{2}h^2\lambda^2 \right)$$

在复平面上, 它的绝对稳定区域的边界为 $|1 + h\lambda + h^2\lambda^2/2| = 1$, 其中设 $h\lambda = x + iy$ 。满足该方程的数对 (x, y) 必须使得下式成立: $f(x, y) = x^4 + y^4 + 2x^2y^2 + 4x^3 + 4xy^2 + 8x^2 + 8x = 0$ 。可以绘出在 $f(x, y) = z$ 时的水平线(相应取 $z=0$), 这一操作可通过下列指令实现:

```

>>f='x.^4+y.^4+2*(x.^2).*(y.^2)+4*x.*y.^2+4*x.^3+8*x.^2+8*x';
>>[x,y]=meshgrid([-2.1:0.1: 0.1],[-2:0.1:2]);
>>contour(x,y,eval(f),{0 0})

```

可以利用 meshgrid 函数在矩形区域 $[-2.1, 0.1] \times [-2, 2]$ 内绘出计算网格, 该网格由 x 方向上均匀分布的 23 个节点, 在 y 轴上均匀分布的 41 个节点构成。利用

contour 函数可以绘出 $f(x, y)$ 与 $z=0$ (设 contour 的输入向量为 $[0 \ 0]$) 相对应的水平线 (可以利用 eval(f) 函数实现)。在图 9.11 中的实线表示 Heun 法的绝对稳定区域, 它大于前向欧拉法的相应区域 (图中以虚线圆的内部区域表示)。在原点 $(0, 0)$ 处, 两条曲线都是关于虚轴的正切。

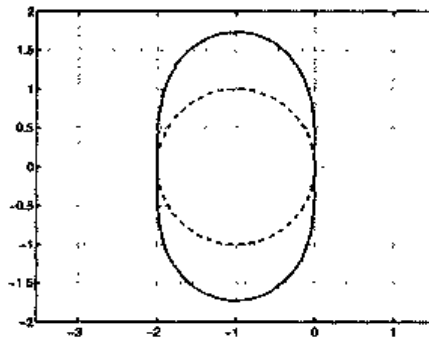


图 9.11 Heun 法(实线)和前向欧拉法(虚线)的绝对稳定区域的边界。
其中边界内部区域即为稳定区域

解 7.18 利用下列指令:

```
>>tspan=[0 1]; y0=0; t=inline('cos(2*y) ','t','y');
>>y='05*asin((exp(4*t)-1)./exp(4*t)+1))';
>>N=2;for k=1:10
[tt,u]=predcor(f,tspan,y0,N,'feonestpe','cnonestep');
t=tt(end);e(k)=abs(u(end)-eval(y));n=2*N;end
>>p=log(abs(e(1:end-1)./e(2:end)))/log(2)
p=
Columns 1 through 7
2.4733 2.2507 2.1223 2.0601 2.0298
2.0148 2.0074
Columns 8 through 9
2.0037 2.0018
```

正如预料的结果一样, 该方法的收敛阶数是 2。但它的计算代价与只具有一阶准确度的前向欧拉法相同。

解 7.19 该二阶微分方程等价于下列一阶系统:

$$\begin{aligned} x' &= z, & z' &= -5z - 6x \end{aligned}$$

且 $x(0)=1, z(0)=0$ 。利用 Heun 法写出下列指令:

```
>>.tspan=[0 5];y0=[1 0];
>>[tt,u]=predcor('fspring',tspan,y0,N,'feonestep','cnonestep');
```

其中, N 是节点的数目, fspring.m 是下列函数:

```
function y=fspring(t,y)
b=5;k=6;
```

```
yy=y; y(1)=yy(2); y(2)=-b*yy(2)-k*yy(1);
```

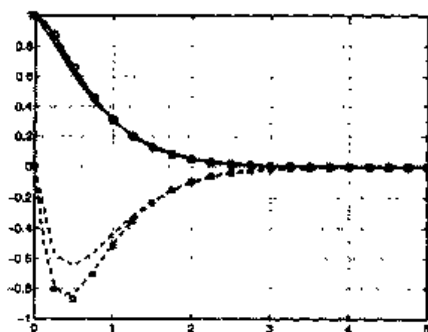


图 9.12 分别取 $N=20$ (细线)和 $N=40$ (粗线)计算得到的 $x(t)$ (实线)和 $x'(t)$ (虚线)的近似值。有圆圈和方块标记的曲线分别表示 $x(t)$ 和 $x'(t)$ 的精确解

在图 9.12 中给出了解的两个部分, 分别取 $N=20, 40$ 来进行计算, 并把它们与精确解 $x(t) = 3e^{-2t} - 2e^{-3t}$ 和它的一阶导数的曲线进行比较。

解 7.20 题设中的二阶差分方程系统可以化为下列形式的一阶系统:

$$\begin{cases} x' = z \\ y' = v \\ z' = 2\omega \sin(\psi) - k^2 x \\ v' = -2\omega \sin(\psi)z - k^2 y \end{cases} \quad (9.5)$$

设在初始时刻 $t_0 = 0$ 时, 摆在位置 $(1, 0)$ 处静止, 则系统(9.5)的初始条件如下:

$$x(0)=1, y(0)=0, z(0)=0, v(0)=0$$

设 $\psi = \pi/4$, 它是意大利北部的平均纬度, 利用前向欧拉法, 采用下列指令:

```
>>[t,y]=feuler('ffocault',[0 300],[1 0 0 0],Nh);
```

其中, Nh 是运算进行的次数, ffocault.m 为下列函数:

```
function y=ffocault(t,y)
1=20; k2=9.8/1; psi=pi/4; omega=7.29*1.e-0.5;
yy=y; y(1)=yy(3); y(2)=yy(4);
y(3)=2*omega*sin(psi)*yy(4)-k2*yy(1);
y(4)=-2*omega*sin(psi)*yy(3)-k2*yy(2);
```

数值实验表明, 即使在 h 取值非常小的情况下, 前向欧拉法也不能给出令人满意的结论。例如, 图 9.13 的左图中给出的图形是在 $N=30000$ 即 $h=1/100$ 时, 计算得到相平面 (x, y) 内的摆的运动情况。正如所分析的那样, 旋转平面是随时间而改变的, 同时振动的幅度也是增加的。利用 Heun 法, 取比较小的 h 可以得到类似的结论。实际上, 这类模型问题有一个纯虚数的系数 λ 。相应的解(正弦曲线)与 t 一样是趋于

无穷的, 而不是趋于零的。

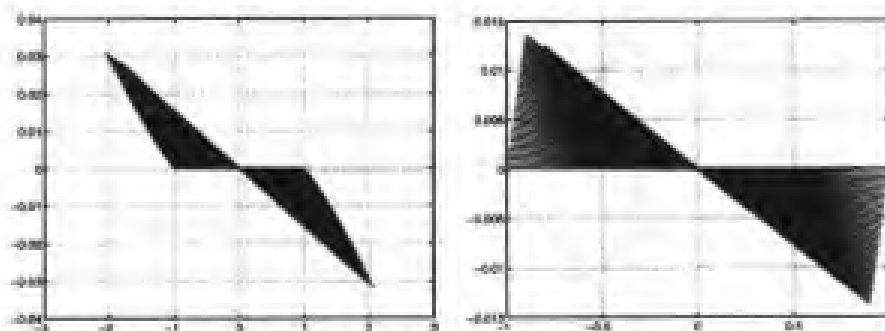


图 9.13 习题 7.20 中利用前向欧拉法(左图)和 3 阶自适应 Runge-Kutta 法(右图)解出的 Foucault 摆在相平面上的轨迹

但是, 前向欧拉法和 Heun 法的绝对稳定区域均不包括虚轴上的点(原点除外), 因此为了确保绝对稳定性, 只能选取无效值 $h=0$ 。

因此为了得到可接受的解, 必须选用一种方法, 它的绝对稳定区域包括虚轴的一部分。3 阶自适应 Runge-Kutta 法就是这样一种方法, MATLAB 中的 ode23 函数就是这种方法的一个实现。可以通过下列指令来调用该函数:

```
>>[t,u]=ode23('ffocault',[0 300],[1 0 0 0]);
```

图 9.13 给出的结果是经过 1022 次迭代得到的。注意其中数值解与解析解是高度一致的。

9.8 第 8 章

解 8.1 可以直接验证 $x^T Ax > 0$ 对于所有的 $x \neq 0$ 均成立。实际上,

$$\begin{aligned}
 & [x_1 \ x_2 \ \dots \ x_{N-1} \ x_N] \begin{bmatrix} 2 & -1 & 0 & \dots & 0 \\ -1 & 2 & \ddots & & \vdots \\ 0 & \ddots & \ddots & -1 & 0 \\ \vdots & & -1 & 2 & -1 \\ 0 & \dots & 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{N-1} \\ x_N \end{bmatrix} \\
 &= 2x_1^2 - 2x_1x_2 + 2x_2^2 - 2x_2x_3 + \dots - 2x_{N-1}x_N + 2x_N^2
 \end{aligned}$$

上面的表达式等于 $(x_1 - x_2)^2 + \dots + (x_{N-1} - x_N)^2 + x_1^2 + x_N^2$, 只要存在一个非零元素 x_i , 该式的值就是大于零的。

解 8.2 验证 $Aq_j = \lambda_j q_j$ 。矩阵和向量的乘积为 $w = Aq_j$, 要 w 等于向量 $\lambda_j q_j$, 可

得:

$$\left\{ \begin{array}{l} 2\sin(j\theta) - \sin(2j\theta) = 2(1 - \cos(j\theta))\sin(j\theta) \\ -\sin(jk\theta) + 2\sin(j(k+1)\theta) - \sin(j(k+2)\theta) = 2(1 - \cos(j\theta))\sin(2j\theta) \\ \quad k=1, \dots, N-2 \\ 2\sin(Nj\theta) - \sin((N-1)j\theta) = 2(1 - \cos(j\theta))\sin(Nj\theta) \end{array} \right.$$

第一个方程是显而易见的, 因为 $\sin(2j\theta) = 2\sin(j\theta)\cos(j\theta)$ 。另外两个方程可以利用下式进行化简:

$$\begin{aligned} \sin(jk\theta) &= \sin((k+1)j\theta)\cos(j\theta) - \cos((k+1)j\theta)\sin(j\theta) \\ \sin(j(k+2)\theta) &= \sin((k+1)j\theta)\cos(j\theta) + \cos((k+1)j\theta)\sin(j\theta) \end{aligned}$$

因为 A 是正定对称阵, 它的条件数是 $K(A) = \lambda_{\max}/\lambda_{\min}$, 即 $K(A) = \lambda_1/\lambda_N = (1 - \cos(N\pi/(N+1)))/(1 - \cos(\pi/(N+1)))$ 。利用余弦函数的二阶 Taylor 展开式, 可以得到 $K(A) \simeq N^2$, 即 $K(A) \simeq h^{-2}$ 。

解 8.3 由于

$$\begin{aligned} u(\bar{x}+h) &= u(\bar{x}) + hu'(\bar{x}) + \frac{h^2}{2}u''(\bar{x}) + \frac{h^3}{6}u'''(\bar{x}) + \frac{h^4}{24}u^{(4)}(\xi_+) \\ u(\bar{x}-h) &= u(\bar{x}) + hu'(\bar{x}) + \frac{h^2}{2}u''(\bar{x}) - \frac{h^3}{6}u'''(\bar{x}) + \frac{h^4}{24}u^{(4)}(\xi_-) \end{aligned}$$

其中, $\xi_+ \in (x, x+h)$, $\xi_- \in (x-h, x)$ 。两个表达式相加可以得到:

$$u(\bar{x}+h) + u(\bar{x}-h) = 2u(\bar{x}) + h^2u''(\bar{x}) + \frac{h^4}{24}(u^{(4)}(\xi_+) + u^{(4)}(\xi_-))$$

于是命题得证。

解 8.4 矩阵是三角矩阵, 元素为 $a_{i,i-1} = -1 - h\frac{\delta}{2}$, $a_{ii} = 2 + h^2\gamma$, $a_{i,i+1} = -1 + h\frac{\delta}{2}$ 。

考虑到边值条件, 方程右侧函数为

$$f = (f(x_1) + \alpha(1 + h\delta/2)/h^2, f(x_2), \dots, f(x_{N-1}), f(x_N) + \beta(1 - h\delta/2)/h^2)^T$$

解 8.5 利用下列指令可以计算出与三个给定的 h 值相对应的解:

```
>>fbvp=inline('1+sin(4*pi*x)','x');
>>[z,uh10]=bvp(0,1,9,0,0.1,fbvp,0,0);
```

```
>>[z,uh20]=bvp(0,1,19,0,0.1,fbvp,0,0);
>>[z,uh40]=bvp(0,1,39,0,0.1,fbvp,0,0);
```

由于我们不知道精确解的值，要估计收敛阶数，可以选取一个比较合适的步长来计算解的近似值(比如取 $h=1/1000$)，然后用这个值来代替精确解。于是有：

```
>>[z,uhex]=bvp(0,1,999,0,0.1,fbvp,0,0);
>>max(abs(uh9-uhex(1:50:end)))
ans=
    8.6782e-04
>>max(abs(uh19-uhex(1:50:end)))
ans=
    2.0422e-04
>>max(abs(uh39-uhex(1:25:end)))
ans=
    5.2789e-05
```

把 h 减小一半，则误差减为原来的 $1/4$ ，于是可以得出关于 h 的收敛阶数为 2。

解 8.6 为使得到的解是单调的(与解析解一样)，要找出一个最大的 h_{crit} ，可以使用下列循环实现：

```
>>fbvp=inline('1+0.*x','x'); for k=3:1000
[z,uh]=bvp(0,1,k,100,0,fbvp,0,1); if sum(diff(uh)>0)==length(uh)-1,
break,end,end
```

改变 $h(=1/(k+1))$ 的值，直到数值解的增量比全都大于零为止。可以利用向量 $\text{diff}(uh)$ 来实现，如果相应的增量比为正，则它的元素为 1，否则为 0。如果所有元素之和等于 uh 的长度减 1，则所有的增量比为正值。上面的循环在 $k=49$ 时停止，即如果取 $\delta=1000$ ， $h=1/500$ ；如果取 $\delta=2000$ ， $h=1/1000$ 。因此可以推测出为了得到一个单调递增的数值解，需要 $h < 2/\delta = h_{\text{crit}}$ 。实际上，关于 h 的这一限制条件也可以通过理论方法证明得到(参见 [QV94])。在图 9.14 中给出了两个不同 h 取值下的数值解，此时取 $\delta=100$ 。

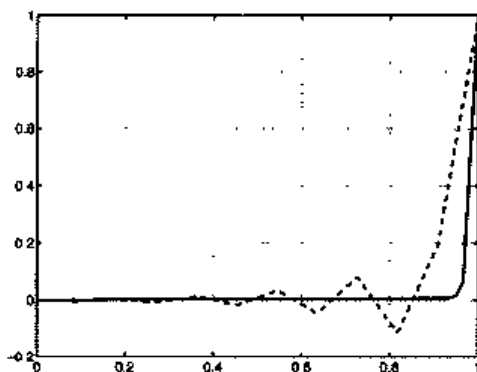


图 9.14 问题 8.6 的数值解(虚线表示 $h=1/10$ 的情况，实线表示 $h=1/60$ 的情况)

解 8.7 为了利用 Neumann 边值条件, 需要对程序 17 作一些修改, 下面的程序 21 就是一种实现形式:

程序 21—neumann: 一个 Neumann 边值问题的逼近

```
function [x,uh]=Neumann(a,b,N,delta,gamma,bvpfun,ua,ub,varargin)
h=(b-a)/(N+1); x=[a:h:b]; e=ones(N+2,1);
A=spdiags([-e-0.5*h*delta 2*e+gamma*h^2-e+0.5*h*delta],
-1:1,N+2,N+2);
f=h^2*feval(bvpfun,'x',varargin{:}); f=f';
A(1,1)=-3/2*h;A(1,2)=2*h;A(1,3)=-1/2*h;f(1)=h^2*ua;
A(N+2,N+2)=3/2*h;A(N+2,N+1)=-2*h;A(N+2,N)=1/2*h;f(N+2)=h^2*ub;
uh=A\f;
```

解 8.8 对于两个小区间 I_{k-1} 和 I_k 使用梯形求积公式, 可得下列近似值:

$$\int_{I_k \cup J_k} f(x) \varphi_k(x) dx = \frac{h}{2} f(x_k) + \frac{h}{2} f(x_k) = hf(x_k)$$

因为 $\varphi_k(x_j) = \delta_{jk}$, $\forall j, k$ 。可见这种方法与利用有限差分法得到的方程右侧是一致的。

解 8.9 因为 $\nabla \phi = (\partial \phi / \partial x, \partial \phi / \partial y)^T$, 因此可得 $\operatorname{div} \nabla \phi = \partial^2 \phi / \partial x^2 + \partial^2 \phi / \partial y^2$, 也就是 Laplacian 算子对 ϕ 运算得到的结果。

解 8.10 为了计算钢板中心的温度, 先求出相应的 Poisson 问题在 $\Delta_x = \Delta_y$ 时的解, 可以利用下列指令:

```
>>k=0;fun=inline('25','x','y');bound=inline('(x==1)','x','y');
>>for N=[10,20,40,80,160],
[u,x,y]=poissonfd(0,0,1,1,N,N,fun,bound);
k=k+1;uc(k)=u(N/2+1,N/2+1);end
```

向量 **uc** 的元素就是在网格尺寸为 h 时计算得到的钢板中心温度的值。即

```
>>uc
2.0168 2.0616 2.0789 2.0859 2.0890
```

于是可以得出在钢板中心温度的近似值是 2.08°C 。在图 9.15 中给出了在不同 h 值下的水平线。

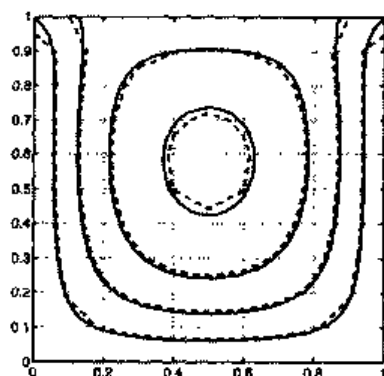


图 9.15 计算得到的温度水平曲线(虚线为 $\Delta_x = \Delta_y = 1/10$ 时的情况, 实线为 $\Delta_x = \Delta_y = 1/80$ 的情况)

参 考 书 目

- [Arn73] Arnold V. (1973) Ordinary Differential Equations. The MIT Press, Cambridge, Massachusetts.
- [AtkS!]] Atkinson K. (1989) An Introduction to Numerical Analysis. ,John Wiley, New York.
- [Axe94] Axelsson O. (1994) Iterative Solution Methods. Cambridge University Press, New York.
- [BB96] Brassar(l G. and Bratley P. (1996) Fundamentals of Algorithmics, l/e. Prentice Hall, New York.
- [BM92] Bernardi C. and Maday Y. (1992) Approximation Spectrales des Problèmes aux Limites Elliptiques. SpringerVerlag, Paris.
- [Bra97] Bra.ess I). (1997) Finite Elements: Theory, Fast Solvers and Applications in Solid Mechanics. Cambridge University Press, Cambridge.
- [BS01] Babuska I. and Strouboulis T. (2001) The Finite Element Method and its Reliability. Oxford University Press.
- [ButS7] Butcher J. (1987) The Numerical Analysis of Ordinary Differential Equations: Runge-Kutta and General Linear Methods. Wiley, Chichester.
- [CHQZ88] Canuto C., Hussaini M. Y., Quarteroni A., and Zang T. A. (1988) Spectral Methods in Fluid Dynamics. Springer, New York.
- [Dav63] Davis P. (1963) Interpolation and Approximation. Blaisdell Pub., New York.
- [DB02] Deuflhard P. and Bornemann F. (2002) Scientific Computing with Ordinary Differential Equations. Springer Verlag, New York.
- [DD95] Davis T. and Duff I. (1995) A Combined Unifrontal/Multifrontal Method for Unsymmetric Sparse Matrices. TR-95-020. University of Florida.
- [Die93] Dierckx P. (1993) Curve and Surface Fitting with Splines. Clarendon Press, New York.
- [DL92] DeVore R. and Lucier J. (1992) Wavelets. Acta Numerica pages 1~56.
- [DR75] Davis P. and Rabinowitz P. (1975) Methods of Numerical, Integration. Academic Press, New York.
- [D583] Dennis J. and Schnabel R. (1983) Numerical Methods for

- Unconstrained Optimization and Nonlinear Equations. Prentice-Hall, Englewood Cliffs, New York.
- [dV89] der Vorst H. V. (1989) High Performance Preconditioning. SIAM J. Sci. Stat. Comput. 10:1174-1185.
- [EEH,196] Eriksson K., Estep D., Hansbo P., and Johnson C. (1996) Computational Differential Equations. Cambridge Univ. Press, Cambridge.
- [EKH02] Etter D., Kuneicky D., and Hull D. (2002) Introduction to MATLAB 6. Prentice Hall.
- [Fun92] Funaro D. (1992) Polynomial Approximation of Differential Equations. Springer-Verlag, Berlin.
- [Gan97] Gautschi W. (1997) Numerical Analysis. AT, Introduction. Birkhäuser, Berlin.
- [GL89] Golub G. and Loan C. V. (1989) Matrix Computations. The John Hopkins Univ. Press, Baltimore and London.
- [Hac85] Hackbusch W. (1985) Multigrid Methods and Applications. Springer-Verlag, Berlin.
- [Hae94] Hackbusch W. (1994) Iterative Solution of Large Sparse Systems of Equations. Springer-Verlag, New York.
- [tlH00] Higham D. and Higham N. (2000) MATLAB Guide. SIAM, Philadelphia.
- [Hig96] Higham N. (1996) Accuracy and Stability of Numerical Algorithms. SIAM Publications, Philadelphia, PA.
- [Hir88] Hirsh C. (1988) Numerical Computation of Internal and External Flows, volume 1. John Wiley and Sons, Chichester.
- [HLR01] thmt B., Lipsman R., and Rosenberg J. (2001) A guide to MATLAB: for Beginners and Experienced Users. Cambridge University Press.
- [IK66] Isaacson E. and Keller It. (1966) Analysis of Numerical Methods. Wiley, New York.
- [IroT0] h'ons B. (1970) A Frontal Solution Program for Finite Element Analysis. Int. J. for Numer. Meth. in Engng. 2:5-32.
- [Krö98] Kröner D. (1998) Finite volume schemes in multidimensions. Pitman Res. Notes Math. Ser., 380, Longman, Harlow.
- [KS99] Karniadakis G. and Sherwin S. (1999) Spectral/hp Element Methods for CFD. Oxford University Press.
- [Lam91] Lalnbert J. (1991) Numerical Methods for Ordinary Differential Systems. John Wiley and Sons, Chichester.
- [Lan03] Langtangen U. (2003) Computational Partial Differential Equations:

- Unconstrained Optimization and Nonlinear Equations. Prentice -Hall, Englewood Cliffs, New York.
- [dV89] der Vorst H. V. (1989) High Performance Preconditioning. SIAM J. Sci. Stat. Comput. 10:1174 1185.
- [EEH,196] Eriksson K., Estep D., Hansbo P., and Johnson C. (1996) Computational Differential Equations. Cambridge Univ. Press, Cambridge.
- [EKH02] Etter D., Kuneicky D., and Hull D. (2002) Introduction to MATLAB 6. Prentice Hall.
- [Fun92] Funaro D. (1992) Polynomial Approximation of Differential Equations. Springer-Verlag, Berlin.
- [Gan97] Gautsehi W. (1997) Numerical Analysis. AT, Introduction. Birkh ä user, Berlin.
- [GL89] Golub G. and Loan C. V. (1989) Matrix Computations. The John Hopkins Univ. Press, Baltimore and London.
- [Hac85] Hackbush W. (1985) Multigrid Methods and Applications. Springer-Verlag, Berlin.
- [Hac94] Hackbush W. (1994) Iterative Solution of Large Sparse Systems of Equations. Springer-Verlag, New York.
- [tlH00] Higham D. and Higham N. (2000) MATLAB Guide. SIAM, Philadelphia.
- [Hig96] Higham N. (1996) Accuracy and Stability of Numerical Algorithms. SIAM Publications, Philadelphia, PA.
- [Hir88] Hirsh C. (1988) Numerical Computation of Internal and External Flows, volume 1. John Wiley and Sons, Chichester.
- [HLR01] thmt B., Lipsman R., and Rosenberg J. (2001) A guide to MATLAB: for Beginners and Experienced Users . Cambridge University Press.
- [IK66] Isaacson E. and Keller It. (1966) Analysis of Numerical Methods. Wiley, New York.
- [IroT0] h'ons B. (1970) A Frontal Solution Program for Finite Element Analysis. Int. J. for Numcr. Meth. in Engng. 2:5 32.
- [Kr ö 98] Kr ö ner D. (1998) Finite volume schemes in multidimensions. Pitman Res. Notes Math. Ser., 380, Longman, Harlow.
- [KS99] Karniadakis G. and Sherwin S. (1999) Spectral/hp Element Methods for CFD. Oxford University Press.
- [Lam91] Lalnbert J. (1991) Numerical Methods for Ordinary Differential Systems. John Wiley and Sons, Chichester.
- [Lan03] Langtangen U. (2003) Computational Partial Differential Equations:

- Numerical Methods and Diffpack Programming. Springer-Verlag, Heidelberg.
- [LeV02] LeVeque R. (2002) Finite Volume Methods for Hyperbolic Problems. Cambridge University Press, Cambridge.
- [Mci67] Meinardus G. (1967) Approximation of Functions: Theory and Numerical Methods. Springer-Verlag, New York.
- [Pan92] Pall V. (1992) Complexity of Computations with Matrices and Polynomials. SIAM Review 34:225~262.
- [PBP02] Prautzsch H., Boehm W., and Paluszny M. (2002) Bezier and B-Spline Techniques. Springer-Verlag.
- [PdDK Ü K83] Piessens R., de Doncker-Kapenga E., Üeberlmber C., and Kahaner D. (1983) QUADPACK: A Subroutine Package for Automatic Integration. Springer-Verlag, Berlin and Heidelberg.
- [QSS00] Quarteroni A., Sacco R., and Saleri F. (2000) Numerical Mathematics, volume 37 of Texts in Applied Mathematics. Springer-Verlag, New York.
- [QV94] Quarteroni A. and Valli A. (1994) Numerical Approximation of Partial Differential Equations. Springer, Berlin and Heidelberg.
- [RRB5] Ralston A. and Rabinowitz P. (1985) A First Course in Numerical Analysis. McGraw-Hill, Singapore. 7th printing.
- [Saa92] Saa Y. (1992) Numerical Methods for Large Eigenvalue Problems. Halstead Press, New York.
- [Saa96] Saad Y. (1996) Iterative Methods for Sparse Linear Systems. PWS Publishing Company, Boston.
- [SR97] Shampine L. F. and Reichelt M. W. (1997) The MATLABODE Suite. SIAM J. Sci. Comp. 18:1~22.
- [TW98] Tveito A. and Winther R. (1998) Introduction to Partial Differential Equations. A Computational Approach. Springer-Verlag.
- [Ü eb97] Üeberhuber C. (1997) Numerical Computation: Methods, Software, and Analysis. Springer-Verlag, Berlin Heidelberg.
- [Urb02] Urban K. (2002) Wavelets in Numerical Simulation. Springer Verlag.
- [Wes91] Wesseling P. (1991) An Introduction to Multigrid Methods. John Wiley and Sons, Chichester.
- [Wi165] Wilkinson J. (1965) The Algebraic Eigenvalue Problem. Clarendon Press, Oxford.

